

PrimeCell® MultiPort Memory Controller (PL172)

Revision: r2p3

Integration Manual



PrimeCell MultiPort Memory Controller (PL172)

Integration Manual

Copyright © 2003, 2004. ARM Limited. All rights reserved.

Release Information

The following changes have been made to this document.

Change history		
Date	Issue	Change
July 2003	A	First release for r2p2
22 March 2004	B	Second release for r2p3

Proprietary Notice

Words and logos marked with © or ™ are registered trademarks or trademarks owned by ARM Limited. Other brands and names mentioned herein may be the trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM in good faith. However, all warranties implied or expressed, including but not limited to implied warranties of merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Confidentiality Status

This document is Open Access. This document has no restriction on distribution.

Product Status

The information in this document is final, that is for a developed product.

Web Address

<http://www.arm.com>

Contents

PrimeCell MultiPort Memory Controller (PL172)

Integration Manual

	Preface	
	About this manual	xvi
	Feedback	xxi
Chapter 1	Introduction	
	1.1 About the PrimeCell MultiPort Memory Controller (PL172)	1-2
	1.2 Changes between r2p2 and r2p3	1-3
Chapter 2	Integration	
	2.1 Integration overview	2-2
	2.2 Synthesis	2-11
	2.3 Integration test	2-16
	2.4 Production test	2-18
	2.5 Functional and timing verification	2-19
	2.6 Customization	2-21
	2.7 Environment variables	2-28
Chapter 3	Static Memory Controller	
	3.1 Static memory device selection	3-2
	3.2 Write-protection	3-3

3.3	Extended wait transfers	3-4
3.4	Memory mapped peripherals	3-5
3.5	Static memory initialization	3-6
3.6	Elimination of floating bytes on the external interface	3-38
3.7	Byte lane control	3-39
3.8	Address connectivity	3-40
3.9	Byte lane control and data bus steering	3-47
Chapter 4	Dynamic Memory Controller	
4.1	Write-protection	4-2
4.2	Access sequencing and memory width	4-3
4.3	Dynamic memory transaction latency	4-4
4.4	Transaction examples	4-37
4.5	Address mapping	4-75
4.6	Dynamic memory controller command descriptions	4-80
4.7	Generic SDRAM initialization example	4-81
4.8	Micron MT48LC4M16A2 SDRAM initialization example	4-83
4.9	Low-power SDRAM initialization example	4-86
4.10	Micron MT28F4M16S2 SyncFlash initialization example	4-90
4.11	Micron SyncFlash commands	4-93
Chapter 5	Common Memory Transactions	
5.1	Summary	5-2
5.2	Write transfer to write-protected chip select	5-3
5.3	Buffer hit transactions	5-4
5.4	Buffer hit transaction waveforms	5-6
Chapter 6	Test Interface Controller	
6.1	About TIC	6-2
6.2	Sequence of events leading to entry into TIC test mode	6-3
Chapter 7	System Connectivity	
7.1	On-chip signals	7-2
7.2	Self-refresh entry	7-24
7.3	Example system	7-28
Chapter 8	Off-chip Connectivity	
8.1	Memory device selection	8-2
8.2	Off-chip signals	8-4
8.3	Clock relationships	8-9
8.4	Pad interface timing	8-11
8.5	Pad interface methodology	8-13
8.6	Clock delay methodology	8-14
8.7	Command delay methodology	8-19
8.8	Clock feedback in MPMC designs	8-31
8.9	Memory clock and feedback clock strategy	8-34

Chapter 9	Power Strategy	
9.1	ASIC silicon process and cell library	9-2
9.2	ASIC design decisions	9-3
9.3	Memory device selection	9-6
9.4	MPMC	9-9
 Chapter 10	 Pad Interface Timing	
10.1	Overview	10-2
10.2	Signal delay	10-3
10.3	Method to reduce delay	10-4
10.4	Methods to reduce skew	10-6
10.5	Methods to minimize the effects of delay and skew	10-7
10.6	Example SDRAM memory timing diagram	10-8
10.7	SDRAM memory timing paths	10-9
 Chapter 11	 Troubleshooting	
11.1	Software	11-2
11.2	System	11-6

List of Tables

PrimeCell MultiPort Memory Controller (PL172)

Integration Manual

	Change history	ii
Table 2-1	Pin counts for systems containing dynamic and static memory and TIC	2-22
Table 2-2	Pin counts for systems containing dynamic and static memory	2-23
Table 2-3	Pin counts for systems containing dynamic memory and TIC	2-23
Table 2-4	Pin counts for systems containing static memory and TIC	2-24
Table 2-5	Pin counts for systems containing dynamic memory	2-25
Table 2-6	Pin counts for systems containing static memory	2-26
Table 3-1	Static memory controller configurations	3-2
Table 3-2	Static memory timing parameters	3-8
Table 3-3	External memory zero wait state read	3-9
Table 3-4	Static memory timing parameters	3-9
Table 3-5	External memory two wait state read	3-10
Table 3-6	Static memory timing parameters	3-11
Table 3-7	External memory two output enable delay state	3-12
Table 3-8	Static memory timing parameters	3-12
Table 3-9	External memory two zero wait state reads	3-13
Table 3-10	Static memory timing parameters	3-14
Table 3-11	External memory zero wait fixed length burst read	3-15
Table 3-12	Static memory timing parameters	3-16
Table 3-13	External memory two wait states fixed length burst read	3-17
Table 3-14	Static memory timing parameters	3-18

Table 3-15	External memory page mode read	3-19
Table 3-16	Static memory timing parameters	3-20
Table 3-17	External memory 32-bit burst read from 8-bit memory	3-21
Table 3-18	Static memory timing parameters	3-23
Table 3-19	External memory zero wait state write	3-24
Table 3-20	Static memory timing parameters	3-24
Table 3-21	External memory two wait state write	3-25
Table 3-22	Static memory timing parameters	3-26
Table 3-23	External memory two write enable delay write	3-27
Table 3-24	Static memory timing parameters	3-28
Table 3-25	External memory two zero wait writes	3-29
Table 3-26	Static memory timing parameters	3-31
Table 3-27	Read followed by write (both zero wait) with no turnarounds	3-32
Table 3-28	Static memory timing parameters	3-33
Table 3-29	Write followed by read (both zero wait) with no turnaround	3-34
Table 3-30	Static memory timing parameters	3-35
Table 3-31	Read followed by a write (all zero wait state) with two turnaround cycles	3-36
Table 3-32	MPMCDATAOUT[31:0] controlled by nMPMCDATAOUTEN[3:0]	3-38
Table 3-33	Little-endian read, 8-bit external bus	3-47
Table 3-34	Little-endian read, 16-bit external bus	3-48
Table 3-35	Little-endian read, 32-bit external bus	3-48
Table 3-36	Little-endian write, 8-bit external bus	3-49
Table 3-37	Little-endian write, 16-bit external bus	3-49
Table 3-38	Little-endian write, 32-bit external bus	3-50
Table 3-39	Big-endian read, 8-bit external bus	3-50
Table 3-40	Big-endian read, 16-bit external bus	3-51
Table 3-41	Big-endian read, 32-bit external bus	3-51
Table 3-42	Big-endian write, 8-bit external bus	3-52
Table 3-43	Big-endian write, 16-bit external bus	3-52
Table 3-44	Big-endian write, 32-bit external bus	3-53
Table 3-45	Little-endian read, 8-bit external bus	3-53
Table 3-46	Little-endian read, 16-bit external bus	3-55
Table 3-47	Little-endian read, 32-bit external bus	3-55
Table 3-48	Little-endian write, 8-bit external bus	3-56
Table 3-49	Little-endian write, 16-bit external bus	3-56
Table 3-50	Little-endian write, 32-bit external bus	3-57
Table 3-51	Big-endian read, 8-bit external bus	3-57
Table 3-52	Big-endian read, 16-bit external bus	3-58
Table 3-53	Big-endian read, 32-bit external bus	3-58
Table 3-54	Big-endian write, 8-bit external bus	3-59
Table 3-55	Big-endian write, 16-bit external bus	3-59
Table 3-56	Big-endian write, 32-bit external bus	3-60
Table 4-1	32-bit chip select width transfer latencies	4-9
Table 4-2	16-bit chip select width transfer latencies	4-23
Table 4-3	Transaction details	4-38
Table 4-4	Single 32-bit read from 32-bit SDR-SDRAM, row open	4-38
Table 4-5	Single 32-bit write to 32-bit SDR-SDRAM, row open	4-41

Table 4-6	Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open	4-43
Table 4-7	Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, bank precharged	4-45
Table 4-8	Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open	4-47
Table 4-9	Burst4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open	4-49
Table 4-10	Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, bank precharged	4-51
Table 4-11	Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, different row open	4-53
Table 4-12	Burst 8 (INCR8) 32-bit read from 32-bit SDR-SDRAM, row open	4-55
Table 4-13	Burst 8 (INCR8/WRAP8) 32-bit write to 32-bit SDR-SDRAM, row open	4-57
Table 4-14	Burst 16 (INCR16) 32-bit read from 32-bit SDR-SDRAM, row open	4-58
Table 4-15	Burst 16 (INCR16/WRAP16) 32-bit write to 32-bit SDR-SDRAM, row open	4-60
Table 4-16	Burst 4 (INCR4) 32-bit read from 16-bit SDR-SDRAM, different row open	4-61
Table 4-17	Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 16-bit SDR-SDRAM, different row open	4-63
Table 4-18	Transaction details	4-64
Table 4-19	Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open	4-64
Table 4-20	Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 0 pad configuration	4-66
Table 4-21	Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 1 pad configuration	4-68
Table 4-22	Two back-to-back burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open	4-70
Table 4-23	Two back-to-back burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open	4-72
Table 4-24	Three back-to-back burst 4 (INCR4) 32-bit accesses from 32-bit SDR-SDRAM, row open	4-73
Table 4-25	Address mapping for 32-bit wide data bus RBC SDR-SDRAM	4-76
Table 4-26	Address mapping for 32-bit wide data bus BRC SDR-SDRAM	4-77
Table 4-27	Address mapping for 16-bit wide data bus RBC SDR-SDRAM and DDR-SDRAM	4-78
Table 4-28	Address mapping for 16-bit wide data bus BRC SDR-SDRAM and DDR-SDRAM	4-79
Table 4-29	Synchronous memory commands used by MPMC	4-80
Table 4-30	Synchronous memory commands programmed by software	4-80
Table 4-31	Required MPMC SDRAM mode register settings	4-82
Table 4-32	Field settings for Micron MT48LC4M16A2 SDRAM	4-83
Table 4-33	Required Micron SDRAM mode register settings	4-84
Table 4-34	Field settings for low-power SDRAM	4-86
Table 4-35	SDRAM mode register settings for low-power SDRAM	4-87
Table 4-36	Required low-power SDRAM extended mode register settings	4-88
Table 4-37	Field settings for Micron MT28S4M16LC SyncFlash	4-90
Table 4-38	SDRAM mode register settings for Micron SyncFlash SDRAM	4-91
Table 4-39	SyncFlash commands	4-93
Table 5-1	Buffer access latency	5-4
Table 5-2	INCR, INCR4, or WRAP4 burst read transfer, buffer hit	5-6
Table 5-3	Three sequential reads, burst type SINGLE, read transfer, buffer	5-7

Table 5-4	Three nonsequential reads, burst type SINGLE, read transfer, buffer	5-8
Table 5-5	INCR8 or WRAP8 burst read transfer, buffer hit	5-9
Table 5-6	Two simultaneous INCR, INCR4, or WRAP4 burst read transfer, buffer	5-11
Table 5-7	Four word INCR, INCR4, or WRAP4 burst write	5-12
Table 5-8	Three sequential write transfers	5-13
Table 5-9	Three nonsequential write transfers	5-14
Table 5-10	INCR8 or WRAP8 burst write	5-15
Table 7-1	Reset signal cycles	7-11
Table 7-2	EBI signal cycles	7-18
Table 7-3	MPMCConfig Register	7-19
Table 7-4	MPMCStaticConfig1 Register	7-20
Table 7-5	MPMCStaticConfig Register	7-21
Table 8-1	Memory device family DC characteristics	8-2
Table 8-2	Memory device I/O cell family	8-3
Table 8-3	Pin counts for system with dynamic and static memory and TIC	8-5
Table 8-4	Pin counts for system with dynamic and static memory	8-5
Table 8-5	Pin counts for system with dynamic memory and TIC	8-5
Table 8-6	Pin counts for system with static memory and TIC	8-6
Table 8-7	Pin counts for system with dynamic memory	8-6
Table 8-8	Pin counts for system with static memory	8-7
Table 8-9	Clock frequencies	8-10
Table 11-1	Software troubleshooting	11-2
Table 11-2	System troubleshooting	11-6

List of Figures

PrimeCell MultiPort Memory Controller (PL172)

Integration Manual

	Key to timing diagram conventions	xviii
Figure 2-1	Example ASIC design flow	2-3
Figure 3-1	External memory zero wait state read timing diagram	3-8
Figure 3-2	External memory two wait state read timing diagram	3-10
Figure 3-3	External memory two output enable delay state read timing diagram	3-11
Figure 3-4	External memory two zero wait state read timing diagram	3-13
Figure 3-5	External memory zero wait fixed length burst read timing diagram	3-15
Figure 3-6	External memory two wait states fixed length burst read timing diagram	3-17
Figure 3-7	External memory page mode read transfer timing diagram	3-19
Figure 3-8	External memory 32-bit burst read from 8-bit memory timing diagram	3-21
Figure 3-9	External memory zero wait state write timing diagram	3-23
Figure 3-10	External memory two wait state write timing diagram	3-25
Figure 3-11	External memory two write enable delay write timing diagram	3-27
Figure 3-12	External memory two zero wait writes timing diagram	3-29
Figure 3-13	Read followed by write (both zero wait) with no turnaround	3-31
Figure 3-14	Write followed by read (both zero wait) with no turnaround	3-34
Figure 3-15	Read followed by a write (all zero wait state) with two turnaround cycles	3-36
Figure 3-16	Memory banks constructed from 8-bit memory	3-41
Figure 3-17	Memory banks constructed from 16-bit memory	3-41
Figure 3-18	Memory bank constructed from 32-bit memory	3-42
Figure 3-19	Typical memory connection diagram	3-43

Figure 3-20	Memory banks constructed from 8-bit memory	3-44
Figure 3-21	Memory banks constructed from 16-bit memory	3-44
Figure 3-22	Memory bank constructed from 32-bit memory	3-45
Figure 3-23	Typical memory connection diagram	3-46
Figure 4-1	Single 32-bit read from 32-bit SDR-SDRAM, row open	4-40
Figure 4-2	Single 32-bit write to 32-bit SDR-SDRAM, row open	4-42
Figure 4-3	Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open	4-44
Figure 4-4	Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, bank precharged	4-46
Figure 4-5	Burst4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open	4-48
Figure 4-6	Burst 4 (INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open	4-50
Figure 4-7	Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, bank precharged	4-52
Figure 4-8	Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, different row open	4-54
Figure 4-9	Burst 8 (INCR8) 32-bit read from 32-bit SDR-SDRAM, row open	4-56
Figure 4-10	Burst 8 (INCR8/WRAP8) 32-bit write to 32-bit SDR-SDRAM memory, row open	4-57
Figure 4-11	Burst 16 (INCR16) 32-bit read from 32-bit SDR-SDRAM, row open	4-59
Figure 4-12	Burst 16 (INCR16/WRAP16) 32-bit write to 32-bit SDR-SDRAM, row open	4-60
Figure 4-13	Burst 4 (INCR4) 32-bit read from 16-bit SDR-SDRAM, different row open	4-62
Figure 4-14	Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 16-bit SDR-SDRAM, different row open	4-63
Figure 4-15	Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open	4-65
Figure 4-16	Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 0 pad configuration	4-67
Figure 4-17	Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 1 pad configuration	4-69
Figure 4-18	Two back-to-back burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open	4-71
Figure 4-19	Two back-to-back burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open	4-72
Figure 4-20	Three back-to-back burst 4 (INCR4) 32-bit accesses from 32-bit SDR-SDRAM, row open	4-74
Figure 5-1	Write transfer to a write-protected chip select	5-3
Figure 5-2	INCR, INCR4, or WRAP4 burst read transfer, buffer hit	5-6
Figure 5-3	Three sequential reads, burst type SINGLE, read transfer, buffer hit	5-7
Figure 5-4	Three nonsequential reads, burst type SINGLE, read transfer, buffer hit	5-8
Figure 5-5	INCR8 or WRAP8 burst read transfer, buffer hit	5-9
Figure 5-6	Two simultaneous INCR, INCR4, or WRAP4 burst read transfer, buffer hit	5-10
Figure 5-7	Four word INCR, INCR4, or WRAP4 burst write	5-12
Figure 5-8	Three sequential write transfers	5-13
Figure 5-9	Three nonsequential write transfers	5-14
Figure 5-10	INCR8 or WRAP8 burst write	5-15
Figure 7-1	MPMC system interconnection diagram	7-3
Figure 7-2	Pad interface input signals	7-6
Figure 7-3	Pad interface output signals	7-7
Figure 7-4	Pad interface output signals	7-8

Figure 7-5	Pad interface output signals	7-9
Figure 7-6	Reset signal timing	7-11
Figure 7-7	EBI connectivity, EBI required	7-17
Figure 7-8	EBI connectivity, EBI not required	7-17
Figure 7-9	EBI signal timing	7-18
Figure 7-10	Hard coding tie-off	7-22
Figure 7-11	Setting tie-off value by a register	7-22
Figure 7-12	Setting tie-off value externally	7-23
Figure 7-13	Setting tie-off value externally using a shared pin	7-23
Figure 7-14	PMU self-refresh	7-25
Figure 7-15	PMU self-refresh waveforms	7-26
Figure 7-16	Forcing AHB masters idle in a multi-layer system	7-27
Figure 7-17	Example system	7-28
Figure 8-1	1:1 HCLK and MPMCCLK clock relationship	8-9
Figure 8-2	1:2 HCLK and MPMCCLK clock relationship	8-9
Figure 8-3	Clock delayed pad interface read timing	8-15
Figure 8-4	Command delayed pad interface timing	8-20
Figure 8-5	Command delay plus zero SDR-SDRAM	8-24
Figure 8-6	Command delayed plus one pad interface timing	8-26
Figure 8-7	Command delay plus one SDR-SDRAM	8-27
Figure 8-8	Command delayed plus two pad interface timing	8-29
Figure 8-9	Command delay plus two SDR-SDRAM	8-30
Figure 8-10	Generating feedback clock off-chip	8-31
Figure 8-11	Generating feedback clock from bidirectional pad	8-32
Figure 8-12	Generating feedback clock on-chip	8-33
Figure 8-13	x8 SDRAM device	8-37
Figure 8-14	x16 SDRAM connection	8-38
Figure 8-15	x32 SDRAM interconnection	8-39
Figure 8-16	Mixed width SDRAM memory devices	8-40
Figure 8-17	SRAM and SDRAM memory devices	8-42
Figure 8-18	Medium performance systems	8-43
Figure 8-19	Lower performance systems	8-44
Figure 10-1	On-chip to off-chip delay for a single signal	10-3
Figure 10-2	On-chip to off-chip delay with skew for a single signal	10-5
Figure 10-3	On-chip to off-chip delay with skew for multiple signals	10-5
Figure 10-4	SDRAM timing diagram	10-8
Figure 10-5	MPMC to SDRAM memory timing path	10-9
Figure 10-6	SDRAM memory to MPMC timing path	10-10

Preface

This preface introduces the PrimeCell *MultiPort Memory Controller* (MPMC) (PL172) Integration Manual. It contains the following sections:

- *About this manual* on page xvi
- *Feedback* on page xxi.

About this manual

This is the integration manual for the PrimeCell MPMC (PL172).

Product revision status

The *rn*pn identifier indicates the revision status of the product described in this manual, where:

rn Identifies the major revision of the product.

pn Identifies the minor revision or modification status of the product.

Intended audience

This manual is written for hardware and software engineers implementing *System-on-Chip* (SoC) designs. It provides information to enable designers to integrate the peripheral into a target system as quickly as possible.

Using this manual

This manual is organized into the following chapters:

Chapter 1 *Introduction*

Read this chapter for an introduction to the MPMC.

Chapter 2 *Integration*

Read this chapter for the integration procedures for the MPMC.

Chapter 3 *Static Memory Controller*

Read this chapter for details of the static memory controller.

Chapter 4 *Dynamic Memory Controller*

Read this chapter for details of the dynamic memory controller.

Chapter 5 *Common Memory Transactions*

Read this chapter for a description of transactions for both dynamic and static memory accesses.

Chapter 6 *Test Interface Controller*

Read this chapter for details of the test interface controller.

Chapter 7 *System Connectivity*

Read this chapter for details of the system connectivity of the MPMC.

Chapter 8 *Off-chip Connectivity*

Read this chapter for details of the off-chip connectivity of the MPMC.

Chapter 9 *Power Strategy*

Read this chapter for details of the power strategy aspects relating to the MPMC.

Chapter 10 *Pad Interface Timing*

Read this chapter for descriptions of signals that interface with the MPMC.

Chapter 11 *Troubleshooting*

Read this chapter for descriptions of typical problems and their suggested remedies.

Conventions

Conventions that this manual can use are described in:

- *Typographical*
- *Timing diagrams* on page xviii
- *Signals* on page xviii
- *Numbering* on page xix.

Typographical

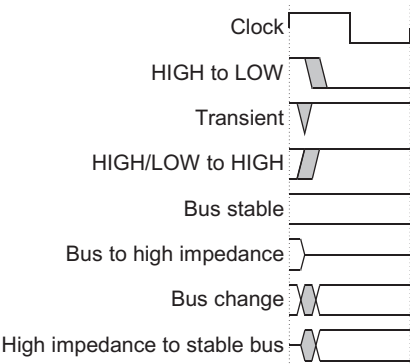
The typographical conventions are:

<i>italic</i>	Highlights important notes, introduces special terminology, denotes internal cross-references, and citations.
bold	Highlights interface elements, such as menu names. Denotes ARM processor signal names. Also used for terms in descriptive lists, where appropriate.
monospace	Denotes text that you can enter at the keyboard, such as commands, file and program names, and source code.
<u>monospace</u>	Denotes a permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
<i>monospace italic</i>	Denotes arguments to monospace text where the argument is to be replaced by a specific value.

monospace bold	Denotes language keywords when used outside example code.
< and >	Angle brackets enclose replaceable terms for assembler syntax where they appear in code or code fragments. They appear in normal font in running text. For example: - MRC p15, 0 <Rd>, <CRn>, <CRm>, <Opcode_2> - The Opcode_2 value selects which register is accessed.

Timing diagrams

The figure named *Key to timing diagram conventions* explains the components used in timing diagrams. Variations, when they occur, have clear labels. You must not assume any timing information that is not explicit in the diagrams.



Key to timing diagram conventions

Signals

The signal conventions are:

Signal level	The level of an asserted signal depends on whether the signal is active-HIGH or active-LOW. Asserted means HIGH for active-HIGH signals and LOW for active-LOW signals.
Prefix A	Denotes <i>Advanced eXtensible Interface</i> (AXI) global and address channel signals.
Prefix B	Denotes AXI write response channel signals.
Prefix C	Denotes AXI low-power interface signals.
Prefix H	Denotes <i>Advanced High-performance Bus</i> (AHB) signals.

Prefix n	Denotes active-LOW signals except in the case of AHB or <i>Advanced Peripheral Bus</i> (APB) reset signals.
Prefix P	Denotes APB signals.
Prefix R	Denotes AXI read channel signals.
Prefix W	Denotes AXI write channel signals.
Suffix n	AHB HRESETn and APB PRESETn reset signals.

Numbering

The numbering convention is:

<size in bits>'<base><number>

This is a Verilog method of abbreviating constant numbers. For example:

- 'h7B4 is an unsized hexadecimal value.
- 'o7654 is an unsized octal value.
- 8'd9 is an eight-bit wide decimal value of 9.
- 8'h3F is an eight-bit wide hexadecimal value of 0x3F. This is equivalent to b00111111.
- 8'b1111 is an eight-bit wide binary value of b00001111.

Further reading

This section lists publications by ARM Limited.

ARM Limited periodically provides updates and corrections to its documentation. See <http://www.arm.com> for current errata sheets, addenda, and the ARM Limited Frequently Asked Questions list.

ARM publications

This manual contains information that is specific to the PrimeCell MPMC (PL172). See the following documents for other relevant information:

- *AMBA® Specification (Rev 2.0)* (ARM IHI 0011)
- *ARM PrimeCell MultiPort Memory Controller (PL172) Design Manual* (PL172 DDES 0000)
- *PrimeCell MultiPort Memory Controller (PL172) Technical Reference Manual* (ARM DDI 0215)

- *AMBA Design Kit Technical Reference Manual* (ARM DDI 0243)
- *ARM PrimeCell External Bus Interface (PL220) Technical Reference Manual* (ARM DDI 0249).

Feedback

ARM Limited welcomes feedback on the PrimeCell MPMC (PL172) and its documentation.

Feedback on this product

If you have any comments or suggestions about this product, contact your supplier giving:

- the product name
- a concise explanation of your comments.

Feedback on this manual

If you have any comments on this manual, send email to errata@arm.com giving:

- the title
- the number
- the relevant page number(s) to which your comments apply
- a concise explanation of your comments.

ARM Limited also welcomes general suggestions for additions and improvements.

Chapter 1

Introduction

This chapter introduces the PrimeCell MPMC (PL172). It contains the following section:

- *About the PrimeCell MultiPort Memory Controller (PL172)* on page 1-2
- *Changes between r2p2 and r2p3* on page 1-3.

1.1 About the PrimeCell MultiPort Memory Controller (PL172)

The MPMC is an *Advanced Microcontroller Bus Architecture* (AMBA Rev 2.0) compliant *System-on-Chip* (SoC) peripheral that is developed, tested, and licensed by ARM Limited. It connects to the *Advanced High-performance Bus* (AHB).

The MPMC is a reusable soft *Intellectual Property* (IP) block that can reduce ASIC development time. It enables faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can be customized for specific customer requirements.

For more information on AMBA, see the *AMBA Specification (Rev 2.0)*.

For more information, and programming data for the MPMC block, see the *PrimeCell MultiPort Memory Controller (PL172) Technical Reference Manual*.

The *ARM PrimeCell MultiPort Memory Controller (PL172) Design Manual* contains information about the implementation and design of the MPMC block that you might require for optional customization of this block.

1.2 Changes between r2p2 and r2p3

There are the following changes to the described functionality:

- It is no longer possible to restart the system using hard reset when it has locked up. If you try this then data might be lost, or is not retained.

Chapter 2

Integration

This chapter describes the integration procedures for the PrimeCell MPMC. It contains the following sections:

- *Integration overview* on page 2-2
- *Synthesis* on page 2-11
- *Integration test* on page 2-16
- *Production test* on page 2-18
- *Functional and timing verification* on page 2-19
- *Customization* on page 2-21
- *Environment variables* on page 2-28.

2.1 Integration overview

The MPMC block is designed to be integrated into an AMBA system through the AHB slave ports and AHB master ports:

- the MPMC reads from and writes to memories through the slave ports
- the master ports are used in test mode.

You must change the decode logic in the AHB decoder to take into account the address location of the MPMC and any other peripherals in the memory map. In addition, you must define the base address of the MPMC on a system-specific basis and change the decode logic accordingly. If you use AMBA TIC methodology for integration testing, any change of address location for the MPMC block must also be reflected in the AMBA TIC test vectors. You can do this by amending the base address declaration for the MPMC integration test program.

The TIC block (MpmcTIC) in the MPMC is used for system testing, through its AHB master ports. The low gate count TIC block enables externally-applied test vectors to be converted into internal AMBA bus transfers. The AMBA test philosophy enables you to test individual modules in the system in isolation by only using transfers from the bus. It does not rely on interaction of other system elements such as the processor core.

Integration of the MPMC is described in more detail in the following sections:

- *Supported design flow*
- *Signal description* on page 2-3
- *AMBA bus connection* on page 2-4
- *Non-AMBA intra-chip connection* on page 2-4
- *Primary input/output connections* on page 2-6
- *Miscellaneous integration aspects* on page 2-7.

2.1.1 Supported design flow

The MPMC is designed to enable integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimization of area, such as gate count, and power consumption, ease of integration has a higher priority to facilitate design reuse.

———— Note ————

You can use logic equivalence checking instead of dynamic functional simulation.

Figure 2-1 on page 2-3 shows an example ASIC design flow.

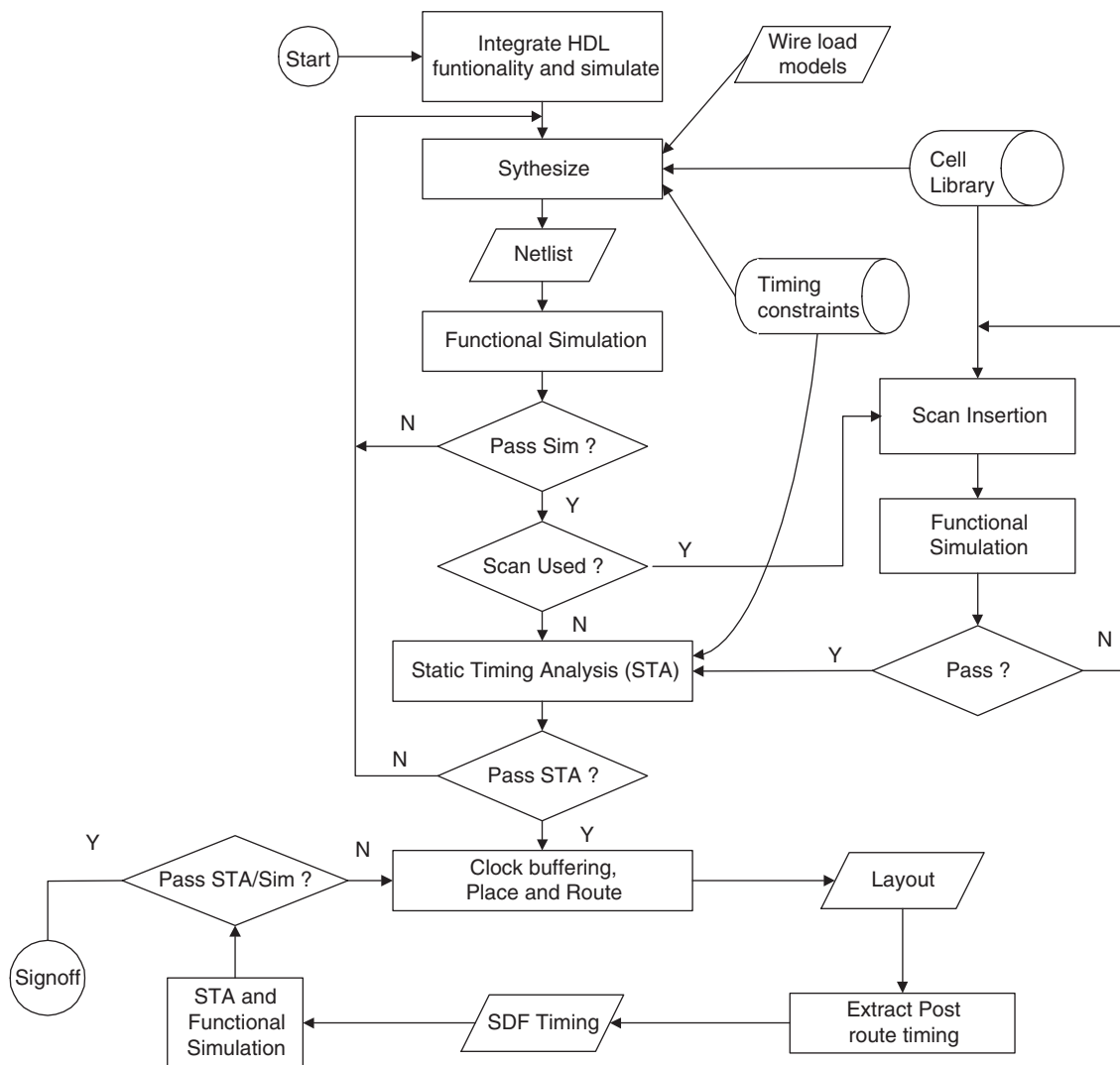


Figure 2-1 Example ASIC design flow

2.1.2 Signal description

For information on the input and output signals used in the MPMC, see the *PrimeCell MultiPort Memory Controller (PL172) Technical Reference Manual*.

2.1.3 AMBA bus connection

Connect the AHB register interface to the AHB bus with the microprocessor so that the microprocessor can program the MPMC.

AHB memory interface 0 is the highest priority interface and must be connected to the AHB bus with high-bandwidth, low-latency AHB masters. AHB memory interface 1 is the next highest priority interface. AHBs with higher numbers are lower priority. The lowest priority depends on the number of AHB memory interface ports in the system. The MPMC supports four AHB memory interfaces, with AHB memory interface 3 having the lowest priority among the 4 ports. You can expand the number of ports up to eight (see *Changing the number of AHB memory interface ports* on page 2-21). For a system with six AHB memory interface ports, AHB memory interface 5 has the lowest priority. Any unused AHB interfaces must have their AHB input signals tied LOW.

If the TIC AHB master interface is required, connect it to the same bus as the microprocessor. The TIC controller must be the highest priority master on this bus. The **MPMCTESTIN** signal must be asserted during reset to enter TIC test mode. The **MPMCTESTIN** signal must also be sent to the clock generation logic so that the appropriate test clock is selected. The TIC controller must not be the default master. Because the TIC master only operates at test speed, it might not function correctly if granted the bus at functional speed.

The MPMC block interfaces with the AMBA AHB interfaces using separate unidirectional read and write data buses, **HRDATA** and **HWDATA**. For more information, see the *AMBA Specification (Rev 2.0)*.

The AHB data buses from the MPMC must be interfaced with other data buses from other peripherals using multiplexers at the chip level.

2.1.4 Non-AMBA intra-chip connection

On power-on reset, the address mirror bit in the MPMCControl Register is asserted and the memory accesses to chip select 0 and chip select 4 are mapped on to chip select 1 for boot purposes. Therefore, the memory data bus width of chip select 1 must be driven on the **MPMCSTCS1MW[1:0]** inputs as follows:

- 00 for 8-bit data bus width
- 01 for 16-bit data bus width
- 10 for 32-bit data bus width.

On power-on reset, the chip select 1 byte lane state information must be driven on **MPMCSTCS1PB** input as follows:

- 0 for 8-bit devices
- 1 for other devices.

These values are used until the **MPMCStConfig1** Register is written to. When the register is programmed, the memory width bits of the register determine the memory width, and the byte lane state bit determines the byte lane state. This feature enables the boot memory width on the **MPMCSTCS1MW[1:0]** pins and the boot memory byte lane state on the **MPMCSTCS1PB** pin to be chosen at reset. The external boot ROM must be connected to chip select 1. This ensures that the chip select 1 always has the correct memory width and byte lane state setting for the respective boot memory device.

On power-on reset the static memory chip select polarity pins, **MPMCSTCS[3:0]POL**, must indicate the polarity of the respective chip selects, 0 for active LOW and 1 for active HIGH. These values are used until the **MPMCStConfig[3:0]** Register is written to. When the corresponding chip select configuration register is programmed, the register value of the chip select polarity bit in the register determines the chip select polarity. This enables the correct chipselect polarity values on these pins to be chosen at reset, and ensures that the correct chipselect polarity is used during boot-up.

On power-on reset the static memory address mode select pin, **MPMCREL1CONFIG**, must indicate the static memory device address connections that are followed while connecting the **MPMCADDROUT[27:0]** lines to the address pins on the static memory devices. This enables the static memory address mode to be selected for the MPMC.

On power-on reset, the endian mode of the MPMC must be driven on the **MPMCBIGENDIAN** pin. This value is used until the **MPMCConfig** Register is written to. When this register is programmed, the register value of the Endian mode bit determines the endian mode.

MPMCSREFREQ and **MPMCSREFACK** must be connected to the *Power Management Unit* (PMU). For more details on power management, see *Power management* on page 2-10.

The MPMC supports connection to an *External Bus Interface* (EBI) to enable external bus arbitration if there are masters on the external bus. In this case, connect **MPMCEBIREQ**, **MPMCEBIGNT**, and **MPMCEBIBACKOFF** to the **EBIREQ**, **EBIGNT**, and **EBIBACKOFF** signals of one port on the EBI. If the MPMC is the only master on the external bus interface, **MPMCEBIGNT** must be tied to HIGH and **MPMCEBIBACKOFF** LOW, to ensure best use of the external bus bandwidth.

The MPMC test mode is controlled by the **MPMCTESTIN** pin. On the AHB, it is assumed that the processor is always requesting access to the bus, but the TIC has the highest priority to bus access. When the MPMC has entered test mode, it is assumed that there are no normal mode accesses. The MPMC can enter test mode only if the sequence of events described below is followed:

1. Apply reset, **HRESETn**, asynchronously.

2. Assert the **MPMCTESTIN** test input to enter test mode.
3. When the reset is removed synchronously, the MPMC enters TIC test mode.
4. The TIC block asserts its **HBUSREQTIC** signal to the arbiter, requesting access to the AHB bus. Because the TIC is the highest priority, **HGRANTTIC** is asserted and the granted TIC takes ownership of the AHB bus.

The **MPMCTESTIN** signal is usually fed to the clock generation logic so that HCLK can be switched to test speed on entry into test mode. **MPMCTESTREQA**, **MPMCTESTREQB**, and **nMPMCWEOUT** are used as the **TESTREQA**, **TESTREQB**, and **TESTACK** inputs of the TIC in test mode.

———— **Note** —————

For more information, see the *PrimeCell MultiPort Memory Controller (PL172) Technical Reference Manual*.

2.1.5 Primary input/output connections

The primary inputs and outputs of MPMC are the external memory bus signals, which must be connected to the appropriate pins of the external memory device. The following points are applicable when connecting the memory devices to the MPMC.

Write enable connection of static memory devices

For static memory banks constructed from 8-bit or non byte-partitioned memory devices, connect the **nMPMCBLSOUT[3:0]** signals to the write enable (**nWE**) inputs of each 8-bit memory. The **nMPMCWEOUT** signal from the MPMC must not be used.

For static memory banks constructed from 16-bit or 32-bit memory devices, connect the **nMPMCBLSOUT[3:0]** signals to the byte select inputs of each memory device. Connect the **nMPMCWEOUT** signal from the MPMC to the write enable inputs of each memory device.

Address connection of static memory devices

If **MPMCREL1CONFIG** is asserted HIGH, the MPMC does not right align the memory address in the case of static memory accesses. For 32 and 16-bit external memory data bus widths, only connect the most significant address bits to the address input of the memory device.

Ignore one least significant address bit, **MPMCADDRROUT[0]**, for 16-bit data bus width and two least significant address bits, **MPMCADDRROUT[1:0]**, for 32-bit data bus width. For example:

- For a 32-bit data bus width memory bank, if the memory device has a 16-bit address (**A[15:0]**), connect **MPMCADDRROUT[17:2]** to **A[15:0]**
- For a 16-bit data bus width memory bank, if the memory device has a 16-bit address (**A[15:0]**), connect **MPMCADDRROUT[16:1]** to **A[15:0]**
- For an 8-bit data bus width memory bank, if the memory device has a 16-bit address (**A[15:0]**), connect **MPMCADDRROUT[15:0]** to **A[15:0]**.

If **MPMCREL1CONFIG** is asserted LOW, the MPMC right aligns the memory address for static memory accesses. For 32, 16, and 8-bit external memory data bus widths, connect the least significant address bits to the corresponding address input of the memory device. For example:

- For a 32-bit data bus width memory bank, if the memory device has a 16-bit address (**A[15:0]**), connect **MPMCADDRROUT[15:0]** to **A[15:0]**
- For a 16-bit data bus width memory bank, if the memory device has a 16-bit address (**A[15:0]**), connect **MPMCADDRROUT[15:0]** to **A[15:0]**
- For a 8-bit data bus width memory bank, if the memory device has a 16-bit address (**A[15:0]**), connect **MPMCADDRROUT[15:0]** to **A[15:0]**.

2.1.6 Miscellaneous integration aspects

Other MPMC integration aspects are described in the following sections:

- *Clocks*
- *Resets* on page 2-9
- *Scan signals* on page 2-9
- *Synchronization* on page 2-9
- *Endianness* on page 2-9
- *Power management* on page 2-10.

Clocks

The MPMC contains seven clock sources at the top level:

- **HCLK**
- **MPMCCLK**
- **MPMCCLKDELAY**
- **MPMCFBCLKIN0**

- **MPMCFBCLKIN1**
- **MPMCFBCLKIN2**
- **MPMCFBCLKIN3**.

MPMCCLK and **HCLK** must be synchronous and rising-edge aligned in both 1:1 and 1:2 clocking modes:

- **HCLK** is the AHB clock used for the AHB side signals and for controlling the accesses to static memory devices.
- **MPMCCLK** is used for the dynamic memory controller logic in the MPMC. This clock also derives the clock inputs of the dynamic memory device, **MPMCCLKOUT[3:0]**. Each bit of **MPMCCLKOUT[3:0]** drives the memory devices connected to one dynamic memory chip select.

The MPMC has two modes of operation with respect to the relationship between the clock and the commands at the inputs of the dynamic memory devices:

- clock delayed mode
- command delayed mode.

If the MPMC has been programmed to be used in the clock delayed mode, that is, MPMCDyRdConfig Register bit 0 is 0, **MPMCCLKOUT[3:0]** must be delayed by a finite value and fed to the clock inputs of the memory devices. This delay on the memory clock must be less than the delay on the command lines of the memory devices.

If the MPMC has been programmed to be used in the command delayed mode, that is, MPMCDyRdConfig Register bit 0 is 1, the MPMC registers the commands to the dynamic memory on **MPMCCLKDELAY**. This is a delayed version of **MPMCCLK**. In this case, connect **MPMCCLKOUT** directly to the memory device without any additional delay element. Obtain **MPMCCLKDELAY** by delaying the **MPMCCLK** for this purpose.

MPMCFBCLKIN[0-3] are the clocks fed back from the memory devices to clock in the read data from the memories. Each of these clocks is used to clock in data for one byte-lane of the memory read data:

- **MPMCFBCLKIN0** clocks in byte 0 of **MPMCDATAIN[31:0]**
- **MPMCFBCLKIN1** clocks in byte 1 of **MPMCDATAIN[31:0]**
- **MPMCFBCLKIN2** clocks in byte 2 of **MPMCDATAIN[31:0]**
- **MPMCFBCLKIN3** clocks in byte 3 of **MPMCDATAIN[31:0]**.

———— **Note** ————

For more information, see the *PrimeCell MultiPort Memory Controller (PL172) Technical Reference Manual*.

Resets

The MPMC block has a two reset pins:

- **HRESETn**, which is the system bus reset
- **nPOR**, which is the power-on reset.

HRESETn and **nPOR** can be asserted asynchronously by the system reset controller, but they must be deasserted synchronous to the rising edge of **HCLK**. The power-on reset signal from the reset controller must be connected to the MPMC **nPOR** signal. The memory controller logic of the MPMC is reset by **nPOR**. However, the AHB interfaces, arbiter, and quadword buffers are controlled by **HRESETn**. The auto-refresh counter is reset by **nPOR**. The use of two resets enables the dynamic memory refresh mechanism to continue in the presence of a soft reset on **HRESETn**. Both the resets must be asserted to reset the whole design.

If the reset controller does not generate a power-on reset signal, connect the **HRESETn** signal to **nPOR**. However, in this situation, some of the functionality is disabled.

Scan signals

SCANENABLE is the scan-mode enable pin of the MPMC. A separate scan input and output is provided for each of the clock domains, so the seven clocks lead to seven scan-chains in the MPMC. The various scan signals are described in the *PrimeCell MultiPort Memory Controller (PL172) Technical Reference Manual*.

Synchronization

HCLK and **MPMCCLK**, the main clock inputs of the MPMC, are assumed to be synchronous and rising edge aligned to each other. Because of this, the design does not require any synchronization logic.

Endianness

The MPMC supports only up to 32-bit accesses on its AHB slave data buses. For a 32-bit system, the MPMC AHB slave interface is sufficiently bi-endian to support 32-bit accesses from a little-endian master and a big-endian master. Memory accesses with **HSIZE[2:0]** less than the memory data bus width are mapped to the correct byte lanes of data in the memory devices, depending on the endian mode of the MPMC. For AHB memory accesses to memories that have 8 or 16-bit data bus widths, data packing and unpacking is done according to the endian mode of the MPMC.

Note

See *Byte lane control and data bus steering* on page 3-47 for more details on endianness configuration.

In systems with wider data buses, data routing must be done external to the MPMC. The endianness of the AHB slave ports of the MPMC is pin or register-controlled. The **MPMCBIGENDIAN** or Endian Mode (N) bit of the **MPMCControl** Register can control the endianness of the AHB slave ports.

Power management

Use the following procedure to enable power management:

1. The system must ensure that the memory subsystem is idle before asserting **MPMCSREFREQ**.
2. When the system is idle, **MPMCSREFREQ** is asserted. The controller closes any open memory banks, and puts the memory into self-refresh mode.
3. **MPMCSREFACK** is asserted to indicate to the PMU that the external memories are in self-refresh state.
4. The PMU self-refresh signals must be connected to the MPMC **MPMCSREFREQ** and **MPMCSREFACK** signals.
5. Deasserting **MPMCSREFREQ** returns the memory to normal operation. See the memory data sheet for refresh requirements.

If the PMU does not provide the required signals to automatically enter self-refresh mode, the MPMC **MPMCSREFREQ** signal must be tied LOW. In this case, self-refresh mode can be entered manually by programming the Self-Refresh (SR) bit in the **MPMCDyControl** Register.

2.2 Synthesis

The MPMC block is synthesized using the Synopsys Design Compiler synthesis tool with the scripts located in the `synopsys` directory. A `README` text file in the `synopsys` directory provides details of the synthesis directory structure used.

The master compile script, `Mpmc.cmd`, synthesizes the MPMC to the chosen target cell library. If scan insertion is used, the master compile script invokes the `Mpmc_scan.cmd` script.

The final area and gate count that are achieved depend on the target cell library used, and synthesis timing constraints and command options.

2.2.1 Constraints

The MPMC TIC module, `MpmcTIC`, is normally used only at a frequency of 10MHz. Because of this, the `MpmcTIC` is synthesized separately at 10MHz while the MPMC Core logic, `MpmcCore`, is synthesized at 156 MHz. At the top level, the `.db` files created from the compiles of `MpmcTIC` and `MpmcCore` are read and the top-level synthesis is done at 156MHz.

The constraints on the different inputs and outputs of the MPMC are described in the following sections:

- *AMBA bus signals*
- *Non-AMBA intra-chip signals*
- *Primary input/output signals* on page 2-12
- *Clocks* on page 2-12
- *Resets* on page 2-12
- *Scan signals* on page 2-12.

AMBA bus signals

The AMBA signals are synthesized at the following constraints:

- 30% input setup on all inputs to the AHB slave ports and master port
- 40% output valid delay on all outputs of the AHB slave ports and master port

Non-AMBA intra-chip signals

The non-AMBA intra-chip signals are synthesized at the following constraints:

- 30% input setup on all non-AMBA intra-chip inputs
- 40% output valid delay on all non-AMBA intra-chip outputs except **MPMCEBIREQ**.

Primary input/output signals

All primary inputs and outputs are synthesized at the following constraints:

- 20% input setup on all primary inputs
- 25% output valid delay on all primary outputs.

A false path is set on **MPMCCLKOUT** to avoid the false violations reported on this path. In addition, a false path is set between the **TESTACK** pin of the MpmcTIC module and the **nMPMCWEOUT** because the MpmcTIC module is synthesized at 10MHz and the logic in this module must not be optimized during the synthesis of the top level. A false path is also set between **MPMCFBCLKIN0-3** and **HCLK**. This is to avoid optimizing these paths because the dynamic memory read data that is registered and driven on the feedback clocks is required to be stable by the next rising edge of **HCLK** and not the immediate rising edge.

Clocks

Various design methodologies are available for clock tree insertion and synthesis. The default MPMC synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering is performed by a separate tool, such as place and route tools. It is possible to modify the scripts if this is not the preferred approach, or if this is not compatible with the design flow being used. It is recommended, however, that clock tree insertion is always performed during the place and route stage and not during synthesis.

Resets

One of several available methods must be used to ensure correct buffering of the reset nets. The reset signals can be assigned infinite drive strength enabling a chip-level integration process to ensure balanced reset trees are used at the top level of the chip or propagating the tree down to all end points. Another method is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block

Scan signals

Because false paths are set from all scan related inputs, the tool must not try to optimize the scan paths.

2.2.2 Synthesis scripts

The synthesis scripts consist of three main categories, and are described in the following sections:

- *Setup scripts*
- *Command script*
- *Constraint scripts* on page 2-14.

Setup scripts

Scripts that are common to several designs are located in the \$GLOBAL/synopsys_shared/scr_common directory. See *Environment variables* on page 2-28 for the list of PrimeCell world environment variables. The following files require several parameters or variables to be changed to target a different cell library:

- The setup script, `setup.scr` defines the following:
 - Synopsys variable settings
 - UNIX directory path for Synopsys read and write caches.
- The `global.scr` script might have to be modified to define:
 - target area goals
 - maximum transition costs.

The `library_avanti_cb18_v1.0.scr` script is included by the master compile script to specify the target library and the cells in the target library that must not be used. It is recommended that an equivalent file be created for the target library to suit library vendor or customer requirements. It is also useful to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. Ensure that the `library_avanti_cb18_v1.0.scr` file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The \$LIB_SYNOP directory must contain the files, or links to the target cell library files, in Synopsys format.

Command script

The master compile command script `Mpmc.cmd` in the `synopsys/scripts` directory analyzes the HDL source before mapping and optimizing to a specific target technology cell library. The `Mpmc.cmd` script includes setup and constraint scripts to set timing and design rule constraints on the design, before optimization. The compiled design is written out both in native Synopsys `.db` format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. In addition, the run-time log is redirected into a separate `log/` directory.

The scan-insertion script `Mpmc_scan.cmd` in the `synopsys/scripts` directory performs scan insertion for the design.

Before synthesis, several variables must be set. The scripts use environment variables, assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

<code>\$PERIPH</code>	This is assigned to <code>topname</code> to define the name of the top-level module in the design.
<code>\$HDL_SOURCE</code>	This is assigned to <code>hdl</code> to define the input HDL source. <code>HDL_SOURCE</code> can be set to either <code>vhdl</code> or <code>verilog</code> to read in the VHDL or Verilog source RTL files.
<code>\$HDL_NETL</code>	This is assigned to <code>hdl_out</code> to define the output format for netlist and SDF files. It can be set to either <code>vhdl</code> or <code>verilog</code> to generate either VHDL or Verilog netlists.

———— **Note** ————

If the environment variables are not required, the scripts can be modified to directly assign values.

Constraint scripts

Operating conditions and signal load/drive values must be set for the chosen technology library in a file equivalent to the `library_avanti_cb18_v1.0.scr` file.

Timing constraints for AHB bus signals are set using the `amba_params.scr`, `ahb_master.scr`, and the `ahb_slave.scr` files. As delivered, the timings for the MPMC core logic are set for a 156MHz clock and, for the MPMC TIC, the timings are set for a 10MHz clock. The MPMC top level is compiled at 156MHz. Adjust these timings according to the target frequency required. The files have been written using parameters and only require amendment of the clock frequency, with all other bus parameters (except clock skew) scaling accordingly.

Clock skew must be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values are used as preliminary values before layout:

- The minus skew is set at 2.5% of the clock period. This is subtracted from the clock period used for the synthesis.
- The plus skew is set at 40% of minus skew. This is used for the plus uncertainty value.

The plus skew derives from best case condition, and is less than the minus skew that derives from worst case condition.

The timing parameters assume the following bus timing budgets:

- 30% input setup on all input ports including those on the AHB slave ports and master port, and the non-AMBA input lines.
- 40% output valid delay on all outputs including those on the AHB slave ports and master port
- 15% output valid delay on all the non-AMBA output lines except **MPMCEBIREQ**. 30% output valid delay has been assumed for **MPMCEBIREQ**.

Timing constraints for non-AMBA MPMC-specific signals are set using the `MpmcCore.scr`, `MpmcTIC.scr`, and `Mpmc.scr` files for the MpmcCore, MpmcTIC and top-level compiles respectively. Sample input and output delay constraints are specified for these signals. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.

Timing exceptions are specified via the `Mpmc_exceptions.scr` file. False paths are specified to **MPMCCLKOUT[3:0]** because it is the gated clock output of the MPMC, and timing constraints are not set for these output clocks. Because the paths in the TIC block are used only at a low frequency of about 10MHz, false paths are set for all the paths in the MpmcTIC module during the top-level compile of the MPMC at 156MHz.

2.3 Integration test

The MPMC integration test vector suite enables the integrator to verify that the MPMC has been connected into the target system correctly. Integration testing is described in the following sections:

- *Integration test strategy*
- *Integration test code.*

2.3.1 Integration test strategy

Integration tests are required to test three classes of signals:

AMBA signals

The AHB signals connected to the AHB slave ports of the MPMC signals are tested with AHB transactions to the MPMC. The vectors required for testing the AHB register port are contained in the `IntRegisterTest.c` file in the `integration/bustest` directory.

The other AHB ports cannot be tested in the integration world because they can result in memory accesses on the external memory bus that is shared between the TIC and the external memory devices. Because the MPMC TIC is used for applying the test vectors in the integration world, the other AHB ports, port 0, 1, 2 and 3, are tested in the verification world. These test vectors are contained in the `IntegrationTest.c` file in the `verification/bustest` directory.

Non-AMBA intra-chip signals

The non-AMBA intra-chip signals of the MPMC are tested by running the integration test vectors contained in the `IntegrationTest.c` file in the `integration/bustest` directory.

Primary inputs and outputs

The primary inputs and outputs, that is, external memory signals, can be tested external to the chip by performing memory transactions. This can be done by running the test vectors contained in the `IntegrationTest.c` file in the `verification/bustest` directory.

2.3.2 Integration test code

The `integration/bustest` directory contains the integration test code files, `IntegrationTest.c` and `IntRegisterTest.c` and the `verification/bustest` directory contains the file `IntegrationTest.c`.

The integration/bustest directory also contains the tests to verify the MPMC TIC block. The test vectors to test the MPMC TIC module are contained in the ResponseTests.c and TransferTests.c files. The file MPMC.c in the integration/bustest directory is used to selectively run the integration tests or TIC tests or all of these tests. As delivered, the integration test vectors are intended to run correctly with the MPMC in stand-alone mode in the integration EASY world, that is, before the MPMC has been integrated into your system.

After integration, the following points must be noted:

- The integration vectors targeting the MPMC AHB slave port (AMBA signals) connectivity do NOT have to be modified by the integrator.
- The integration test vectors targeting the intra-chip signals of the MPMC are required to be modified to drive a 1 or a 0 on the intra-chip inputs and to read the intra-chip outputs. The test vectors that must be modified are indicated in the IntegrationTest.c file in the integration/bustest directory.

2.4 Production test

The MPMC design fully supports scan test and *Automatic Test Pattern Generation* (ATPG) methodology. Depending on the clocking arrangement used, the number of chip level scan chains required can vary. As delivered, the MPMC is synthesized with seven scan chains.

If scan insertion is to be performed post-synthesis, it must be done when the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach can be used for synthesis where the RTL is compiled to create netlist with a scan test D flips-flops and provisionally connected scan chains. After scan insertion, logical equivalence must again be proven.

Extraction of test vectors is done when the whole chip design has been integrated. The quoted fault coverage, except variation resulting from the use of a sparse cell library, is met by extracting test vectors around the complete chip while running the automatically generated vectors.

2.5 Functional and timing verification

It is essential to verify the functionality and the timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout, and at completion before sign-off of the ASIC. Verification is described in the following sections:

- *Functional verification*
- *Timing verification.*

2.5.1 Functional verification

Functional verification tests the functionality of the design. The netlist must also be checked for functional equivalence to the RTL at all the key stages of the design flow. The functional verification can be done using one of the following methods:

Formal verification

Formal verification compares RTL with RTL, and RTL with netlist. The functionality of the netlist or RTL can be compared to that of the RTL by use of a logic equivalence-checking tool, such as Synopsys Formality or Chrysalis. Functionality of the netlist and RTL can be compared in less time using this method compared to gate-level simulations.

Dynamic simulation

Verification tests are run on the RTL and netlist to confirm functionality of the design. Gate-level simulations can compare the netlist functionality to that of the RTL. The same test vectors used to verify the RTL are used to verify the netlist functionality, although it takes more time.

2.5.2 Timing verification

It is essential to verify timing performance of the netlist against the RTL at all the key stages of the design flow. Use of timing verification tools is a useful check for errors in the manual definition of false paths. The following methods can be used to perform timing verification:

Static timing analysis

Static timing verification is an efficient way of verifying timing performance of a design block, but can be pessimistic unless false paths are defined, to exclude these paths from analysis.

Dynamic simulation

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in the design. Results can be optimistic because worst case timing paths might not be exercised.

2.6 Customization

You can modify the MPMC to suit specific configurations, but using the design without modification provides the fastest integration. If you do modify the design, ensure that the design is fully functional by changing the functional test program and the testbenches where required. Synthesis must be run again to ensure that timing is met. For more information about these modifications, see the *ARM PrimeCell MultiPort Memory Controller (PL172) Design Manual*.

Some functionality of the MPMC cannot be removed without significant changes to its design. For example, the four quadword buffers in the MPMC cannot be removed because the dynamic memory controller assumes that the buffers are always enabled for dynamic memory accesses. Also, the buffer control logic is closely linked with the memory request generation logic and it is extremely difficult to separate these functionalities.

Note

The gate counts that can be achieved depend on the target cell library used, and synthesis timing constraints and command options.

Some of the customization methods possible in the MPMC are described in the following sections, together with estimates of the gate count reduction or addition, and the RTL updates required for each customization.

- *Changing the number of AHB memory interface ports*
- *Reducing the memory data bus width* on page 2-22
- *Removing the TIC* on page 2-23
- *Removing the static memory controller* on page 2-23
- *Removing the dynamic memory controller* on page 2-24
- *Removing the TIC and static memory controller* on page 2-25
- *Removing the TIC and dynamic memory controller* on page 2-26
- *Reducing the memory device support* on page 2-27.

2.6.1 Changing the number of AHB memory interface ports

The MPMC currently implements four AHB memory interface ports, but the number of ports can easily be changed. The design can support up to eight ports with minor modifications to the RTL.

Gate count change

Each AHB interface logic takes approximately 4000 gates:

- adding <number> AHB ports increases the gate count by <number> x4000 gates

- removing <number> AHB ports decreases the gate count by <number> x 4000 gates.

RTL updates

If <number> AHB ports are required, instantiate the AHB memory interface block <number> times in the AHB interface top-level block.

To verify the change, apply input test vectors at all the AHB ports and verify the operation of the AHB. If the number of AHB ports is more than four, more test cases must be added to check the multiple AHB port functionality. If the number of AHB ports is less than four, some of the test cases can be discarded.

2.6.2 Reducing the memory data bus width

Reducing the data bus width of the MPMC from 32 bits to 16 bits can reduce the pin counts. Table 2-1 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic and static memory and a TIC. The MPMC currently supports a 32-bit data bus.

Table 2-1 Pin counts for systems containing dynamic and static memory and TIC

Data bus	Control	Address	Total
16	32	28	76
32	32	28	92

Gate count change

There is no change in gate count.

RTL updates

No RTL changes are required for pin count reduction. Only the lower 16 bits have to be bonded out.

The MPMC does not require any additional verification for this change because the RTL is not changed. The verification test cases for 32-bit memories can be discarded.

2.6.3 Removing the TIC

If TIC testing is not required in a system, the MPMC TIC interface can be removed. Table 2-2 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic and static memory.

Table 2-2 Pin counts for systems containing dynamic and static memory

Data bus	Control	Address	Total
16	29	28	73
32	29	28	89

The **MPMCTESTIN**, **MPMCTESTREQA**, and **MPMCTESTREQB** signals are not required because the TIC is not used.

Gate count change

Removing the TIC module from the MPMC reduces the gate count by approximately 2000 gates.

RTL updates

To exclude the TIC controller, remove the MpmcTIC instantiation from the MPMC top-level block. Ensure that the external memory interface signals shared by the static memory and dynamic memory and TIC function correctly.

The verification test code of the TIC module can be discarded if this change is made.

2.6.4 Removing the static memory controller

If the static memory is not accessed through the MPMC in a system, it can be removed. Table 2-3 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic memory and TIC.

Table 2-3 Pin counts for systems containing dynamic memory and TIC

Data bus	Control	Address	Total
16	23	15	54
32	23	15	70

The **MPMCADDR[27:15]** address signals are not required. The following static memory signals are not required:

- **nMPMCSTCSOUT[3:0]**
- **nMPMCBLSOUT[3:0]**
- **nMPMCOEOUT**.

Gate count change

Removing the static memory controller functionality reduces the gate count by approximately 15000 gates.

RTL updates

To remove the static memory controller from the MPMC, remove the instantiation of the static memory controller and the memory bus granting logic from the top level of the MPMC block. Ensure that the signals driven by the static memory controller to the AHB interfaces, quadword buffers, and the pad interface are tied off to their inactive values for proper functioning of the RTL.

All the verification tests used to verify the static memory controller in the MPMC can be discarded if this change is made. The verification test cases used to test the static memory accesses interspersed with dynamic memory accesses can also be discarded.

2.6.5 Removing the dynamic memory controller

If the dynamic memory is not accessed through the MPMC in a system, the static memory controller in the MPMC can be removed. Table 2-4 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing static memory and a TIC.

Table 2-4 Pin counts for systems containing static memory and TIC

Data bus	Control	Address	Total
16	11	28	55
32	11	28	71

The following dynamic memory signals are not required:

- **MPMCCLKOUT[3:0]**
- **MPMCFBCLKIN[3:0]**
- **nMPMCDYCSOUT[3:0]**
- **MPMCCKEOUT[3:0]**
- **nMPMCRASOUT**

- **nMPMCCASOUT**
- **nMPMCRPOUT**
- **MPMCDQMOUT[3:0]**.

Gate count change

Removing the dynamic memory controller functionality reduces the gate count by approximately 38000 gates.

RTL updates

To remove the dynamic memory controller from the MPMC, remove the instantiation of the dynamic memory controller and the memory bus granting logic from the MPMC memory controller block. Ensure that the signals driven by the dynamic memory controller to the AHB interfaces, quadword buffers, and the pad interface are tied off to their inactive values for proper functioning of the RTL.

All the verification tests used to verify the dynamic memory controller in the MPMC can be discarded if this change is made.

2.6.6 Removing the TIC and static memory controller

If the static memory is not accessed through the MPMC in a system, and if the TIC is not used, the static memory controller and the MpmcTIC modules in the MPMC can be removed. Table 2-5 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic memory.

Table 2-5 Pin counts for systems containing dynamic memory

Data bus	Control	Address	Total
16	20	15	51
32	20	15	67

The **MPMCADDRROUT[27:15]** address signals are not required. The following static memory and TIC signals are not required:

- **nMPMPMCSTCSOUT[3:0]**
- **nMPMCBLSOUT[3:0]**
- **nMPMCOEOUT**
- **MPMCTESTIN**
- **MPMCTESTREQA**
- **MPMCTESTREQB**.

Gate count change

Removing the TIC and static memory controller reduces the gate count by approximately 18000 gates.

RTL updates

To remove the static memory controller and the TIC from the MPMC, remove the instantiation of the static memory controller from the MPMC memory controller block and the instantiation of the TIC module from the MPMC top-level block. Ensure that the signals driven by the static memory controller to the AHB interfaces, quadword buffers, and the pad interface are tied off to their inactive values for proper functioning of the RTL. In addition, tie off the TIC related inputs to the MPMC core logic to their inactive values.

All the verification tests used to verify the static memory controller and the TIC interface in the MPMC can be discarded if this change is made. The verification test cases used to test the static memory accesses interspersed with dynamic memory accesses can also be discarded

2.6.7 Removing the TIC and dynamic memory controller

If the dynamic memory is not accessed through the MPMC in a system, and if the TIC is not used, the dynamic memory controller and the MpmcTIC modules in the MPMC can be removed. Table 2-6 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing static memory.

Table 2-6 Pin counts for systems containing static memory

Data bus	Control	Address	Total
16	8	28	52
32	8	28	68

The following dynamic memory and TIC signals are not required:

- **MPMCCLKOUT[3:0]**
- **MPMCFBCLKIN[3:0]**
- **nMPMCDYCSOUT[3:0]**
- **MPMCCKEOUT[3:0]**
- **nMPMCRASOUT**
- **nMPMCCASOUT**
- **nMPMCRPOUT**
- **MPMCTESTIN**

- **MPMCTESTREQA**
- **MPMCTESTREQB**.

Gate count change

Removing the TIC and dynamic memory controller reduces the gate count by approximately 40000 gates.

RTL updates

To remove the dynamic memory controller and the TIC from the MPMC, remove the instantiation of the dynamic memory controller from the MPMC memory controller block and the instantiation of the TIC module from the MPMC top-level block. Ensure that the signals driven by the dynamic memory controller to the AHB interfaces, quadword buffers, and pad interface are tied off to their inactive values for proper functioning of the RTL. In addition, tie off the TIC related inputs to the MPMC core logic to their inactive values.

All the verification tests used to verify the dynamic memory controller and the TIC interface in the MPMC can be discarded if this change is made. The verification test cases used to test the static memory accesses interspersed with dynamic memory accesses can also be discarded.

2.6.8 Reducing the memory device support

Some of the address pins can be removed depending on the memory devices to be supported. For example, if 32Kx32, 32Kx16, or smaller devices static memory devices are to be supported, address lines **MPMCADDRROUT[27:15]** are not required. Removing these address lines does not reduce the number of dynamic memory devices supported.

Gate count change

There is no change in gate count.

RTL updates

No RTL changes are required for pin count reduction.

The MPMC does not require any additional verification for this change because the RTL is not changed. The verification test cases for memories that target larger memory devices can be discarded.

2.7 Environment variables

Simulation and synthesis for the MPMC are controlled by the following environment variables:

GLOBAL The path to the shared files is defined by the GLOBAL variable.

For example:

```
setenv GLOBAL /h/arm/global
```

PERIPH The peripheral name is defined by the PERIPH variable.

For example:

```
setenv PERIPH Mpmc
```

SIMULATOR

The simulator type is defined by the SIMULATOR variable.

It can take the following values:

- `modelsim`, for ModelSim simulations
- `LDV`, for LDV simulations.

For example:

```
setenv SIMULATOR modelsim
```

```
setenv SIMULATOR LDV
```

TEST_METH

The scan methodology adopted during synthesis is defined by the TEST_METH variable.

It can take the following values:

- `noscan`, that creates a netlist without scan structures
- `scanready`, that creates a netlist with scan F/Fs but without scan routing
- `scaninsert`, that creates a fully-routed scan netlist.

For example:

```
setenv TEST_METH noscan
```

```
setenv TEST_METH scanready
```

```
setenv TEST_METH scaninsert
```

Note

All the scan inputs and outputs are specified at the top level of the MPMC. Therefore, for a scan-insertion synthesis of the MPMC, the scan-ready synthesis of MpmcCore and MpmcTIC must be done, followed by the scan-insertion synthesis at the top level.

HDL_SOURCE

The source code type is defined by the HDL_SOURCE variable.

It can take the following values:

- vhd1, for VHDL RTL
- verilog, for Verilog RTL.

For example:

```
setenv HDL_SOURCE vhd1
setenv HDL_SOURCE verilog
```

HDL_NETL

The netlist type is defined by the HDL_NETL variable.

It can take the following values:

- vhd1, for VHDL netlist
- verilog, for Verilog netlist.

For example:

```
setenv HDL_NETL vhd1
setenv HDL_NETL verilog
```

TEST_ENV The test environment is defined by the TEST_ENV variable.

It can take the following values:

- BUSTALK, for functional vector simulations
- TICTALK, for integration vector simulations.

For example:

```
setenv TEST_ENV BUSTALK
setenv TEST_ENV TICTALK
```

LIB_VHDL The location of the vhd1 cell library views is defined by the LIB_VHDL variable.

For example:

```
setenv LIB_VHDL /AVANT/cb18/v1.0/vital/cb18os120/zero
```

DIRS_VERILOG

The location of the verilog cell library views is defined by the DIRS_VERILOG variable.

For example:

```
setenv DIRS_VERILOG /AVANT/cb18/v1.0/verilog/cb18os120/zero
```

FILES_VERILOG

The location of the verilog *User Defined Primitive* (UDP) file is defined by the FILES_VERILOG variable.

For example:

```
setenv FILES_VERILOG  
/AVANT/cb18/v1.0/verilog/cb18os120/zero/mtb_verilog.v
```

LIB_SYNOP

The location of the synopsys target library files is defined by the LIB_SYNOP variable.

For example:

```
setenv LIB_SYNOP /AVANT/cb18/v1.0/synopsys/cb18os120/1999.05/models
```

AMBA

This environment variable defines the AMBA bus to which the peripheral is connected. Because the MPMC is an AHB peripheral, set this environment variable to AHB. This variable is used by makefiles in the MPMC database to select settings for the integration world.

For example:

```
setenv AMBA AHB
```


Chapter 3

Static Memory Controller

This chapter describes the static memory controller. It contains the following sections:

- *Static memory device selection* on page 3-2
- *Write-protection* on page 3-3
- *Extended wait transfers* on page 3-4
- *Memory mapped peripherals* on page 3-5
- *Static memory initialization* on page 3-6
- *Elimination of floating bytes on the external interface* on page 3-38
- *Byte lane control* on page 3-39
- *Address connectivity* on page 3-40
- *Byte lane control and data bus steering* on page 3-47.

3.1 Static memory device selection

Table 3-1 shows suggested configurations for the static memory controller with different types of memory devices. These fields are found in the MPMCStaticConfig[n] Register. For more information, see the *PrimeCell MPMC (PL172) Technical Reference Manual*.

Table 3-1 Static memory controller configurations

Device	Write protect	Page mode	Buffer
ROM	Enabled	Disabled	Disabled ^a
Page mode ROM	Enabled	Enabled	Enabled ^a
Extended wait ROM	Enabled	Disabled	Disabled ^a
SRAM	Disabled (or enabled) ^b	Disabled	Disabled ^a
Page mode SRAM	Disabled (or enabled) ^b	Enabled	Enabled ^a
Extended wait SRAM	Disabled (or enabled) ^b	Disabled	Disabled ^a
Flash	Disabled (or enabled) ^b	Disabled	Disabled ^c
Page mode Flash	Disabled (or enabled) ^b	Enabled	Enabled ^c
Extended wait Flash	Disabled (or enabled) ^b	Disabled	Disabled ^a
Memory mapped peripheral	Disabled (or enabled) ^b	Disabled	Disabled

- a. Enabling the buffers means that any access causes the buffer to be used. Depending on the application, this can provide performance improvements. Generally, devices without async-page-mode support work better with the buffer disabled. Depending on the application, this can provide performance improvements.
- b. SRAM and FLASH memory devices can be write protected if required.
- c. Buffering must be disabled when performing FLASH memory commands and during writes.

Note

- Buffering enables the transaction order to be rearranged to improve memory performance. If the transaction order is important, the buffers must be disabled.
- Extended wait and Page mode cannot be enabled simultaneously.

3.2 Write-protection

Each static memory chip select can be configured for write-protection. Usually SRAM is unprotected and ROM devices must be write-protected (to avoid potential bus conflict when performing a write access to ROM), but the P field in the MPMCStaticConfig Register can be set to write-protect SRAM as well as ROM devices. If a write access is made to a write-protected memory bank, an error is indicated by the **HRESP[1:0]** signal. If a write access is made to a memory bank containing ROM devices and the chip select is not write-protected, an error indication is not returned, and the write access proceeds as normal. This might lead to bus conflict.

3.3 Extended wait transfers

The static memory controller supports extremely long transfer times. In normal use the memory transfers are timed using the MPMCStaticWaitRd and MPMCStaticWaitWr Registers. These registers enable transfers with up to 32 wait states. However, if an extremely slow static memory device has to be accessed, the MPMCStaticConfig Extended Wait (EW) bit can be enabled. When this bit is enabled, the MPMCStaticExtendedWait Register times both the read and write transfers. This register enables transfers to have up to 16368 wait states.

Note

- Using extremely long transfer times might mean that SDRAM devices are not refreshed correctly.
 - Very slow transfers can severely degrade system performance because the external memory interface is tied up for long periods of time. This has detrimental effects on time critical services, such as interrupt latency and low latency devices, such as video controllers.
-

3.4 Memory mapped peripherals

Some systems use external peripherals that can be accessed using the static memory interface.

Because of the way many of these peripherals function, the read and write transfers to them must not be buffered. Therefore the buffer must be disabled.

3.5 Static memory initialization

Static memory must be initialized as required after *Power-On Reset (nPOR)* by programming the relevant registers in the MPMC, and the configuration registers in the external static memory device. The MPMC static memory configuration registers are described in the following sections:

- *Access sequencing and memory width*
- *Wait state generation*
- *Static memory read control* on page 3-7
- *Static memory write control* on page 3-22.

3.5.1 Access sequencing and memory width

The data width of each external memory bank must be configured by programming the appropriate bank configuration register, MPMCStaticConfig[n]. When the external memory bus is narrower than the transfer initiated from the current AMBA bus master, the internal bus transfer takes several external bus transfers to complete. For example, if bank 0 is configured as 8-bit wide memory and a 32-bit read is initiated, the AMBA AHB bus stalls while the MPMC reads four consecutive bytes from the memory. During these accesses the static memory controller block demultiplexes the four bytes into one 32-bit word on the AMBA AHB bus.

3.5.2 Wait state generation

Each bank of the MPMC must be configured for external transfer wait states in read and write accesses. This is achieved by programming the appropriate fields of the bank control registers:

- MPMCStaticConfig[n]
- MPMCStaticWaitWen[n]
- MPMCStaticWaitOen[n]
- MPMCStaticWaitRd[n]
- MPMCStaticWaitWr[n]
- MPMCStaticWaitPage[n]
- MPMCStaticWaitTurn[n]
- MPMCStaticExtendedWait.

The number of cycles in which an AMBA transfer completes is controlled by two additional factors:

- access width
- external memory width.

Each bank of the MPMC has a programmable enable for the extended wait (EW):

- The MPMCStaticWaitRd[n] Register WAITRD wait state field can be programmed to select from 1 to 32 wait states for read memory accesses to SRAM and ROM, or the initial read access to page mode devices.
- The MPMCStaticWaitWr[n] Register WAITWR wait state field can be programmed to select from 1 to 32 wait states for write access to SRAM.
- The MPMCStaticWaitPage[n] Register can be programmed to select from 1 to 32 wait states for page mode accesses.

3.5.3 Static memory read control

The static memory read controls are described in the following sections:

- *Output enable programmable delay*
- *ROM, SRAM, and Flash*
- *Asynchronous page mode read* on page 3-18.

Output enable programmable delay

You can program the delay between the assertion of the chip select and the output enable from 0 to 15 cycles using the WAITOEN bits of the MPMCStaticWaitOen[n] Registers. This reduces the power consumption for memories that cannot provide valid output data immediately after the chip select has been asserted. The output enable is always deasserted at the same time as the chip select, at the end of the transfer.

ROM, SRAM, and Flash

The MPMC uses the same read timing control for ROM, SRAM, and Flash devices. Each read starts with the assertion of the appropriate memory bank chip select signals, **nMPMCSTCSOUT[n]** and memory address signals, **MPMCADDRROUT[27:0]**. The read access time is determined by the number of wait states programmed for the WAITRD field of the MPMCStaticWaitRd[n] Register. The WAITTURN field in the MPMCStaticWaitTurn[n] Register determines the number of bus turnaround wait states added between external read and write transfers.

Figure 3-1 shows an external memory read transfer with the minimum zero wait states (WAITRD = 0). A minimum of five AHB wait states are inserted during all single-read transfers. See Table 3-2 for the programmed values.

Table 3-2 Static memory timing parameters

Timing parameter	Value
WAITRD	0
WAITOEN	0
WAITPAGE	NA
WAITWR	NA
WAITWEN	NA
WAITTURN	NA

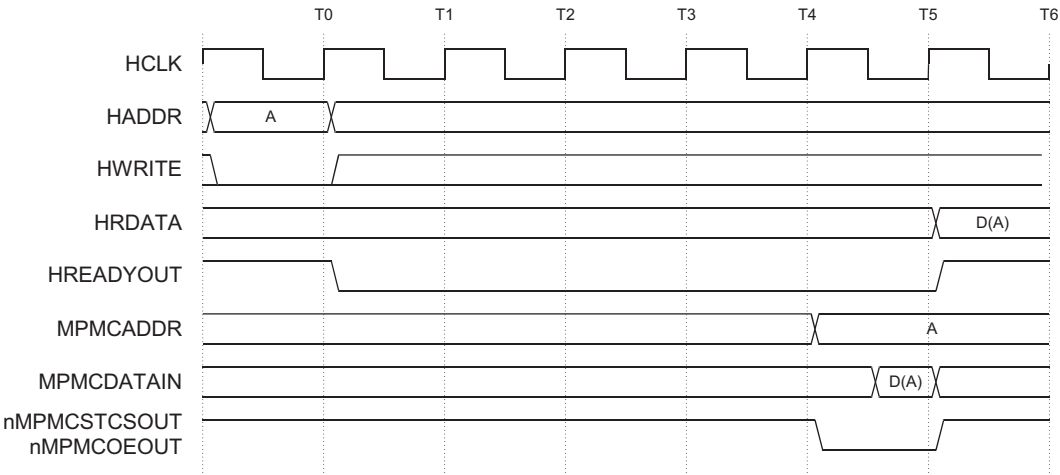


Figure 3-1 External memory zero wait state read timing diagram

Table 3-3 describes the transactions in Figure 3-1 on page 3-8.

Table 3-3 External memory zero wait state read

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory address, chip select, and control signals submitted to static memory.
T5-T6	Read data returned from the static memory. Data is provided to AHB.

Figure 3-2 on page 3-10 shows an external memory read transfer with two wait states ($WAITRD = 2$). Seven AHB wait states are inserted during the transfer, five for the standard read access, and an additional two because of the programmed wait states added ($WAITRD$). See Table 3-4 for the programmed values.

Table 3-4 Static memory timing parameters

Timing parameter	Value
WAITRD	2
WAITOEN	0
WAITPAGE	NA
WAITWR	NA
WAITWEN	NA
WAITTURN	NA

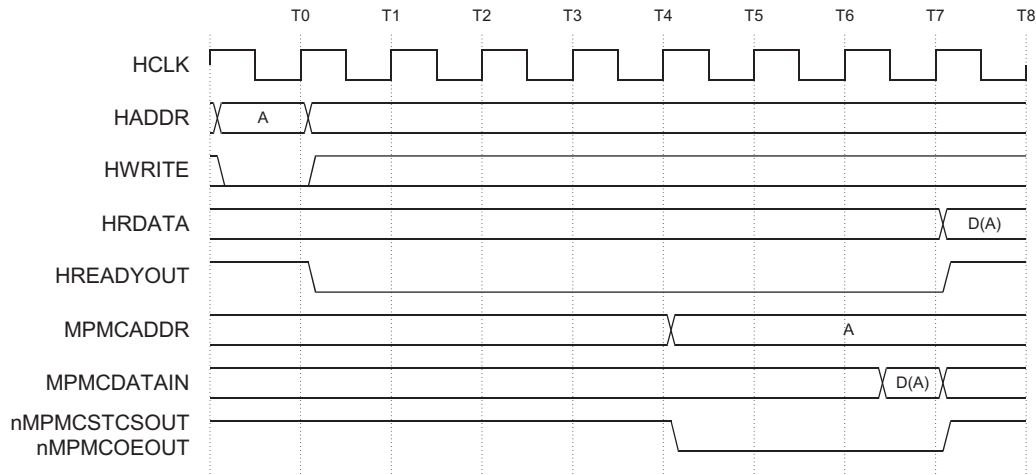


Figure 3-2 External memory two wait state read timing diagram

Table 3-5 describes the transactions for Figure 3-2.

Table 3-5 External memory two wait state read

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory address, chip select, and control signals submitted to static memory.
T5-T6	Read wait state 1.
T6-T7	Read wait state 2.
T7-T8	Read data returned from the static memory. Data is provided to the AHB.

Figure 3-3 shows an external memory read transfer with two output enable delay states (WAITOEN = 2). Seven AHB wait states are inserted during the transfer, five for the standard read, and an additional two because of the output enable delay states added. See Table 3-6 for the programmed values.

Table 3-6 Static memory timing parameters

Timing parameter	Value
WAITRD	2
WAITOEN	2
WAITPAGE	NA
WAITWR	NA
WAITWEN	NA
WAITTURN	NA

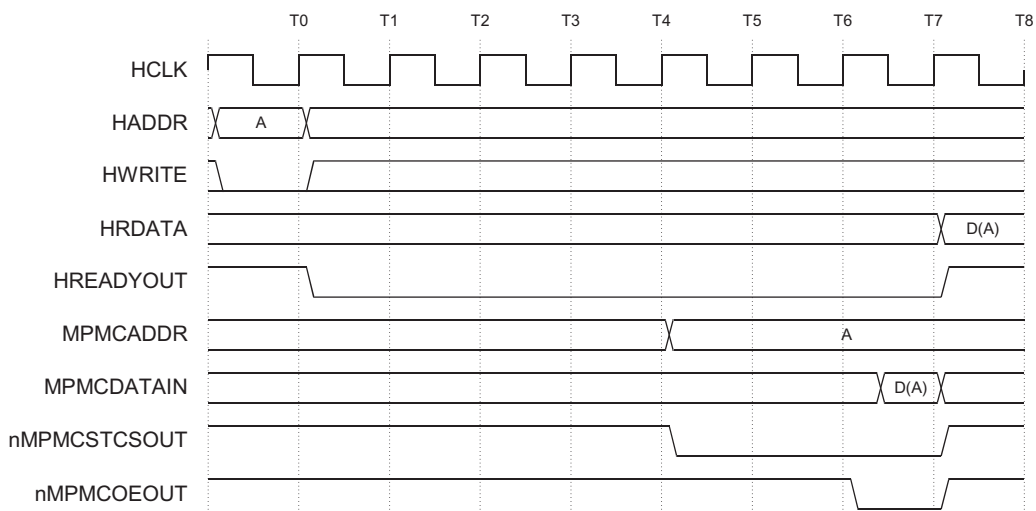


Figure 3-3 External memory two output enable delay state read timing diagram

Table 3-7 describes the transactions for Figure 3-3 on page 3-11.

Table 3-7 External memory two output enable delay state

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory address, chip select, and control signals submitted to static memory. Static memory output enable inactive.
T5-T6	Static memory output enable inactive.
T6-T7	Static memory output enable active.
T7-T8	Read data returned from the static memory. Data is provided to the AHB.

Figure 3-4 on page 3-13 shows external memory read transfers with zero wait states (WAITRD = 0). These are nonsequential transfers. All transfers are treated as separate reads, so have the minimum of six AHB wait states added. See Table 3-8 for the programmed values.

Table 3-8 Static memory timing parameters

Timing parameter	Value
WAITRD	0
WAITOEN	0
WAITPAGE	NA
WAITWR	NA
WAITWEN	NA
WAITTURN	NA

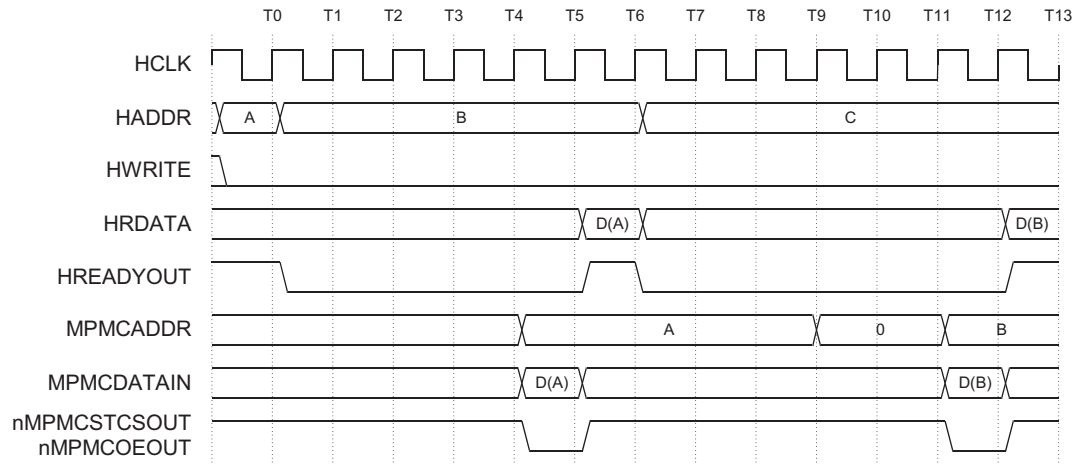
**Figure 3-4 External memory two zero wait state read timing diagram**

Table 3-9 describes the transactions for Figure 3-4.

Table 3-9 External memory two zero wait state reads

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory address, chip select, and control signals submitted to static memory.
T5-T6	Read data returned from the static memory. Data is provided to the AHB.
T6-T7	AHB address provided to memory controller. AHB transaction processing.
T7-T8	Arbitration of AHB memory ports.
T8-T9	Raise EBI bus request.
T9-T10	Register EBI grant.

Table 3-9 External memory two zero wait state reads (continued)

Cycle	Description
T10-T11	Arbitration of AHB memory ports.
T11-T12	Static memory address, chip select, and control signals submitted to static memory.
T12-T13	Read data returned from the static memory. Data is provided to the AHB.

Figure 3-5 on page 3-15 shows a burst of zero wait state reads with the burst length specified. Because the length of the burst is known, it is possible to hold the chip select asserted during the whole burst, and generate the external transfers before the current AHB transfer has completed. Therefore, the first read has five AHB wait states added, and the three following sequential reads have zero AHB wait states added because of the automatic generation of the external transfers.

Note

INCR bursts are treated as INCR4 bursts.

See Table 3-10 for the programmed values.

Table 3-10 Static memory timing parameters

Timing parameter	Value
WAITRD	0
WAITOEN	0
WAITPAGE	NA
WAITWR	NA
WAITWEN	NA
WAITTURN	NA

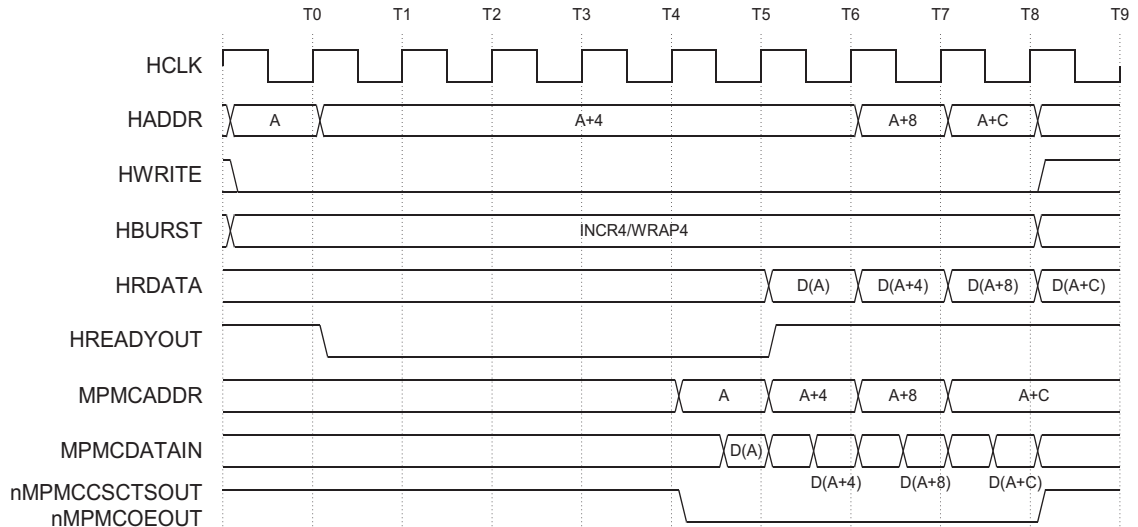


Figure 3-5 External memory zero wait fixed length burst read timing diagram

Table 3-11 describes the transactions for Figure 3-5.

Table 3-11 External memory zero wait fixed length burst read

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory read 0 address, chip select, and control signals submitted to static memory.
T5-T6	Static memory read 1 address, chip select, and control signals submitted to static memory. Read data 0 returned from the static memory. Read data 0 is provided to the AHB.

Table 3-11 External memory zero wait fixed length burst read (continued)

Cycle	Description
T6-T7	Static memory read 2 address, chip select, and control signals submitted to static memory. Read data 1 returned from the static memory. Read data 1 is provided to the AHB.
T7-T8	Static memory read 3 address, chip select, and control signals submitted to static memory. Read data 2 returned from the static memory. Read data 2 is provided to the AHB.
T8-T9	Read data 3 returned from the static memory. Read data 3 is provided to the AHB.

Figure 3-6 on page 3-17 shows a burst of two wait state reads with the length specified. The WAITRD value is used for all transfers in the burst, with the first read having seven AHB wait states inserted, and all sequential transfers having two AHB wait states. See Table 3-12 for the programmed values.

Table 3-12 Static memory timing parameters

Timing parameter	Value
WAITRD	2
WAITOEN	0
WAITPAGE	NA
WAITWR	NA
WAITWEN	NA
WAITTURN	NA

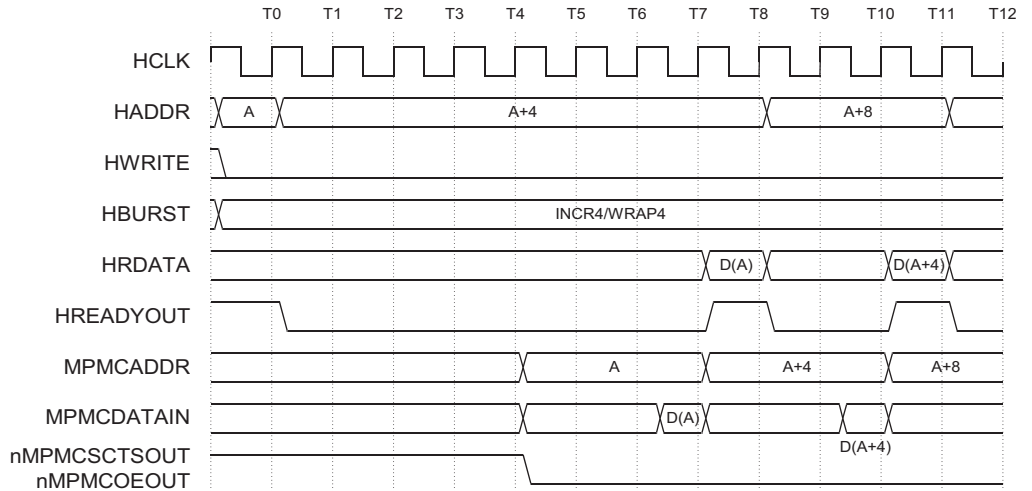


Figure 3-6 External memory two wait states fixed length burst read timing diagram

Table 3-13 describes the transactions for Figure 3-6.

Table 3-13 External memory two wait states fixed length burst read

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory address, chip select, and control signals submitted to static memory.
T5-T6	Read wait state 1.
T6-T7	Read wait state 2.
T7-T8	Read data 0 returned from the static memory. Read data 0 is provided to the AHB. Static memory transfer 1, address, chip select, and control signals submitted to static memory.
T8-T9	Read wait state 1.

Table 3-13 External memory two wait states fixed length burst read (continued)

Cycle	Description
T9-T10	Read wait state 2.
T10-T11	Read data 1 returned from the static memory. Read data1 is provided to the AHB. Static memory transfer 2, address, chip select, and control signals submitted to static memory.
T11-T12	Read wait state 1.

Asynchronous page mode read

The MPMC supports asynchronous page mode read of up to four memory transfers by updating address bits A[1] and A[0]. This feature increases the bandwidth by using a reduced access time for the read accesses that are in page mode. The first read access takes MPMCStaticWaitRd and WAITRD cycles. Subsequent read accesses that are in page mode take MPMCStaticWaitPage and WAITPAGE cycles. The chip select and output enable lines are held during the burst, and only the lower two address bits change between subsequent accesses. At the end of the burst the chip select and output enable lines are deasserted together.

Figure 3-7 on page 3-19 shows an external memory page mode read transfer with two initial wait states, and one sequential wait state. The first read has seven AHB wait states inserted, and the following, up to 3, sequential transfers have only one AHB wait state. This gives increased performance over the equivalent nonpage mode ROM timing shown in Figure 3-6 on page 3-17. See Table 3-14 for the programmed values.

Table 3-14 Static memory timing parameters

Timing parameter	Value
WAITRD	2
WAITOEN	0
WAITPAGE	1
WAITWR	NA
WAITWEN	NA
WAITTURN	NA

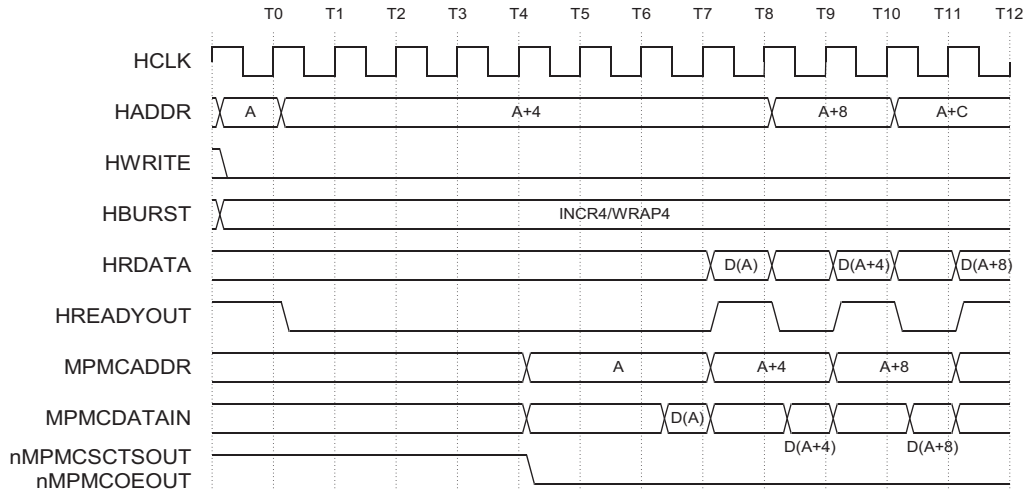


Figure 3-7 External memory page mode read transfer timing diagram

Table 3-15 describes the transactions for Figure 3-7.

Table 3-15 External memory page mode read

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory transfer 0, address, chip select, and control signals submitted to static memory.
T5-T6	Read wait state 1.
T6-T7	Read wait state 2.
T7-T8	Read data 0 returned from the static memory. Read data 0 is provided to the AHB. Static memory transfer 1, address, chip select, and control signals submitted to static memory.
T8-T9	Read page mode wait state1.

Table 3-15 External memory page mode read (continued)

Cycle	Description
T9-T10	Read data 1 returned from the static memory. Read data1 is provided to the AHB. Static memory transfer 2, address, chip select, and control signals submitted to static memory.
T10-T11	Read page mode wait state1.
T11-T12	Read data 2 returned from the static memory. Read data 2 is provided to the AHB. Static memory transfer 3, address, chip select, and control signals submitted to static memory.

Figure 3-8 on page 3-21 shows a 32-bit read from an 8-bit page mode ROM device, causing four burst reads to be performed. A total of eight AHB wait states are added during this transfer:

- five for the first external read
- one for each of the subsequent reads.

WAITRD and WAITPAGE are zero. See Table 3-16 for the programmed values.

Table 3-16 Static memory timing parameters

Timing parameter	Value
WAITRD	0
WAITOEN	0
WAITPAGE	0
WAITWR	NA
WAITWEN	NA
WAITTURN	NA

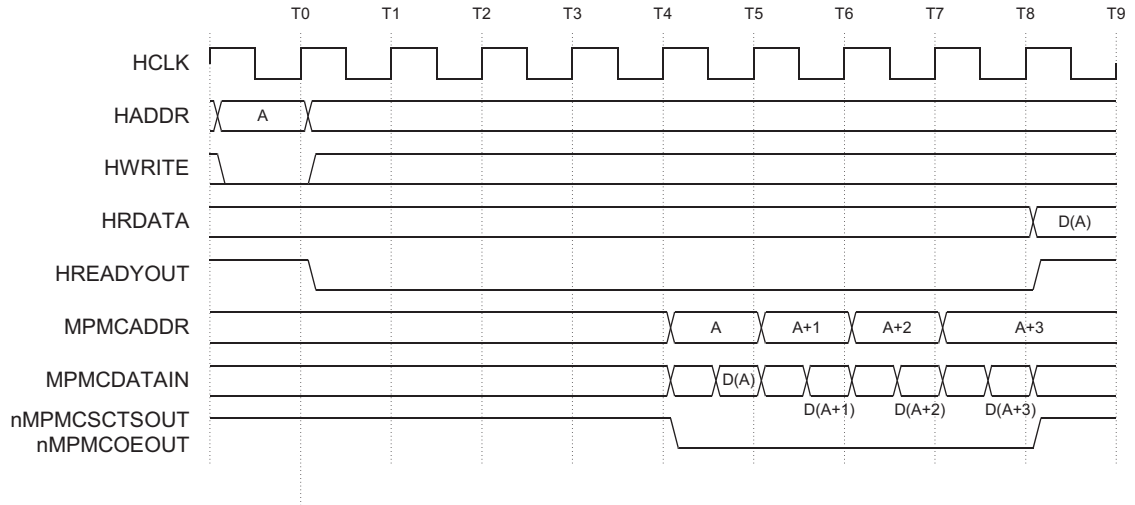


Figure 3-8 External memory 32-bit burst read from 8-bit memory timing diagram

Table 3-17 describes the transactions for Figure 3-8.

Table 3-17 External memory 32-bit burst read from 8-bit memory

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory transfer 0, address, chip select, and control signals submitted to static memory.
T5-T6	Static memory transfer 1, address, chip select, and control signals submitted to static memory. Read data byte 0 returned from the static memory.

Table 3-17 External memory 32-bit burst read from 8-bit memory (continued)

Cycle	Description
T6-T7	Static memory transfer 2, address, chip select, and control signals submitted to static memory. Read data byte 1 returned from the static memory.
T7-T8	Static memory transfer 3, address, chip select, and control signals submitted to static memory. Read data byte2 returned from the static memory.
T8-T9	Read data byte 3 returned from the static memory. Read data 32-bit word is provided to the AHB.

3.5.4 Static memory write control

Write timing is described in the following sections:

- *Write enable programmable delay*
- *SRAM*
- *Flash memory* on page 3-30.

Write enable programmable delay

The delay between the assertion of the chip select and the write enable is programmable from 0 to 15 cycles using the WAITWEN bits of the MPMCStaticWaitWen0-3 Registers. This reduces the power consumption for memories. The write enable is asserted on the rising edge of **HCLK** after the assertion of the chip select for zero wait states. The write enable is always deasserted a cycle before the chip select, at the end of the transfer. **nMPMCBLSOUT** has the same timing as **nMPMCWEOUT** for writes to 8-bit devices that use the byte lane selects instead of the write enables.

SRAM

Write timing for SRAM starts with assertion of the appropriate memory bank chip selects **nMPMCSTCSOUT[n]** and address signals **MPMCADDRROUT[27:0]**. The write access time is determined by the number of wait states programmed for the WAITWR field of the MPMCStaticWaitWr Register. The WAITTURN field in the bank control register determines the number of bus turnaround wait states added between external read and write transfers.

Figure 3-9 shows a single external memory write transfer with minimum zero wait states ($WAITWR = 0$). One wait state is added. See Table 3-18 for the programmed values.

Table 3-18 Static memory timing parameters

Timing parameter	Value
WAITRD	NA
WAITOEN	NA
WAITPAGE	NA
WAITWR	0
WAITWEN	0
WAITTURN	NA

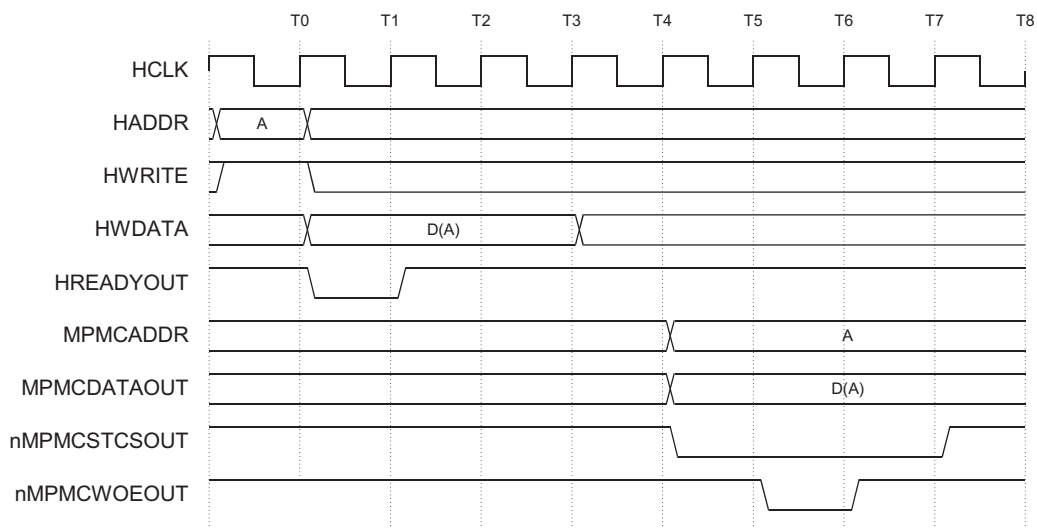


Figure 3-9 External memory zero wait state write timing diagram

Table 3-19 describes the transactions for Figure 3-9 on page 3-23.

Table 3-19 External memory zero wait state write

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory transfer 0, address, chip select, and control signals submitted to static memory. Write data is read from the AHB memory port. Write enable inactive.
T5-T6	Write enable taken active. Write data submitted to static memory. Static memory writes the data.
T6-T7	Static memory writes the data. Write enable taken inactive.
T7-T8	Static memory control signals taken inactive.

Figure 3-10 on page 3-25 shows a single external memory write transfer with two wait states (WAITWR = 2). One AHB wait state is added. See Table 3-20 for the programmed values.

Table 3-20 Static memory timing parameters

Timing parameter	Value
WAITRD	NA
WAITOEN	NA
WAITPAGE	NA
WAITWR	2
WAITWEN	0
WAITTURN	NA

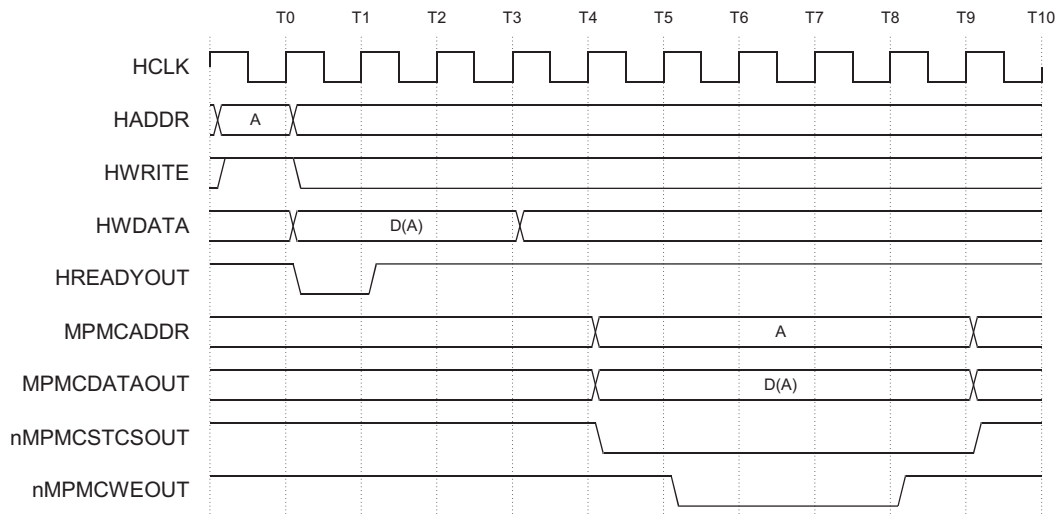
**Figure 3-10 External memory two wait state write timing diagram**

Table 3-21 describes the transactions for Figure 3-10.

Table 3-21 External memory two wait state write

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory transfer 0, address, chip select, and control signals submitted to static memory. Write data is read from the AHB memory port. Write enable inactive.
T5-T6	Write enable taken active. Write data submitted to static memory.
T6-T7	Wait state 1.

Table 3-21 External memory two wait state write (continued)

Cycle	Description
T7-T8	Wait state 2.
T8-T9	Static memory writes the data. Write enable taken inactive.
T9-T10	Static memory control signals taken inactive.

Figure 3-11 on page 3-27 shows a single external memory write transfer with two write enable delay states (WAITWEN = 2). One wait state is added. See Table 3-22 for the programmed values.

Table 3-22 Static memory timing parameters

Timing parameter	Value
WAITRD	NA
WAITOEN	NA
WAITPAGE	NA
WAITWR	2
WAITWEN	2
WAITTURN	NA

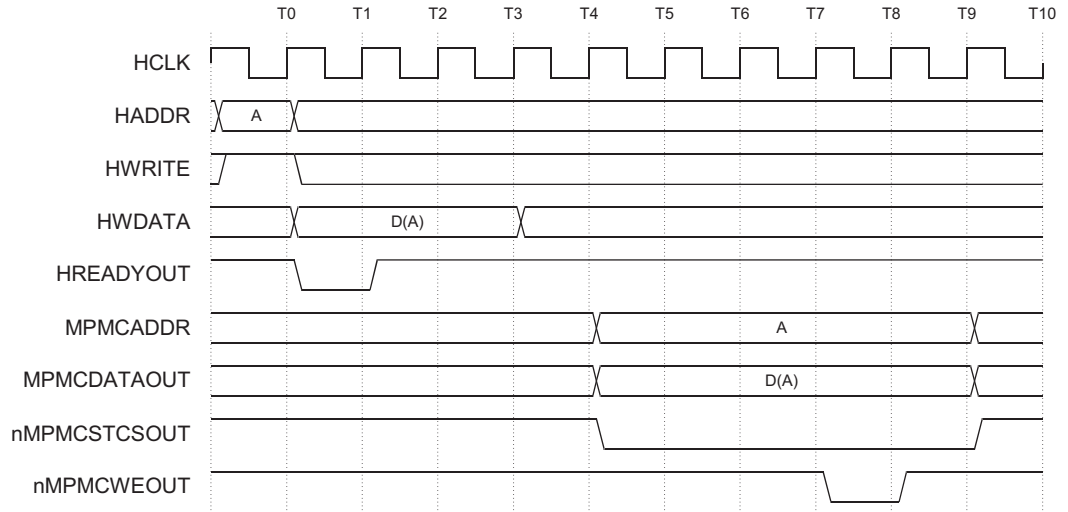


Figure 3-11 External memory two write enable delay write timing diagram

Table 3-23 describes the transactions for Figure 3-11.

Table 3-23 External memory two write enable delay write

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory transfer 0, address, chip select, and control signals submitted to static memory. Write data is read from the AHB memory port. Write enable inactive.
T5-T6	Write data submitted to static memory. Write enable wait state 1.
T6-T7	Write enable wait state 2.

Table 3-23 External memory two write enable delay write (continued)

Cycle	Description
T7-T8	Write enable taken active.
T8-T9	Static memory writes the data. Write enable taken inactive.
T9-T10	Static memory control signals taken inactive.

Figure 3-12 on page 3-29 shows two external memory write transfers with zero wait states (WAITWR = 0). Four AHB wait states are added to the second write, because this can only be started when the first write has completed. This is the timing of any sequence of write transfers, nonsequential to nonsequential, or nonsequential to sequential, with any value of **HBURST**. The maximum speed of write transfers is controlled by the external timing of the write enable relative to the chip select, so all external writes must take two cycles to complete, the cycle that write enable is asserted, and the cycle that write enable is deasserted. See Table 3-24 for the programmed values.

Table 3-24 Static memory timing parameters

Timing parameter	Value
WAITRD	NA
WAITOEN	NA
WAITPAGE	NA
WAITWR	0
WAITWEN	0
WAITTURN	0

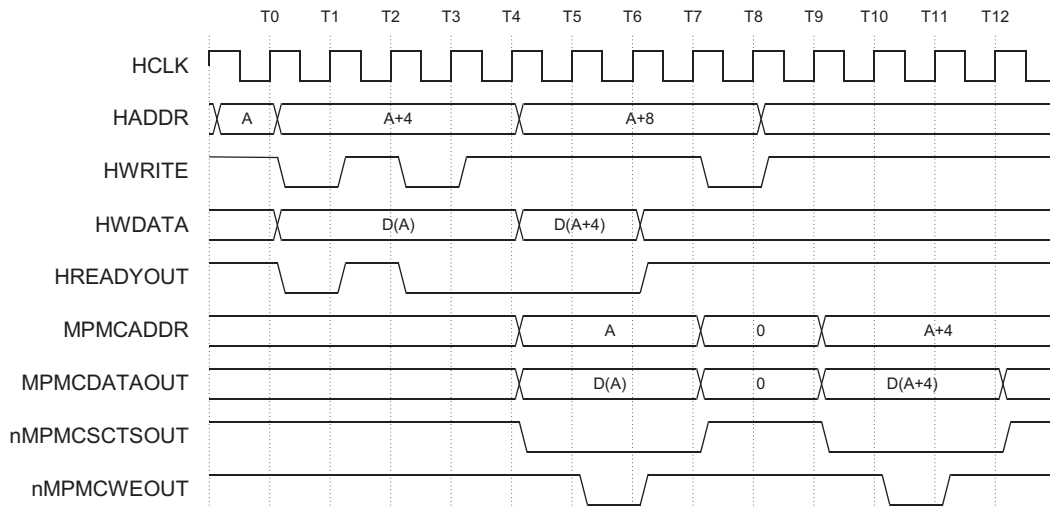
**Figure 3-12 External memory two zero wait writes timing diagram**

Table 3-25 describes the transactions for Figure 3-12.

Table 3-25 External memory two zero wait writes

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory transfer 0, address, chip select, and control signals submitted to static memory. Write data 0 is read from the AHB memory port. Write enable inactive.
T5-T6	Write enable taken active. Write data submitted to static memory.
T6-T7	Static memory writes data 0. Write enable taken inactive. Write data 1 is read from AHB memory port.

Table 3-25 External memory two zero wait writes (continued)

Cycle	Description
T7-T8	Static memory control signals taken inactive.
T8-T9	Memory controller processing.
T9-T10	Static memory transfer 1, address, chip select, and control signals submitted to static memory. Write enable inactive. Write data submitted to static memory.
T10-T11	Write enable taken active.
T11-T12	Static memory writes data 1. Write enable taken inactive.

Flash memory

Write timing for flash memory devices is the same as for SRAM devices.

3.5.5 Bus turnaround

The MPMC can be configured for each memory bank to use external bus turnaround cycles between read and write memory accesses. The WAITTURN field can be programmed for 1 to 16 bus turnaround wait states. This is to avoid bus contention on the external memory data bus. Bus turnaround cycles are generated between external bus transfers as follows:

- read to read (different memory banks)
- read to write (same memory bank)
- read to write (different memory banks).

Figure 3-13 on page 3-31 shows a zero wait read followed by a zero wait write with default turnaround between the transfers of two cycles because of the timing of the AHB transfers. Standard AHB wait states are added to the transfers, five for the read, and three for the write. See Table 3-26 on page 3-31 for the programmed values.

Table 3-26 Static memory timing parameters

Timing parameter	Value
WAITRD	0
WAITOEN	0
WAITPAGE	NA
WAITWR	0
WAITWEN	0
WAITTURN	0

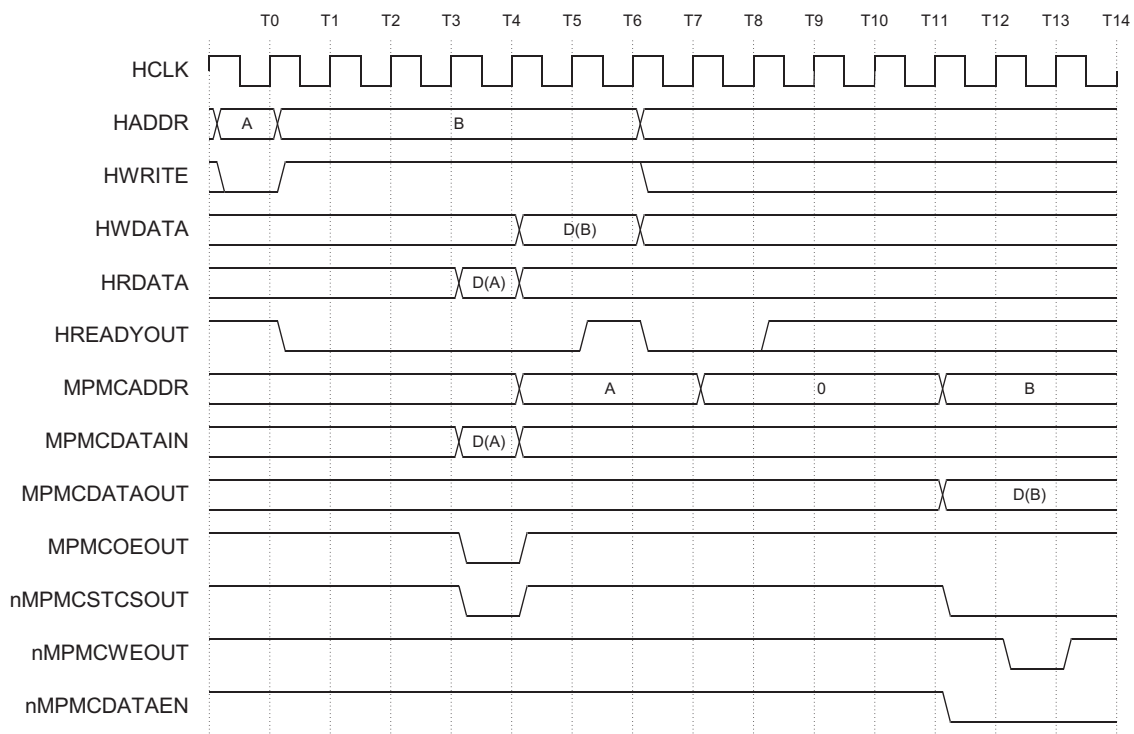
**Figure 3-13 Read followed by write (both zero wait) with no turnaround**

Table 3-27 describes the transactions for Figure 3-13 on page 3-31.

Table 3-27 Read followed by write (both zero wait) with no turnarounds

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Arbitration of AHB memory ports.
T3-T4	Read data returned from the static memory. Data is provided to the AHB. Static memory address, chip select, and control signals submitted to static memory.
T4-T5	AHB write address provided to memory controller.
T5-T6	Memory controller processing.
T6-T7	Memory controller processing.
T7-T8	Static memory transfer address, chip select, and control signals submitted to static memory. Write data is read from AHB memory port. Write enable inactive.
T8-T9	Write enable taken active. Write data submitted to static memory.
T9-T10	Static memory control signals taken inactive.
T10-T11	Memory controller processing.
T11-T12	Static memory transfer 1, address, chip select, and control signals submitted to static memory. Write enable inactive. Write data submitted to static memory.
T12-T13	Write enable taken active.
T13-T14	Static memory writes data 1. Write enable taken inactive.

Figure 3-14 on page 3-34 shows a zero wait write followed by a zero wait read with default turnaround between the transfers of one cycle. Three wait states are added to the write transfer, but five are added to the read transfer. The five wait states for the read transfer consist of two parts:

- two wait states to enable the previous write access to complete
- standard three wait states for the read transfer.

See Table 3-28 for the programmed values.

Table 3-28 Static memory timing parameters

Timing parameter	Value
WAITRD	0
WAITOEN	0
WAITPAGE	NA
WAITWR	0
WAITWEN	0
WAITTURN	0

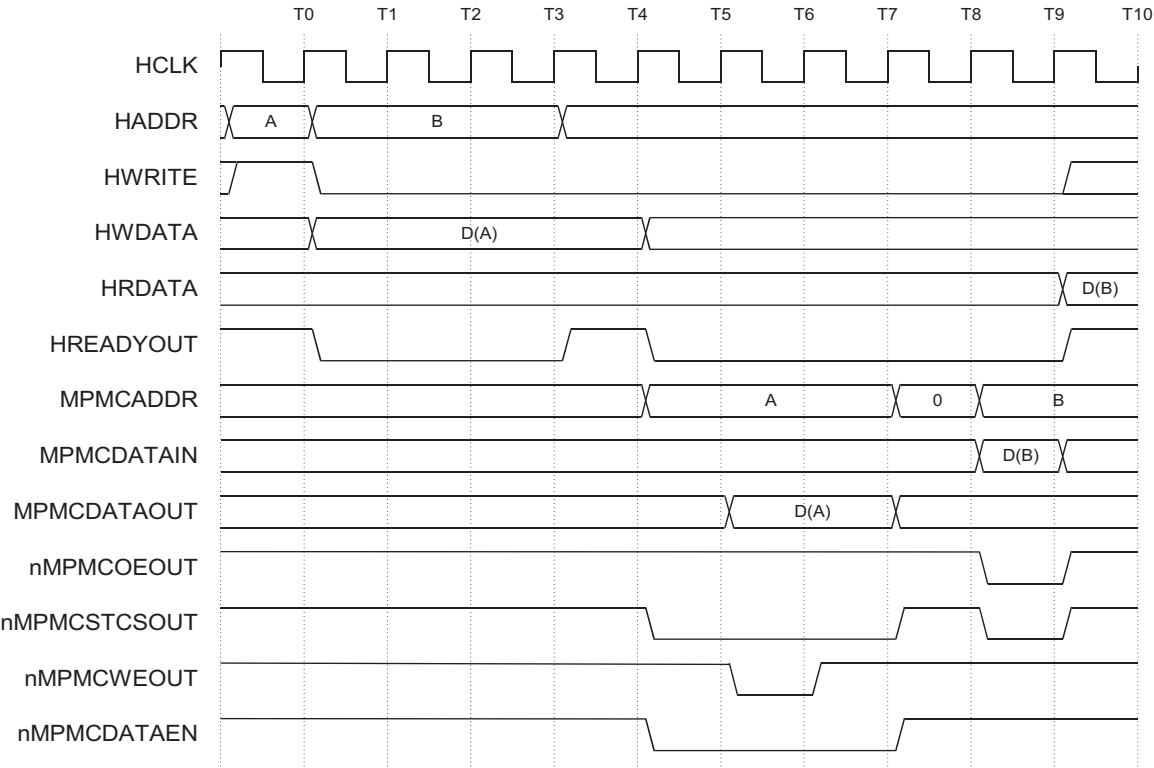


Figure 3-14 Write followed by read (both zero wait) with no turnaround

Table 3-29 describes the transactions for Figure 3-14.

Table 3-29 Write followed by read (both zero wait) with no turnaround

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.

Table 3-29 Write followed by read (both zero wait) with no turnaround (continued)

Cycle	Description
T4-T5	Static memory address, chip select, and control signals submitted to static memory. Write data is read from AHB memory port. Write enable inactive. AHB read address provided to memory controller
T5-T6	Write enable taken active. Write data submitted to static memory.
T6-T7	Static memory writes the data. Write enable taken inactive.
T7-T8	Static memory control signals taken inactive.
T8-T9	Static memory address, chip select, and control signals submitted to static memory.
T9-T10	Read data returned from the static memory. Data is provided to the AHB.

Figure 3-15 on page 3-36 shows a zero wait read followed by a zero wait write with two turnaround cycles added. The standard minimum of three AHB wait states are added to the read transfer, and two wait states are added to the write (as for any read-write transfer sequence). See Table 3-30 for the programmed values.

Table 3-30 Static memory timing parameters

Timing parameter	Value
WAITRD	0
WAITOEN	0
WAITPAGE	NA
WAITWR	0
WAITWEN	0
WAITTURN	2

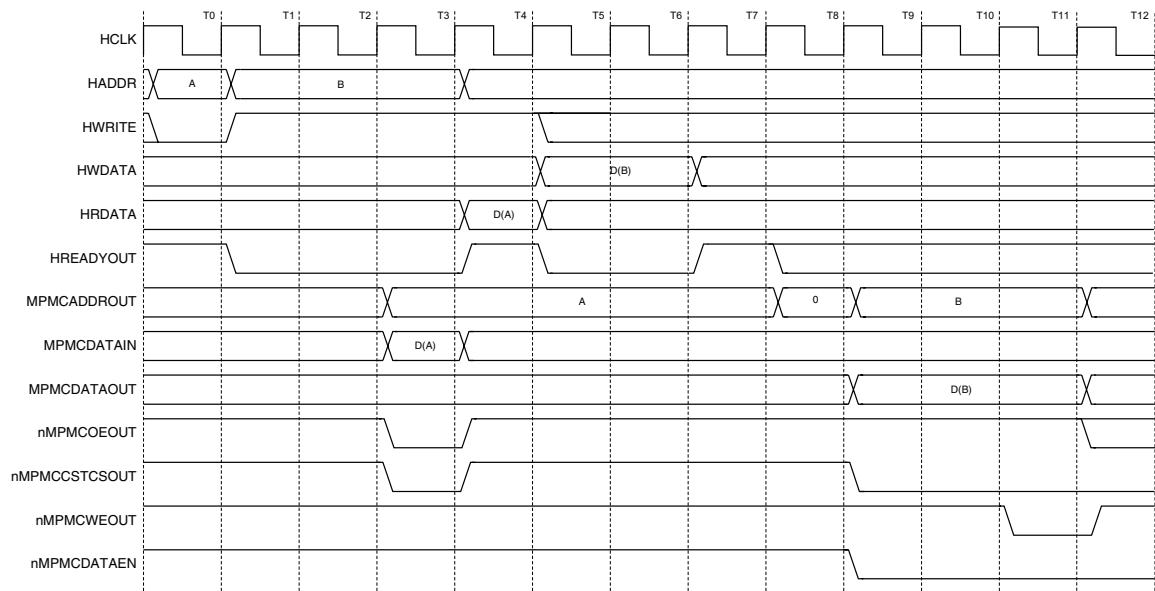


Figure 3-15 Read followed by a write (all zero wait state) with two turnaround cycles

Table 3-31 describes the transactions for Figure 3-15.

Table 3-31 Read followed by a write (all zero wait state) with two turnaround cycles

Cycle	Description
T0	AHB address provided to memory controller
T0-T1	AHB transaction processing
T1-T2	Arbitration of AHB memory ports
T2-T3	Raise EBI bus request.
T3-T4	Register EBI grant.
T4-T5	Static memory address, chip select, and control signals submitted to static memory.
T5-T6	Read data returned from the static memory. Data is provided to the AHB. AHB write address provided to memory controller.
T6-T7	Turn around cycle 1.

Table 3-31 Read followed by a write (all zero wait state) with two turnaround cycles

Cycle	Description
T7-T8	Turn around cycle 2.
T8-T9	Static memory transfer address, chip select, and control signals submitted to static memory. Write enable inactive.
T9-T10	Memory controller processing.
T10-T11	Write enable taken active. Write data submitted to static memory.
T11-T12	Static memory write the data. Write enable taken inactive.

3.6 Elimination of floating bytes on the external interface

The MPMC uses the programmed external memory width of each bank and the endianness to determine which remaining bytes it requires to drive to ensure that the external bus is never floating. The input/output pads for the external write data bus lines are controlled by **nMPMCDATAOUTEN[3:0]**, see Table 3-32. Data is driven out on **MPMCDATAOUT[31:0]** when **nMPMCDATAOUTEN** is asserted LOW.

Table 3-32 MPMCDATAOUT[31:0] controlled by nMPMCDATAOUTEN[3:0]

nMPMCDATAOUTEN	MPMCDATAOUT
[3]	[31:24]
[2]	[23:16]
[1]	[15:8]
[0]	[7:0]

3.7 Byte lane control

The MPMC generates byte lane control signals **nMPMCBLSOUT[3:0]** according to:

- little or big-endian operation
- AMBA transfer width, indicated by **HSIZE[2:0]**
- external memory bank data bus width, defined in each control register
- the decoded **HADDR[1:0]** value for write accesses only.

Word transfers are the largest size transfers supported by the MPMC. Any access attempted with a size greater than a word causes an ERROR response to be generated. Each memory chip select can be 8, 16, or 32 bits wide. The type of memory used determines how the **nMPMCWEOUT** and **nMPMCBLSOUT** signals are connected to provide byte, halfword and word access. For read accesses, control the **nMPMCBLSOUT** signals by driving them either all HIGH, or all LOW. This is done by programming the Byte Lane State (PB) bit in the MPMCStaticConfig[n] Register. *Memory banks constructed from 8-bit or non byte-partitioned memory devices* on page 3-40 and *Memory banks constructed from 16 or 32-bit memory devices* on page 3-40 explain why different connections are required, in respect of **nMPMCWEOUT** and **nMPMCBLSOUT[3:0]**, for different memory configurations.

3.8 Address connectivity

The static memory address output signal **MPMCADDROUT[27:0]** can be right justified.

3.8.1 Memory banks constructed from 8-bit or non byte-partitioned memory devices

For memory banks constructed from 8-bit or non byte-partitioned memory devices, it is important that the Byte Lane State (PB) bit is cleared to 0 in the respective memory bank control register. This forces all **nMPMCBLSOUT[3:0]** lines HIGH during a read access because the byte lane selects are connected to the device write enables.

Figure 3-16 on page 3-41 shows 8-bit memory being used to configure memory banks that are 8, 16, and 32 bits wide. In each of these configurations, the **nMPMCBLSOUT[3:0]** signals are connected to write enable (**nWE**) inputs of each 8-bit memory. The **nMPMCWEOUT** signal from the MPMC is not used. For write transfers, the relevant **nMPMCBLSOUT[3:0]** byte lane signals are asserted LOW, and steer the data to the addressed bytes. For read transfers, all the **nMPMCBLSOUT[3:0]** lines are deasserted HIGH, enabling the external bus to be defined for at least the width of the accessed memory.

3.8.2 Memory banks constructed from 16 or 32-bit memory devices

For memory banks constructed from 16 or 32-bit memory devices, it is important that the Byte Lane Select (PB) bit is set to 1 in the respective memory bank control register. This asserts all **nMPMCBLSOUT[3:0]** lines LOW during a read access because, during a read, all bytes of the devices must be selected to avoid undriven byte lanes on the read data value. In the case of 16 and 32-bit wide memory devices, byte select signals exist and these must be appropriately controlled. See Figure 3-17 on page 3-41 and Figure 3-18 on page 3-42.

Figure 3-19 on page 3-43 shows connections for a typical memory system with different data width memory devices.

3.8.3 Address connectivity, address right justified

Figure 3-16 on page 3-41 to Figure 3-19 on page 3-43 show how to connect the memory controller to static memory where the address is right justified. Right justification reduces the number of address pins and is selected by setting the **MPMCREL1CONFIG** signal LOW.

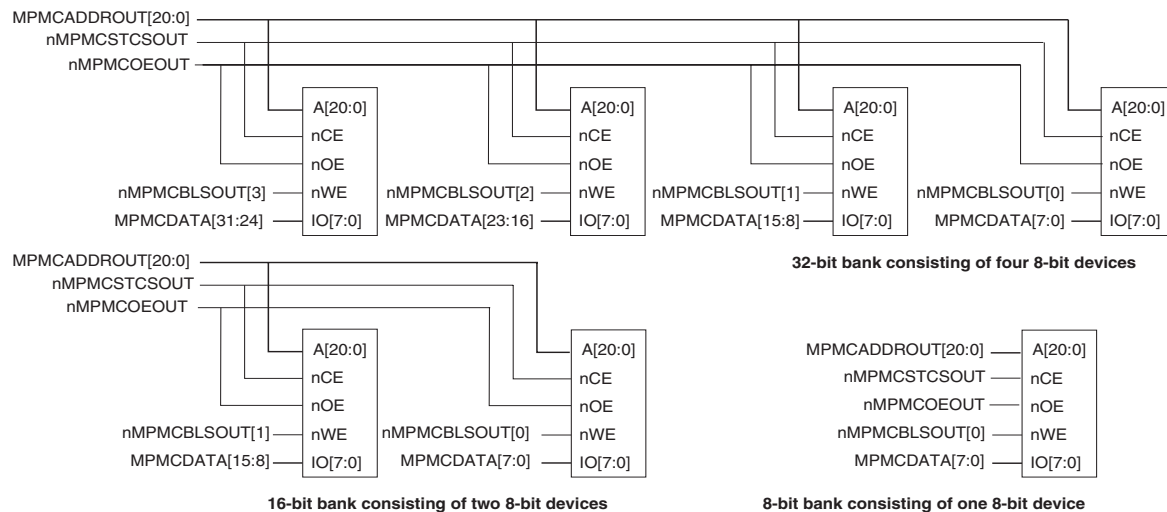


Figure 3-16 Memory banks constructed from 8-bit memory

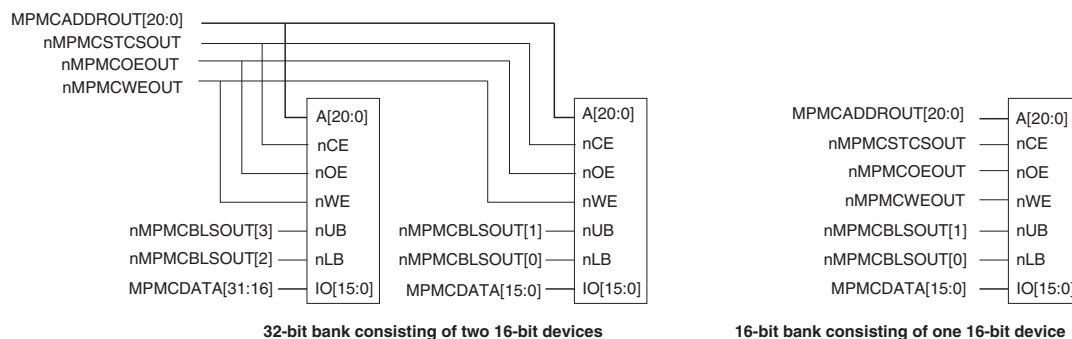
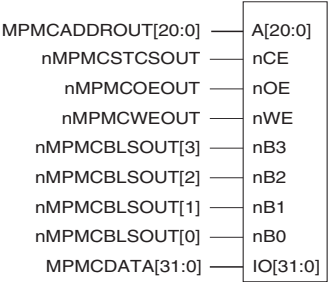


Figure 3-17 Memory banks constructed from 16-bit memory



32-bit bank consisting of one 32-bit device

Figure 3-18 Memory bank constructed from 32-bit memory

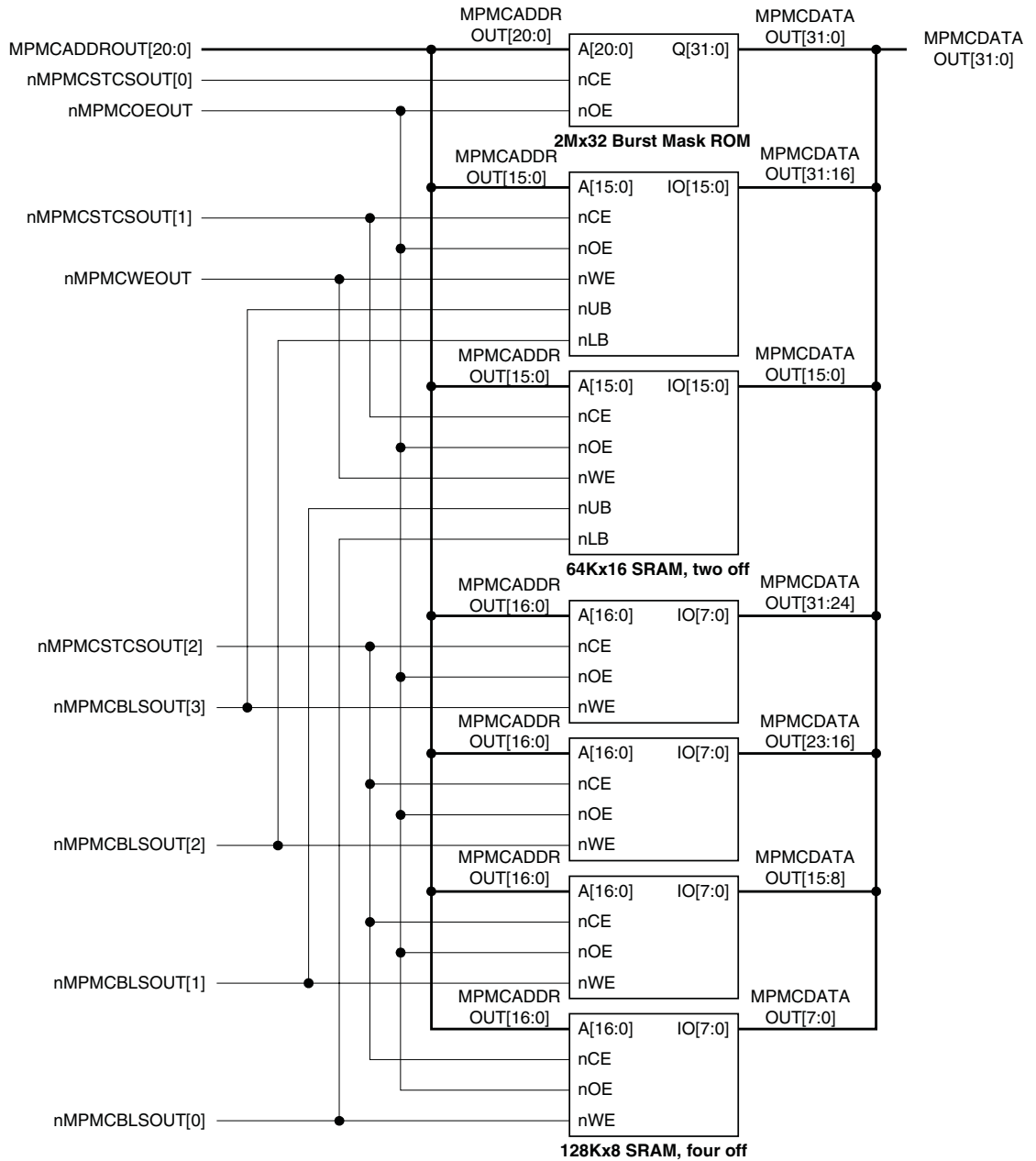


Figure 3-19 Typical memory connection diagram

3.8.4 Address connectivity, address not right justified

Figure 3-21 to Figure 3-23 on page 3-46 show how to connect the memory controller to static memory where the address is not right justified. Not right justifying the address might increase the number of pins and is selected by setting the **MPMCREL1CONFIG** signal HIGH.

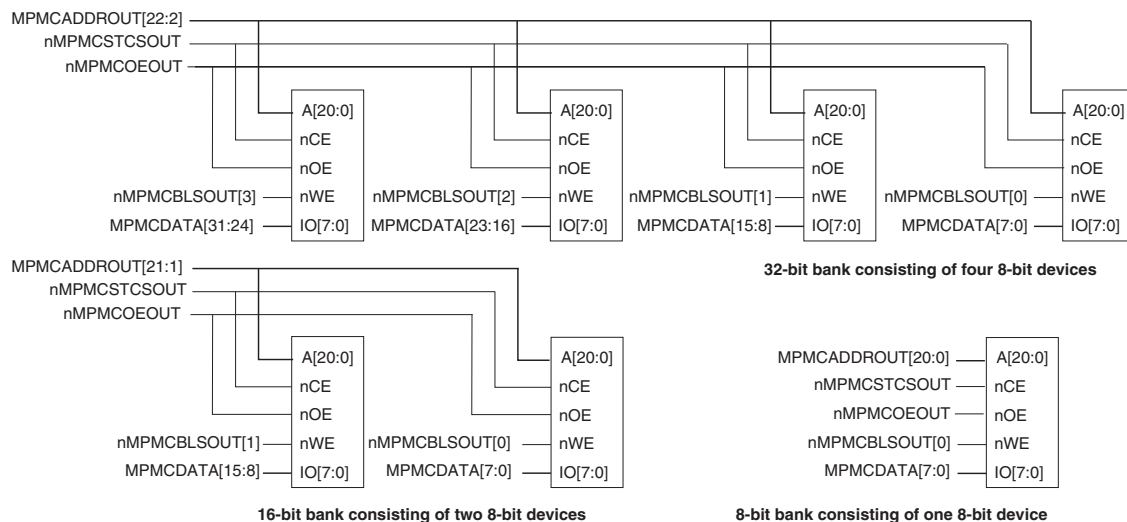


Figure 3-20 Memory banks constructed from 8-bit memory

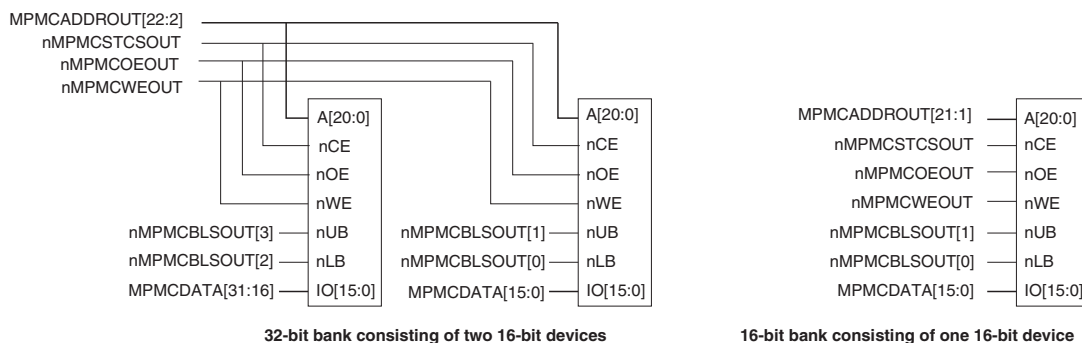
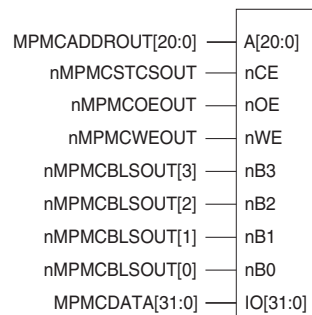


Figure 3-21 Memory banks constructed from 16-bit memory



32-bit bank consisting of one 32-bit device

Figure 3-22 Memory bank constructed from 32-bit memory

Figure 3-19 on page 3-43 shows connections for a typical memory system with different data width memory devices.

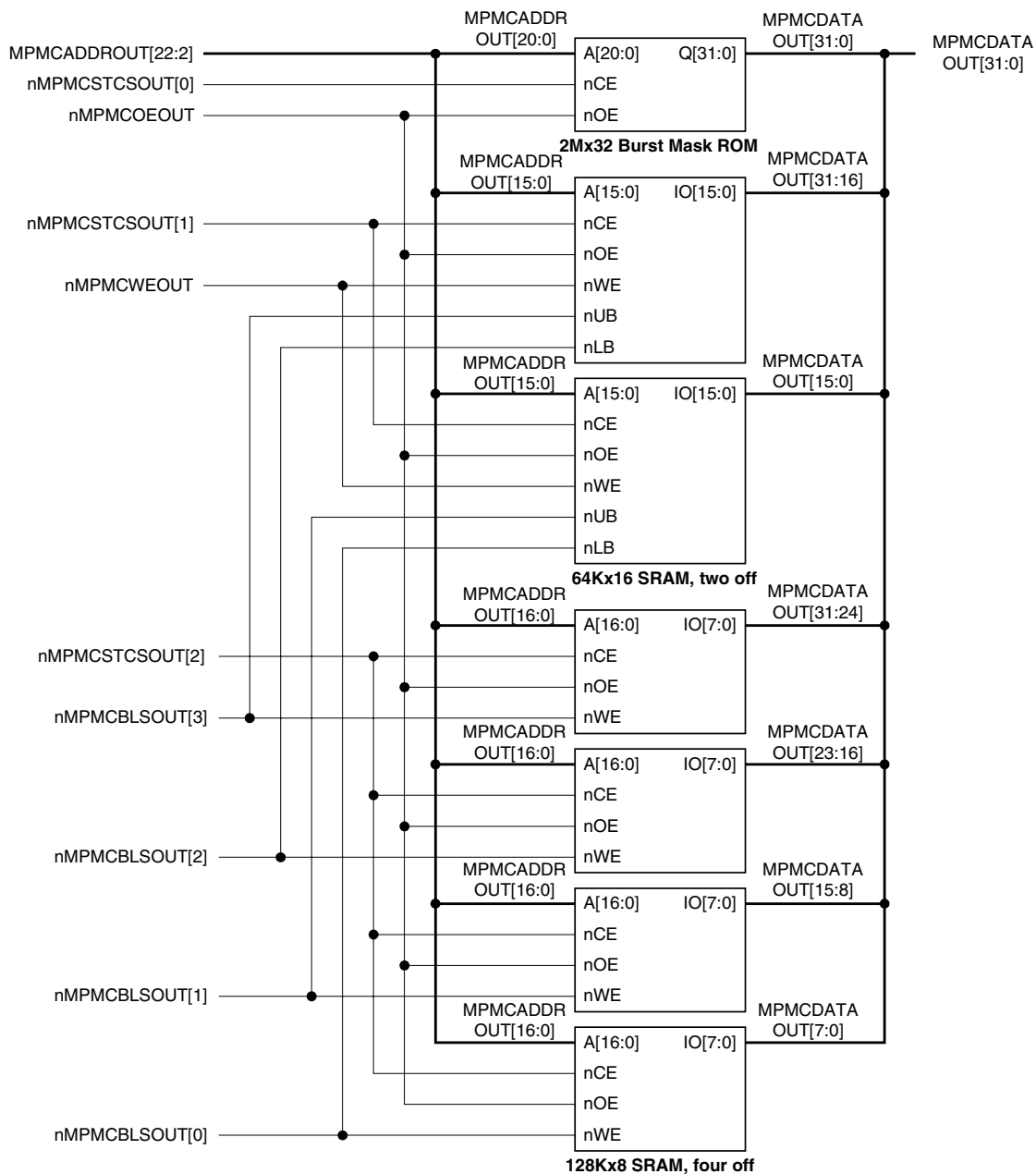


Figure 3-23 Typical memory connection diagram

3.9 Byte lane control and data bus steering

This section describes the byte lane control and data bus steering for little and big-endian configurations. These configurations are described in more detail in the following subsections:

- *Address right justified*
- *Address not right justified* on page 3-53.

3.9.1 Address right justified

Table 3-33 to Table 3-44 on page 3-53 show the relationship of signals **HSIZE[2:0]**, **HADDR[1:0]**, **MPMCADDR[1:0]**, and **nMPMCBLSOUT[3:0]** and mapping of data between the AHB system data bus and the external memory data bus. This mapping applies to both the static and dynamic memory controllers.

Table 3-33 Little-endian read, 8-bit external bus

Internal transfer width	Access: Read, little-endian, 8-bit external bus				External data mapping on to system data bus (HRDATA to MPMCDATA)			
	HSIZE [2:0]	HADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	11	0	[7:0]	-	-	-
			10	0	-	[7:0]	-	-
			01	0	-	-	[7:0]	-
			00	0	-	-	-	[7:0]
Halfword (2 transfers)	001	1-	11	0	[7:0]	-	-	-
			10	0	-	[7:0]	-	-
Halfword (2 transfers)	001	0-	01	0	-	-	[7:0]	-
			00	0	-	-	-	[7:0]
Byte	000	11	11	0	[7:0]	-	-	-
Byte	000	10	10	0	-	[7:0]	-	-
Byte	000	01	01	0	-	-	[7:0]	-
Byte	000	00	00	0	-	-	-	[7:0]

Table 3-34 Little-endian read, 16-bit external bus

Internal transfer width	Access: Read, little-endian, 16-bit external bus				External data mapping on to system data bus (HRDATA to MPMC DATA)			
	HSIZE [2:0]	HADDR [1:0]	MPMCADDR OUT[0]	nMPMCBLS OUT[1:0]	[31:4]	[23:16]	[15:8]	[7:0]
Word (2 transfers)	010	--	1 0	00 00	[15:8] -	[7:0] -	- [15:8]	- [7:0]
Halfword	001	1-	1	00	[15:8]	[7:0]	-	-
Halfword	001	0-	0	00	-	-	[15:8]	[7:0]
Byte	000	11	1	01	[15:8]	-	-	-
Byte	000	10	1	10	-	[7:0]	-	-
Byte	000	01	0	01	-	-	[15:8]	-
Byte	000	00	0	10	-	-	-	[7:0]

Table 3-35 Little-endian read, 32-bit external bus

Internal transfer width	Access: Read, little-endian, 32-bit external bus			External data mapping on to system data bus (HRDATA to MPMC DATA)			
	HSIZE [2:0]	HADDR [1:0]	nMPMCBLS OUT[3:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	0000	[31:24]	[23:16]	[15:8]	[7:0]
Halfword (2 transfers)	001	1-	0011	[31:24]	[23:16]	-	-
Halfword (2 transfers)	001	0-	1100	-	-	[15:8]	[7:0]
Byte	000	11	0111	[31:24]	-	-	-
Byte	000	10	1011	-	[23:16]	-	-
Byte	000	01	1101	-	-	[15:8]	-
Byte	000	00	1110	-	-	-	[7:0]

Table 3-36 Little-endian write, 8-bit external bus

Internal transfer width	Access: Write, little-endian, 8-bit external bus				System data mapping on to external data bus (MPMCDATA to HRDATA)			
	HSIZE [2:0]	HADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	11	0	-	-	-	[31:24]
			10	0	-	-	-	[23:16]
			01	0	-	-	-	[15:8]
			00	0	-	-	-	[7:0]
Halfword (2 transfers)	001	1-	11	0	-	-	-	[31:24]
			10	0	-	-	-	[23:16]
Halfword (2 transfers)	001	0-	01	0	-	-	-	[15:8]
			00	0	-	-	-	[7:0]
Byte	000	11	11	0	-	-	-	[31:24]
Byte	000	10	10	0	-	-	-	[23:16]
Byte	000	01	01	0	-	-	-	[15:8]
Byte	000	00	00	0	-	-	-	[7:0]

Table 3-37 Little-endian write, 16-bit external bus

Internal transfer width	Access: Write, little-endian, 16-bit external bus				System data mapping on to external data bus (MPMCDATA to HRDATA)			
	HSIZE [2:0]	HADDR [1:0]	MPMCADDR OUT[0]	nMPMCBLS OUT[1:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (2 transfers)	010	--	1	00	-	-	[31:24]	[23:16]
			0	00	-	-	[15:8]	[7:0]
Halfword	001	1-	1	00	-	-	[31:24]	[23:16]
Halfword	001	0-	0	00	-	-	[15:8]	[7:0]
Byte	000	11	1	01	-	-	[31:24]	-
Byte	000	10	1	10	-	-	-	[23:16]
Byte	000	01	0	01	-	-	[15:8]	-
Byte	000	00	0	10	-	-	-	[7:0]

Table 3-38 Little-endian write, 32-bit external bus

Internal transfer width	Access: Write, little-endian, 32-bit external bus			System data mapping on to external data bus (MPMCDATA to HRDATA)			
	HSIZE [2:0]	HADDR [1:0]	nMPMCBLS OUT[3:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word	010	--	0000	[31:24]	[23:16]	[15:8]	[7:0]
Halfword	001	1-	0011	[31:24]	[23:16]	-	-
Halfword	001	0-	1100	-	-	[15:8]	[7:0]
Byte	000	11	0111	[31:24]	-	-	-
Byte	000	10	1011	-	[23:16]	-	-
Byte	000	01	1101	-	-	[15:8]	-
Byte	000	00	1110	-	-	-	[7:0]

Table 3-39 Big-endian read, 8-bit external bus

Internal transfer width	Access: Read, big-endian, 8-bit external bus				External data mapping on to system data bus (HRDATA to MPMCDATA)			
	HSIZE [2:0]	HADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	11	0	-	-	-	[7:0]-
			10	0	-	-	[7:0]	-
			01	0	-	[7:0]	-	-
			00	0	[7:0]	-	-	-
Halfword (2 transfers)	001	1-	11	0	-	-	-	[7:0]-
			10	0	-	-	[7:0]	-
Halfword (2 transfers)	001	0-	01	0	-	[7:0]	-	-
			00	0	[7:0]	-	-	-
Byte	000	11	11	0	-	-	-	[7:0]
Byte	000	10	10	0	-	-	[7:0]	-
Byte	000	01	01	0	-	[7:0]	-	-
Byte	000	00	00	0	[7:0]	-	-	-

Table 3-40 Big-endian read, 16-bit external bus

Internal transfer width	Access: Read, Big-endian, 16-bit external bus				External data mapping on to system data bus (HRDATA to MPMCDATA)			
	HSIZE [2:0]	HADDR [1:0]	MPMCADDR OUT[0]	nMPMCBLS OUT[1:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (2 transfers)	010	--	1 0	00 00	- [15:8]	- [7:0]	[15:8] -	[7:0] -
Halfword	001	1-	1	00	-	-	[15:8]	[7:0]
Halfword	001	0-	0	00	[15:8]	[7:0]	-	-
Byte	000	11	1	10	-	-	-	[7:0]
Byte	000	10	1	01	-	-	[15:8]	-
Byte	000	01	0	10	-	[7:0]	-	-
Byte	000	00	0	01	[15:8]	-	-	-

Table 3-41 Big-endian read, 32-bit external bus

Internal transfer width	Access: Read, big-endian, 32-bit external bus			External data mapping on to system data bus (HRDATA to MPMCDATA)			
	HSIZE [2:0]	HADDR [1:0]	nMPMCBLS OUT[3:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	0000	[31:24]	[23:16]	[15:8]	[7:0]
Halfword (2 transfers)	001	1-	1100	-	-	[15:8]	[7:0]
Halfword (2 transfers)	001	0-	0011	[31:24]	[23:16]	-	-
Byte	000	11	1110	-	-	-	[7:0]
Byte	000	10	1101	-	-	[15:8]	-
Byte	000	01	1011	-	[23:16]	-	-
Byte	000	00	0111	[31:24]	-	-	-

Table 3-42 Big-endian write, 8-bit external bus

Internal transfer width	Access: Write, big-endian, 8-bit external bus				System data mapping on to external data bus (MPMCDATA to HRDATA)			
	H SIZE [2:0]	H ADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	11	0	-	-	-	[7:0]
			10	0	-	-	-	[15:8]
			01	0	-	-	-	[23:16]
			00	0	-	-	-	[31:24]
Halfword (2 transfers)	001	1-	11	0	-	-	-	[7:0]
			10	0	-	-	-	[15:8]
Halfword (2 transfers)	001	0-	01	0	-	-	-	[23:16]
			00	0	-	-	-	[31:24]
Byte	000	11	11	0	-	-	-	[7:0]
Byte	000	10	10	0	-	-	-	[15:8]
Byte	000	01	01	0	-	-	-	[23:16]
Byte	000	00	00	0	-	-	-	[31:24]

Table 3-43 Big-endian write, 16-bit external bus

Internal transfer width	Access: Write, big-endian, 16-bit external bus				System data mapping on to external data bus (MPMCDATA to HRDATA)			
	H SIZE [2:0]	H ADDR [1:0]	MPMCADDR OUT[0]	nMPMCBLS OUT[1:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (2 transfers)	010	--	1	00	-	-	[15:8]	[7:0]
			0	00	-	-	[31:24]	[23:16]
Halfword	001	1-	1	00	-	-	[15:8]	[7:0]
Halfword	001	0-	0	00	-	-	[31:24]	[23:16]
Byte	000	11	1	10	-	-	-	[7:0]
Byte	000	10	1	01	-	-	[15:8]	-
Byte	000	01	0	10	-	-	-	[23:16]
Byte	000	00	0	01	-	-	[31:24]	-

Table 3-44 Big-endian write, 32-bit external bus

Internal transfer width	Access: Write, big-endian, 32-bit external bus			System data mapping on to external data bus (MPMCDATA to HRDATA)			
	HSIZE [2:0]	HADDR [1:0]	nMPMCBLS OUT[3:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word	010	--	0000	[31:24]	[23:16]	[15:8]	[7:0]
Halfword	001	1-	1100	-	-	[15:8]	[7:0]
Halfword	001	0-	0011	[31:24]	[23:16]	-	-
Byte	000	11	1110	-	-	-	[7:0]
Byte	000	10	1101	-	-	[15:8]	-
Byte	000	01	1011	-	[23:16]	-	-
Byte	000	00	0111	[31:24]	-	-	-

3.9.2 Address not right justified

Table 3-45 to Table 3-56 on page 3-60 show the relationship of signals **HSIZE[2:0]**, **HADDR[1:0]**, **MPMCADDR[1:0]**, and **nMPMCBLSOUT[3:0]** and mapping of data between the AHB system data bus and the external memory data bus. This mapping applies to both the static and dynamic memory controllers.

Table 3-45 Little-endian read, 8-bit external bus

Internal transfer width	Access: Read, little-endian, 8-bit external bus				External data mapping on to system data bus (HRDATA to MPMCDATA)			
	HSIZE [2:0]	HADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	11	0	[7:0]	-	-	-
			10	0	-	[7:0]	-	-
			01	0	-	-	[7:0]	-
			00	0	-	-	-	[7:0]
Halfword (2 transfers)	001	1-	11	0	[7:0]	-	-	-
			10	0	-	[7:0]	-	-
Halfword (2 transfers)	001	0-	01	0	-	-	[7:0]	-
			00	0	-	-	-	[7:0]

Table 3-45 Little-endian read, 8-bit external bus (continued)

Internal transfer width	Access: Read, little-endian, 8-bit external bus				External data mapping on to system data bus (HRDATA to MPMCDATA)			
	HSIZE [2:0]	HADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[0]	[31:24]	[23:16]	[15:8]	[7:0]
Byte	000	11	11	0	[7:0]	-	-	-
Byte	000	10	10	0	-	[7:0]	-	-
Byte	000	01	01	0	-	-	[7:0]	-
Byte	000	00	00	0	-	-	-	[7:0]

Table 3-46 Little-endian read, 16-bit external bus

Internal transfer width	Access: Read, little-endian, 16-bit external bus				External data mapping on to system data bus (HRDATA to MPMCDATA)			
	H SIZE [2:0]	H ADDR [2:1]	MPMC ADDR OUT[1:0]	nMPMC BLS OUT[1:0]	[31:4]	[23:16]	[15:8]	[7:0]
Word (2 transfers)	010	--	1- 0-	00 00	[15:8] - -	[7:0] -	- [15:8]	- [7:0]
Halfword	001	1-	1-	00	[15:8]	[7:0]	-	-
Halfword	001	0-	0-	00	-	-	[15:8]	[7:0]
Byte	000	11	1-	01	[15:8]	-	-	-
Byte	000	10	1-	10	-	[7:0]	-	-
Byte	000	01	0-	01	-	-	[15:8]	-
Byte	000	00	0-	10	-	-	-	[7:0]

Table 3-47 Little-endian read, 32-bit external bus

Internal transfer width	Access: Read, little-endian, 32-bit external bus				External data mapping on to system data bus (HRDATA to MPMCDATA)			
	H SIZE [2:0]	H ADDR [1:0]	MPMC ADDR ROUT[1:0]	nMPMC BLS OUT[3:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	--	0000	[31:24]	[23:16]	[15:8]	[7:0]
Halfword (2 transfers)	001	1-	--	0011	[31:24]	[23:16]	-	-
Halfword (2 transfers)	001	0-	--	1100	-	-	[15:8]	[7:0]
Byte	000	11	--	0111	[31:24]	-	-	-
Byte	000	10	--	1011	-	[23:16]	-	-
Byte	000	01	--	1101	-	-	[15:8]	-
Byte	000	00	--	1110	-	-	-	[7:0]

Table 3-48 Little-endian write, 8-bit external bus

Internal transfer width	Access: Write, little-endian, 8-bit external bus				System data mapping on to external data bus (MPMCDATA to HRDATA)			
	H SIZE [2:0]	H ADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	11	0	-	-	-	[31:24]
			10	0	-	-	-	[23:16]
			01	0	-	-	-	[15:8]
			00	0	-	-	-	[7:0]
Halfword (2 transfers)	001	1-	11	0	-	-	-	[31:24]
			10	0	-	-	-	[23:16]
Halfword (2 transfers)	001	0-	01	0	-	-	-	[15:8]
			00	0	-	-	-	[7:0]
Byte	000	11	11	0	-	-	-	[31:24]
Byte	000	10	10	0	-	-	-	[23:16]
Byte	000	01	01	0	-	-	-	[15:8]
Byte	000	00	00	0	-	-	-	[7:0]

Table 3-49 Little-endian write, 16-bit external bus

Internal transfer width	Access: Write, little-endian, 16-bit external bus				System data mapping on to external data bus (MPMCDATA to HRDATA)			
	H SIZE [2:0]	H ADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[1:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (2 transfers)	010	--	1-	00	-	-	[31:24]	[23:16]
			0-	00	-	-	[15:8]	[7:0]
Halfword	001	1-	1-	00	-	-	[31:24]	[23:16]
Halfword	001	0-	0-	00	-	-	[15:8]	[7:0]
Byte	000	11	1-	01	-	-	[31:24]	-
Byte	000	10	1-	10	-	-	-	[23:16]
Byte	000	01	0-	01	-	-	[15:8]	-
Byte	000	00	0-	10	-	-	-	[7:0]

Table 3-50 Little-endian write, 32-bit external bus

Internal transfer width	Access: Write, little-endian, 32-bit external bus				System data mapping on to external data bus (MPMCDATA to HRDATA)			
	H SIZE [2:0]	H ADDR [1:0]	MPMCADDR [1:0]	nMPMCBLS OUT[3:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word	010	--	--	0000	[31:24]	[23:16]	[15:8]	[7:0]
Halfword	001	1-	--	0011	[31:24]	[23:16]	-	-
Halfword	001	0-	--	1100	-	-	[15:8]	[7:0]
Byte	000	11	--	0111	[31:24]	-	-	-
Byte	000	10	--	1011	-	[23:16]	-	-
Byte	000	01	--	1101	-	-	[15:8]	-
Byte	000	00	--	1110	-	-	-	[7:0]

Table 3-51 Big-endian read, 8-bit external bus

Internal transfer width	Access: Read, big-endian, 8-bit external bus				External data mapping on to system data bus (HRDATA to MPMCDATA)			
	H SIZE [2:0]	H ADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	11	0	-	-	-	[7:0]-
			10	0	-	-	[7:0]	-
			01	0	-	[7:0]	-	-
			00	0	[7:0]	-	-	-
Halfword (2 transfers)	001	1-	11	0	-	-	-	[7:0]-
			10	0	-	-	[7:0]	-
Halfword (2 transfers)	001	0-	01	0	-	[7:0]	-	-
			00	0	[7:0]	-	-	-
Byte	000	11	11	0	-	-	-	[7:0]
Byte	000	10	10	0	-	-	[7:0]	-
Byte	000	01	01	0	-	[7:0]	-	-
Byte	000	00	00	0	[7:0]	-	-	-

Table 3-52 Big-endian read, 16-bit external bus

Internal transfer width	Access: Read, Big-endian, 16-bit external bus				External data mapping on to system data bus (HRDATA to MPMCDATA)			
	H SIZE [2:0]	H ADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[1:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (2 transfers)	010	--	1-0-	0000	- [15:8]	- [7:0]	[15:8] -	[7:0] -
Halfword	001	1-	1-	00	-	-	[15:8]	[7:0]
Halfword	001	0-	0-	00	[15:8]	[7:0]	-	-
Byte	000	11	1-	10	-	-	-	[7:0]
Byte	000	10	1-	01	-	-	[15:8]	-
Byte	000	01	0-	10	-	[7:0]	-	-
Byte	000	00	0-	01	[15:8]	-	-	-

Table 3-53 Big-endian read, 32-bit external bus

Internal transfer width	Access: Read, big-endian, 32-bit external bus				External data mapping on to system data bus (HRDATA to MPMCDATA)			
	H SIZE [2:0]	H ADDR [1:0]	MPMCADDR [1:0]	nMPMCBLS OUT[3:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	--	0000	[31:24]	[23:16]	[15:8]	[7:0]
Halfword (2 transfers)	001	1-	--	1100	-	-	[15:8]	[7:0]
Halfword (2 transfers)	001	0-	--	0011	[31:24]	[23:16]	-	-
Byte	000	11	--	1110	-	-	-	[7:0]
Byte	000	10	--	1101	-	-	[15:8]	-
Byte	000	01	--	1011	-	[23:16]	-	-
Byte	000	00	--	0111	[31:24]	-	-	-

Table 3-54 Big-endian write, 8-bit external bus

Internal transfer width	Access: Write, big-endian, 8-bit external bus				System data mapping on to external data bus (MPMCDATA to HRDATA)			
	H SIZE [2:0]	H ADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (4 transfers)	010	--	11	0	-	-	-	[7:0]
			10	0	-	-	-	[15:8]
			01	0	-	-	-	[23:16]
			00	0	-	-	-	[31:24]
Halfword (2 transfers)	001	1-	11	0	-	-	-	[7:0]
			10	0	-	-	-	[15:8]
Halfword (2 transfers)	001	0-	01	0	-	-	-	[23:16]
			00	0	-	-	-	[31:24]
Byte	000	11	11	0	-	-	-	[7:0]
Byte	000	10	10	0	-	-	-	[15:8]
Byte	000	01	01	0	-	-	-	[23:16]
Byte	000	00	00	0	-	-	-	[31:24]

Table 3-55 Big-endian write, 16-bit external bus

Internal transfer width	Access: Write, big-endian, 16-bit external bus				System data mapping on to external data bus (MPMCDATA to HRDATA)			
	H SIZE [2:0]	H ADDR [1:0]	MPMCADDR OUT[1:0]	nMPMCBLS OUT[1:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word (2 transfers)	010	--	1-	00	-	-	[15:8]	[7:0]
			0-	00	-	-	[31:24]	[23:16]
Halfword	001	1-	1-	00	-	-	[15:8]	[7:0]
Halfword	001	0-	0-	00	-	-	[31:24]	[23:16]
Byte	000	11	1-	10	-	-	-	[7:0]
Byte	000	10	1-	01	-	-	[15:8]	-
Byte	000	01	0-	10	-	-	-	[23:16]
Byte	000	00	0-	01	-	-	[31:24]	-

Table 3-56 Big-endian write, 32-bit external bus

Internal transfer width	Access: Write, big-endian, 32-bit external bus			System data mapping on to external data bus (MPMCDATA to HRDATA)			
	HSIZE [2:0]	HADDR [1:0]	nMPMCBLS OUT[3:0]	[31:24]	[23:16]	[15:8]	[7:0]
Word	010	--	0000	[31:24]	[23:16]	[15:8]	[7:0]
Halfword	001	1-	1100	-	-	[15:8]	[7:0]
Halfword	001	0-	0011	[31:24]	[23:16]	-	-
Byte	000	11	1110	-	-	-	[7:0]
Byte	000	10	1101	-	-	[15:8]	-
Byte	000	01	1011	-	[23:16]	-	-
Byte	000	00	0111	[31:24]	-	-	-

Chapter 4

Dynamic Memory Controller

This chapter describes the dynamic memory controller. It contains the following sections:

- *Write-protection* on page 4-2
- *Access sequencing and memory width* on page 4-3
- *Dynamic memory transaction latency* on page 4-4
- *Transaction examples* on page 4-37
- *Address mapping* on page 4-75
- *Dynamic memory controller command descriptions* on page 4-80
- *Generic SDRAM initialization example* on page 4-81
- *Micron MT48LC4M16A2 SDRAM initialization example* on page 4-83
- *Low-power SDRAM initialization example* on page 4-86
- *Micron MT28F4M16S2 SyncFlash initialization example* on page 4-90
- *Micron SyncFlash commands* on page 4-93.

4.1 Write-protection

Each dynamic memory chip select can be configured for write-protection by setting the relevant bit in the write-protect (P) field in the MPMCDynamicConfig Register. If a write access is performed to a write-protected memory bank, an ERROR response is generated on the **HRESP[1:0]** signal.

4.2 Access sequencing and memory width

The data width of each external chip select must be configured by programming the appropriate MPMCDynamicConfig[n] Register. When the chip select data bus width is narrower than the transfer initiated from the current AMBA bus master, the internal bus transfer takes several external bus transfers to complete. For example, if chip select 4 is configured as 16-bit wide memory and a 32-bit read is initiated, the AHB bus stalls while the MPMC reads two consecutive words from the memory. During these accesses the memory controller block demultiplexes the two 16-bit words into one 32-bit word and places the result onto the AHB bus.

Word transfers are the widest transfers supported by the MPMC. Any access attempted with a size larger than a word generates an error response.

4.3 Dynamic memory transaction latency

Table 4-1 on page 4-9 and Table 4-2 on page 4-23 show the MPMC transfer latencies. The values in the tables assume CAS latency 2 SDRAM, where CAS, RAS, and Precharge (t_{RP}) are two clock cycles. The AHB latency heading shows the latency seen by the AHB interface, for the case where the request is dealt with immediately. Memory latency indicates the amount of time it takes for the AHB request to go active until the memory transfer has completed at the SDRAM.

4.3.1 Transaction latency formulae

This section provides transfer latency formulas for memory transfers assuming a 32-bit wide dynamic memory chip select, and a 1:1 **HCLK** to **MPMCCLK** clock ratio.

The latency formulas use the following parameters:

PrechargeLatency

SDRAM memory device precharge latency. This parameter is required if the read transaction is to a row that is different to the one already open.

ActivateLatency

SDRAM memory devices activate latency (RAS to CAS latency). This parameter is required if the read transaction is to a precharged bank.

CASLatency

SDRAM memory device CAS latency. This parameter is required for all read transactions.

BurstLength

AHB burst length. This parameter is not required for write transactions.

AlignedIncr

Aligned incrementing AHB burst. If the AHB burst is an aligned INCR, INCR4, INCR8, or INCR16 burst, this is true (1). Otherwise this is false (0).

Arbitration

Arbitration delay. For unaligned INCR4, INCR8, INCR16, WRAP4, WRAP8, and WRAP16 AHB bursts this parameter is false (0).

n

Number of bursts submitted to the memory controller.

Read transfer latency

This section provides the dynamic memory read transfer latencies.

Single read transfer latency formula

Single read transfer latency formula:

$$\text{ReadLatency} = 7 + \text{PrechargeLatency} + \text{ActivateLatency} + \text{CASLatency} - (2 \times \text{AlignedIncr}) + \text{BurstLength}$$

Note

- PrechargeLatency is only required for the case where a different row is open.
- ActivateLatency is only required for the case where a different row is open, or if the bank is precharged.

Back-to-back read transfer on same AHB port transfer latency formula

Back-to-back read transfer latency formula:

$$\text{ReadLatency} = n(7 + \text{PrechargeLatency} + \text{ActivateLatency} + \text{CASLatency} - \{2 \times \text{AlignedIncr} + \text{BurstLength}\}) + (n-1) \times \text{Arbitration}$$

Note

- PrechargeLatency is only required for the case where a different row is open.
- ActivateLatency is only required for the case where a different row is open, or if the bank is precharged.

Simultaneous read transfers on multiple AHB ports transfer latency formula

For two AHB ports performing INCR4 bursts the transfer latency is:

$$\text{ReadLatency} = 4 + n(\text{CASLatency} + \text{AHBABurstLength} + \text{AHBBBurstLength})$$

For two AHB ports performing INCR8, or INCR16 bursts the transfer latency is:

$$\text{ReadLatency} = 4 + n(\text{CASLatency} + \text{AHBABurstLength} + \text{CASLatency} + \text{AHBBBurstLength})$$

Note

In the case of WRAP8 or WRAP16 parallel transfers, the latency is the same for both single and multiple AHB ports submitting requests.

Read transaction latency examples

The following read transaction latency examples assume:

- PrechargeLatency. SDRAM precharge latency is 2.
- ActivateLatency. SDRAM activate latency is 2
- CASLatency. SDRAM CAS latency is 2.

Single INCR4 read from a different row from the one currently open

This example is for a single aligned INCR4 read from a different row from the one currently open. The SDRAM memory bank has to be precharged, and activated before the read command can be submitted:

$$\text{ReadLatency} = 7 + \text{PrechargeLatency} + \text{ActivateLatency} + \text{CASLatency} - (2 \times \text{AlignedIncr}) + \text{BurstLength}$$

$$\text{ReadLatency} = 7 + 2 + 2 + 2 - (2 \times 1) + 4 = 15 \text{ clock cycles.}$$

Two back-to-back INCR16 reads on the same AHB port from an open row

This example is for two aligned back-to-back INCR16 reads that are on the same AHB port. The SDRAM memory row is open, therefore only read commands require to be submitted:

$$\text{ReadLatency} = n(7 + \text{CASLatency} - \{2 \times \text{AlignedIncr}\} + \text{BurstLength}) + (n-1) \times \text{Arbitration}$$

$$\text{ReadLatency} = 2 \times (7 + 2 - \{2 \times 1\} + 16) + (1-1) \times 1 = 47 \text{ clock cycles.}$$

Two back-to-back WRAP8 reads on the same AHB port from an open row

This example is for two back-to-back WRAP8 reads that are submitted to the same AHB port. The SDRAM memory row is open, therefore only a read command require to be submitted:

$$\text{ReadLatency} = n(7 + \text{CASLatency} - \{2 \times \text{AlignedIncr} + \text{BurstLength}\}) + (n-1) \times \text{Arbitration}$$

$$\text{ReadLatency} = 2 \times (7 + 2 - \{2 \times 0\} + 8) + (2 - 1) \times 0 = 34 \text{ clock cycles.}$$

Two simultaneous INCR16 reads on the multiple AHB ports from an open row

This example is for two aligned back-to-back INCR16 reads that are on different AHB ports. The SDRAM memory row is open, therefore only a read command requires to be submitted:

$$\text{ReadLatency} = 4 + n(\text{CASLatency} + \text{AHBABurstLength} + \text{CASLatency} + \text{AHBBBurstLength})$$

$\text{ReadLatency} = 4 + (3 + 16 + 3 + 16) = 42$ clock cycles.

Write transfer latency

This section provides the dynamic memory write transfer latencies.

Single write transfer latency formula

Single write transfer AHB latency, the latency seen by the AHB interface, formula:

$$\text{AHBWriteLatency} = 1 + \text{BurstLength}$$

Single write transfer memory latency formula:

$$\text{MemoryWriteLatency} = (1 + \text{BurstLength}) + (4 + \text{PrechargeLatency} + \text{ActivateLatency} + \text{BurstLength})$$

Note

- PrechargeLatency is only required for the case where a different row is open.
- ActivateLatency is only required for the case where a different row is open, or if the bank is precharged.

Back-to-back write transfer on same AHB port

Back-to-back write transfer AHB latency, the latency seen by the AHB interface, formula:

$$\text{AHBWriteLatency} = n(2 + \text{BurstLength}) - 1$$

Back-to-back write transfer memory latency formula if the writes are not in-page:

$$\text{MemoryWriteLatency} = n(2 + \text{BurstLength}) - 1 + n(4 + \text{PrechargeLatency} + \text{ActivateLatency} + \text{BurstLength})$$

Note

- PrechargeLatency is only required for the case where a different row is open.
- ActivateLatency is only required for the case where a different row is open, or if the bank is precharged.

Back-to-back write transfer memory latency formula if the writes are in-page:

$$\text{MemoryWriteLatency} = n(2 + \text{BurstLength}) - 1 + (4 + \text{PrechargeLatency} + \text{ActivateLatency} + n \times \text{BurstLength})$$

Note

- PrechargeLatency is only required for the case where a different row is open.
 - ActivateLatency is only required for the case where a different row is open, or if the bank is precharged.
-

Simultaneous write transfer on multiple AHB ports

Simultaneous write transfer AHB latency, the latency seen by the AHB interface, formula:

$$\text{AHBWriteLatency} = n(2 + \text{BurstLength}) - 1$$

Write transaction latency examples

The following read transaction latency examples assume:

- PrechargeLatency. SDRAM precharge latency is 2.
- ActivateLatency. SDRAM activate latency is 2.
- CASLatency. SDRAM CAS latency is 2.

Single INCR4 write to a different row from the one currently open

This example is for a single INCR4 write to a different row from the one currently open. The SDRAM memory bank has to be precharged, and activated before the write command can be submitted:

$$\text{AHBWriteLatency} = 1 + \text{BurstLength}$$

$$\text{AHBWriteLatency} = 1 + 4 = 5 \text{ clock cycles}$$

$$\text{MemoryWriteLatency} = (1 + \text{BurstLength}) + (4 + \text{PrechargeLatency} + \text{ActivateLatency} + \text{BurstLength})$$

$$\text{MemoryWriteLatency} = 1 + 4 + 4 + 2 + 2 + 4 = 17 \text{ clock cycles.}$$

Two back-to-back WRAP4 write on the same AHB port, to an open row

This example is for two back-to-back WRAP4 write transactions to an open row. The SDRAM memory row is open, therefore only write commands require to be submitted:

$$\text{AHBWriteLatency} = n(2 + \text{BurstLength}) - 1$$

$$\text{AHBWriteLatency} = 2 \times (2 + 4) - 1 = 11 \text{ clock cycles}$$

$$\text{MemoryWriteLatency} = n(2 + \text{BurstLength}) - 1 + (4 + n \times \text{BurstLength})$$

$$\text{MemoryWriteLatency} = 2 \times (2 + 4) - 1 + (4 + 2 \times 4) = 23 \text{ clock cycles.}$$

4.3.2 Latency tables

Table 4-1 shows the 32-bit chip select width transfer latencies.

Note

These tables only provide latency information on the 1:1 **HCLK** to **MPMCCLK** ratio.

Table 4-1 32-bit chip select width transfer latencies

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
1 x 32-bit read, page open	SINGLE	Read	10	10
1 x 32-bit read, bank precharged	SINGLE	Active, Read	12	12
1 x 32-bit read, incorrect page open	SINGLE	Precharge, Active, Read	14	14
1 x 32-bit write, page open	SINGLE	Write	2	7
1 x 32-bit write, bank precharged	SINGLE	Active, Write	2	9
1 x 32-bit write, incorrect page open	SINGLE	Precharge, Active, Write	2	11
1 x 32-bit read, page open, port a 1 x 32-bit read, page open, port a (same quad word) HTRANS = N, N	SINGLE, SINGLE	Read	13 D0 at 10th clk D1 at 13th clk	13
1 x 32-bit read, page open, port a 1 x 32-bit read, page open, port a (same quad word) HTRANS = N, S	INCR	Read	11 clks D0 at 10th clk D1 at 11th clk	11
1 x 32-bit read, page open, port a 1 x 32-bit read, page open, port a (different quad word)	SINGLE, SINGLE	2 x Read	21 D0 at 10th clk D1 at 21th clk	21
1 x 32-bit read, page open, port a 1 x 32-bit read, page open, port a (different quad word)	INCR	2 x Read	21 D0 at 10th clk D1 at 21th clk	21
1 x 32-bit read, page open, port a 1 x 32-bit read, page open, port b (different quad word)	SINGLE, SINGLE	Read	14 D0 at 10th clk D1 at 14th clk	14

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
1 x 32-bit write, page open, port a 1 x 32-bit write, page open, port a (same quad word) HTRANS = N,N	SINGLE, SINGLE	Write	5 D0 at 2nd clk D1 at 5th clk	11 D0 at 10th clk D1 at 14th clk
1 x 32-bit write, page open, port a 1 x 32-bit write, page open, port a (same quad word) HTRANS = N,S	INCR	Write	3 D0 at 2nd clk D1 at 3rd clk	9 D0 at 8th clk D1 at 9th clk
1 x 32-bit write, page open, port a 1 x 32-bit write, page open, port a (different quad word)	SINGLE, SINGLE	2 x Write	5 D0 at 2nd clk D1 at 5th clk	12 D0 at 8th clk D1 at 12th clk
1 x 32-bit write, page open, port a 1 x 32-bit write, page open, port a (different quad word)	INCR	2 x Write	5 D0 at 2nd clk D1 at 5th clk	12 D0 at 8th clk D1 at 12th clk
1 x 32-bit write, page open, port a 1 x 32-bit write, page open, port b (different quad word)	SINGLE, SINGLE	Write	5 D0 (AHB0) at 2nd clk D1 (AHB3) at 5th clk	13 clks
1 x 32-bit read, page open, port a (same quad word) 1 x 32-bit write, page open, port a (same quad word) 1 x 32-bit read, page open, port a (same quad word)	SINGLE, SINGLE	Read, Write, Read	16 D0 Read at 10th D1 Write 13th D2 Read 16th	21 D1 at memory bus at 21st clk
1 x 32-bit read, page open streaming in page accesses 8 word at AHB side, HTRAN = N,N,N,....	SINGLE	Read, Read...	39 clks for 8 reads D0 at 10th clk D1 at 13th clk D2 at 16th clk D3 at 19th clk D4 at 30th clk D5 at 33th clk D6 at 36th clk D7 at 39th clk	39 clks Average: 4.85 clks

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
1 x 32-bit read, page open streaming in page accesses 8 word at AHB side, HTRAN = N,S,S,....	INCR	Read, Read...	26 clks for 8 reads D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk D4 at 23th clk D5 at 24th clk D6 at 25th clk D7 at 26th clk	26 clks Average: 3.25
1 x 32-bit read, page open, random in page accesses	SINGLE	Read, Read... Two reads from different quad words and two reads from same quad words - (N,N+N,S)/4	21+13 = 34 clk D0 at 10 D1 at 21 D2 at 31 D3 at 34	34
1 x 32-bit read, page open, random in page accesses	INCR	Read, Read... Two read from different quad words and two reads from same quad words - (N,N+N,S)/4	21+11 = 32 clk D0 at 10 D1 at 21 D2 at 31 D3 at 32	32
1 x 32-bit write, page open streaming in page accesses	SINGLE	Write, Write...	23 clks for 8 writes	28 clks Average: 3.5
1 x 32-bit write, page open streaming in page accesses	INCR	Write, Write...	9 clks for 8 writes.	21 clks for 8 data Average: 2.625
1 x 32-bit write, page open random in page accesses	SINGLE	Write, Write... Two writes to different quad words and two writes to same quad words - (N,N+N,S)/4	11 clks for 4writes	20 clks for 4 data Average: 5

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
1 x 32-bit write, page open random in page accesses	INCR	Write, Write... Two writes to different quad words and two writes to same quad words - (N,N+N,S)/4	9 (2+3+3+1) clks for 4 write	17 clks for 4 data Average: 4.25
4 x 32-bit read, page open	INCR	Read	13 D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk	13
4 x 32-bit read, page open	INCR4	Read	11 D0 at 8th clk D1 at 9th clk D2 at 10th clk D3 at 11th clk	11
4 x 32-bit read, page open	WRAP4	Read	13 D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk	13
4 x 32-bit read, bank precharged	INCR	Active, Read	15 D0 at 12th clk D1 at 13th clk D2 at 14th clk D3 at 15th clk	15
4 x 32-bit read, bank precharged	INCR4	Active, Read	13 D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk	13
4 x 32-bit read, bank precharged	WRAP4	Active, Read	15 D0 at 12th clk D1 at 13th clk D2 at 14th clk D3 at 15th clk	15

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
4 x 32-bit read, incorrect page open	INCR	Precharge, Active, Read	17 D0 at 14th clk D1 at 15th clk D2 at 16th clk D3 at 17th clk	17
4 x 32-bit read, incorrect page open	INCR4	Precharge, Active, Read	15 D0 at 12th clk D1 at 13th clk D2 at 14th clk D3 at 15th clk	15
4 x 32-bit read, incorrect page open	WRAP4	Precharge, Active, Read	17 D0 at 14th clk D1 at 15th clk D2 at 16th clk D3 at 17th clk	17
4 x 32-bit write, page open	INCR	Write	5	13
4 x 32-bit write, page open	INCR4	Write	5	13
4 x 32-bit write, page open	WRAP4	Write	5	13
4 x 32-bit write, bank precharged	INCR	Active, Write	5	15
4 x 32-bit write, bank precharged	INCR4	Active, Write	5	15
4 x 32-bit write, bank precharged	WRAP4	Active, Write	5	15
4 x 32-bit write, incorrect page open	INCR	Precharge, Active, Write	5	17
4 x 32-bit write, incorrect page open	INCR4	Precharge, Active, Write	5	17
4 x 32-bit write, incorrect page open	WRAP4	Precharge, Active, Write	5	17

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port a	INCR	2 x Read	27 clks D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk D4 at 24th clk D5 at 25th clk D6 at 26th clk D7 at 27th clk	27
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port a	INCR4	2 x Read	23 clks D0 at 8th clk D1 at 9th clk D2 at 10th clk D3 at 11th clk D4 at 20th clk D5 at 21th clk D6 at 22th clk D7 at 23th clk	23
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port a	WRAP4	2 x Read	26 clks D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk D4 at 23th clk D5 at 24th clk D6 at 25th clk D7 at 26th clk	26
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port b	INCR	2 x Read	19 D0 at 10th clk D4 at 16h clk	19
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port b	INCR4	2 x Read	15 D0 at 8th clk D7 at 15th clk	15
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port b	WRAP4	2 x Read	18	18
4 x 32-bit write, page open, port a 4 x 32-bit write, page open, port a HTRANS = N,S,S,S,S,S,S,S	INCR	2 x Write	9	21 clocks D0 at 13th clk D7 at 21st clk

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
4 x 32-bit write, page open, port a 4 x 32-bit write, page open, port a	INCR4	2 x Write	11	23
4 x 32-bit write, page open, port a 4 x 32-bit write, page open, port a	WRAP4	2 x Write	11	23
4 x 32-bit write, page open, port a 4 x 32-bit write, page open, port b	INCR	2 x Write	11	21
4 x 32-bit write, page open, port a 4 x 32-bit write, page open, port b	INCR4	2 x Write	11	21
4 x 32-bit write, page open, port a 4 x 32-bit write, page open, port b	WRAP4	2 x Write	11 (AHB0) D0 at 2nd clk D3 at 5th clk AHB3 D4 at 8th clk D7 at 11th clk	21 D0 at 14th clk D7 at 21st clk
4 x 32-bit write, page open, port a 4 x 32-bit read, page open, port a	INCR	Write, Read, <u>Read</u>	32	39
4 x 32-bit write, page open, port a 4 x 32-bit read, page open, port a	INCR4	Write, Read, <u>Read</u>	29	36
4 x 32-bit write, page open, port a 4 x 32-bit read, page open, port a	WRAP4	Write, Read, <u>Read</u>	32	39
4 x 32-bit read, page open continuous in page accesses TRANS = N,S,S,S,S,S,S,S	INCR	Read, Read...	26 clks for 8 reads	26 clks for 8 read Average: 3.25 clks
4 x 32-bit read, page open continuous in page accesses TRANS = N,S,S,S,N,S,S,S	INCR4	Read, Read...	23 clks for 8 reads	23 clks for 8 read Average: 2.875 clks
4 x 32-bit read, page open continuous in page accesses TRANS = N,S,S,S,N,S,S,S	WRAP4	Read, Read...	26 clks for 8 reads	26 clks for 8 read Average: 3.25 clks
4 x 32-bit write, page open continuous in page accesses	INCR	Write, Write...	9	21 clks for 8 write Average 2.625
4 x 32-bit write, page open continuous in page accesses	INCR4	Write, Write...	11	23 clks for 8 write Average: 2.875

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
4 x 32-bit write, page open continuous in page accesses	WRAP4	Write, Write...	11	23 clks for 8 write Average: 2.875
8 x 32-bit read, page open	INCR8	2 x Read	15 clks D0 at 8th clk D1 at 9th clk D2 at 10th clk D3 at 11th clk D4 at 12th clk D5 at 13th clk D6 at 14th clk D7 at 15th clk	15
8 x 32-bit read, page open	WRAP8	2 x Read	17 clks D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk D4 at 14h clk D5 at 15th clk D6 at 16th clk D7 at 17th clk	17
8 x 32-bit read, bank precharged	INCR8	Active, 2 x Read	17 clks D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk D4 at 14h clk D5 at 15th clk D6 at 16th clk D7 at 17th clk	17
8 x 32-bit read, bank precharged	WRAP8	Active, 2 x Read	19 clks D0 at 12th clk D1 at 13th clk D2 at 14th clk D3 at 15th clk D4 at 16th clk D5 at 17th clk D6 at 18th clk D7 at 19th clk	19

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
8 x 32-bit read, incorrect page open	INCR8	Precharge, Active, 2 x Read	19 clks D0 at 12th clk D1 at 13th clk D2 at 14th clk D3 at 15th clk D4 at 16h clk D5 at 17th clk D6 at 18th clk D7 at 19th clk	19
8 x 32-bit read, incorrect page open	WRAP8	Precharge, Active, 2 x Read	21 clks D0 at 14th clk D1 at 15th clk D2 at 16th clk D3 at 17th clk D4 at 18h clk D5 at 19th clk D6 at 20th clk D7 at 21th clk	21
8 x 32-bit write, page open	INCR8	2 x Write	9	21
8 x 32-bit write, page open	WRAP8	2 x Write	9	21
8 x 32-bit write, bank precharged	INCR8	Active, 2 x Write	9	23
8 x 32-bit write, bank precharged	WRAP8	Active, 2 x Write	9	23
8 x 32-bit write, incorrect page open	INCR8	Precharge, Active, 2 x Write	9	25
8 x 32-bit write, incorrect page open	WRAP8	Precharge, Active, 2 x Write	9	25
8 x 32-bit read, page open, port a 8 x 32-bit read, page open, port a	INCR8	4 x Read	31 clks D0 at 8th clk D7 at 15th clk D8 at 24th clk D15 at 31st clk	31
8 x 32-bit read, page open, port a 8 x 32-bit read, page open, port a	WRAP8	4 x Read	34 clks D0 at 10th clk D7 at 17th clk D8 at 27th clk D15 at 34th clk	34

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
8 x 32-bit read, page open, port a 8 x 32-bit read, page open, port b	INCR8	4 x Read	26 D0 at 8th clk D7 at 15th clk D8 at 19th clk D15 at 26th clk	26
8 x 32-bit read, page open, port a 8 x 32-bit read, page open, port b	WRAP8	4 x Read	34	34
8 x 32-bit write, page open, port a 8 x 32-bit write, page open, port a	INCR8	4 x Write	19	39
8 x 32-bit write, page open, port a 8 x 32-bit write, page open, port a	WRAP8	4 x Write	19	39
8 x 32-bit write, page open, port a 8 x 32-bit write, page open, port b	INCR8	4 x Write	19	33
8 x 32-bit write, page open, port a 8 x 32-bit write, page open, port b	WRAP8	4 x Write	19	33
8 x 32-bit read, page open, port a 8 x 32-bit write, page open, port a 8 x 32-bit read, page open, port a	INCR8	2x Read, 2 x Write, 2 x Read	41	48
8 x 32-bit read, page open, port a 8 x 32-bit write, page open, port a 8 x 32-bit read, page open, port a	WRAP8	2x Read, 2 x Write, 2 x Read	44	54
8 x 32-bit read, page open continuous in page accesses	INCR8	Read, Read...	31	31 for 16 data Average: 1.94
8 x 32-bit read, page open continuous in page accesses	WRAP8	Read, Read...	34	34 for 16 data Average: 2.125
8 x 32-bit write, page open continuous in page accesses	INCR8	Write, Write...	19	39 for 16 data Average: 2.43
8 x 32-bit write, page open continuous in page accesses	WRAP8	Write, Write...	19	39 for 16 data Average: 2.43
16 x 32-bit read, page open	INCR16	4 x Read	23 clks D0 at 8th clk D15 at 23rd clk	23

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
16 x 32-bit read, page open	WRAP16	4 x Read	25 clks D0 at 10th clk D15 at 25th clk	25
16 x 32-bit read, bank precharged	INCR16	Active, 4 x Read	25 clks D0 at 10th clk D15 at 25th clk	25
16 x 32-bit read, bank precharged	WRAP16	Active, 4 x Read	27 clks D0 at 12th clk D15 at 27th clk	27
16 x 32-bit read, incorrect page open	INCR16	Precharge, Active, 4 x Read	27 clks D0 at 12th clk D15 at 27th clk	27
16 x 32-bit read, incorrect page open	WRAP16	Precharge, Active, 4 x Read	29 clks D0 at 14th clk D15 at 29th clk	29
16 x 32-bit write, page open	INCR16	4 x Write	17	37
16 x 32-bit write, page open	WRAP16	4 x Write	17	37
16 x 32-bit write, bank precharged	INCR16	Active, 4 x Write	17	39
16 x 32-bit write, bank precharged	WRAP16	Active, 4 x Write	17	39
16 x 32-bit write, incorrect page open	INCR16	Precharge, Active, 4 x Write	17	43
16 x 32-bit write, incorrect page open	WRAP16	Precharge, Active, 4 x Write	17	43
16 x 32-bit read, page open, port a 16 x 32-bit read, page open, port a	INCR16	8 x Read	47 clks D0 at 8th clk D15 at 23rd clk D16 at 32nd clk D31 at 47th clk	47
16 x 32-bit read, page open, port a 16 x 32-bit read, page open, port a	WRAP16	8 x Read	50 clks D0 at 10th clk D15 at 25rd clk D16 at 35st clk D31 at 50th clk	50

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
16 x 32-bit read, page open, port a 16 x 32-bit read, page open, port b	INCR16	8 x Read	42	42
16 x 32-bit read, page open, port a 16 x 32-bit read, page open, port b	WRAP16	8 x Read	50	50
16 x 32-bit write, page open, port a 16 x 32-bit write, page open, port a	INCR16	8 x Write	35	54 clks D0 at 23rd clk D32 at 54th clk
16 x 32-bit write, page open, port a 16 x 32-bit write, page open, port a	WRAP16	8 x Write	35	54 clks D0 at 23rd clk D32 at 54th clk
16 x 32-bit write, page open, port a 16 x 32-bit write, page open, port b	INCR16	8 x Write	35	57
16 x 32-bit write, page open, port a 16 x 32-bit write, page open, port b	WRAP16	8 x Write	35	57
16 x 32-bit read, page open, port a 16 x 32-bit write, page open, port a 16 x 32-bit read, page open, port a	INCR16	4 x Read, 4 x Write, 4 x Read	65	80
16 x 32-bit read, page open, port a 16 x 32-bit write, page open, port a 16 x 32-bit read, page open, port a	WRAP16	4 x Read, 4 x Write, 4 x Read	68	86
16 x 32-bit read, page open continuous in page accesses	INCR16	Read, Read...	47 clks for 32 reads	47 clks Average: 1.47
16 x 32-bit read, page open continuous in page accesses	WRAP16	Read, Read...	50 clks for 32 reads	50 clks Average: 1.56
16 x 32-bit write, page open continuous in page accesses	INCR16	Write, Write...	35 clks for 32 write Average:1.09	54 clks Average: 1.69
16 x 32-bit write, page open continuous in page accesses	WRAP16	Write, Write...	35 clks for 32 write Average:1.09	54 clks Average: 1.69
1 x 16-bit read, page open	SINGLE	Read	10	10
1 x 16-bit write, page open	SINGLE	Write	2	7
4 x 16-bit read, page open	INCR	Read	13 clks D0 at 10th clk D3 at 13th clk	13

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
4 x 16-bit read, page open	INCR4	Read	13 clks D0 at 10th clk D3 at 13th clk	13
4 x 16-bit read, page open	WRAP4	Read	13 clks D0 at 10th clk D3 at 13th clk	13
4 x 16-bit write, page open	INCR	Write	5	11 clk D0, D1 at 10th clk D2, D3 at 11th clk
4 x 16-bit write, page open	INCR4	Write	5	11 clk D0, D1 at 10th clk D2, D3 at 11th clk
4 x 16-bit write, page open	WRAP4	Write	5	11 clk D0, D1 at 10th clk D2, D3 at 11th clk
8 x 16-bit read, page open	INCR8	Read	17	17
8 x 16-bit read, page open	WRAP8	Read	17	17
8 x 16-bit write, page open	INCR8	Write	9	17
8 x 16-bit write, page open	WRAP8	Write	9	17
16 x 16-bit read, page open	INCR16	2 x Read	25	25
16 x 16-bit read, page open	WRAP16	2 x Read	25	25
16 x 16-bit write, page open	INCR16	2 x Write	17	29
16 x 16-bit write, page open	WRAP16	2 x Write	17	29
1 x 8-bit read, page open	SINGLE	Read	10	10
1 x 8-bit write, page open	SINGLE	Write	2	7
4 x 8-bit read, page open	INCR	Read	13	13
4 x 8-bit read, page open	INCR4	Read	13	13
4 x 8-bit read, page open	WRAP4	Read	13	13
4 x 8-bit write, page open	INCR	Write	5	10
4 x 8-bit write, page open	INCR4	Write	5	10

Table 4-1 32-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
4 x 8-bit write, page open	WRAP4	Write	5	10
8 x 8-bit read, page open	INCR8	Read	17	17
8 x 8-bit read, page open	WRAP8	Read	17	17
8 x 8-bit write, page open	INCR8	Write	9	15
8 x 8-bit write, page open	WRAP8	Write	9	15
16 x 8-bit read, page open	INCR16	Read	25 D0 at 10th clk D7 at 17th clk D16 at 25th clk	25
16 x 8-bit read, page open	WRAP16	Read	25 D0 at 10th clk D7 at 17th clk D16 at 25th clk	25
16 x 8-bit write, page open	INCR16	Write	17	25 D0,D1,D2,D3 at 22nd clk D4,D5,D6,D7 at 23rd clk D8,D9,D10,D11 at 24th clk D12,D13,D14,D15 at 25th clk
16 x 8-bit write, page open	WRAP16	Write	17	25 D0,D1,D2,D3 at 22nd clk D4,D5,D6,D7 at 23rd clk D8,D9,D10,D11 at 24th clk D12,D13,D14,D15 at 25th clk

Table 4-2 on page 4-23 shows the 16-bit chip select width transfer latencies.

Table 4-2 16-bit chip select width transfer latencies

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
1 x 32-bit read, page open	SINGLE	Read	10	10
1 x 32-bit read, bank precharged	SINGLE	Active, Read	12	12
1 x 32-bit read, incorrect page open	SINGLE	Precharge, Active, Read	14	14
1 x 32-bit write, page open	SINGLE	Write	2	7
1 x 32-bit write, bank precharged	SINGLE	Active, Write	2	9
1 x 32-bit write, incorrect page open	SINGLE	Precharge, Active, Write	2	11
1 x 32-bit read, page open, port a 1 x 32-bit read, page open, port a (same quad word) HTRANS = N, N	SINGLE, SINGLE	Read	13 D0 at 10th clk D1 at 13th clk	13
1 x 32-bit read, page open, port a 1 x 32-bit read, page open, port a (same quad word) Note: HTRANS = N, S	INCR	Read	11 clks D0 at 10th clk D1 at 11th clk	11
1 x 32-bit read, page open, port a 1 x 32-bit read, page open, port a (different quad word)	SINGLE, SINGLE	2 x Read	21 D0 at 10th clk D1 at 21th clk	21
1 x 32-bit read, page open, port a 1 x 32-bit read, page open, port a (different quad word)	INCR	2 x Read	21 D0 at 10th clk D1 at 21th clk	21
1 x 32-bit read, page open, port a 1 x 32-bit read, page open, port b (different quad word)	SINGLE, SINGLE	Read	14 D0 at 10th clk D1 at 14th clk	14
1 x 32-bit write, page open, port a 1 x 32-bit write, page open, port a (same quad word) HTRANS = N,N	SINGLE, SINGLE	Write	5 D0 at 2nd clk D1 at 5th clk	11 D0 at 10th clk D1 at 14th clk
1 x 32-bit write, page open, port a 1 x 32-bit write, page open, port a (same quad word) HTRANS = N,S	INCR	Write	3 D0 at 2nd clk D1 at 3rd clk	9 D0 at 8th clk D1 at 9th clk

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
1 x 32-bit write, page open, port a 1 x 32-bit write, page open, port a (different quad word)	SINGLE, SINGLE	2 x Write	5 D0 at 2nd clk D1 at 5th clk	12 D0 at 8th clk D1 at 12th clk
1 x 32-bit write, page open, port a 1 x 32-bit write, page open, port a (different quad word)	INCR	2 x Write	5 D0 at 2nd clk D1 at 5th clk	12 D0 at 8th clk D1 at 12th clk
1 x 32-bit write, page open, port a 1 x 32-bit write, page open, port b (different quad word)	SINGLE, SINGLE	Write	5 D0 (AHB0) at 2nd clk D1 (AHB3) at 5th clk	13 clks
1 x 32-bit read, page open, port a (same quad word) 1 x 32-bit write, page open, port a (same quad word) 1 x 32-bit read, page open, port a (same quad word)	SINGLE, SINGLE	Read, Write, Read	16 D0 Read at 10th D1 Write 13th D2 Read 16th	21 D1 at memory bus at 21st clk
1 x 32-bit read, page open streaming in page accesses 8 word at AHB side, HTRAN = N,N,N,....	SINGLE	Read, Read...	39 clks for 8 reads D0 at 10th clk D1 at 13th clk D2 at 16th clk D3 at 19th clk D4 at 30th clk D5 at 33th clk D6 at 36th clk D7 at 39th clk	39 clks Average: 4.85 clks
1 x 32-bit read, page open streaming in page accesses 8 word at AHB side, HTRAN = N,S,S,....	INCR	Read, Read...	26 clks for 8 reads D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk D4 at 23th clk D5 at 24th clk D6 at 25th clk D7 at 26th clk	26 clks Average: 3.25

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
1 x 32-bit read, page open, random in page accesses	SINGLE	Read, Read... Two reads from different quad words and two reads from same quad words - (N,N+N,S)/4	21+13 = 34 clk D0 at 10 D1 at 21 D2 at 31 D3 at 34	34
1 x 32-bit read, page open, random in page accesses	INCR	Read, Read... Two read from different quad words and two reads from same quad words - (N,N+N,S)/4	21+11 = 32 clk D0 at 10 D1 at 21 D2 at 31 D3 at 32	32
1 x 32-bit write, page open streaming in page accesses	SINGLE	Write, Write...	23 clks for 8 writes	28 clks Average: 3.5
1 x 32-bit write, page open streaming in page accesses	INCR	Write, Write...	9 clks for 8 writes.	21 clks for 8 data Average: 2.625
1 x 32-bit write, page open random in page accesses	SINGLE	Write, Write... Two writes to different quad words and two writes to same quad words - (N,N+N,S)/4	11 clks for 4writes	20 clks for 4 data Average: 5
1 x 32-bit write, page open random in page accesses	INCR	Write, Write... Two writes to different quad words and two writes to same quad words - (N,N+N,S)/4	9 (2+3+3+1) clks for 4 write	17 clks for 4 data Average: 4.25
4 x 32-bit read, page open	INCR	Read	13 D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk	13

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
4 x 32-bit read, page open	INCR4	Read	11 D0 at 8th clk D1 at 9th clk D2 at 10th clk D3 at 11th clk	11
4 x 32-bit read, page open	WRAP4	Read	13 D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk	13
4 x 32-bit read, bank precharged	INCR	Active, Read	15 D0 at 12th clk D1 at 13th clk D2 at 14th clk D3 at 15th clk	15
4 x 32-bit read, bank precharged	INCR4	Active, Read	13 D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk	13
4 x 32-bit read, bank precharged	WRAP4	Active, Read	15 D0 at 12th clk D1 at 13th clk D2 at 14th clk D3 at 15th clk	15
4 x 32-bit read, incorrect page open	INCR	Precharge, Active, Read	17 D0 at 14th clk D1 at 15th clk D2 at 16th clk D3 at 17th clk	17
4 x 32-bit read, incorrect page open	INCR4	Precharge, Active, Read	15 D0 at 12th clk D1 at 13th clk D2 at 14th clk D3 at 15th clk	15

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
4 x 32-bit read, incorrect page open	WRAP4	Precharge, Active, Read	17 D0 at 14th clk D1 at 15th clk D2 at 16th clk D3 at 17th clk	17
4 x 32-bit write, page open	INCR	Write	5	13
4 x 32-bit write, page open	INCR4	Write	5	13
4 x 32-bit write, page open	WRAP4	Write	5	13
4 x 32-bit write, bank precharged	INCR	Active, Write	5	15
4 x 32-bit write, bank precharged	INCR4	Active, Write	5	15
4 x 32-bit write, bank precharged	WRAP4	Active, Write	5	15
4 x 32-bit write, incorrect page open	INCR	Precharge, Active, Write	5	17
4 x 32-bit write, incorrect page open	INCR4	Precharge, Active, Write	5	17
4 x 32-bit write, incorrect page open	WRAP4	Precharge, Active, Write	5	17
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port a	INCR	2 x Read	27 clks D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk D4 at 24th clk D5 at 25th clk D6 at 26th clk D7 at 27th clk	27
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port a	INCR4	2 x Read	23 clks D0 at 8th clk D1 at 9th clk D2 at 10th clk D3 at 11th clk D4 at 20th clk D5 at 21th clk D6 at 22th clk D7 at 23th clk	23

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port a	WRAP4	2 x Read	26 clks D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk D4 at 23th clk D5 at 24th clk D6 at 25th clk D7 at 26th clk	26
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port b	INCR	2 x Read	19 D0 at 10th clk D4 at 16h clk	19
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port b	INCR4	2 x Read	15 D0 at 8th clk D7 at 15th clk	15
4 x 32-bit read, page open, port a 4 x 32-bit read, page open, port b	WRAP4	2 x Read	18	18
4 x 32-bit write, page open, port a 4 x 32-bit write, page open, port a HTRANS = N,S,S,S,S,S,S,S	INCR	2 x Write	9	21 clocks D0 at 13th clk D7 at 21st clk
4 x 32-bit write, page open, port a 4 x 32-bit write, page open, port a	INCR4	2 x Write	11	23
4 x 32-bit write, page open, port a 4 x 32-bit write, page open, port a	WRAP4	2 x Write	11	23
4 x 32-bit write, page open, port a 4 x 32-bit write, page open, port b	INCR	2 x Write	11	21
4 x 32-bit write, page open, port a 4 x 32-bit write, page open, port b	INCR4	2 x Write	11	21
4 x 32-bit write, page open, port a 4 x 32-bit Write, page open, port b	WRAP4	2 x Write	11 (AHB0) D0 at 2nd clk D3 at 5th clk AHB3 D4 at 8th clk D7 at 11th clk	21 D0 at 14th clk D7 at 21st clk

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
4 x 32-bit write, page open, port a 4 x 32-bit read, page open, port a	INCR	Write, Read, <u>Read</u>	32	39
4 x 32-bit write, page open, port a 4 x 32-bit read, page open, port a	INCR4	Write, Read, <u>Read</u>	29	36
4 x 32-bit write, page open, port a 4 x 32-bit read, page open, port a	WRAP4	Write, Read, <u>Read</u>	32	39
4 x 32-bit read, page open continuous in page accesses TRANS = N,S,S,S,S,S,S,S	INCR	Read, Read...	26 clks for 8 reads	26 clks for 8 read Average: 3.25 clks
4 x 32-bit read, page open continuous in page accesses TRANS = N,S,S,S,N,S,S,S	INCR4	Read, Read...	23 clks for 8 reads	23 clks for 8 read Average: 2.875 clks
4 x 32-bit read, page open continuous in page accesses TRANS = N,S,S,S,N,S,S,S	WRAP4	Read, Read...	26 clks for 8 reads	26 clks for 8 read Average: 3.25 clks
4 x 32-bit write, page open continuous in page accesses	INCR	Write, Write...	9	21 clks for 8 write Average: 2.625
4 x 32-bit write, page open continuous in page accesses	INCR4	Write, Write...	11	23 clks for 8 write Average: 2.875
4 x 32-bit write, page open continuous in page accesses	WRAP4	Write, Write...	11	23 clks for 8 write Average: 2.875
8 x 32-bit read, page open	INCR8	2 x Read	15 clks D0 at 8th clk D1 at 9th clk D2 at 10th clk D3 at 11th clk D4 at 12th clk D5 at 13th clk D6 at 14th clk D7 at 15th clk	15

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
8 x 32-bit read, page open	WRAP8	2 x Read	17 clks D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk D4 at 14h clk D5 at 15th clk D6 at 16th clk D7 at 17th clk	17
8 x 32-bit read, bank precharged	INCR8	Active, 2 x Read	17 clks D0 at 10th clk D1 at 11th clk D2 at 12th clk D3 at 13th clk D4 at 14h clk D5 at 15th clk D6 at 16th clk D7 at 17th clk	17
8 x 32-bit read, bank precharged	WRAP8	Active, 2 x Read	19 clks D0 at 12th clk D1 at 13th clk D2 at 14th clk D3 at 15th clk D4 at 16th clk D5 at 17th clk D6 at 18th clk D7 at 19th clk	19
8 x 32-bit read, incorrect page open	INCR8	Precharge, Active, 2 x Read	19 clks D0 at 12th clk D1 at 13th clk D2 at 14th clk D3 at 15th clk D4 at 16h clk D5 at 17th clk D6 at 18th clk D7 at 19th clk	19

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
8 x 32-bit read, incorrect page open	WRAP8	Precharge, Active, 2 x Read	21 clks D0 at 14th clk D1 at 15th clk D2 at 16th clk D3 at 17th clk D4 at 18h clk D5 at 19th clk D6 at 20th clk D7 at 21th clk	21
8 x 32-bit write, page open	INCR8	2 x Write	9	21
8 x 32-bit write, page open	WRAP8	2 x Write	9	21
8 x 32-bit write, bank precharged	INCR8	Active, 2 x Write	9	23
8 x 32-bit write, bank precharged	WRAP8	Active, 2 x Write	9	23
8 x 32-bit write, incorrect page open	INCR8	Precharge, Active, 2 x Write	9	25
8 x 32-bit write, incorrect page open	WRAP8	Precharge, Active, 2 x Write	9	25
8 x 32-bit read, page open, port a 8 x 32-bit read, page open, port a	INCR8	4 x Read	31 clks D0 at 8th clk D7 at 15th clk D8 at 24th clk D15 at 31st clk	31
8 x 32-bit read, page open, port a 8 x 32-bit read, page open, port a	WRAP8	4 x Read	34 clks D0 at 10th clk D7 at 17th clk D8 at 27th clk D15 at 34th clk	34
8 x 32-bit read, page open, port a 8 x 32-bit read, page open, port b	INCR8	4 x Read	26 D0 at 8th clk D7 at 15th clk D8 at 19th clk D15 at 26th clk	26
8 x 32-bit read, page open, port a 8 x 32-bit read, page open, port b	WRAP8	4 x Read	34	34

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
8 x 32-bit write, page open, port a 8 x 32-bit write, page open, port a	INCR8	4 x Write	19	39
8 x 32-bit write, page open, port a 8 x 32-bit write, page open, port a	WRAP8	4 x Write	19	39
8 x 32-bit write, page open, port a 8 x 32-bit write, page open, port b	INCR8	4 x Write	19	33
8 x 32-bit write, page open, port a 8 x 32-bit write, page open, port b	WRAP8	4 x Write	19	33
8 x 32-bit read, page open, port a 8 x 32-bit write, page open, port a 8 x 32-bit read, page open, port a	INCR8	2x Read, 2 x Write, 2 x Read	41	48
8 x 32-bit read, page open, port a 8 x 32-bit write, page open, port a 8 x 32-bit read, page open, port a	WRAP8	2x Read, 2 x Write, 2 x Read	44	54
8 x 32-bit read, page open continuous in page accesses	INCR8	Read, Read...	31	31 for 16 data Average: 1.94
8 x 32-bit read, page open continuous in page accesses	WRAP8	Read, Read...	34	34 for 16 data Average: 2.125
8 x 32-bit write, page open continuous in page accesses	INCR8	Write, Write...	19	39 for 16 data Average: 2.43
8 x 32-bit write, page open continuous in page accesses	WRAP8	Write, Write...	19	39 for 16 data Average: 2.43
16 x 32-bit read, page open	INCR16	4 x Read	23 clks D0 at 8th clk D15 at 23rd clk	23
16 x 32-bit read, page open	WRAP16	4 x Read	25 clks D0 at 10th clk D15 at 25th clk	25
16 x 32-bit read, bank precharged	INCR16	Active, 4 x Read	25 clks D0 at 10th clk D15 at 25th clk	25
16 x 32-bit read, bank precharged	WRAP16	Active, 4 x Read	27 clks D0 at 12th clk D15 at 27th clk	27

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
16 x 32-bit read, incorrect page open	INCR16	Precharge, Active, 4 x Read	27 clks D0 at 12th clk D15 at 27th clk	27
16 x 32-bit read, incorrect page open	WRAP16	Precharge, Active, 4 x Read	29 clks D0 at 14th clk D15 at 29th clk	29
16 x 32-bit write, page open	INCR16	4 x Write	17	37
16 x 32-bit write, page open	WRAP16	4 x Write	17	37
16 x 32-bit write, bank precharged	INCR16	Active, 4 x Write	17	39
16 x 32-bit write, bank precharged	WRAP16	Active, 4 x Write	17	39
16 x 32-bit write, incorrect page open	INCR16	Precharge, Active, 4 x Write	17	43
16 x 32-bit write, incorrect page open	WRAP16	Precharge, Active, 4 x Write	17	43
16 x 32-bit read, page open, port a 16 x 32-bit read, page open, port a	INCR16	8 x Read	47 clks D0 at 8th clk D15 at 23rd clk D16 at 32nd clk D31 at 47th clk	47
16 x 32-bit read, page open, port a 16 x 32-bit read, page open, port a	WRAP16	8 x Read	50 clks D0 at 10th clk D15 at 25rd clk D16 at 35st clk D31 at 50th clk	50
16 x 32-bit read, page open, port a 16 x 32-bit read, page open, port b	INCR16	8 x Read	42	42
16 x 32-bit read, page open, port a 16 x 32-bit read, page open, port b	WRAP16	8 x Read	50	50
16 x 32-bit write, page open, port a 16 x 32-bit write, page open, port a	INCR16	8 x Write	35	54 clks D0 at 23rd clk D32 at 54th clk
16 x 32-bit write, page open, port a 16 x 32-bit write, page open, port a	WRAP16	8 x Write	35	54 clks D0 at 23rd clk D32 at 54th clk

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
16 x 32-bit write, page open, port a 16 x 32-bit write, page open, port b	INCR16	8 x Write	35	57
16 x 32-bit write, page open, port a 16 x 32-bit write, page open, port b	WRAP16	8 x Write	35	57
16 x 32-bit read, page open, port a 16 x 32-bit write, page open, port a 16 x 32-bit read, page open, port a	INCR16	4 x Read, 4 x Write, 4 x Read	65	80
16 x 32-bit read, page open, port a 16 x 32-bit write, page open, port a 16 x 32-bit read, page open, port a	WRAP16	4 x Read, 4 x Write, 4 x Read	68	86
16 x 32-bit read, page open continuous in page accesses	INCR16	Read, Read...	47 clks for 32 reads	47 clks Average: 1.47
16 x 32-bit read, page open continuous in page accesses	WRAP16	Read, Read...	50 clks for 32 reads	50 clks Average: 1.56
16 x 32-bit write, page open continuous in page accesses	INCR16	Write, Write...	35 clks for 32 write Average:1.09	54 clks Average: 1.69
16 x 32-bit write, page open continuous in page accesses	WRAP16	Write, Write...	35 clks for 32 write Average:1.09	54 clks Average: 1.69
1 x 16-bit read, page open	SINGLE	Read	10	10
1 x 16-bit write, page open	SINGLE	Write	2	7
4 x 16-bit read, page open	INCR	Read	13 clks D0 at 10th clk D3 at 13th clk	13
4 x 16-bit read, page open	INCR4	Read	13 clks D0 at 10th clk D3 at 13th clk	13
4 x 16-bit read, page open	WRAP4	Read	13 clks D0 at 10th clk D3 at 13th clk	13
4 x 16-bit write, page open	INCR	Write	5	11 clk D0, D1 at 10th clk D2, D3 at 11th clk

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
4 x 16-bit write, page open	INCR4	Write	5	11 clk D0, D1 at 10th clk D2, D3 at 11th clk
4 x 16-bit write, page open	WRAP4	Write	5	11 clk D0, D1 at 10th clk D2, D3 at 11th clk
8 x 16-bit read, page open	INCR8	Read	17	17
8 x 16-bit read, page open	WRAP8	Read	17	17
8 x 16-bit write, page open	INCR8	Write	9	17
8 x 16-bit write, page open	WRAP8	Write	9	17
16 x 16-bit read, page open	INCR16	2 x Read	25	25
16 x 16-bit read, page open	WRAP16	2 x Read	25	25
16 x 16-bit write, page open	INCR16	2 x Write	17	29
16 x 16-bit write, page open	WRAP16	2 x Write	17	29
1 x 8-bit read, page open	SINGLE	Read	10	10
1 x 8-bit write, page open	SINGLE	Write	2	7
4 x 8-bit read, page open	INCR	Read	13	13
4 x 8-bit read, page open	INCR4	Read	13	13
4 x 8-bit read, page open	WRAP4	Read	13	13
4 x 8-bit write, page open	INCR	Write	5	10
4 x 8-bit write, page open	INCR4	Write	5	10
4 x 8-bit write, page open	WRAP4	Write	5	10
8 x 8-bit read, page open	INCR8	Read	17	17
8 x 8-bit read, page open	WRAP8	Read	17	17
8 x 8-bit write, page open	INCR8	Write	9	15
8 x 8-bit write, page open	WRAP8	Write	9	15

Table 4-2 16-bit chip select width transfer latencies (continued)

Access type	Burst type	SDRAM commands	AHB latency	Memory latency
16 x 8-bit read, page open	INCR16	Read	25 D0 at 10th clk D7 at 17th clk D16 at 25th clk	25
16 x 8-bit read, page open	WRAP16	Read	25 D0 at 10th clk D7 at 17th clk D16 at 25th clk	25
16 x 8-bit write, page open	INCR16	Write	17	25 D0,D1,D2,D3 at 22nd clk D4,D5,D6,D7 at 23rd clk D8,D9,D10,D11 at 24th clk D12,D13,D14,D15 at 25th clk
16 x 8-bit write, page open	WRAP16	Write	17	25 D0,D1,D2,D3 at 22nd clk D4,D5,D6,D7 at 23rd clk D8,D9,D10,D11 at 24th clk D12,D13,D14,D15 at 25th clk

4.4 Transaction examples

The following section provides examples of dynamic memory transactions, and contains the following subsections:

- *Single 32-bit read from SDR-SDRAM, row open on page 4-38*
- *Single 32-bit write to SDR-SDRAM, row open on page 4-41*
- *Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open on page 4-43*
- *Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, bank precharged on page 4-45*
- *Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open on page 4-47*
- *Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open on page 4-49*
- *Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, bank precharged on page 4-51*
- *Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, different row open on page 4-53*
- *Burst 8 (INCR8) 32-bit read from 32-bit SDR-SDRAM, row open on page 4-55*
- *Burst 8 (INCR8/WRAP8) 32-bit write to 32-bit SDR-SDRAM, row open on page 4-56*
- *Burst 16 (INCR16) 32-bit read from 32-bit SDR-SDRAM, row open on page 4-58*
- *Burst 16 (INCR16/WRAP16) 32-bit write to 32-bit SDR-SDRAM, row open on page 4-59*
- *Burst 4 (INCR4) 32-bit read from 16-bit SDR-SDRAM, different row open on page 4-61*
- *Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 16-bit SDR-SDRAM, different row open on page 4-62*
- *Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, with transfer latency 3 on page 4-64*
- *Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 0 pad configuration on page 4-66*

- *Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 1 pad configuration on page 4-68*
- *Two back-to-back burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open on page 4-70*
- *Two back-to-back burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open on page 4-72*
- *Three back-to-back burst 4 (INCR4) 32-bit accesses from 32-bit SDR-SDRAM, row open on page 4-73.*

Unless explicitly specified the following transactions use the parameters in Table 4-3.

Table 4-3 Transaction details

Parameter	Value
CAS latency	Two cycles
RAS latency	Two cycles
Precharge delay (t _{RP})	Two cycles
Memory controller state	Idle when request goes active
Pad interface scheme	Clock delayed
Clock ratio (HCLK:MPMCCLK)	1:1

4.4.1 Single 32-bit read from SDR-SDRAM, row open

Figure 4-1 on page 4-40 and Table 4-4 show an example of a single 32-bit AHB read from 32-bit wide SDR-SDRAM. The row to be read from (in SDRAM) is open.

Table 4-4 Single 32-bit read from 32-bit SDR-SDRAM, row open

Cycle	Description
T0	AHB address provided to memory controller
T0-T1	AHB transaction processing
T1-T2	Arbitration of AHB memory ports
T2-T3	Memory controller processing
T3-T4	Read command submitted to SDRAM

Table 4-4 Single 32-bit read from 32-bit SDR-SDRAM, row open

Cycle	Description
T4-T5	SDRAM CAS access latency
T5-T6	Read data capture
T6-T7	Internal memory controller processing
T7-T8	Internal memory controller processing
T8-T9	Internal memory controller processing
T9-T10	Provide read data to AHB

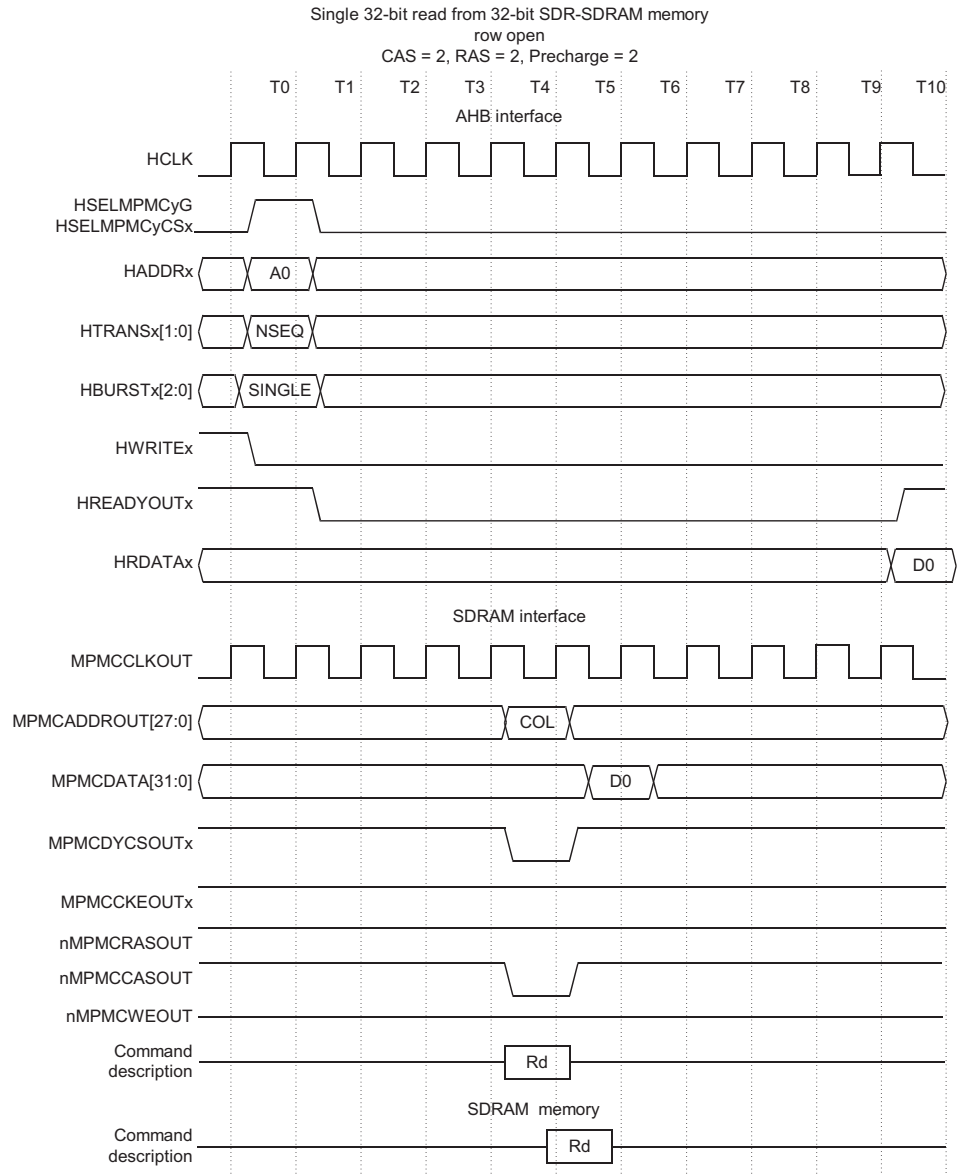


Figure 4-1 Single 32-bit read from 32-bit SDR-SDRAM, row open

4.4.2 Single 32-bit write to SDR-SDRAM, row open

Figure 4-2 on page 4-42 and Table 4-5 show an example of a single 32-bit AHB write to 32-bit wide SDR-SDRAM. The row to be written to (in SDRAM) is open.

Table 4-5 Single 32-bit write to 32-bit SDR-SDRAM, row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	AHB write data provided to memory controller. Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T6	Internal memory controller processing.
T6-T7	Write command submitted to SDRAM.



4.4.3 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open

Figure 4-3 on page 4-44 and Table 4-6 show an example of a burst 4, 32-bit AHB read from 32-bit wide SDR-SDRAM. The row to be read from (in SDRAM) is open.

Table 4-6 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T4	Read command submitted to SDRAM.
T4-T5	SDRAM CAS access latency.
T5-T6	Read data 0 capture.
T6-T7	Read data 1 capture.
T7-T8	Read data 2 capture. Provide read data 0 to AHB.
T8-T9	Read data 3 capture. Provide read data 1 to AHB.
T9-T10	Provide read data 2 to AHB.
T10-T11	Provide read data 3 to AHB.

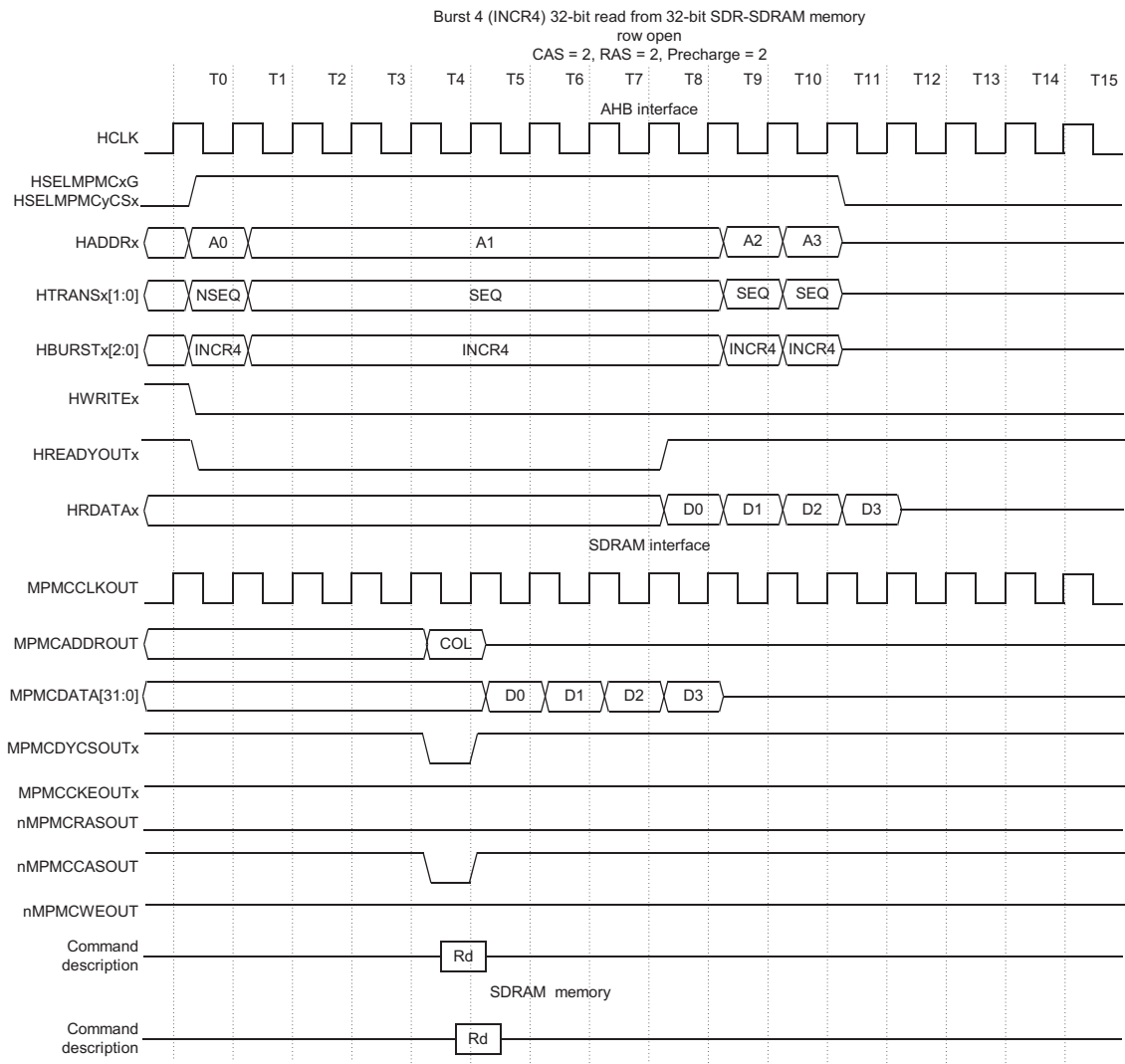


Figure 4-3 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open

4.4.4 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, bank precharged

Figure 4-4 on page 4-46 and Table 4-7 show an example of a burst 4, 32-bit AHB read from 32-bit wide SDR-SDRAM. The row to be read from (in SDRAM) is precharged.

Table 4-7 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, bank precharged

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T4	Active command submitted to SDRAM.
T4-T5	SDRAM active command latency.
T5-T6	Read command submitted to SDRAM.
T6-T7	SDRAM CAS access latency.
T7-T8	Read data 0 capture.
T8-T9	Read data 1 capture.
T9-T10	Read data 2 capture. Provide read data 0 to AHB.
T10-T11	Read data 3 capture. Provide read data 1 to AHB.
T11-T12	Provide read data 2 to AHB.
T12-T13	Provide read data 3 to AHB.

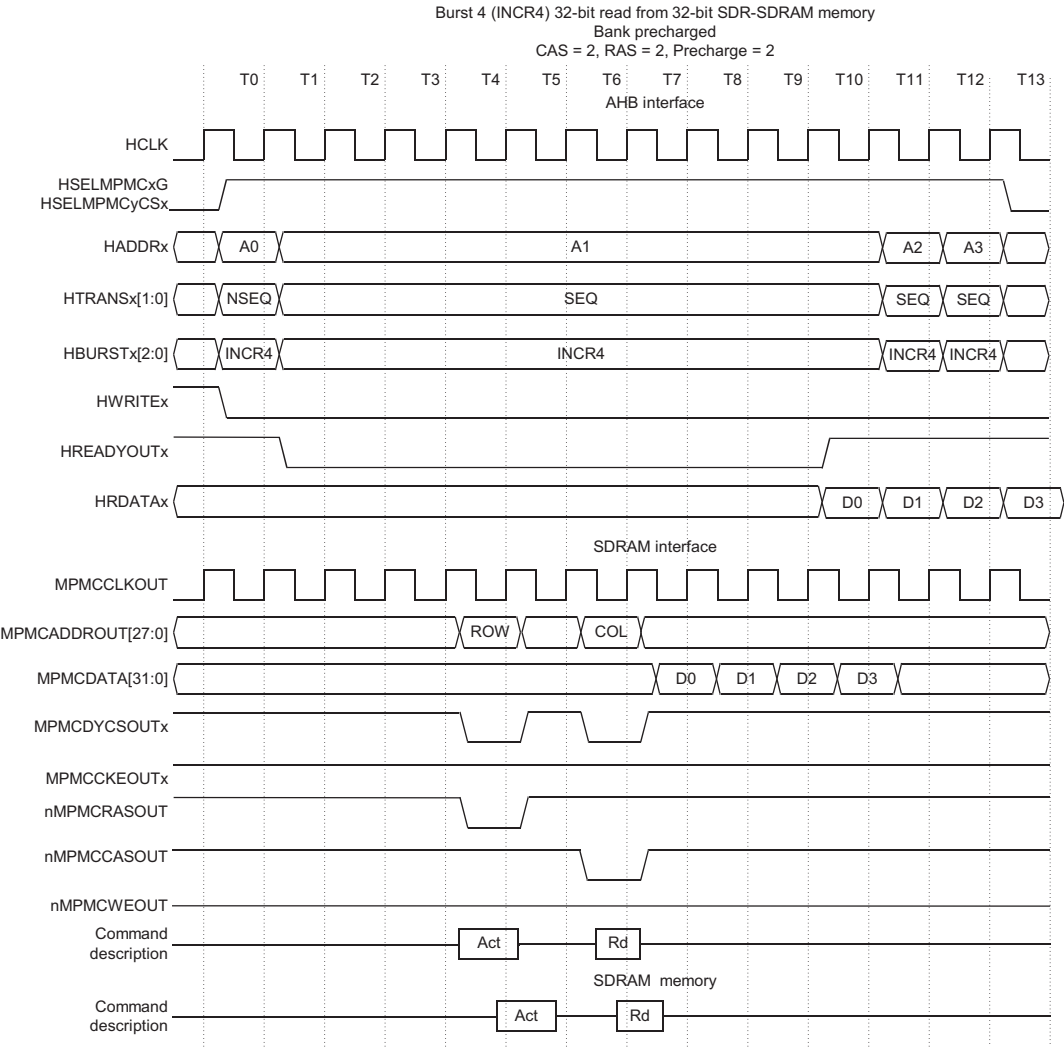


Figure 4-4 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, bank precharged

4.4.5 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open

Figure 4-5 on page 4-48 and Table 4-8 show an example of a burst 4, 32-bit AHB read from 32-bit wide SDR-SDRAM. A different row from the one that is required to be read from is open.

Table 4-8 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T4	Precharge command submitted to SDRAM.
T4-T5	SDRAM precharge command latency.
T5-T6	Active command submitted to SDRAM.
T6-T7	SDRAM active command latency.
T7-T8	Read command submitted to SDRAM.
T8-T9	SDRAM CAS access latency.
T9-T10	Read data 0 capture.
T10-T11	Read data 1 capture.
T11-T12	Read data 2 capture. Provide read data 0 to AHB.
T12-T13	Read data 3 capture. Provide read data 1 to AHB.
T13-T14	Provide read data 2 to AHB.
T14-T15	Provide read data 3 to AHB.

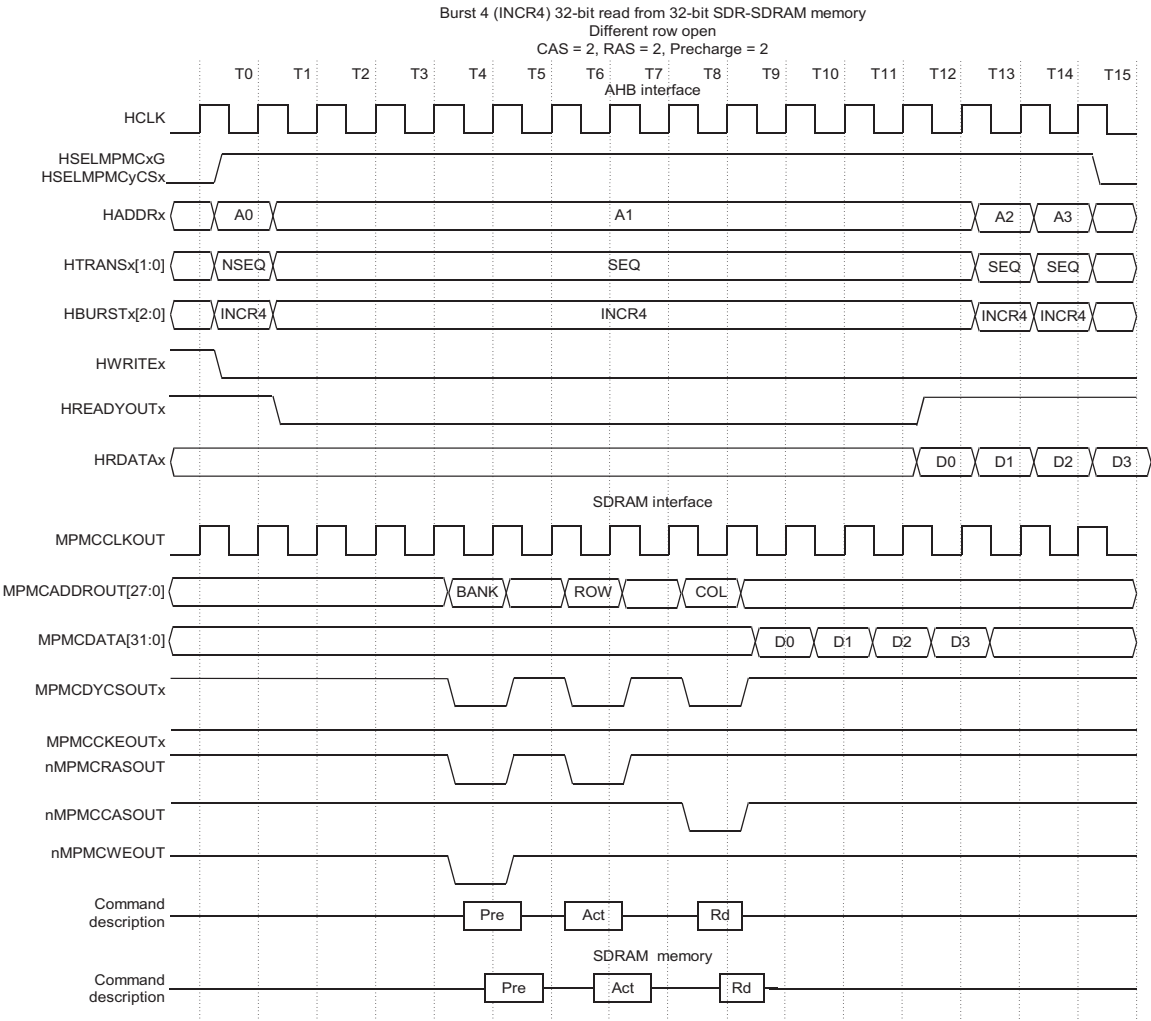


Figure 4-5 Burst4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open

4.4.6 Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open

Figure 4-6 on page 4-50 and Table 4-9 show an example of a burst 4, 32-bit AHB write to 32-bit wide SDR-SDRAM. The row to be written to (in SDRAM) is open.

Table 4-9 Burst4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	AHB write data 0 provided to memory controller. Arbitration of AHB memory ports.
T2-T3	AHB write data 1 provided to memory controller.
T3-T4	AHB write data 2 provided to memory controller.
T4-T5	AHB write data 3 provided to memory controller.
T5-T9	Internal memory controller processing.
T9-T10	Write data 0 submitted to SDRAM.
T10-T11	Write data 1 submitted to SDRAM.
T11-T12	Write data 2 submitted to SDRAM.
T12-T13	Write data 3 submitted to SDRAM.

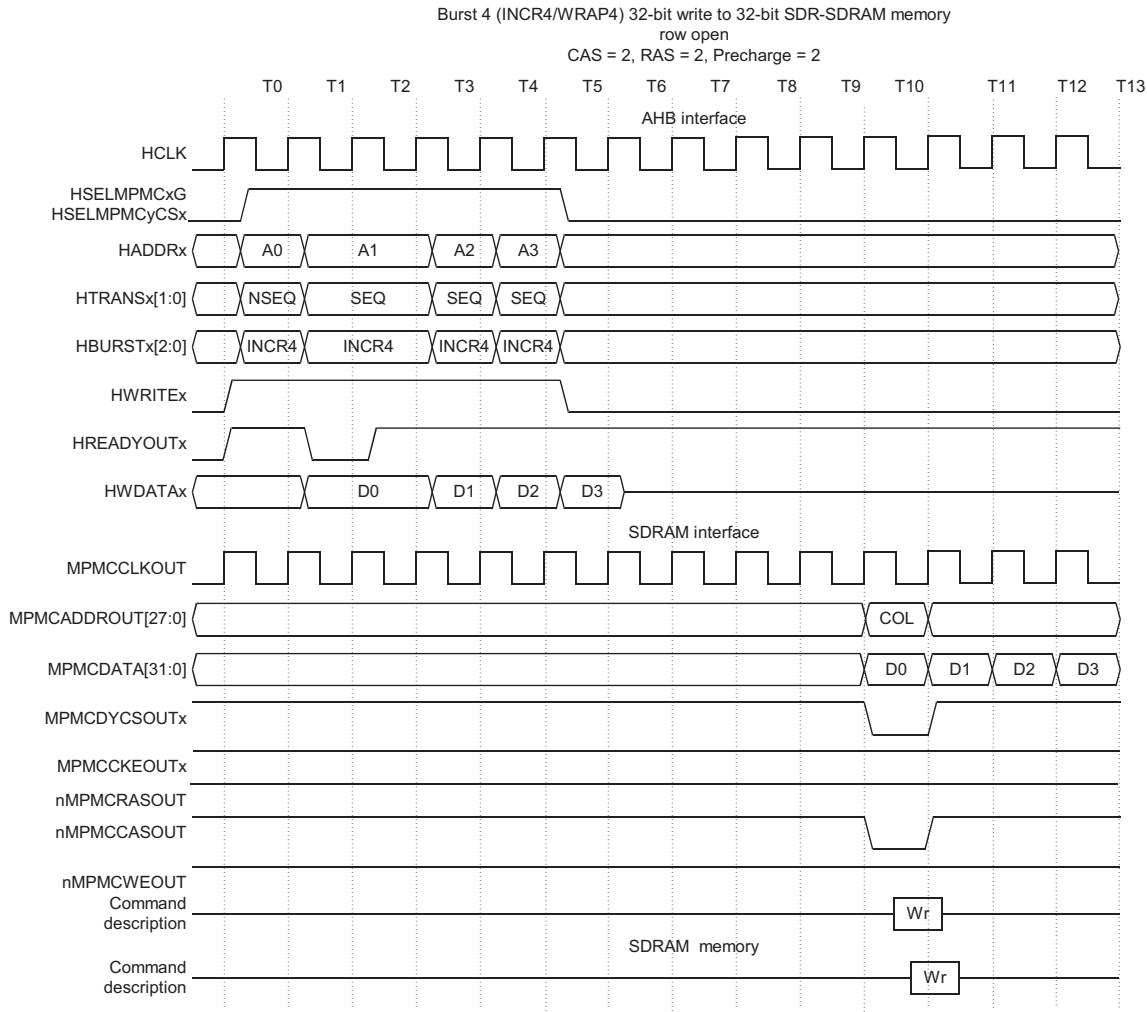


Figure 4-6 Burst 4 (INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open

4.4.7 Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, bank precharged

Figure 4-7 on page 4-52 and Table 4-10 show an example of a burst 4, 32-bit AHB write to 32-bit wide SDR-SDRAM. The bank to be written to (in SDRAM) is precharged.

Table 4-10 Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, bank precharged

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	AHB write data provided to memory controller. Arbitration of AHB memory ports.
T2-T3	AHB write data 1 provided to memory controller.
T3-T4	AHB write data 2 provided to memory controller.
T4-T5	AHB write data 3 provided to memory controller.
T5-T9	Internal memory controller processing.
T9-T10	Active command submitted to SDRAM.
T10-T11	SDRAM active command latency.
T11-T12	Write data 0 submitted to SDRAM.
T12-T13	Write data 1 submitted to SDRAM.
T13-T14	Write data 2 submitted to SDRAM.
T14-T15	Write data 3 submitted to SDRAM.

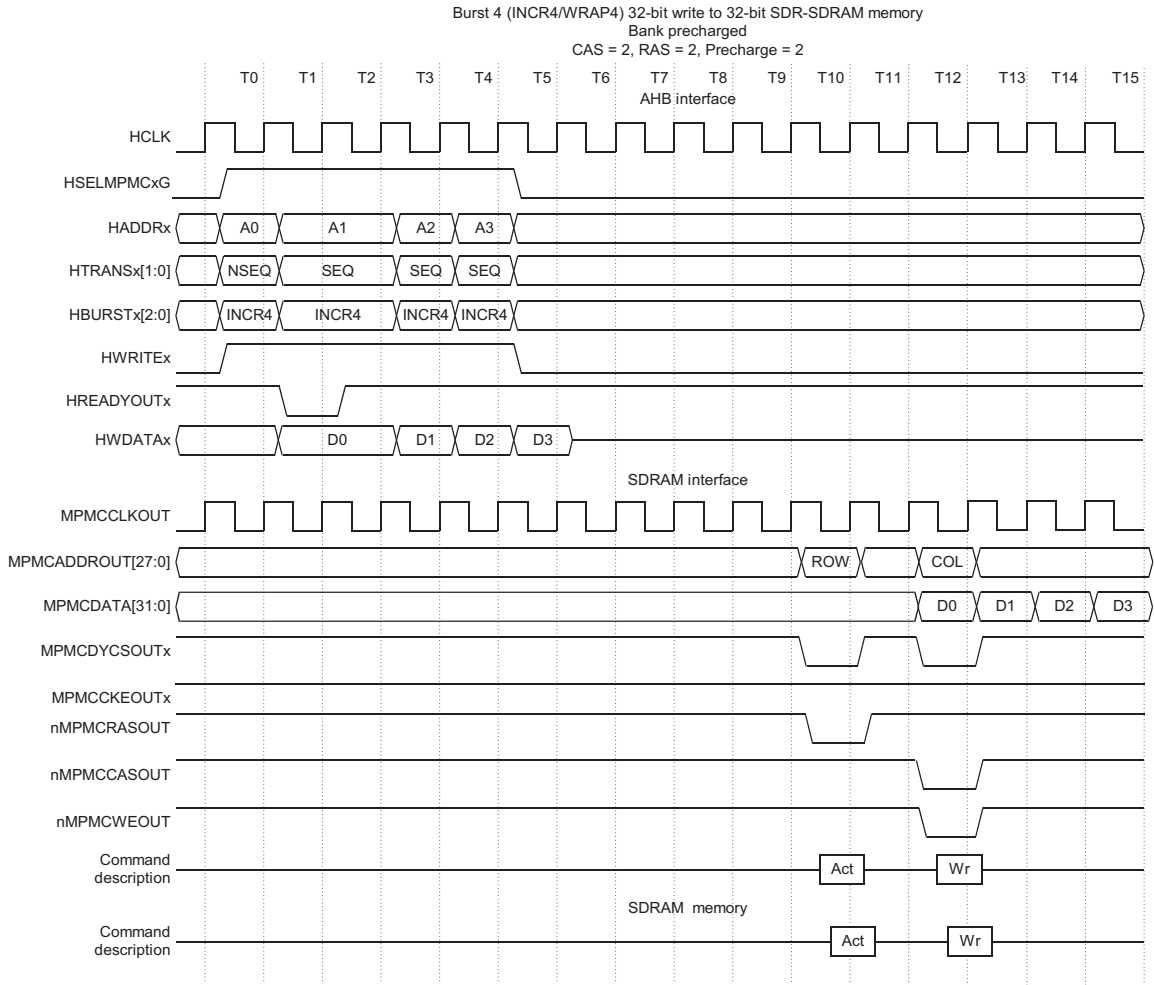


Figure 4-7 Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, bank precharged

4.4.8 Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, different row open

Figure 4-8 on page 4-54 and Table 4-11 show an example of a burst 4, 32-bit AHB write to 32-bit wide SDR-SDRAM. A different row to the one being written to is open.

Table 4-11 Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, different row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	AHB write data provided to memory controller. Arbitration of AHB memory ports.
T2-T3	AHB write data 1 provided to memory controller.
T3-T4	AHB write data 2 provided to memory controller.
T4-T5	AHB write data 3 provided to memory controller.
T5-T9	Internal memory controller processing.
T9-T10	Precharge command submitted to SDRAM.
T10-T11	SDRAM precharge command latency.
T11-T12	Active command submitted to SDRAM.
T12-T13	SDRAM active command latency.
T13-T14	Write data 0 submitted to SDRAM.
T14-T15	Write data 1 submitted to SDRAM.
T15-T16	Write data 2 submitted to SDRAM.
T16-T17	Write data 3 submitted to SDRAM.

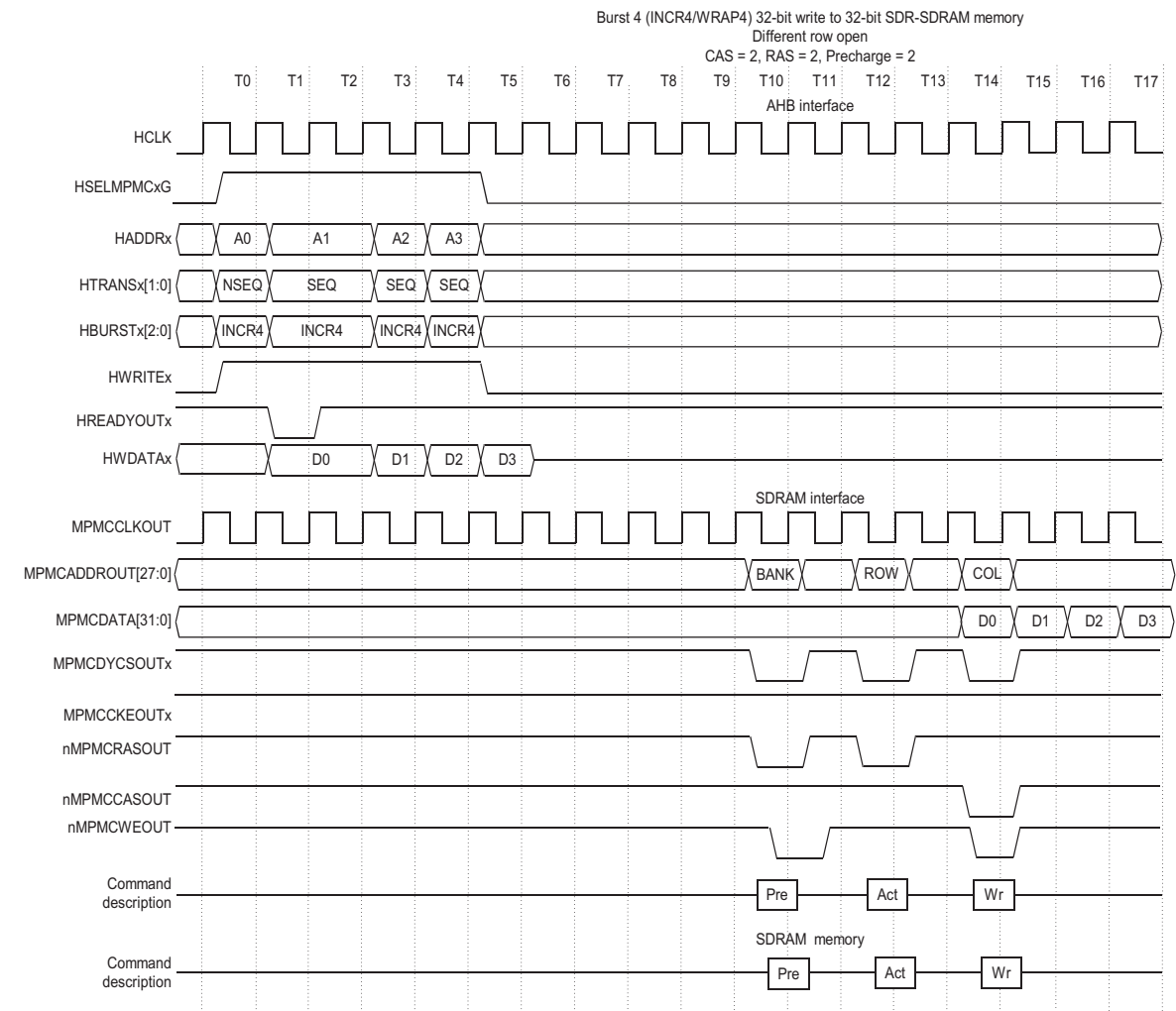


Figure 4-8 Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, different row open

4.4.9 Burst 8 (INCR8) 32-bit read from 32-bit SDR-SDRAM, row open

Figure 4-9 on page 4-56 and Table 4-12 show an example of a burst 8, 32-bit AHB read from 32-bit wide SDR-SDRAM. The row to be read from (in SDRAM) is open.

Table 4-12 Burst 8 (INCR8) 32-bit read from 32-bit SDR-SDRAM, row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T4	Read command submitted to SDRAM.
T4-T5	SDRAM CAS access latency.
T5-T6	Read data 0 capture.
T6-T7	Read data 1 capture.
T7-T8	Read data 2 capture. Provide read data 0 to AHB.
T8-T9	Read data 3 capture. Provide read data 1 to AHB.
T9-T10	Read data 4 capture. Provide read data 2 to AHB.
T10-T11	Read data 5 capture. Provide read data 3 to AHB.
T11-T12	Read data 6 capture. Provide read data 4 to AHB.
T12-T13	Read data 7 capture. Provide read data 5 to AHB.
T13-T14	Provide read data 6 to AHB.
T14-T15	Provide read data 7 to AHB.

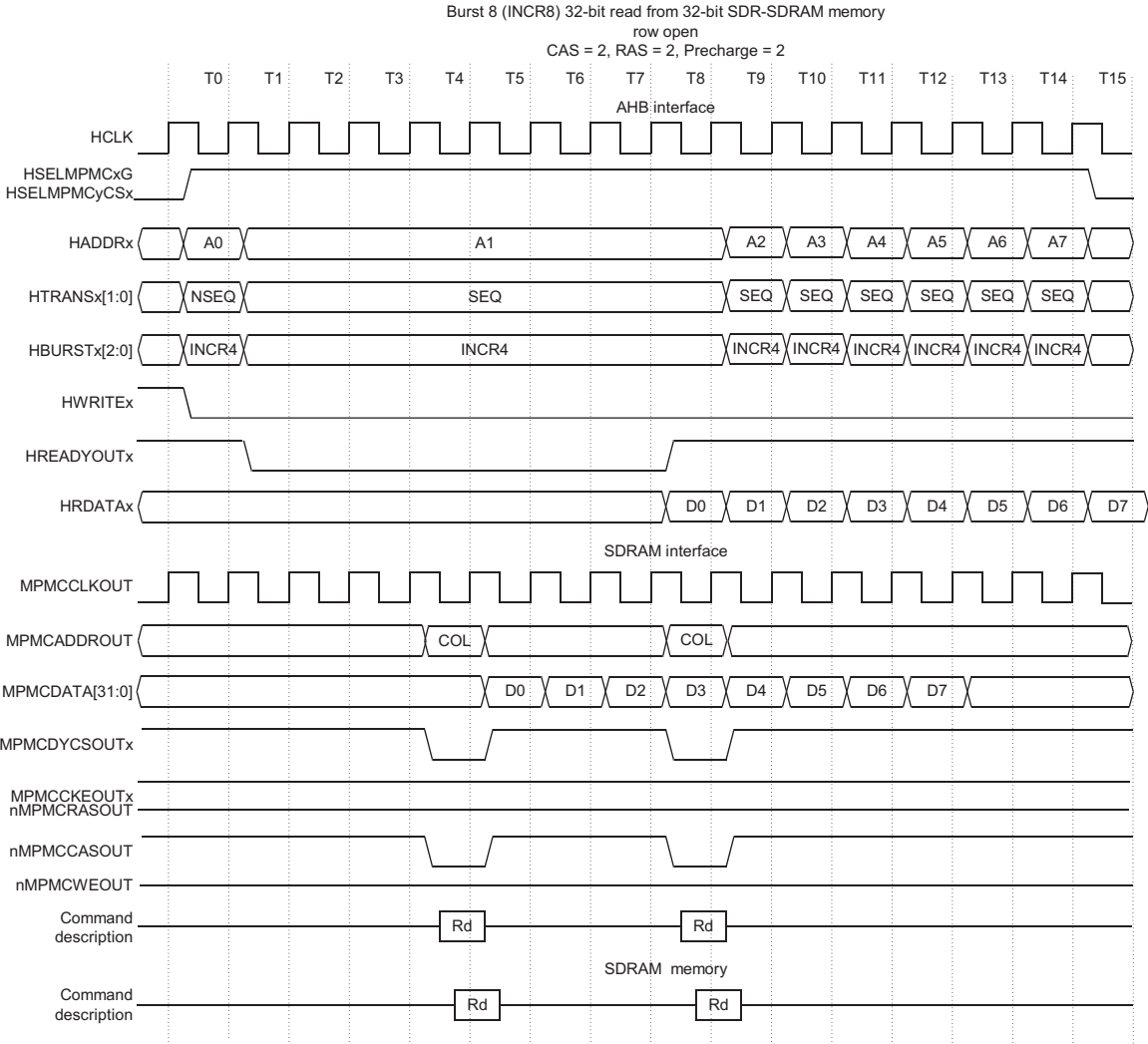


Figure 4-9 Burst 8 (INCR8) 32-bit read from 32-bit SDR-SDRAM, row open

4.4.10 Burst 8 (INCR8/WRAP8) 32-bit write to 32-bit SDR-SDRAM, row open

Figure 4-10 on page 4-57 and Table 4-13 on page 4-57 show an example of a burst 8, 32-bit AHB write to 32-bit wide SDR-SDRAM. The row to be written to (in SDRAM) is open.

Note

Figure 4-10 and Table 4-13 do not show the data write to memory.

Table 4-13 Burst 8 (INCR8/WRAP8) 32-bit write to 32-bit SDR-SDRAM, row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	AHB write data 0 provided to memory controller. Arbitration of AHB memory ports.
T2-T9	Write data 1-7 provided to memory controller.

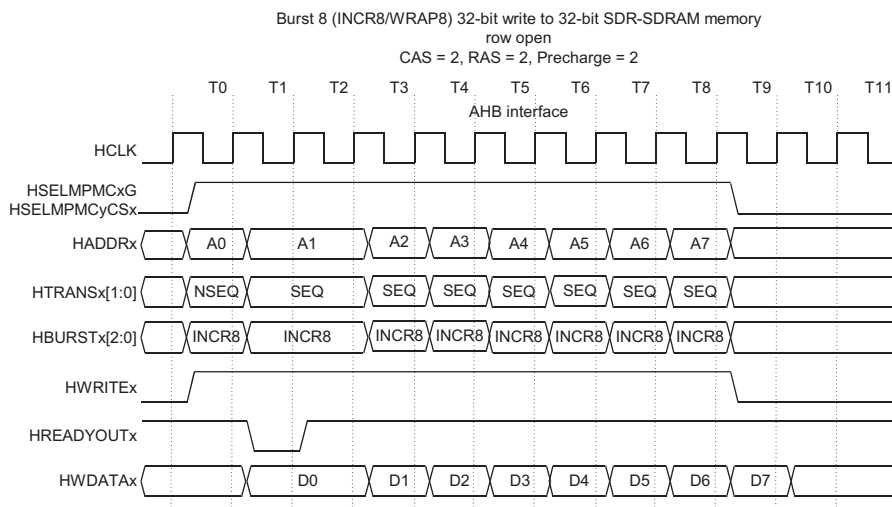


Figure 4-10 Burst 8 (INCR8/WRAP8) 32-bit write to 32-bit SDR-SDRAM memory, row open

4.4.11 Burst 16 (INCR16) 32-bit read from 32-bit SDR-SDRAM, row open

Figure 4-11 on page 4-59 and Table 4-14 show an example of a burst 16, 32-bit AHB read from 32-bit wide SDR-SDRAM. The row to be read from (in SDRAM) is open.

Table 4-14 Burst 16 (INCR16) 32-bit read from 32-bit SDR-SDRAM, row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T4	Read command submitted to SDRAM.
T4-T5	SDRAM CAS access latency.
T5-T6	Read data 0 capture.
T6-T7	Read data 1 capture.
T7-T8	Read data 2 capture. Provide read data 0 to AHB.
T8-T9	Read data 3 capture. Provide read data 1 to AHB.
T9-T10	Read data 4 capture. Provide read data 2 to AHB.
T10-T11	Read data 5 capture. Provide read data 3 to AHB.
T11-T12	Read data 6 capture. Provide read data 4 to AHB.
T12-T13	Read data 7 capture. Provide read data 5 to AHB.
T13-T14	Read data 8 capture. Provide read data 6 to AHB.
T14-T15	Read data 9 capture. Provide read data 7 to AHB.
T15-T16	Read data 10 capture. Provide read data 8 to AHB.
T16-T17	Read data 11 capture. Provide read data 9 to AHB.
T17-T18	Read data 12 capture. Provide read data 10 to AHB.
T18-T19	Read data 13 capture. Provide read data 11 to AHB.
T19-T20	Read data 14 capture. Provide read data 12 to AHB.
T20-T21	Read data 15 capture. Provide read data 12 to AHB.
T21-T22	Provide read data 14 to AHB.
T22-T23	Provide read data 15 to AHB.

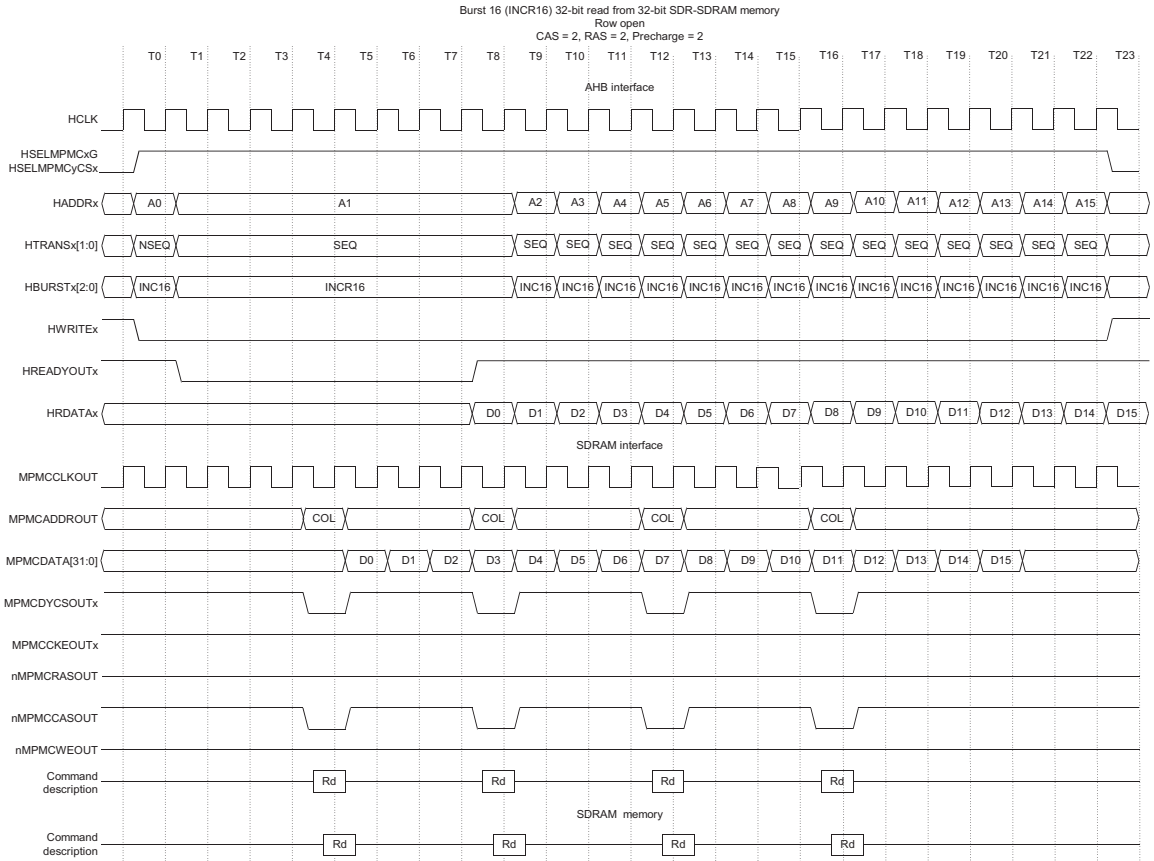


Figure 4-11 Burst 16 (INCR16) 32-bit read from 32-bit SDR-SDRAM, row open

4.4.12 Burst 16 (INCR16/WRAP16) 32-bit write to 32-bit SDR-SDRAM, row open

Figure 4-12 on page 4-60 and Table 4-15 on page 4-60 show an example of a burst 8, 32-bit AHB write to 32-bit wide SDR-SDRAM. The row to be written to (in SDRAM) is open.

Note

Figure 4-12 on page 4-60 and Table 4-15 on page 4-60 do not show the data write to memory.

Table 4-15 Burst 16 (INCR16/WRAP16) 32-bit write to 32-bit SDR-SDRAM, row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	AHB write data 0 provided to memory controller. Arbitration of AHB memory ports.
T2-T17	Write date 1-15 provided to memory controller.

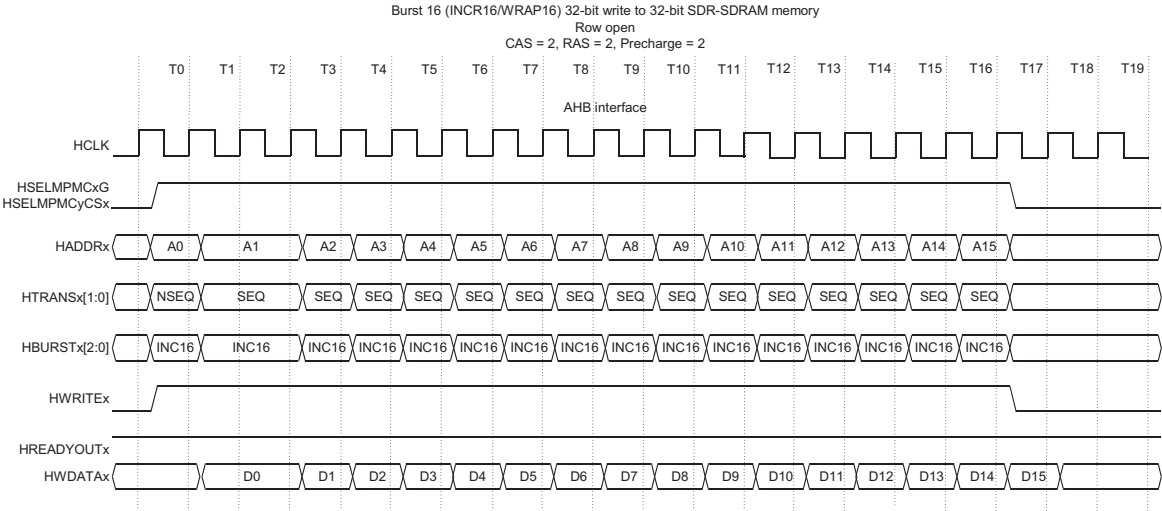


Figure 4-12 Burst 16 (INCR16/WRAP16) 32-bit write to 32-bit SDR-SDRAM, row open

4.4.13 Burst 4 (INCR4) 32-bit read from 16-bit SDR-SDRAM, different row open

Figure 4-13 on page 4-62 and Table 4-16 show an example of a burst 4, 32-bit AHB read from 16-bit wide SDR-SDRAM. A different row from the one that is required to be read from is open.

Table 4-16 Burst 4 (INCR4) 32-bit read from 16-bit SDR-SDRAM, different row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T4	Precharge command submitted to SDRAM.
T4-T5	SDRAM precharge command latency.
T5-T6	Active command submitted to SDRAM.
T6-T7	SDRAM active command latency.
T7-T8	Read command submitted to SDRAM.
T8-T9	SDRAM CAS access latency.
T9-T10	16 bits of read data 0 captured.
T10-T11	16 bits of read data 0 captured.
T11-T12	16 bits of read data 1 captured.
T12-T13	16 bits of read data 1 captured. Provide read data 0 to AHB.
T13-T14	16 bits of read data 2 captured.
T14-T15	16 bits of read data 2 captured. Provide read data 1 to AHB.
T15-T16	16 bits of read data 3 captured.
T16-T17	16 bits of read data 3 captured. Provide read data 2 to AHB.
T17-T18	Internal memory controller processing.
T18-T19	Provide read data 3 to AHB.

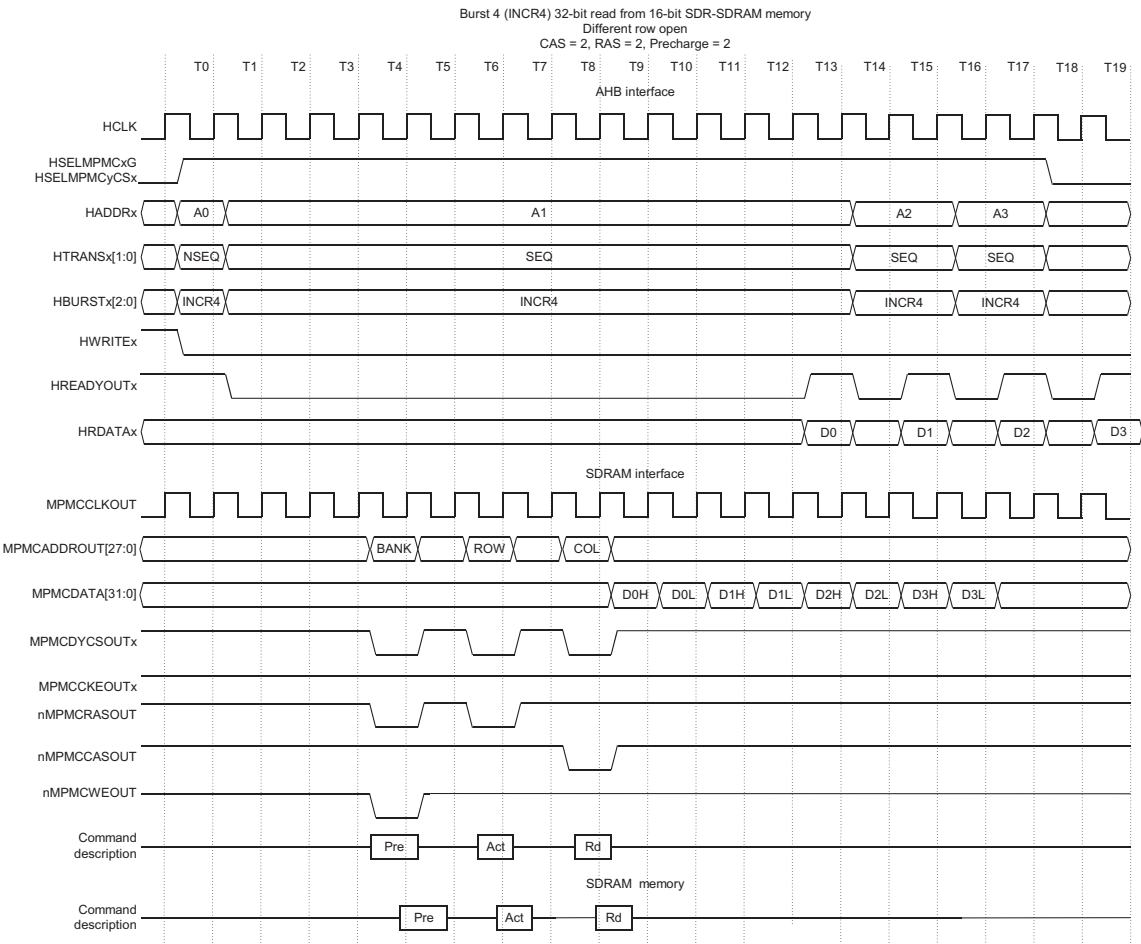


Figure 4-13 Burst 4 (INCR4) 32-bit read from 16-bit SDR-SDRAM, different row open

4.4.14 Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 16-bit SDR-SDRAM, different row open

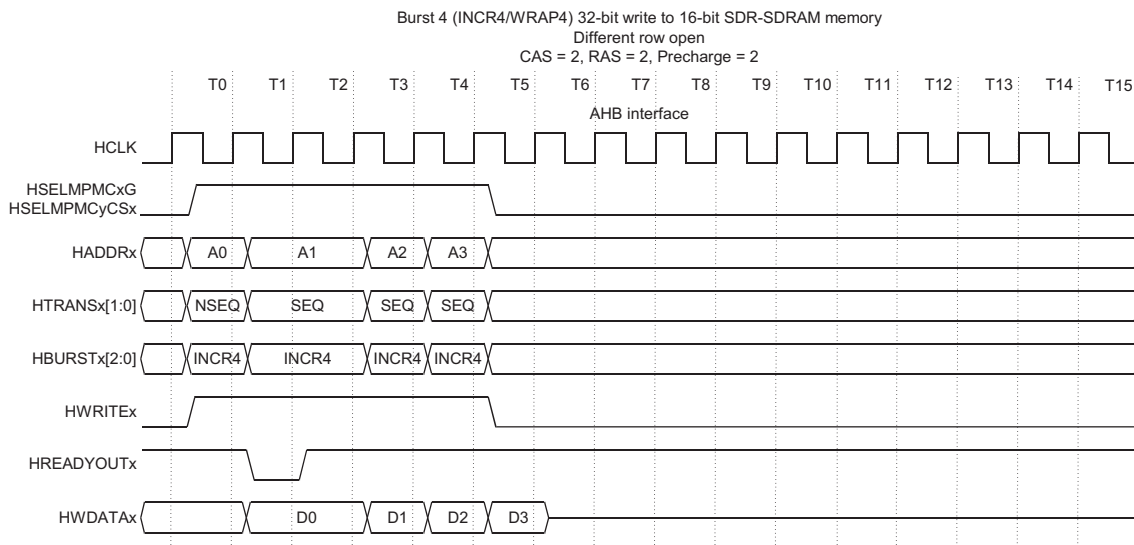
Figure 4-14 on page 4-63 and Table 4-17 on page 4-63 show an example of a burst 4, 32-bit AHB write to 16-bit wide SDR-SDRAM. A different row to the one being written to is open.

Note

Figure 4-14 on page 4-63 and Table 4-17 on page 4-63 do not show the data write to memory.

Table 4-17 Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 16-bit SDR-SDRAM, different row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	AHB write data provided to memory controller. Arbitration of AHB memory ports.
T2-T5	Write data 1-3 provided to memory controller.

**Figure 4-14 Burst 4 (INCR/INCR4/WRAP4) 32-bit write to 16-bit SDR-SDRAM, different row open**

4.4.15 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, with transfer latency 3

Table 4-18 and Table 4-19 show an example of a burst 4, 32-bit AHB read from 32-bit wide SDR-SDRAM. A different row from the one that must be read from is open.

Table 4-18 Transaction details

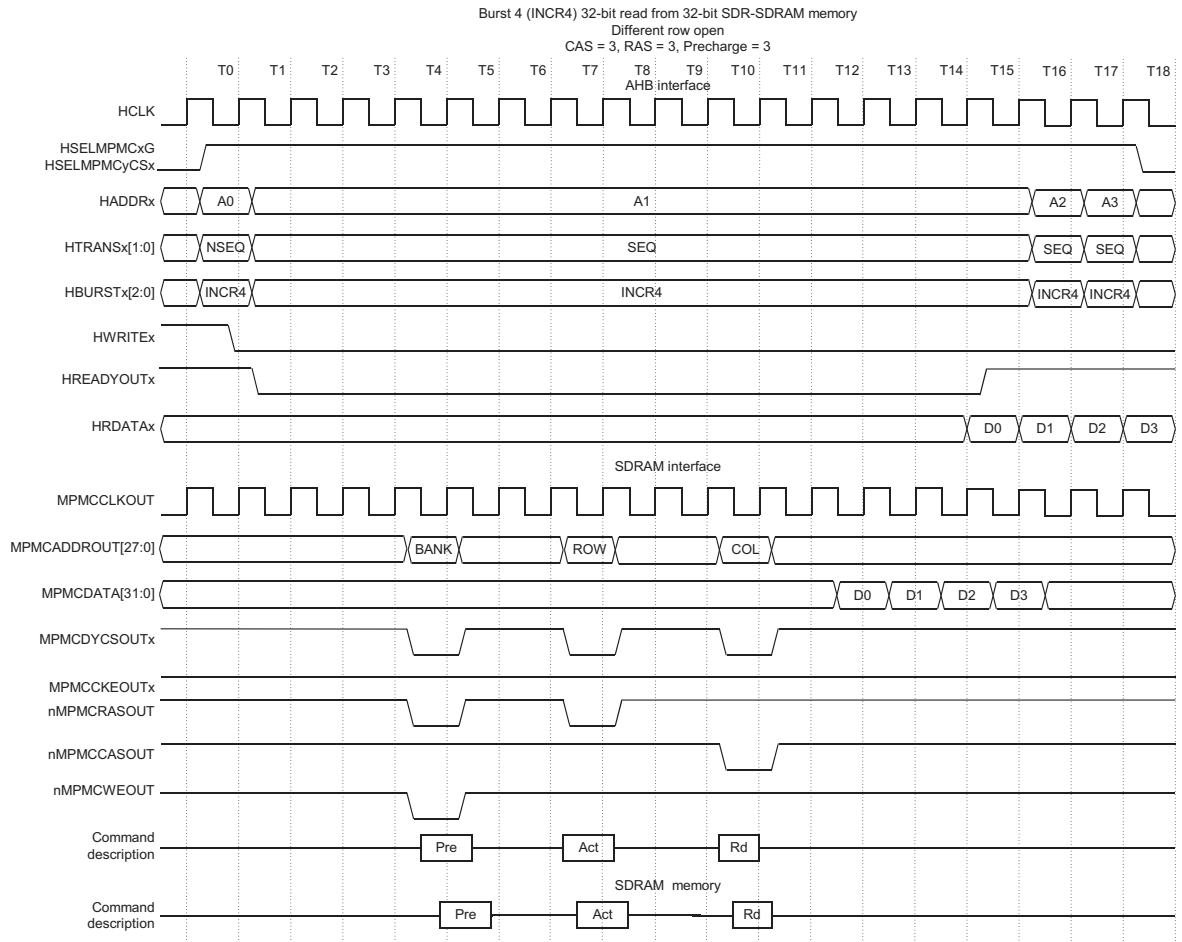
Parameter	Value
CAS latency	Three cycles
RAS latency	Three cycles
Precharge delay (t _{RP})	Three cycles

Table 4-19 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T4	Precharge command submitted to SDRAM.
T4-T5	SDRAM precharge command latency.
T5-T6	SDRAM precharge command latency.
T6-T7	Active command submitted to SDRAM.
T7-T8	SDRAM active command latency.
T8-T9	SDRAM active command latency.
T9-T10	Read command submitted to SDRAM.
T10-T11	SDRAM CAS access latency.
T11-T12	SDRAM CAS access latency.
T12-T13	SDRAM CAS access latency.
T13-T14	Read data 0 capture.
T14-T15	Read data 1 capture. Provide read data 0 to AHB.

Table 4-19 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open

Cycle	Description
T15-T16	Read data 2 capture. Provide read data 1 to AHB.
T16-T17	Read data 3 capture. Provide read data 2 to AHB.
T17-T18	Provide read data 3 to AHB.

**Figure 4-15 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open**

4.4.16 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 0 pad configuration

Figure 4-16 on page 4-67 and Table 4-20 show an example of a burst 4, 32-bit AHB read from 32-bit wide SDR-SDRAM. A different row from the one that is required to be read from is open. The command delay 0 pad configuration is used. Data is returned a cycle later in the command delayed-plus-zero mode compared to the clock delay mode.

Table 4-20 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 0 pad configuration

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T4	Precharge command submitted to SDRAM.
T4-T5	SDRAM precharge command latency.
T5-T6	Active command submitted to SDRAM.
T6-T7	SDRAM active command latency.
T7-T8	Read command submitted to SDRAM.
T8-T9	Pad delay 1.
T9-T10	SDRAM CAS access latency.
T10-T11	Read data 0 capture.
T11-T12	Read data 1 capture.
T12-T13	Read data 2 capture. Provide read data 0 to AHB.
T13-T14	Read data 3 capture. Provide read data 1 to AHB.
T14-T15	Provide read data 2 to AHB.
T15-T16	Provide read data 3 to AHB.

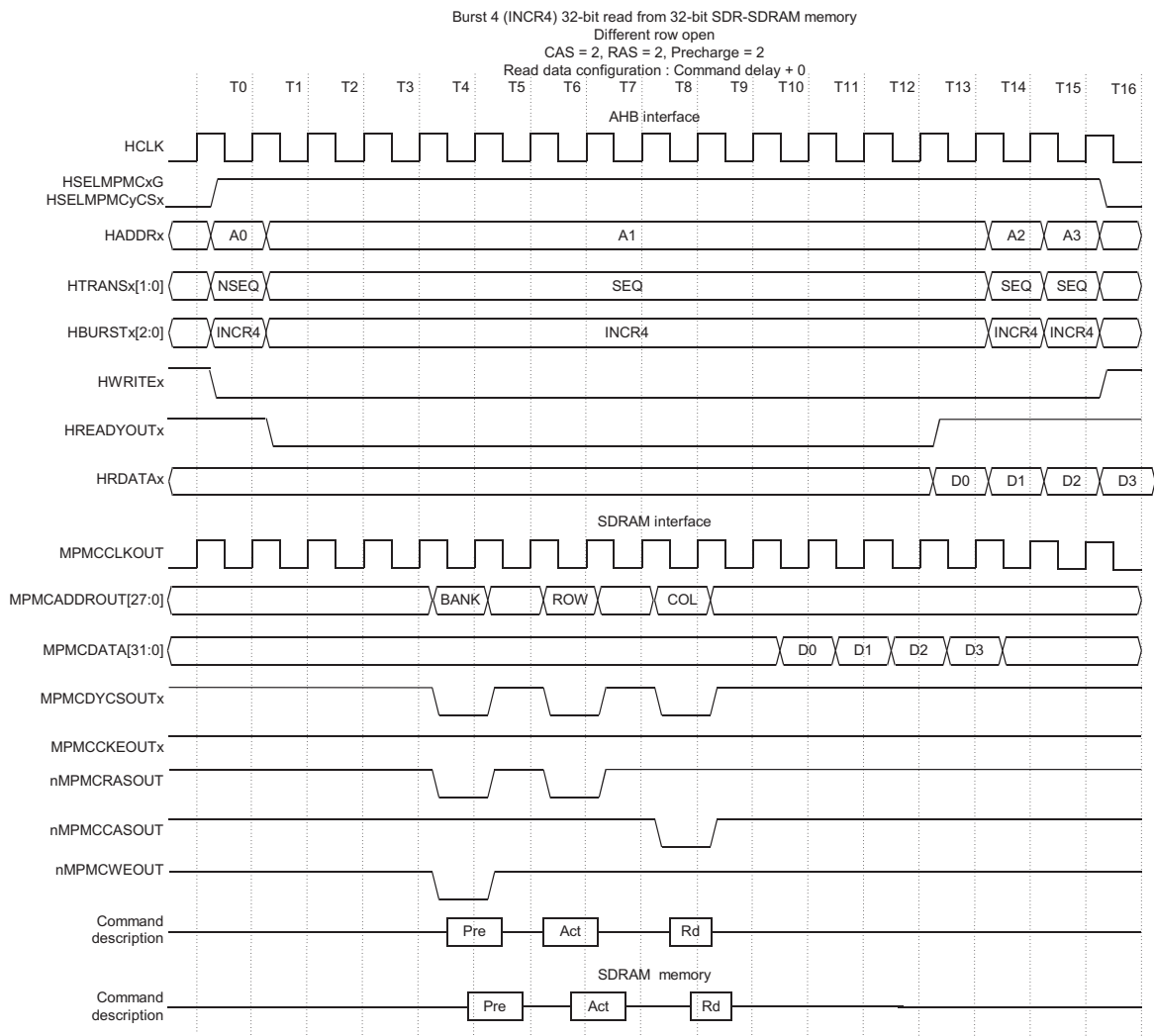


Figure 4-16 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 0 pad configuration

4.4.17 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 1 pad configuration

Figure 4-17 on page 4-69 and Table 4-21 show an example of a burst 4, 32-bit AHB read from 32-bit wide SDR-SDRAM. A different row from the one that is required to be read from is open. The command delay 1 pad configuration is used. Data is returned two cycles later in the command delayed-plus-one mode compared to the clock delay mode.

Table 4-21 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 1 pad configuration

Cycle	Description
T0	AHB address provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T4	Precharge command submitted to SDRAM.
T4-T5	SDRAM precharge command latency.
T5-T6	Active command submitted to SDRAM.
T6-T7	SDRAM active command latency.
T7-T8	Read command submitted to SDRAM.
T8-T9	Pad delay 1.
T9-T10	Pad delay 2.
T10-T11	SDRAM CAS access latency.
T11-T12	Read data 0 capture.
T12-T13	Read data 1 capture.
T13-T14	Read data 2 capture. Provide read data 0 to AHB.
T14-T15	Read data 3 capture. Provide read data 1 to AHB.
T15-T16	Provide read data 2 to AHB.
T16-T17	Provide read data 3 to AHB.

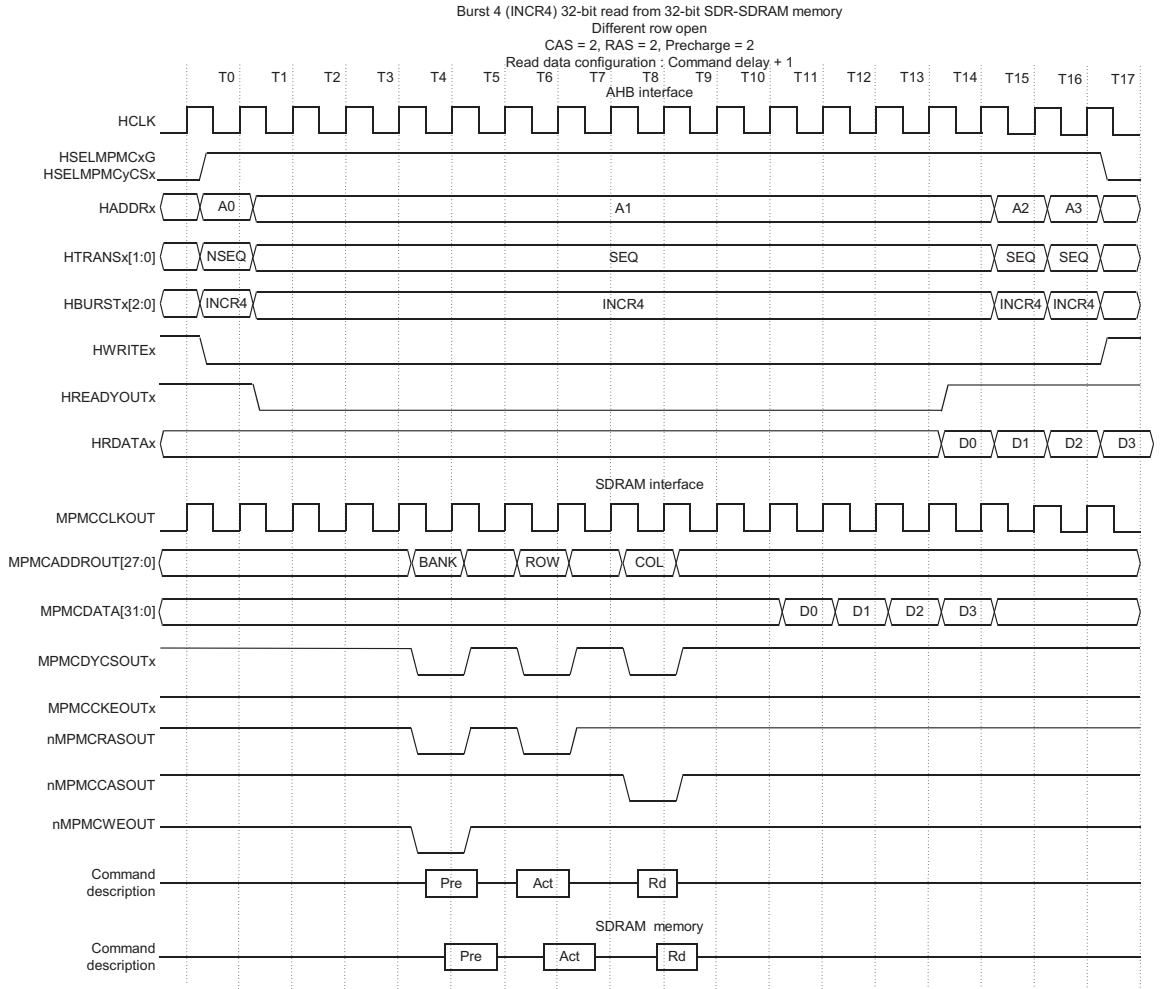


Figure 4-17 Burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, different row open, command delay 1 pad configuration

4.4.18 Two back-to-back burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open

Figure 4-18 on page 4-71 and Table 4-22 show an example of two back-to-back burst 4, 32-bit AHB read from 32-bit wide SDR-SDRAM. The row to be read from (in SDRAM) is open.

Table 4-22 Two back-to-back burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open

Cycle	Description
T0	AHB address 1 provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T4	Read command submitted to SDRAM.
T4-T5	SDRAM CAS access latency.
T5-T6	Read data 0 capture.
T6-T7	Read data 1 capture.
T7-T8	Read data 2 capture. Provide read data 0 to AHB.
T8-T9	Read data 3 capture. Provide read data 1 to AHB.
T9-T10	Provide read data 2 to AHB.
T10-T11	Provide read data 3 to AHB. AHB address 2 provided to memory controller.
T11-T12	AHB transaction processing.
T12-T13	Arbitration of AHB memory ports.
T13-T14	Memory controller processing.
T14-T15	Read command submitted to SDRAM.
T15-T16	SDRAM CAS access latency.
T16-T17	Read data 0 capture.
T17-T18	Read data 1 capture.
T18-T19	Read data 2 capture.
T19-T20	Read data 3 capture. Provide read data 0 to AHB.
T20-T21	Provide read data 1 to AHB.
T21-T22	Provide read data 2 to AHB.
T22-T23	Provide read data 3 to AHB.

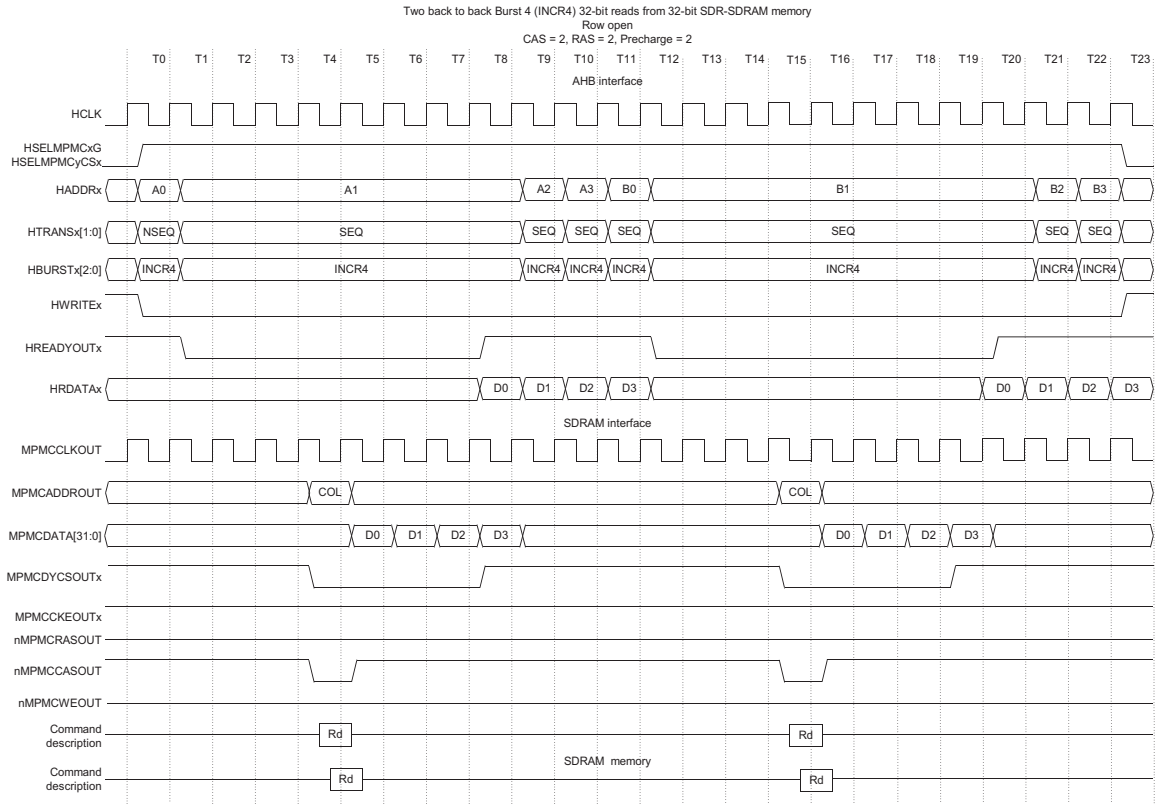


Figure 4-18 Two back-to-back burst 4 (INCR4) 32-bit read from 32-bit SDR-SDRAM, row open

4.4.19 Two back-to-back burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open

Figure 4-19 and Table 4-23 show an example of two back-to-back burst 4, 32-bit AHB write to 32-bit wide SDR-SDRAM. The row to be written to (in SDRAM) is open.

Table 4-23 Two back-to-back burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open

Cycle	Description
T0	AHB address 1 provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	AHB write data provided to memory controller. Arbitration of AHB memory ports.
T2-T8	AHB write data provided to memory controller.

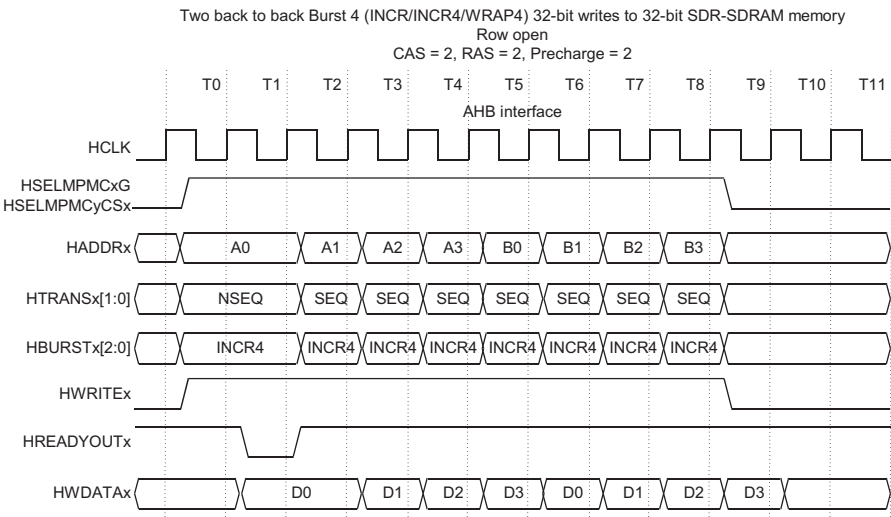


Figure 4-19 Two back-to-back burst 4 (INCR/INCR4/WRAP4) 32-bit write to 32-bit SDR-SDRAM, row open

4.4.20 Three back-to-back burst 4 (INCR4) 32-bit accesses from 32-bit SDR-SDRAM, row open

Figure 4-20 on page 4-74 and Table 4-24 show an example of three back-to-back burst 4, 32-bit AHB accesses from 32-bit wide SDR-SDRAM. The first burst 4 access is a read, the next is a write and this is followed by another read. The row to be read from (in SDRAM) is open.

Table 4-24 Three back-to-back burst 4 (INCR4) 32-bit accesses from 32-bit SDR-SDRAM, row open

Cycle	Description
T0	AHB address 1 provided to memory controller.
T0-T1	AHB transaction processing.
T1-T2	Arbitration of AHB memory ports.
T2-T3	Memory controller processing.
T3-T4	Read command submitted to SDRAM.
T4-T5	SDRAM CAS access latency.
T5-T6	SDRAM CAS access latency.
T6-T7	Read data 0 capture.
T7-T8	Read data 1 capture. Provide read data 0 to AHB.
T8-T9	Read data 2 capture. Provide read data 1 to AHB.
T9-T10	Read data 3 capture. Provide read data 2 to AHB.
T10-T11	Provide read data 3 to AHB. AHB address 2 provided to memory controller.
T11-T12	AHB transaction processing.
T12-T13	Arbitration of AHB memory ports.
T13-T14	Memory controller processing.
T14-T15	Write data 0 submitted to SDRAM. AHB address 3 provided to memory controller.
T15-T16	Write data 1 submitted to SDRAM.
T16-T17	Write data 2 submitted to SDRAM.
T17-T18	Write data 3 submitted to SDRAM.
T18-T19	Read command submitted to SDRAM.
T19-T20	SDRAM CAS access latency.
T20-T21	SDRAM CAS access latency.

Table 4-24 Three back-to-back burst 4 (INCR4) 32-bit accesses from 32-bit SDR-SDRAM, row open (continued)

Cycle	Description
T21-T22	Read data 0 capture.
T22-T23	Read data 1 capture. Provide read data 0 to AHB.
T23-T24	Read data 2 capture. Provide read data 1 to AHB.
T24-T25	Read data 3 capture. Provide read data 2 to AHB.
T25-T26	Provide read data 3 to AHB.

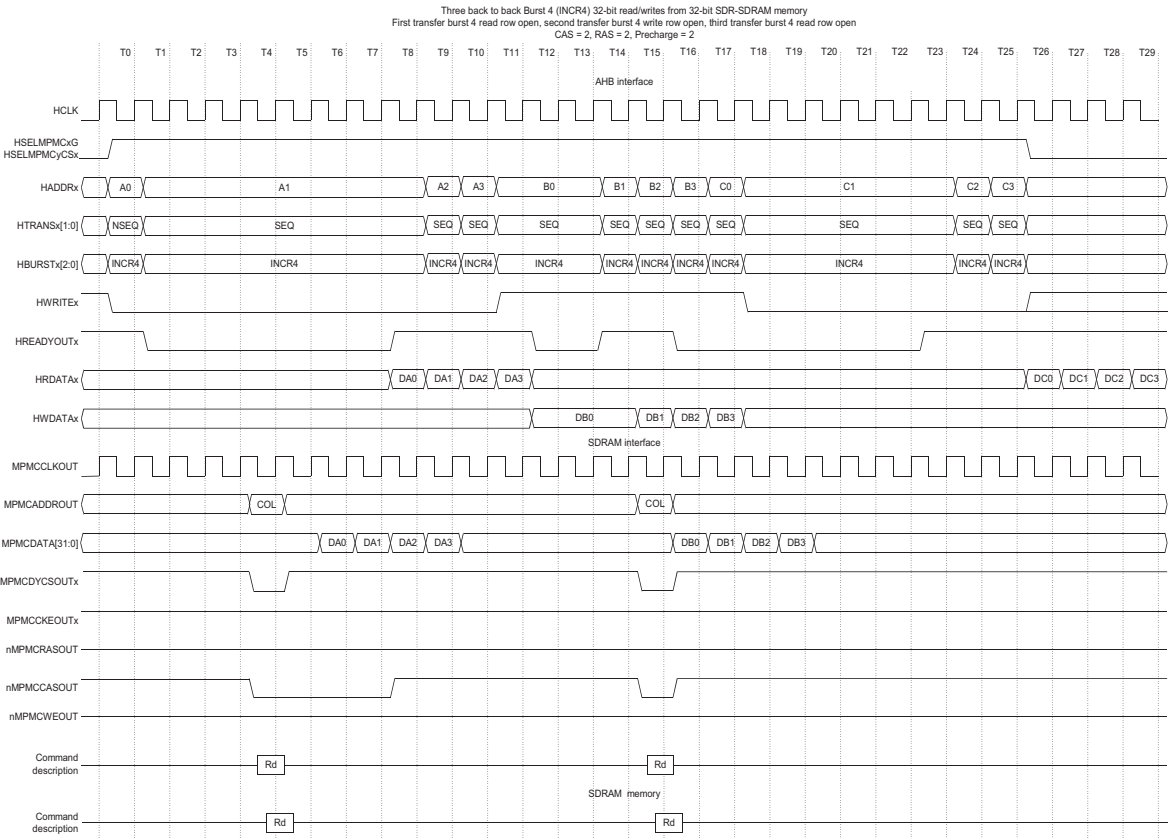


Figure 4-20 Three back-to-back burst 4 (INCR4) 32-bit accesses from 32-bit SDR-SDRAM, row open

4.5 Address mapping

The tables in this section provide the mapping of AHB address bus addresses to the external dynamic memory address **MPMCADDROUT[14:0]** for various memory configurations and bus widths. Select the address mapping by programming the *Address Mapping* (AM) bits in the **MPMCDynamicConfig[0-3]** Registers. The outputs from the MPMC tables indicate the address mapping out of the MPMC:

MPMC output address (MPMCADDROUT)

Indicates the address lines output from the MPMC.

AHB address to row address

Indicates the input **HADDR** address bits used from the AHB transfer for the row access.

AHB address to column address

Indicates the input **HADDR** address bits used from the AHB transfer for the column access.

Memory device connections

Indicates the device signals that must be connected to the MPMC AddrOut lines.

Note

For *32-bit wide data bus address mappings for RBC SDR-SDRAM* on page 4-76 to *16-bit wide data bus address mappings for BRC SDR-SDRAM and DDR-SDRAM* on page 4-78:

- ****** indicates that the bit is controlled by the SDRAM controller. The SDRAM controller always transfers 32-bits of data at a time. For chip selects with a 16-bit wide data bus, the SDRAM controller performs two transfers a column transfer with the lowest bit set to 0 and a column transfer with the lowest bit set to 1.
- **BA**, **BA0**, and **BA1** indicate the bank address signals (normally address bit 10). The MPMC does not use the Auto Precharge (AP) command.

Separate tables are provided for two different address mapping schemes, *Row, Bank, Column* (RBC), or *Bank, Row, Column* (BRC), and for 32-bit and 16-bit wide buses:

- *32-bit wide data bus address mappings for RBC SDR-SDRAM* on page 4-76. These address mappings are used for 32-bit data bus chip select with SDR-SDRAM memory devices. The row-bank-column address mapping scheme enables memory accesses to be efficiently performed to nearby memory regions.

- *32-bit wide data bus address mappings for BRC SDR-SDRAM* on page 4-77. These address mappings are used for 32-bit data bus chip select with SDR-SDRAM, low-power SDR-SDRAM, or Micron SyncFlash memory devices. The bank-row-column address mapping scheme enables the low-power SDR-SDRAM and Micron SyncFlash memory features to be used efficiently.
- *16-bit wide data bus address mappings for RBC SDR-SDRAM and DDR-SDRAM* on page 4-78. These address mappings are used for 16-bit data bus chip select with SDR-SDRAM memory devices. The row-bank-column address mapping scheme enables memory accesses to be efficiently performed to nearby memory regions.
- *16-bit wide data bus address mappings for BRC SDR-SDRAM and DDR-SDRAM* on page 4-78. These address mappings are used for 16-bit data bus chip select with SDR-SDRAM, low-power SDR-SDRAM, or Micron SyncFlash memory devices. The bank-row-column address mapping scheme enables the low-power SDR-SDRAM and Micron SyncFlash memory features to be used efficiently.

4.5.1 32-bit wide data bus address mappings for RBC SDR-SDRAM

Table 4-25 lists the address mapping for 32-bit wide data bus RBC SDR-SDRAM.

Table 4-25 Address mapping for 32-bit wide data bus RBC SDR-SDRAM

Memory type	Row MPMCADDRROUT [12:0]	Column MPMCADDRROUT [12:0]	BA1 MPMCADDRROUT [14]	BA0 MPMCADDRROUT [13]
16MB, 1MBx16	[21:11]	[9:2]	-	10
16MB, 2MBx8	[22:12]	[10:2]	11	-
64MB, 2MBx32	[22:12]	[9:2]	11	10
64MB, 4MBx16	[23:12]	[9:2]	11	10
64MB, 8MBx8	[24:13]	[10:2]	11	12
128MB, 4MBx32	[23:12]	[9:2]	11	10
128MB, 8MBx16	[24:13]	[10:2]	11	12
128MB, 16MBx8	[25:14]	[11:2]	13	12
256MB, 8MBx32	[24:12]	[9:2]	11	10
256MB, 16MBx16	[25:13]	[10:2]	11	12

Table 4-25 Address mapping for 32-bit wide data bus RBC SDR-SDRAM (continued)

Memory type	Row MPMCADDR0UT [12:0]	Column MPMCADDR0UT [12:0]	BA1 MPMCADDR0UT [14]	BA0 MPMCADDR0UT [13]
256MB, 32MBx8	[26:14]	[11:2]	13	12
512MB, 32MBx16	[26:14]	[11:2]	13	12
512MB, 64MBx8	[27:15]	[12:2]	13	14

4.5.2 32-bit wide data bus address mappings for BRC SDR-SDRAM

Table 4-26 lists the address mapping for 32-bit wide data bus BRC SDR-SDRAM.

Table 4-26 Address mapping for 32-bit wide data bus BRC SDR-SDRAM

Memory type	Row MPMCADDR0UT [12:0]	Column MPMCADDR0UT [12:0]	BA1 MPMCADDR0UT [14]	BA0 MPMCADDR0UT [13]
16MB, 1MBx16	[20:10]	[9:2]	21	-
16MB, 2MBx8	[21:11]	[10:2]	-	22
64MB, 2MBx32	[20:10]	[9:2]	21	22
64MB, 4MBx16	[21:10]	[9:2]	23	22
64MB, 8MBx8	[22:11]	[10:2]	23	24
128MB, 4MBx32	[21:10]	[9:2]	23	22
128MB, 8MBx16	[22:11]	[10:2]	23	24
128MB, 16MBx8	[23:12]	[11:2]	25	24
256MB, 8MBx32	[22:10]	[9:2]	23	24
256MB, 16MBx16	[23:11]	[10:2]	25	24
256MB, 32MBx8	[24:12]	[11:2]	25	26
512MB, 32MBx16	[24:12]	[11:2]	25	26
512MB, 64MBx8	[25:13]	[12:2]	27	26

4.5.3 16-bit wide data bus address mappings for RBC SDR-SDRAM and DDR-SDRAM

Table 4-27 lists the address mapping for 32-bit wide data bus RBC SDR-SDRAM and DDR-SDRAM.

Table 4-27 Address mapping for 16-bit wide data bus RBC SDR-SDRAM and DDR-SDRAM

Memory type	Row MPMCADDRROUT [12:0]	Column MPMCADDRROUT [12:0]	BA1 MPMCADDRROUT [14]	BA0 MPMCADDRROUT [13]
16MB, 1MBx16	[20:10]	[8:2] ^a	9	-
16MB, 2MBx8	[21:11]	[9:2] ^a	-	10
64MB, 4MBx16	[22:11]	[8:2] ^a	9	10
64MB, 8MBx8	[23:12]	[9:2] ^a	11	10
128MB, 8MBx16	[23:12]	[9:2] ^a	11	10
128MB, 16MBx8	[24:13]	[10:2] ^a	11	12
256MB, 16MBx16	[24:12]	[9:2] ^a	11	10
256MB, 32MBx8	[25:13]	[10:2] ^a	11	12
512MB, 32MBx16	[25:13]	[10:2] ^a	11	12
512MB, 64MBx8	[26:14]	[11:2] ^a	13	12

a. MPMCADDRROUT[1] and MPMCADDRROUT[0] are controlled by the SDRAM controller. The SDRAM controller always transfers 32 bits of data at a time. For chip selects with a 16-bit wide data bus the SDRAM controller performs two transfers, a column transfer with the lowest bit set to 0 and a column transfer with the lowest bit set to 1.

4.5.4 16-bit wide data bus address mappings for BRC SDR-SDRAM and DDR-SDRAM

Table 4-28 on page 4-79 lists the address mapping for 16-bit wide data bus BRC SDR-SDRAM and DDR-SDRAM.

Table 4-28 Address mapping for 16-bit wide data bus BRC SDR-SDRAM and DDR-SDRAM

Memory type	Row MPMCADDRROUT [12:0]	Column MPMCADDRROUT [12:0]	BA1 MPMCADDRROUT [14]	BA0 MPMCADDRROUT [13]
16MB, 1MBx16	[19:9]	[8:2] ^a	-	20
16MB, 2MBx8	[20:10]	[9:2] ^a	21	-
64MB, 4MBx16	[20:9]	[8:2] ^a	21	22
64MB, 8MBx8	[21:10]	[9:2] ^a	23	22
128MB, 8MBx16	[21:10]	[9:2] ^a	23	22
128MB, 16MBx8	[22:11]	[10:2] ^a	23	24
256MB, 16MBx16	[22:10]	[9:2] ^a	23	24
256MB, 32MBx8	[23:11]	[10:2] ^a	25	24
512MB, 32MBx16	[23:11]	[10:2] ^a	25	24
512MB, 64MBx8	[24:12]	[11:2] ^a	25	26

- a. **MPMCADDRROUT[1]** and **MPMCADDRROUT[0]** are controlled by the SDRAM controller. The SDRAM controller always transfers 32 bits of data at a time. For chip selects with a 16-bit wide data bus the SDRAM controller performs two transfers, a column transfer with the lowest bit set to 0 and a column transfer with the lowest bit set to 1.

4.6 Dynamic memory controller command descriptions

Table 4-29 and Table 4-30 show the memory commands supported by the dynamic memory controller block in the MPMC.

The commands listed in Table 4-29 are generated automatically.

Table 4-29 Synchronous memory commands used by MPMC

Mnemonic	Operation
ACT	Opens an SDRAM row
REF	CAS before RAS style refresh
SREF	Self-refresh
PRE	Precharge, closes a bank
RD	Read from an open row, row left open
WR	Write to an open row, row left open
RDA	Read followed by precharge
WRA	Write followed by precharge

The commands in Table 4-30 are generated under software control by programming the SDRAM initialization (I) and deep-sleep mode (DP) fields of the MPMCDynamicControl Register.

Table 4-30 Synchronous memory commands programmed by software

Mnemonic	Operation
MRS	Mode register set, programs SDRAM mode register
NOP	No operation, used during the SDRAM initialization sequence
PALL	Precharge all, used during the SDRAM initialization sequence
DSM	Deep sleep mode, for low-power SDRAM

4.7 Generic SDRAM initialization example

On **nPOR** software must initialize the MPMC and each of the dynamic memories connected to the controller. Check the dynamic memory data sheet for the start up procedure. A generic example initialization sequence is:

1. Wait 100ms after the power is applied and the clocks have stabilized.
2. Set the SDRAM Initialization (I) value to NOP in the MPMCDynamicControl Register. This automatically issues a NOP command to the SDRAM memories.
3. Wait 200ms.
4. Set the SDRAM Initialization (I) value to PALL in the MPMCDynamicControl Register. This automatically issues a precharge all instruction (PRE-ALL) to the SDRAM memories. This precharges all banks and places the device into the all banks idle state.
5. Perform a number of refresh cycles by writing 1 into the MPMCDynamicRefresh Register. This provides a memory refresh every 16 AHB clock cycles.
6. Wait until eight SDRAM refresh cycles have occurred (128 AHB clock cycles).
7. Program the operational value into the MPMCDynamicRefresh Register.
8. Program the operational value into the MPMCDynamicRasCas Register.
9. Program the operational values into the MPMCDynamicConfig Register. The buffers must be disabled during initialization.
10. Set the SDRAM initialization value (I) to MODE in the MPMCDynamicControl Register.
11. Program the SDRAM memories mode register. The mode register enables the following parameters (see Table 4-31 on page 4-82) to be programmed:
 - burst length
 - burst type
 - CAS latency
 - operating mode
 - write burst mode.

A read transaction from the SDRAM memory programs the mode register. The address of the transfer contains the value to be programmed. The mapping from AHB address bus, HADDR, to the SDRAM memories address lines depends on the address mapping value selected in the MPMCDynamicConfig Register. The row address bits contain the value to be programmed. The bank select signals BA0 and BA1 must both be 0 to program the mode register.

Table 4-31 Required MPMC SDRAM mode register settings

Field	Value
Burst length	4 for a 32-bit wide external data bus, or 8 for a 16-bit wide external data bus
Burst type	Sequential
CAS latency	Dependent on operating frequency
Operating mode	Standard operation
Write burst mode	Programmed burst length

Note

- The AHB memory port must be used to perform this transaction.
 - When initializing the memory device the appropriate chip select must be activated. Therefore, depending on the AHB decoder address map the address programmed might require modifying.
12. Set the SDRAM initialization value (I) to NORMAL in the MPMCDynamicControl Register.
 13. Enable the buffers in MPMCDynamicConfig Register. The SDRAM is now ready for normal operation.

4.8 Micron MT48LC4M16A2 SDRAM initialization example

Use this procedure to initialize two Micron MT48LC4M16A2 SDRAM (64MB, 4MBx16) devices, speed grade -8E, configured to provide a 32-bit bus. **HCLK** and **MPMCLK** are 100MHz:

1. Wait 100ms after the power is applied and the clocks have stabilized.
2. Set the SDRAM Initialization (I) value to NOP in the MPMCDynamicControl Register. This automatically issues a NOP to the SDRAM memories.
3. Set the SDRAM Initialization (I) value to PALL in the MPMCDynamicControl Register. This automatically issues a precharge all instruction (PRE-ALL) to the SDRAM memories. This precharges all the banks and places the device into the all banks idle state.
4. Perform a number of refresh cycles by writing 2 into the MPMCDynamicRefresh Register. This provides a memory refresh every 32 AHB clock cycles.
5. Wait until two SDRAM refresh cycles have occurred (64 AHB clock cycles).
6. Program the operational value into the MPMCDynamicRefresh Register. This device requires a memory refresh every 15.625 μ s. With a 100MHz **HCLK** the refresh register must be programmed with $(15.625\mu\text{s} \times 100\text{MHz})/16 = 97$.
7. Program the operational value into the MPMCDynamicRasCas Register. The -8E speed grade devices support CAS latency 2 at 100MHz operation. Therefore 0x0202 must be programmed into the register.
8. Program the operational values into the MPMCDynamicConfig Register. The buffers must be disabled during initialization. Table 4-32 shows how the fields must be set for this memory device.

Table 4-32 Field settings for Micron MT48LC4M16A2 SDRAM

Field	Value
Memory Device (MD)	SDRAM (00)
Address Mapping (AM)	32-bit bus, 64MB, 4Mx16 devices, RBC mapping (10000101)
Buffer Enable (B)	Disabled (0)
Write Protect (P)	Writes not protected (0)
Column Width (CW)	8 (010)
Number of Banks (NB)	Four banks (1)
Row Width (RW)	12 (01)

The value is 0x14804280.

———— **Note** ————

- The AHB memory port must be used to perform this transaction.
- When initializing the memory device the appropriate chip select must be activated. Therefore, depending on the AHB decoder address map the address programmed might require modifying.

9. Set the SDRAM Initialization (I) value to MODE in the MPMCDynamicControl Register.
10. Program the SDRAM memories mode register. The mode register enables the following parameters (see Table 4-33) to be programmed:
- burst length
 - burst type
 - CAS latency
 - operating mode
 - write burst mode.

A read transaction from the SDRAM memory programs the mode register. The address of the transfer contains the value to be programmed. The mapping from AHB address bus, HADDR, to the SDRAM memories address lines depends on the address mapping value selected in the MPMCDynamicConfig Register, in this case 32-bit, 64M, 4Mx16, RBC (10000101). The row address bits contain the value to be programmed. The bank select signals **BA0** and **BA1** must both be 0 to program the mode register.

Table 4-33 Required Micron SDRAM mode register settings

Field	Value	Micron device
Burst length	4 (for 32-bit data bus)	A[2:0]=010
Burst type	Sequential	A[3]=0
CAS latency	2 (for -8E device @ 100MHz)	A[6:4]=01
Operating mode	Standard operation	A[8:7]=0
Write burst mode	Programmed burst length	A[9]=0
Reserved (0)	0	A[11:10]=0

The value required to program the Micron SDRAM mode register is 0x22. The HADDR to SDRAM memory address mapping is 32-bit 64M SDRAM (4Mx16, RBC). *Address mapping* on page 4-75 shows the address mapping tables. The SDRAM memory row address bits are mapped to **HADDR[23:12]**. The SDRAM memory bank address bits are mapped to **HADDR[11:10]**. Therefore, the address to be accessed is 0x22000.

Note

- The AHB memory port must be used to perform this transaction.
- When initializing the memory device the appropriate chip select must be activated. Therefore, depending on the AHB decoder address map the address programmed might require modifying.

-
11. Set the SDRAM Initialization (I) value to NORMAL in the MPMCDynamicControl Register.
 12. Enable the buffers in MPMCDynamicConfig Register. The SDRAM is now ready for normal operation.

4.9 Low-power SDRAM initialization example

Use this procedure to initialize Infineon HYB/E 25L128160AC-A 128MB (8Mx16) devices configured to provide a 16-bit bus. **HCLK** and **MPMCLK** are 100MHz:

- 1. Wait 100ms after the power is applied and the clocks have stabilized.
- 2. Set the SDRAM Initialization (I) value to PALL in the MPMCDynamicControl Register. This automatically issues a precharge all instruction (PRE-ALL) to the SDRAM memories. This precharges all the banks and places the device into the all banks idle state.
- 3. Perform a number of refresh cycles by writing 2 into the MPMCDynamicRefresh Register. This provides a memory refresh every 32 AHB clock cycles.
- 4. Wait until eight SDRAM refresh cycles have occurred (256 AHB clock cycles).
- 5. Program the operational value into the MPMCDynamicRefresh Register. This device requires a memory refresh every 16μs. Therefore with a 100MHz **HCLK** the refresh register must be programmed with $(16\mu s \times 100MHz)/16 = 97$.
- 6. Program the operational value into the MPMCDynamicRasCas Register. The -8 speed grade devices support CAS latency 2 at 100MHz operation. Therefore 0x0202 must be programmed into the register.
- 7. Program the operational values into the MPMCDynamicConfig Register. The buffers must be disabled during initialization. Table 4-34 shows how the fields must be set for this memory device.

Table 4-34 Field settings for low-power SDRAM

Field	Value
Memory Device (MD)	Low-power SDRAM (01)
Address Mapping (AM)	16-bit bus, 128M, 8Mx16 devices, BRC mapping (00101001)
Buffer Enable (B)	Disabled (0)
Write Protect (P)	Writes not protected (0)
Column Width (CW)	9 (011)
Number of Banks (NB)	Four banks (1)
Row Width (RW)	12 (01)

The value is 0x14C01488.

Note

- The AHB memory port must be used to perform this transaction.
 - When initializing the memory device the appropriate chip select must be activated. Therefore, depending on the AHB decoder address map the address programmed might require modifying.
-

8. Set the SDRAM initialization value (I) to MODE in the MPMCDynamicControl Register.
9. Program the SDRAM memories mode register. The mode register enables the following parameters (see Table 4-35) to be programmed:
 - burst length
 - burst type
 - CAS latency
 - operating mode.

A read transaction from the SDRAM memory programs the mode register. The address of the transfer contains the value to be programmed. The mapping from AHB address bus, HADDR, to the SDRAM memories address lines depends on the address mapping value selected in the MPMCDynamicConfig Register, in this case 16-bit, 128M, 8Mx16, BRC. The row address bits contain the value to be programmed. The bank select signals **BA0** and **BA1** must both be 0 to program the mode register.

Table 4-35 SDRAM mode register settings for low-power SDRAM

Option	Value	Micron device
Burst length	8 (for 32-bit data bus)	A[2:0]=011
Burst type	Sequential	A[3]=0
CAS latency	2 (for -8E device @ 100MHz)	A[6:4]=010
Operating mode	Standard operation	A[11:7]=00000

Therefore, 0x23 must be programmed in the low-power SDRAM mode register. The HADDR to SDRAM memory address mapping is 16-bit 128M SDRAM (8Mx16, BRC). *Address mapping* on page 4-75 shows the address mapping tables. The SDRAM memory row address bits are mapped to **HADDR[21:10]**. The SDRAM memory bank address bits are mapped to **HADDR[23:22]**. The address to be accessed is therefore 0x08C00.

Note

- The AHB memory port must be used to perform this transaction.
- When initializing the memory device the appropriate chip select must be activated. Therefore, depending on the AHB decoder address map the address programmed might require modifying.

10. Program the low-power SDRAM memories extended mode register. The mode register enables the following parameters (see Table 4-36) to be programmed:
- temperature compensated self-refresh
 - partial array self-refresh.

The bank select signals **BA1** and **BA0** must be 1, 0 to select the extended mode register. A read transaction from the SDRAM memory programs the mode register. The address of the transfer contains the value to be programmed. The mapping from AHB address bus, HADDR, to the SDRAM memories address lines depends on the address mapping value selected in the MPMCDynamicConfig Register, in this case 16-bit, 128M, 8Mx16, BRC. The row address bits contain the value to be programmed.

Table 4-36 Required low-power SDRAM extended mode register settings

Option	Value	Micron device
Partial array self-refresh	All banks	A[2:0]=000
Temperature compensated self-refresh	70°C	A[4:3]=00

The value required to program the low-power SDRAM extended mode register is 0x00. The HADDR to SDRAM memory address mapping is 16-bit 128M SDRAM (8Mx16, BRC). *Address mapping* on page 4-75 shows the address mapping tables. The SDRAM memory row address bits are mapped to **HADDR[21:10]**. The SDRAM memory bank address bits are mapped to **HADDR[23:22]**. Therefore, the address to be accessed is 0x800000.

Note

- The AHB memory port must be used to perform this transaction.
- When initializing the memory device the appropriate chip select must be activated. Therefore, depending on the AHB decoder address map, the address programmed might require modifying.

11. Set the SDRAM initialization value (I) to NORMAL in the MPMCDynamicControl Register.

12. Enable the buffers in MPMCDynamicConfig Register. The SDRAM is now ready for normal operation.

4.10 Micron MT28F4M16S2 SyncFlash initialization example

Use this procedure to initialize Micron MT28S4M16LC SyncFlash (64MB, 4Mx16) devices configured to provide a 16-bit bus. **HCLK** and **MPMCLK** are 100MHz:

- 1. Wait for the power to be applied and the clocks have stabilized.
- 2. Set the RP value in the MPMCDynamicControl Register HIGH.
- 3. Wait 100µs.
- 4. Program the operational value into the MPMCDynamicRasCas Register. The -10 speed grade devices support CAS latency 3 at 100MHz operation. Therefore, 0x0303 must be programmed into the register.
- 5. Program the operational values into the MPMCDynamicConfig Register. The buffers must be disabled during initialization. Table 4-37 shows how the fields must be set for this memory device.

Table 4-37 Field settings for Micron MT28S4M16LC SyncFlash

Field	Value
Memory Device (MD)	Micron SyncFlash (10)
Address Mapping (AM)	16-bit bus, 64M, 4Mx16 devices, BRC mapping(00100101)
Buffer Enable (B	Disabled (0)
Write Protect (P)	Writes not protected (0)
Column Width (CW)	8 (010)
Number of Banks (NB)	Four banks (1)
Row Width (RW)	12 (01)

The value is 0x14801290.

Note

- The AHB memory port must be used to perform this transaction.
- When initializing the memory device the appropriate chip select must be activated. Therefore, depending on the AHB decoder address map the address programmed might require modifying.

- 6. Set the SDRAM Initialization (I) value to MODE in the MPMCDynamicControl Register.

7. Program the SyncFlash memories mode register. The mode register enables the following parameters (see Table 4-31 on page 4-82) to be programmed:
 - burst length
 - burst type
 - CAS latency
 - operating mode
 - write burst mode.

A read transaction from the SDRAM memory programs the mode register. The address of the transfer contains the value to be programmed. The mapping from AHB address bus, HADDR, to the SDRAM memories address lines depends on the address mapping value selected in the MPMCDynamicConfig Register, in this case 16-bit, 64M, 4Mx16, BRC. The row address bits contain the value to be programmed. The bank select signals **BA0** and **BA1** must both be 0 to program the mode register.

Table 4-38 SDRAM mode register settings for Micron SyncFlash SDRAM

Option	Value	Micron device
Burst length	8 (for 32-bit data bus)	A[2:0]=011
Burst type	Sequential	A[3]=0
CAS latency	3 (for -10 device @100MHz)	A[6:4]=011
Operating mode	Standard operation	A[8:7]=00
Write burst mode	Programmed burst length	A[9]=0
Reserved (0)	0	A[11:10]=00

Note

- The AHB memory port must be used to perform this transaction.
- When initializing the memory device the appropriate chip select must be activated. Therefore, depending on the AHB decoder address map the address programmed might require modifying.

8. The value required to program the SyncFlash memories mode register is 0x033. The HADDR to SDRAM memory address mapping is 16-bit 64M SDRAM (4Mx16, BRC). *Address mapping* on page 4-75 shows the address mapping tables. The SDRAM memory row address bits are mapped to **HADDR[20:9]**. The SDRAM memory bank address bits are mapped to **HADDR[22:21]**. Therefore, the address used must be 0x06600.

Note

- The AHB memory port must be used to perform this transaction.
 - When initializing the memory device the appropriate chip select must be activated. Therefore, depending on the AHB decoder address map the address programmed might require modifying.
-

9. Set the SDRAM Initialization (I) value to NORMAL in the MPMCDynamicControl Register.
10. Enable the buffers in MPMCDynamicConfig Register. The SDRAM is now ready for normal operation.

4.11 Micron SyncFlash commands

Table 4-39 shows the Micron SyncFlash commands supported by the MPMC.

Table 4-39 SyncFlash commands

SyncFlash operation	AHB access	HADDR	HWDATA
Read device configuration	Read	CMD1 = 1 CMD0 = 0 Other = row + CA address	N/A
Read status register	Read	CMD1 = 1 CMD0 = 1 Other = row address	N/A
Clear status register	Write	CMD1 = 1 Other = don't care	0xFFFF50XX (X = don't care)
Erase setup/confirm	Write	CMD1 = 1 Other = bank and row address	0xFFFF20D0 (X = don't care)
Program setup/program	Write	CMD1 = 0 Other = row, column and bank address	Din, the data to be written into the array
Protect block/confirm	Write	CMD1 = 1 Other = bank and row address	0xFFFF6001 (X = don't care)
Protect device/confirm	Write	CMD1 = 1 Other = bank address	0xFFFF60F1 (X = don't care)
Unprotect blocks/confirm	Write	CMD1 = 1 Other = bank address	0xFFFF60D0 (X = don't care)
Erase nvmode register	Write	CMD1 = 1 Other = bank address	0xFFFF30C0 (X = don't care)
Program nvmode register	Write	CMD1 = 1 Other = bank address	0xFFFFA0XX (X = don't care)

To execute the SyncFlash command set, read or write transfers must be performed while the address is set appropriately. The commands can be found in Table 4-39. CMD1 in the table indicates, HADDR, address bit 27, CMD0 indicates address bit 26. For example, to perform a SyncFlash Protect block or confirm command, a write transfer to the memory must be performed to an address where HADDR bit 27 is HIGH, and the rest of the address indicates the bank and row address.

Note

- The MPMC only supports 16-bit wide SyncFlash memory devices and a 16-bit wide chip select. All writes must be halfword writes and the data must be replicated on the upper and lower halfword lanes. See the *AMBA Specification (Rev 2.0)*.
- The buffers must be disabled while executing SyncFlash commands, and enabled for normal operation.

4.11.1 Micron SyncFlash programming example

This section describes how some Micron SyncFlash commands can be performed.

Reading the ID register of the manufacturer

To read the ID register of the manufacturer the read device configuration command must be performed. This command requires a read to be performed with the AHB address configured so that:

- the appropriate chip select is selected
- the CMD1 bit is HIGH, and the CMD0 bit is LOW
- the low order address bits are LOW to select the ID register of the manufacturer.

Chapter 5

Common Memory Transactions

This chapter describes common transactions for static and dynamic memory accesses. It contains the following sections:

- *Summary* on page 5-2
- *Write transfer to write-protected chip select* on page 5-3
- *Buffer hit transactions* on page 5-4
- *Buffer hit transaction waveforms* on page 5-6.

5.1 Summary

The values in this chapter assume CAS latency 2 SDRAM, where CAS, RAS, and Precharge (t_{RP}) are 2 clock cycles.

The latency heading shows the latency seen by the AHB interface, where the request is dealt with immediately.

5.2 Write transfer to write-protected chip select

Figure 5-1 shows a write transaction to a write-protected bank. The MPMC responds by placing an ERROR response on **HRESP[1:0]**.

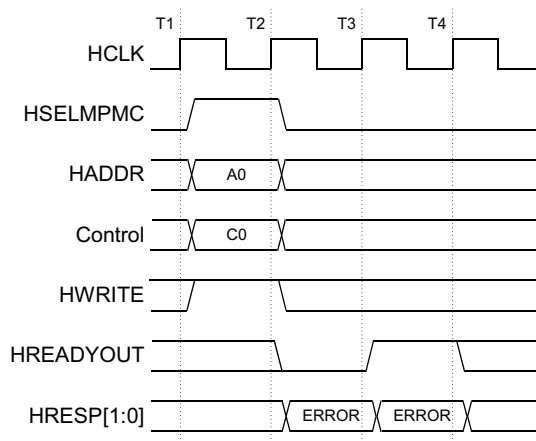


Figure 5-1 Write transfer to a write-protected chip select

5.3 Buffer hit transactions

This section provides the access times for read and write transfers that hit the buffers.

Both static and dynamic transfers that hit the buffer have the timings provided below.

The latency values provided show the latency seen by the AHB interface, where the request is dealt with immediately. However, for write accesses the data is written to memory later.

5.3.1 Read access timing

Nonsequential transfers (HTRANS=NONSEQ) have a latency of two cycles. Sequential transfers (HTRANS=SEQ), including burst transfers, have a transfer latency of one cycle if they hit a previously accessed buffer.

5.3.2 Write access timing

Nonsequential transfers (HTRANS=NONSEQ) have a latency of two cycles. Sequential transfers (HTRANS=SEQ) including burst transfers have a transfer latency of one cycle.

Table 5-1 shows the buffer access latency.

Table 5-1 Buffer access latency

Access type	Burst types	Buffer commands	Latency
Four-word burst read. All transfers hit the same data buffer.	INCR, INCR4, WRAP4	Read x 4	5
Three sequential read transfer. All transfers hit the same data buffer.	SINGLE	Read x 3	4
Three nonsequential reads.	SINGLE	Read x 3	6
Read.	SINGLE	Read x 1	2
Eight-word burst read.	INCR8, WRAP8	Read x 8	9
Two simultaneous four-word burst read transfers from different AHB ports.	INCR, INCR4, WRAP4	Read x 4 (x2)	10
Four-word burst write. All transfers hit the same data buffer.	INCR, INCR4, WRAP4	Write x 4	5

Table 5-1 Buffer access latency (continued)

Access type	Burst types	Buffer commands	Latency
Three sequential write transfer. All transfers hit the same data buffer.	SINGLE	Write x 3	4
Three nonsequential writes.	SINGLE	Write x 3	6
Write.	SINGLE	Write x 1	2
Eight-word burst write.	INCR8, WRAP8	Write x 8	9
Two simultaneous four-word burst write transfer from different AHB ports.	INCR, INCR4, WRAP4	Write x 4 (x2)	10

5.4 Buffer hit transaction waveforms

Figure 5-2 and Table 5-2 show a four word INCR burst, or INCR4, or WRAP4 burst read transfers that hit the buffer. The waveform shows the case where all transfers hit the same buffer.

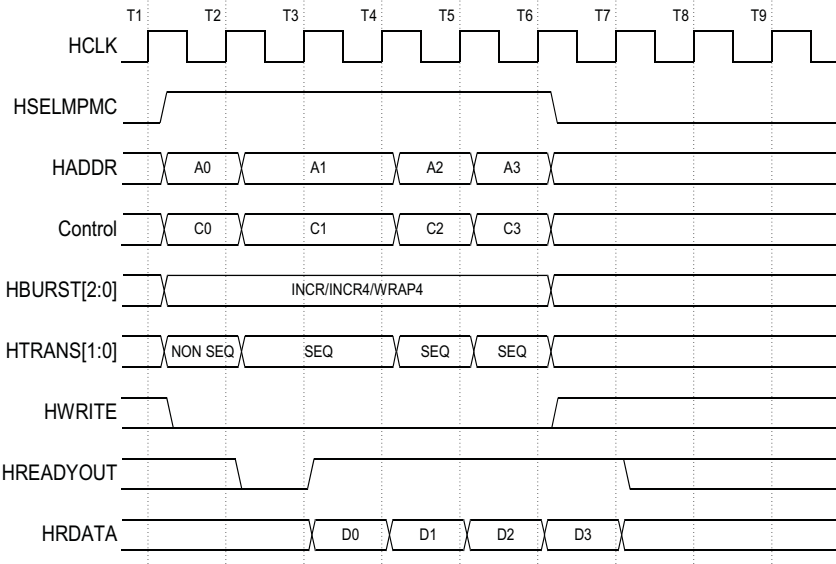


Figure 5-2 INCR, INCR4, or WRAP4 burst read transfer, buffer hit

Table 5-2 INCR, INCR4, or WRAP4 burst read transfer, buffer hit

Cycle	Description
T0	AHB address provided to memory controller
T0-T1	AHB transaction processing
T1-T2	Provide read data zero to AHB
T2-T3	Provide read data one to AHB
T3-T4	Provide read data two to AHB
T4-T5	Provide read data three to AHB

Figure 5-3 on page 5-7 and Table 5-3 on page 5-7 show three sequential reads, of burst type SINGLE from a buffer. The waveform shows the case where all transfers hit the same buffer.

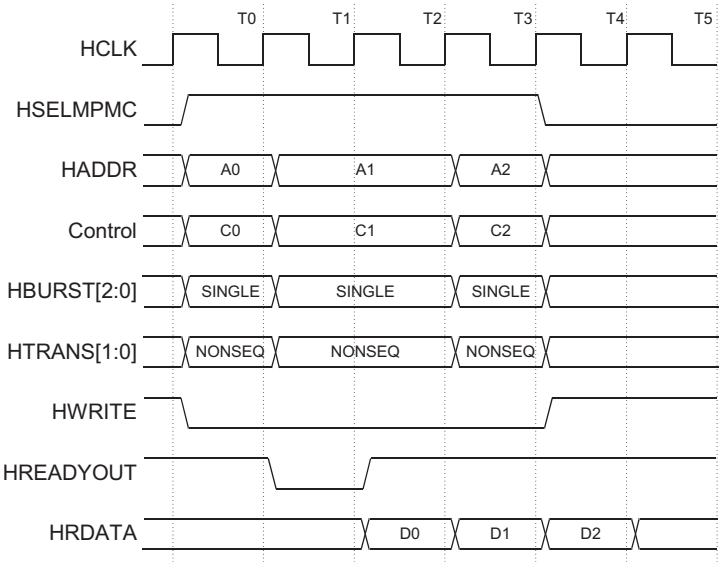


Figure 5-3 Three sequential reads, burst type SINGLE, read transfer, buffer hit

Table 5-3 Three sequential reads, burst type SINGLE, read transfer, buffer

Cycle	Description
T0	AHB address provided to memory controller
T0-T1	AHB transaction processing
T1-T2	Provide read data zero to AHB
T2-T3	Provide read data one to AHB
T3-T4	Provide read data two to AHB

Figure 5-4 on page 5-8 and Table 5-4 on page 5-8 show three nonsequential reads, of burst type SINGLE from a buffer.

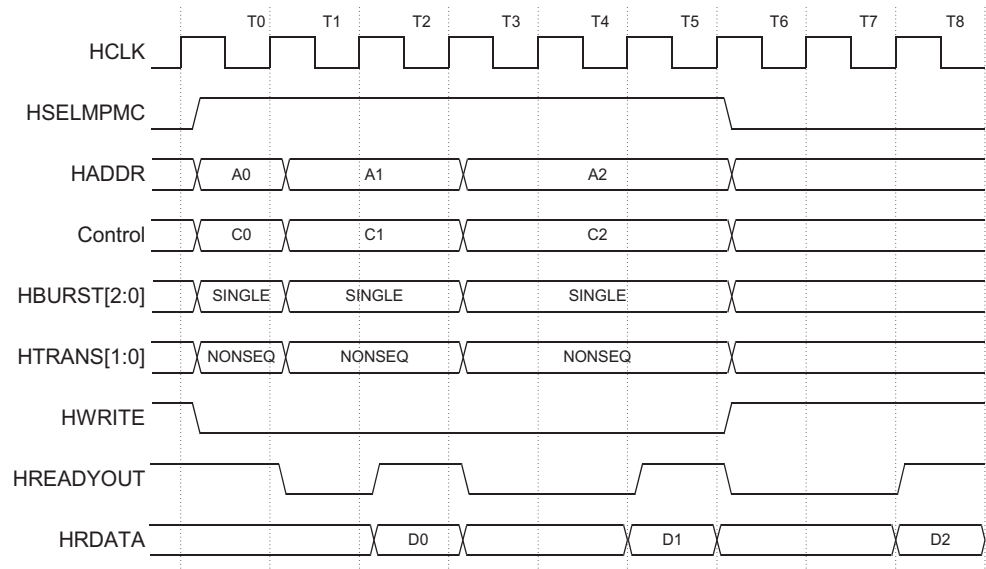
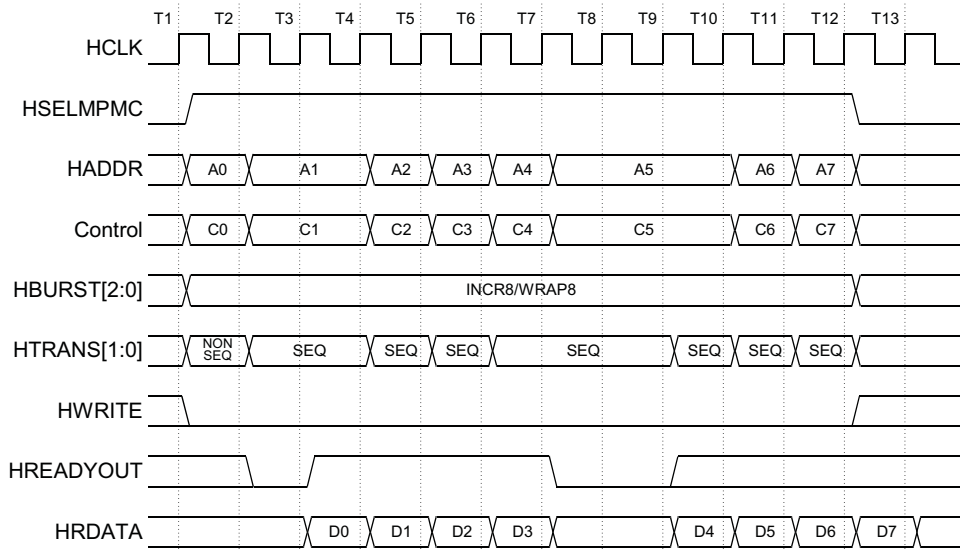


Figure 5-4 Three nonsequential reads, burst type SINGLE, read transfer, buffer hit

Table 5-4 Three nonsequential reads, burst type SINGLE, read transfer, buffer

Cycle	Description
T0	AHB address provided to memory controller
T0-T1	AHB transaction processing
T1-T2	Provide read data zero to AHB
T2-T3	AHB transaction processing
T3-T4	AHB transaction processing
T4-T5	Provide read data one to AHB
T5-T6	AHB transaction processing
T6-T7	AHB transaction processing
T7-T8	Provide read data two to AHB

Figure 5-5 on page 5-9 and Table 5-5 on page 5-9 show an INCR8, or WRAP8 burst read transfers that hit the buffer. The waveform shows the case where all transfers hit the same buffer.

**Figure 5-5 INCR8 or WRAP8 burst read transfer, buffer hit****Table 5-5 INCR8 or WRAP8 burst read transfer, buffer hit**

Cycle	Description
T0	AHB address provided to memory controller
T0-T1	AHB transaction processing
T1-T2	Provide read data zero to AHB
T2-T3	Provide read data one to AHB
T3-T4	Provide read data two to AHB
T4-T5	Provide read data three to AHB
T5-T6	Provide read data four to AHB
T6-T7	Provide read data five to AHB
T7-T8	Provide read data six to AHB
T8-T9	Provide read data seven to AHB

Figure 5-6 on page 5-10 and Table 5-6 on page 5-11 show two simultaneous INCR, INCR4, or WRAP4 burst read transfers that hit the buffer. The waveform shows the case where all transfers hit the same buffer.

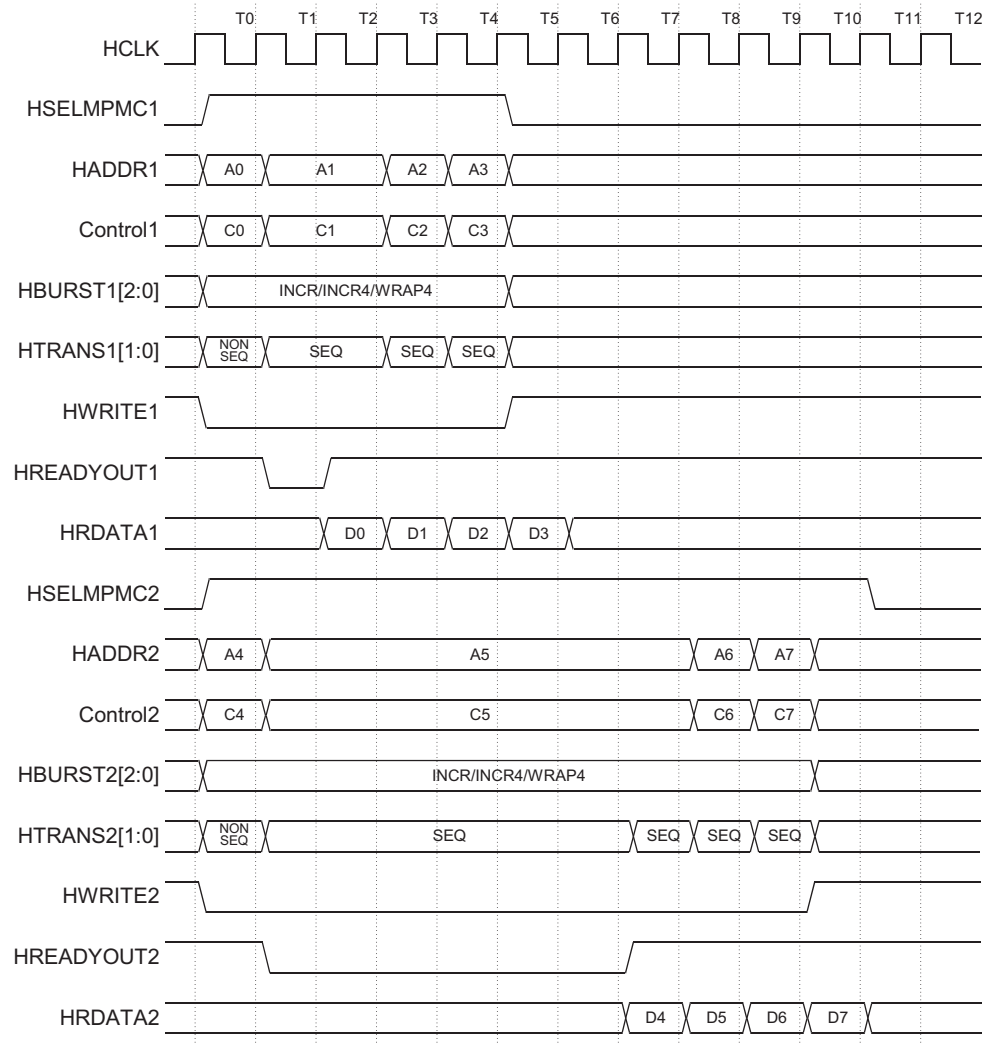


Figure 5-6 Two simultaneous INCR, INCR4, or WRAP4 burst read transfer, buffer hit

Table 5-6 Two simultaneous INCR, INCR4, or WRAP4 burst read transfer, buffer

Cycle	Description
T0	AHB port one address provided to memory controller. AHB port two address provided to memory controller.
T0-T1	AHB port one transaction processing.
T1-T2	Provide read data zero to AHB.
T2-T3	Provide read data one to AHB.
T3-T4	Provide read data two to AHB.
T4-T5	Provide read data three to AHB.
T5-T6	AHB port two transaction processing.
T6-T7	Provide read data four to AHB.
T7-T8	Provide read data five to AHB.
T8-T9	Provide read data six to AHB.
T9-T10	Provide read data seven to AHB.

Figure 5-7 on page 5-12 and Table 5-7 on page 5-12 show a four word INCR, INCR4, or WRAP4 burst write transfer to a buffer. The waveform shows the case where all transfers hit the same buffer. The data is written to memory later.

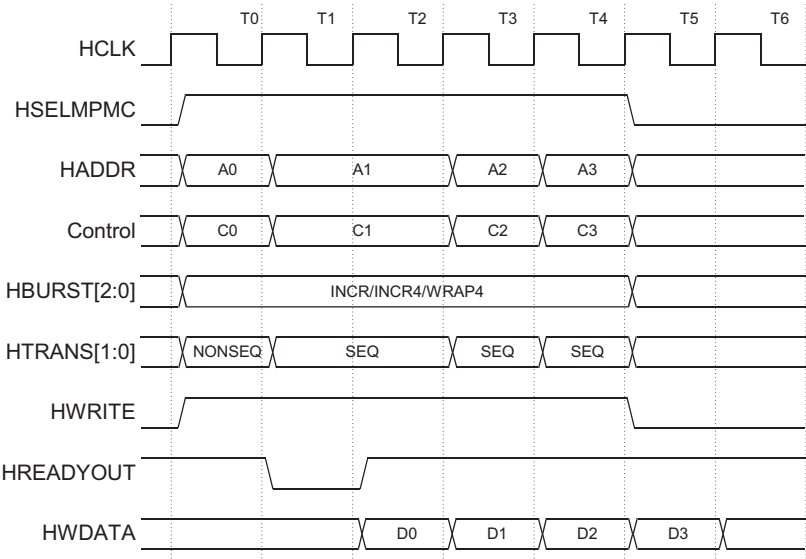


Figure 5-7 Four word INCR, INCR4, or WRAP4 burst write

Table 5-7 Four word INCR, INCR4, or WRAP4 burst write

Cycle	Description
T0	AHB address provided to memory controller
T0-T1	AHB transaction processing
T1-T2	Provide write data zero to AHB
T2-T3	Provide write data one to AHB
T3-T4	Provide write data two to AHB
T4-T5	Provide write data three to AHB

Figure 5-8 on page 5-13 and Table 5-8 on page 5-13 show three sequential write transfers to a buffer. The waveform shows the case where all transfers hit the same buffer. The data is written to memory later.

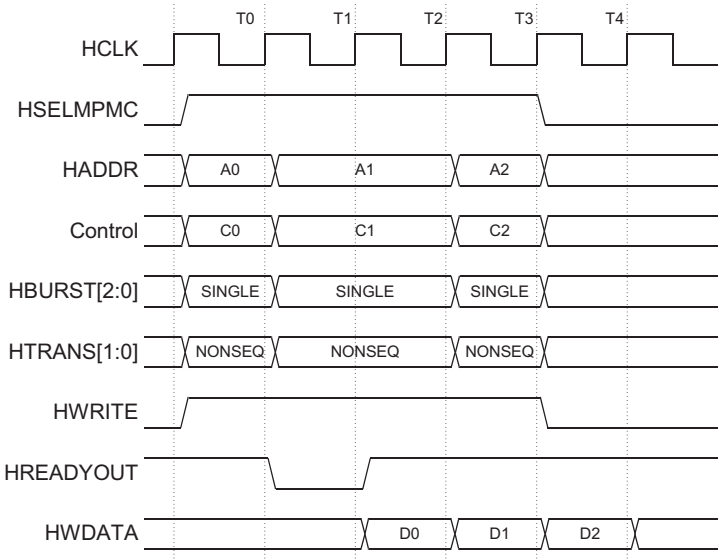


Figure 5-8 Three sequential write transfers

Table 5-8 Three sequential write transfers

Cycle	Description
T0	AHB address provided to memory controller
T0-T1	AHB transaction processing
T1-T2	Provide write data zero to AHB
T2-T3	Provide write data one to AHB
T3-T4	Provide write data two to AHB

Figure 5-9 on page 5-14 and Table 5-9 on page 5-14 show three nonsequential write transfers to a buffer. The data is written to memory later.

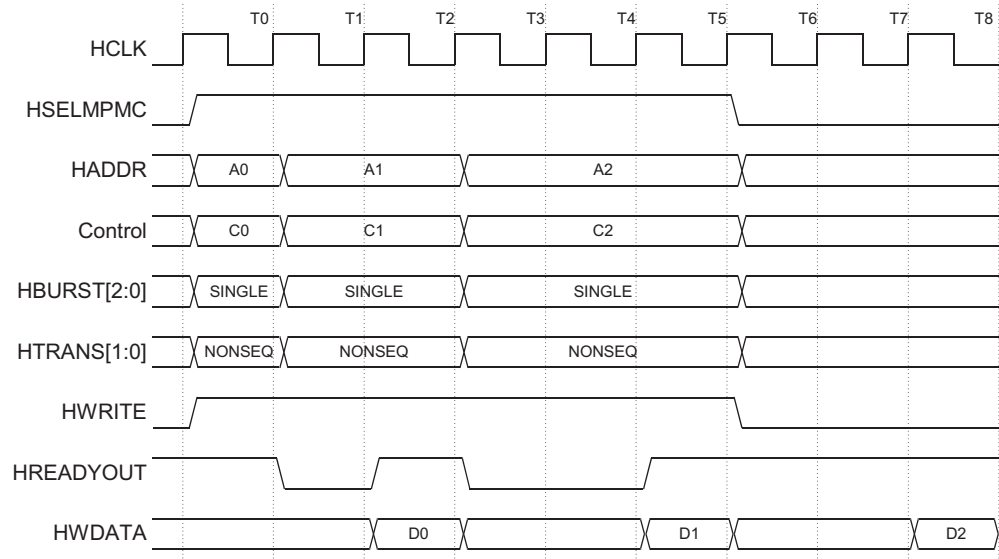


Figure 5-9 Three nonsequential write transfers

Table 5-9 Three nonsequential write transfers

Cycle	Description
T0	AHB address provided to memory controller
T0-T1	AHB transaction processing
T1-T2	Provide write data zero to AHB
T2-T3	AHB transaction processing
T3-T4	AHB transaction processing
T4-T5	Provide write data one to AHB
T5-T6	AHB transaction processing
T6-T7	AHB transaction processing
T7-T8	Provide write data two to AHB

Figure 5-10 on page 5-15 and Table 5-10 on page 5-15 show an INCR8 or WRAP8 burst write transfer to a buffer. The waveform shows the case where all transfers hit the same buffer. The data is written to memory later.

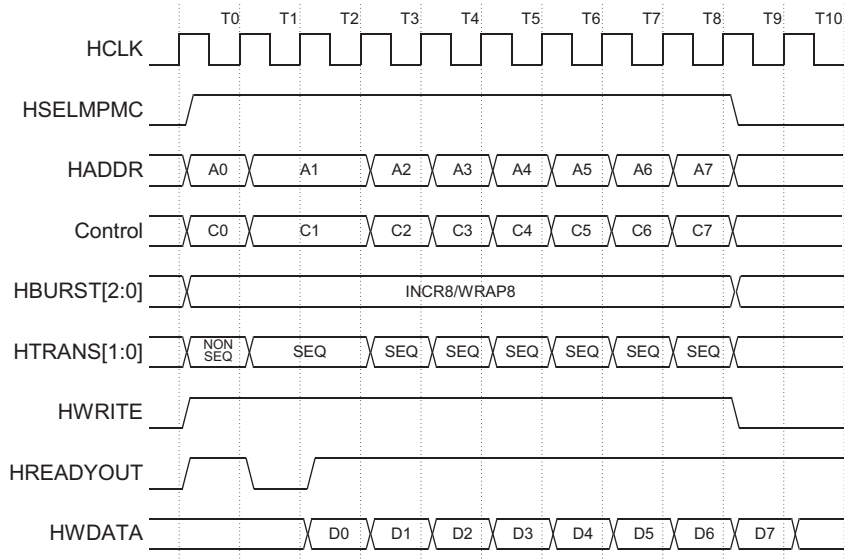


Figure 5-10 INCR8 or WRAP8 burst write

Table 5-10 INCR8 or WRAP8 burst write

Cycle	Description
T0	AHB address provided to memory controller
T0-T1	AHB transaction processing
T1-T2	Provide write data zero to AHB
T2-T3	Provide write data one to AHB
T3-T4	Provide write data two to AHB.
T4-T5	Provide write data three to AHB
T5-T6	Provide write data four to AHB
T6-T7	Provide write data five to AHB
T7-T8	Provide write data six to AHB
T8-T9	Provide write data seven to AHB

Chapter 6

Test Interface Controller

This chapter describes the *Test Interface Controller* (TIC). It contains the following sections:

- *About TIC* on page 6-2
- *Sequence of events leading to entry into TIC test mode* on page 6-3.

6.1 About TIC

The TIC converts externally applied test vectors into internal transfers on the TIC AHB bus. A three-wire external handshake protocol is used, with two inputs, **MPMCTESTREQA** and **MPMCTESTREQB**, controlling the type of vector that is applied and a single output, **MPMCTESTACK**, that indicates when the next vector can be applied. The **MPMCTESTIN** input is used to indicate that TIC test mode must be entered. Typically, the TIC is the highest priority AMBA bus master, ensuring test access under all conditions. TIC testing requires a 32-bit data bus. Therefore, if TIC test support is required for systems with a 16-bit **MPMCDATA** data bus, the upper 16 bits of the data bus, **MPMCDATA[31:16]**, must be multiplexed onto other pads during test mode.

Note

- For more information on TIC testing, see the *AMBA Design Kit Technical Reference Manual*.
 - The TIC is used for production testing of some hard macro ARM processors.
 - **MPMCTESTIN** is used by the MPMC to enter test mode. The system can use this signal to switch **HCLK** to test speed.
 - The TIC must not be the default master because the TIC only operates at the test speed. The TIC might not function correctly if granted the bus at normal operating frequency.
-

6.2 Sequence of events leading to entry into TIC test mode

The following description assumes that, in a typical system, the processor is always requesting access to the external bus, but the TIC has the highest priority to bus access. A typical sequence of events in using the TIC to apply test patterns is:

1. Apply reset asynchronously.
2. Assert the **MPMCTESTIN** test input to enter test mode.
3. When the reset is removed asynchronously, the MPMC enters TIC test mode.
4. The TIC block asserts its **HBUSREQTIC** signal to the arbiter, requesting access to the AHB bus. Because the TIC is the highest priority, **HGRANTTIC** is asserted and the granted TIC takes ownership of the AHB bus.

Chapter 7

System Connectivity

This chapter describes the system connectivity aspects relating to the MPMC. It contains the following sections:

- *On-chip signals* on page 7-2
- *Self-refresh entry* on page 7-24
- *Example system* on page 7-28.

7.1 On-chip signals

The on-chip signals are described in the following sections:

- *AHB register interface connectivity*
- *AHB memory interface connectivity* on page 7-4
- *TIC test connectivity* on page 7-4
- *Pad interface connectivity* on page 7-4
- *Reset controller connectivity* on page 7-10
- *Boot methodologies* on page 7-12
- *EBI connectivity* on page 7-15
- *Endianness* on page 7-19
- *Static memory configuration* on page 7-19
- *Methodologies in setting configuration tie-offs* on page 7-21.

7.1.1 AHB register interface connectivity

The AHB register interface must be connected to the AHB bus with the microprocessor so that the microprocessor can program the MPMC.

Figure 7-1 on page 7-3 shows the MPMC system interconnection diagram.

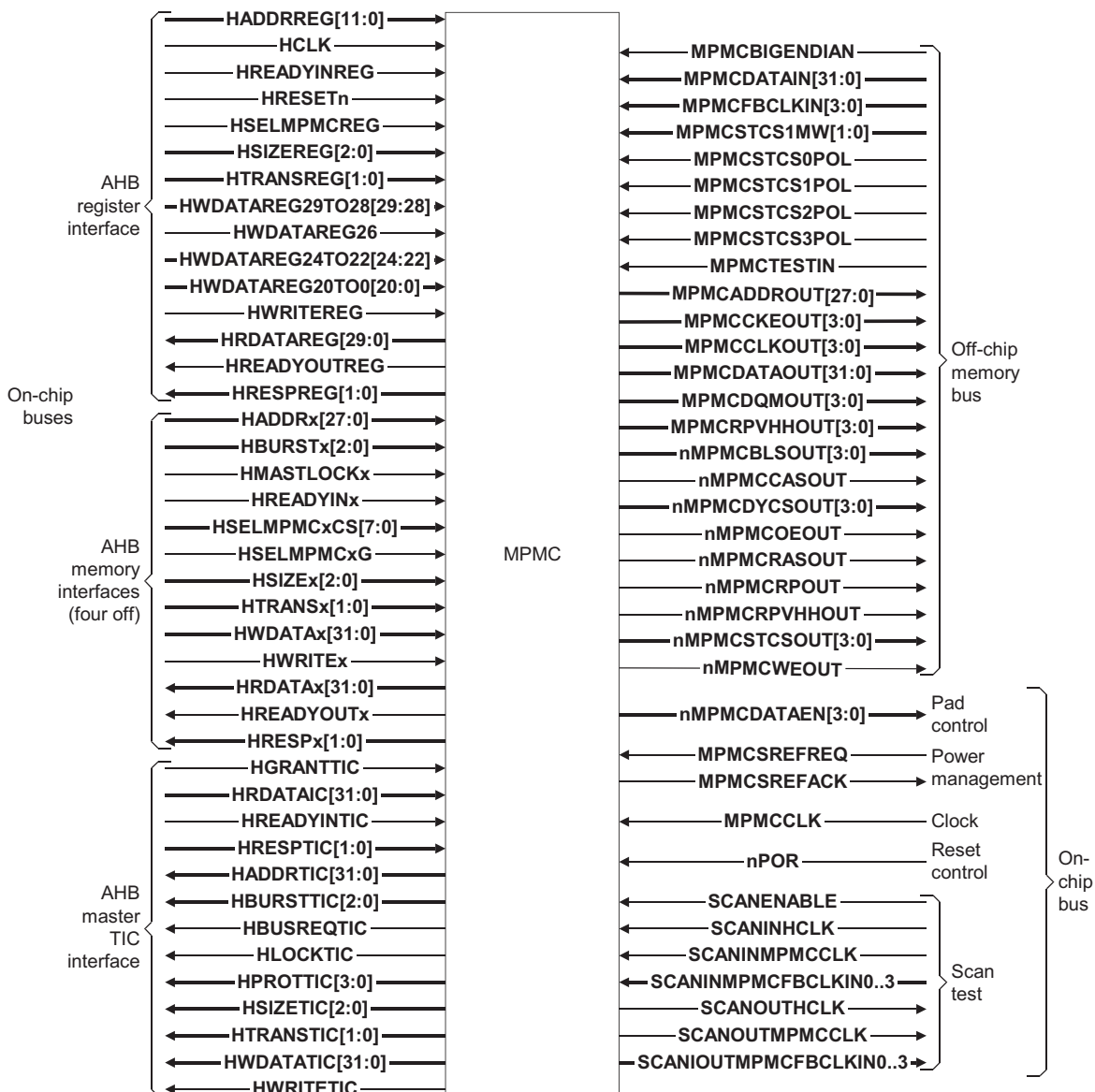


Figure 7-1 MPMC system interconnection diagram

7.1.2 AHB memory interface connectivity

AHB memory interface 0 is the highest priority interface and is normally connected to the AHB bus with high-bandwidth, low-latency AHB masters. AHB memory interface 1 is the next highest priority interface. Priorities having greater numbers are lower priority. The lowest priority is system-dependent. Any unused AHB interfaces must have their AHB input signals tied LOW.

7.1.3 TIC test connectivity

The TIC AHB master interface, if required, must be connected to the same bus as the microprocessor. The TIC must be the highest priority master on this bus. The **MPMCTESTIN** signal must be asserted during reset to enter TIC test mode. The **MPMCTESTIN** signal must also be sent to the clock generation logic so that the appropriate test clock is selected.

———— **Note** ————

- The TIC must not be the default master. Because the TIC master only operates at test speed, it might not function correctly if granted the bus at functional speed.
- The TIC must be granted the bus in test mode. This is normally performed by making the TIC the highest priority master on the AHB bus.

7.1.4 Pad interface connectivity

The following sections describe the pad interface connectivity for:

- *Static memory*
- *Dynamic memory*
- *TIC* on page 7-5.

Static memory

The static memory output signals are generated using **HCLK**, and then combined with the dynamic memory and TIC output signals. See Figure 7-3 on page 7-7 and Figure 7-5 on page 7-9.

The static memory input signals are registered by **HCLK**. See Figure 7-2 on page 7-6.

Dynamic memory

The SDR-SDRAM memory output signals are generated either using **MPMCCLK** in the clock delayed pad interface methodology, or by **MPMCCLKDELAY** clock using the command delayed pad interface methodology, see *Pad interface methodology* on

page 8-13. The output signals are then combined with the static memory and TIC output signals. See Figure 7-3 on page 7-7, Figure 7-4 on page 7-8, and Figure 7-5 on page 7-9.

The SDR-SDRAM memory input signals are first registered by the feedback clocks, **MPMCFBCLKIN[3:0]**. The read data is then captured onto the positive edge of **HCLK**. See Figure 7-2 on page 7-6

TIC

The TIC output signals are generated using **HCLK**, and then combined with the static memory and dynamic controller output signals. See Figure 7-3 on page 7-7.

The TIC input signals are registered by **HCLK**. See Figure 7-2 on page 7-6.

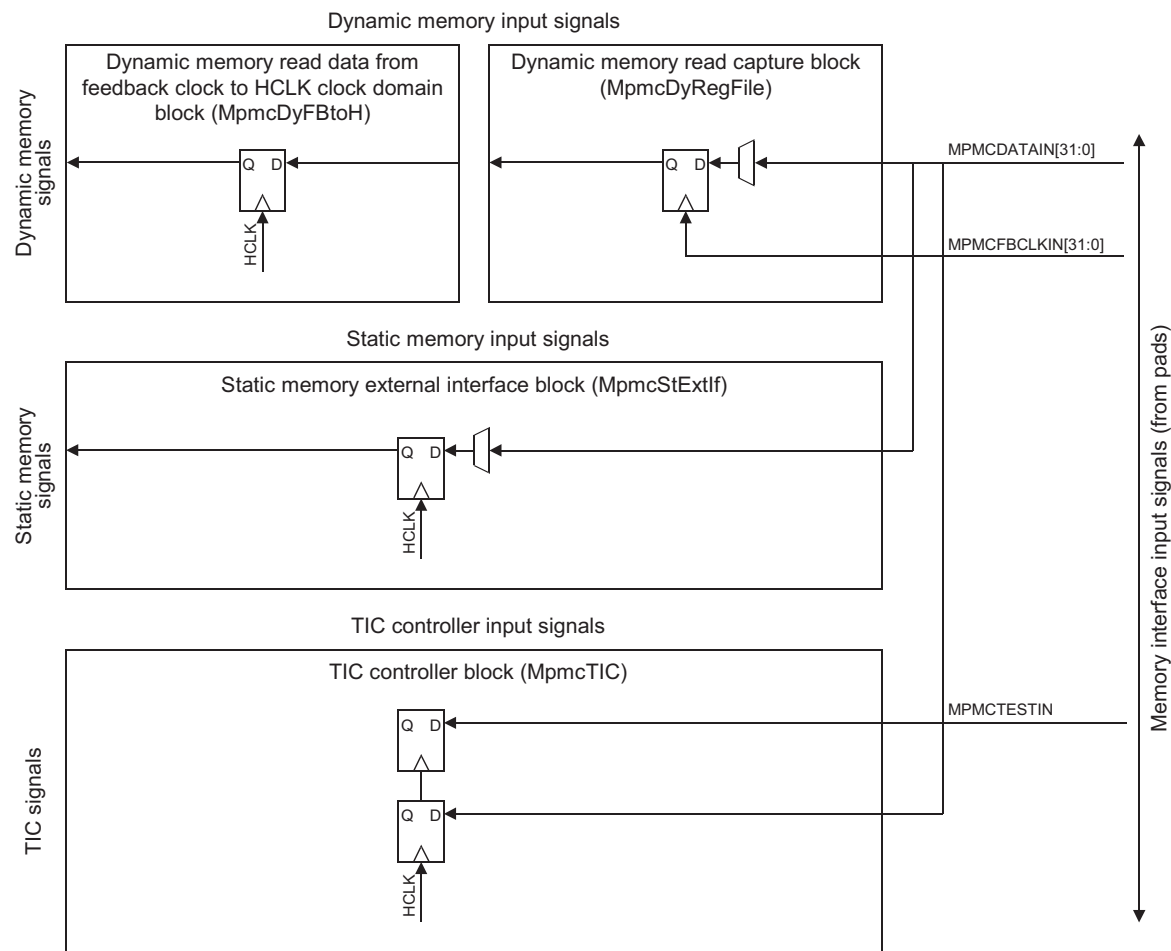


Figure 7-2 Pad interface input signals

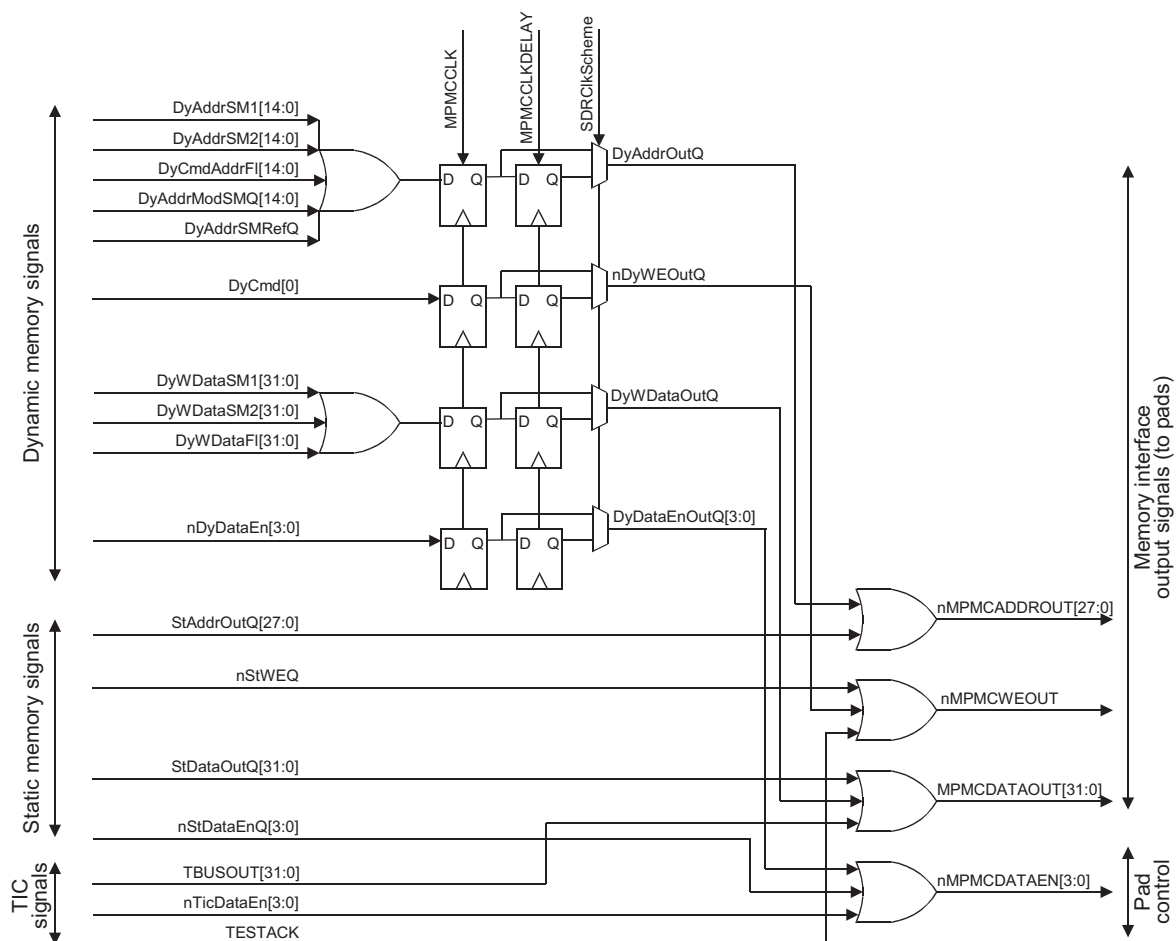


Figure 7-3 Pad interface output signals

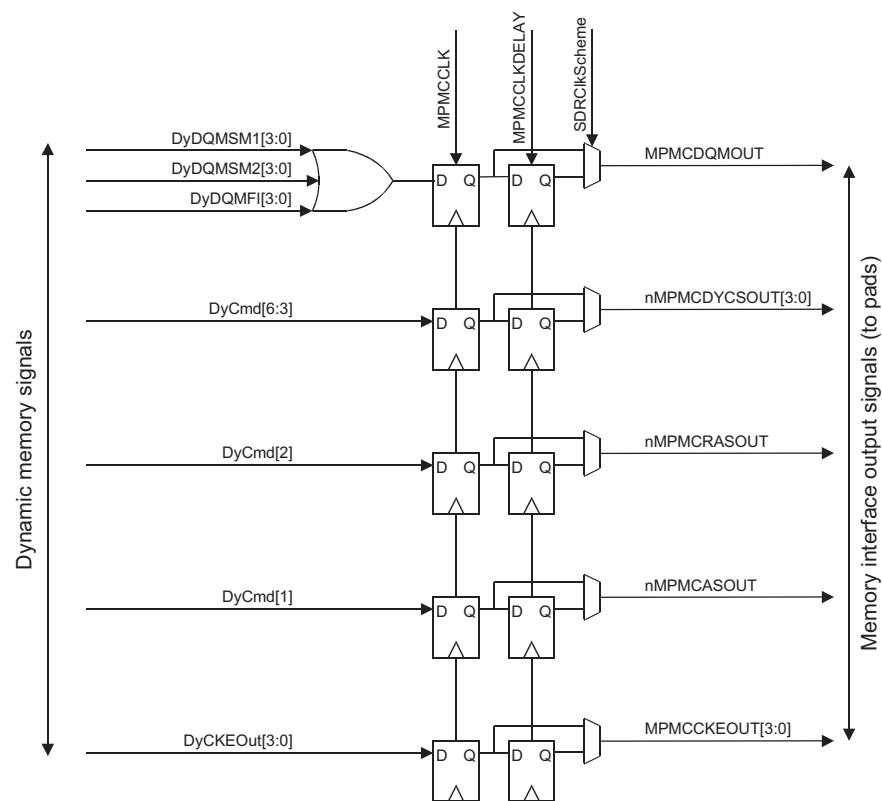


Figure 7-4 Pad interface output signals

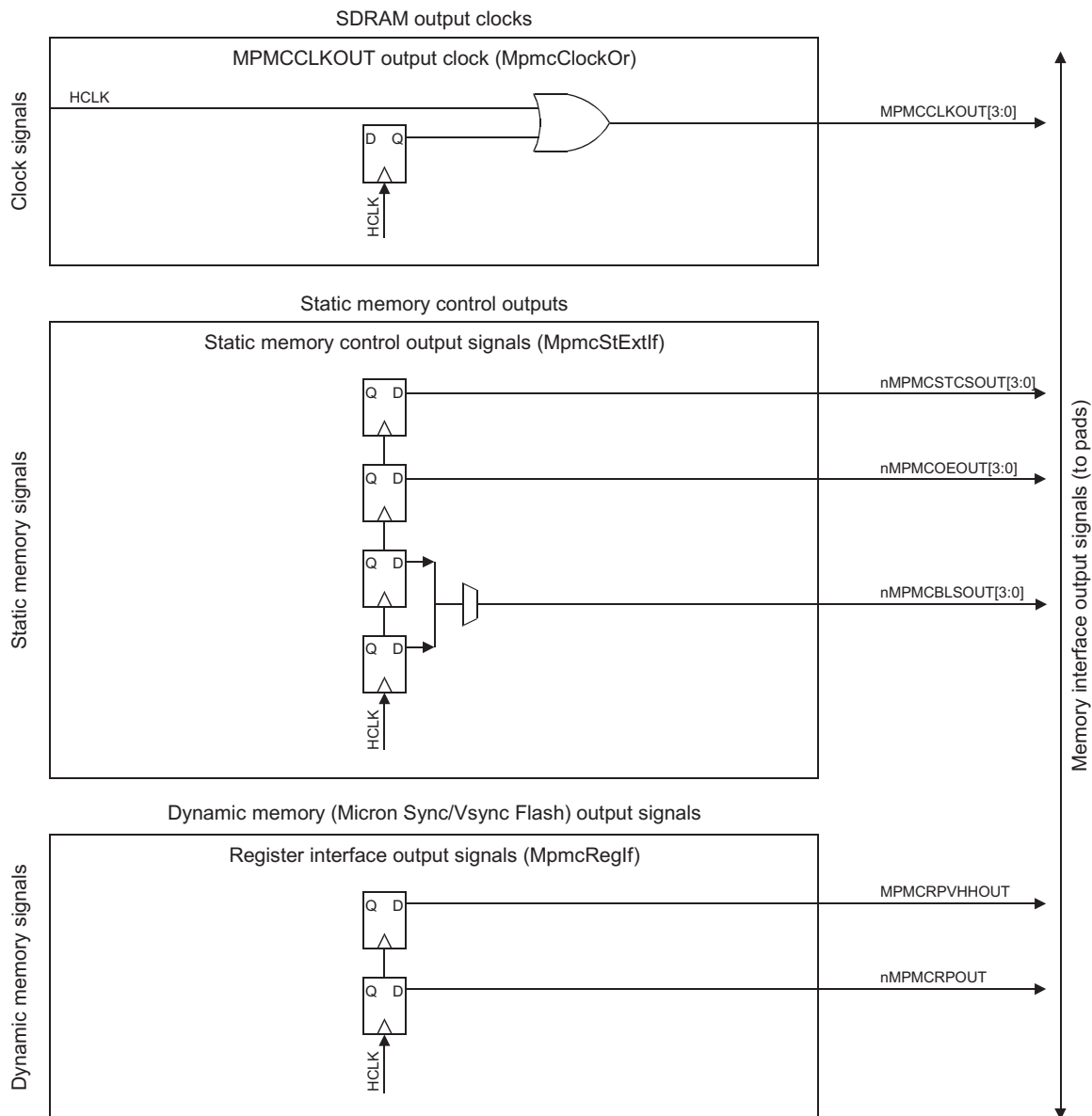


Figure 7-5 Pad interface output signals

7.1.5 Reset controller connectivity

There are two reset signals:

- power-on reset, **nPOR**
- soft reset, **HRESETn**.

Both **nPOR** and **HRESETn** are active LOW reset signals. Both signals can be asynchronously asserted, but must be synchronously deasserted with respect to **HCLK** and **MPMCCLK**:

- On power-on reset, both **nPOR** and **HRESETn** must go active.
- On soft reset, **HRESETn** must go active.

Having two reset signals enables the system to be booted using soft reset if the system is in a low-power state, and the SDRAM is in self-refresh mode.

Note

PrimeCell designs use asynchronous reset. The minimum amount of time the reset signals must be enabled depends on the cell library.

nPOR reset signal

nPOR resets the majority of the logic in the MPMC. If power-on reset is performed, the MPMC registers must be initialized.

HRESETn reset signal

HRESETn sets the MPMCControl Register, Enable (E) field, ensuring that the MPMC is enabled and can be booted from. Any outstanding memory requests and memory transactions in progress are lost when **HRESETn** is asserted.

HRESETn does not reset any of the other programmer-visible registers. The MPMC does not have to be re-initialized over soft reset.

For more information on the effects of the reset signals, see Figure 7-6 on page 7-11 and Table 7-1 on page 7-11.

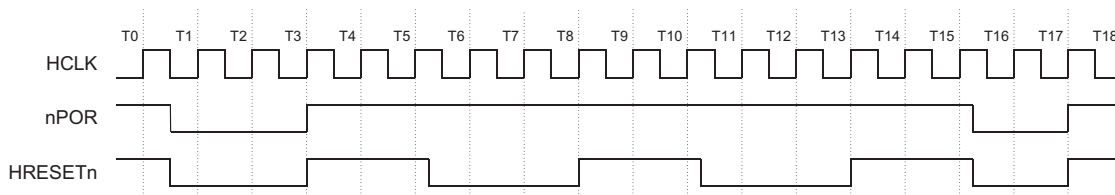


Figure 7-6 Reset signal timing

Table 7-1 Reset signal cycles

Cycle	Description
T0-T1	Power-on reset is asserted asynchronously. Power-on reset requires both nPOR and HRESETn to be asserted.
T1-T2	Power-on reset active.
T2-T3	Power-on reset active.
T3-T4	Power-on reset is deasserted synchronously. Power-on reset exit requires both nPOR and HRESETn to be deasserted.
T4-T5	Normal operation.
T5-T6	Soft reset, HRESETn , is asserted asynchronously.
T6-T7	Soft reset active.
T7-T8	Soft reset active.
T8-T9	Soft reset is deasserted synchronously.
T9-T10	Normal operation.
T10-T11	Soft reset, HRESETn , is asserted asynchronously.
T11-T12	Soft reset active.
T12-T13	Soft reset active.
T13-T14	Soft reset is deasserted synchronously.
T14-T15	Normal operation.
T15-T16	Power-on reset is asserted asynchronously. Power-on reset requires both nPOR and HRESETn to be asserted.
T16-T17	Power-on reset active.
T17-T18	Power-on reset is deasserted synchronously. Power-on reset exit requires both nPOR and HRESETn to be deasserted.

Systems with hard reset only

Systems with hard reset only must assert both **nPOR** and **HRESETn** on reset. All the register values in the MPMC are then lost on reset. The MPMC must be re-initialized on exiting reset. See *Boot methodologies* for more information on restarting the system using power-on reset during self-refresh.

7.1.6 Boot methodologies

The following boot methodologies are available:

- *System first powered up*
- *Soft reset performed on self-refresh exit*
- *Hard reset performed on self-refresh exit on page 7-13*
- *Restarting the system when it has locked up using soft reset, possible data loss on page 7-13*
- *Restarting the system when it has locked up using hard reset, possible data loss on page 7-14*
- *Restarting the system when it has locked up using soft reset, retaining SDRAM data on page 7-15.*

System first powered up

The following procedure can be used when the system is first powered up:

1. Both **nPOR** and **HRESETn** must go active.
2. Both **nPOR** and **HRESETn** must go inactive synchronously to the clock.
3. The MPMC must be initialized.
4. The memory devices must be initialized.

Soft reset performed on self-refresh exit

The following procedure can be used when a soft reset is performed on a self-refresh exit:

1. SDRAM self-refresh mode entered.
2. Power management unit disables clocks to save power.
3. Power management unit restarts system by performing a soft reset, asserting and then deasserting **HRESETn**.
4. On deassertion of **HRESETn** the MPMC can be used.

Note

The MPMC and memory devices do not have to be initialized.

5. The CPU reads a system status bit to determine the mode of the system.
6. Self-refresh mode is exited by:
 - performing a memory transaction
 - using the self-refresh hardware signals
 - using the software register fields.

Hard reset performed on self-refresh exit

The following procedure can be used when a hard reset is performed on a self-refresh exit:

1. SDRAM self-refresh mode entered.
2. Power management unit disables clocks to save power.
3. Power management unit powers off part of the ASIC, including the MPMC, to save additional power. The MPMC register contents are lost.
4. Power management unit restarts the system by re-enabling power.
5. The power management unit performs a hard reset, asserting both **nPOR** and **HRESETn**, and then deasserting both reset signals.

Note

The MPMC must be initialized before accessing dynamic memory. The memory devices do not have to be initialized.

6. The CPU reads a system status bit to determine the mode of the system.
7. The CPU initializes the MPMC.
8. Self-refresh mode is exited, by using the software register fields.

Restarting the system when it has locked up using soft reset, possible data loss

The following soft reset procedure can be used to restart the system when it has locked up:

1. Power management unit performs a soft reset, by asserting **HRESETn**.

Note

During reset the MPMC does not perform auto-refresh commands and the memory contents of SDRAM might be corrupted.

2. Power management unit exits soft reset, by deasserting **HRESETn**.
3. On deassertion of **HRESETn** the MPMC can be used.

Note

The MPMC and memory devices do not have to be initialized.

4. The CPU reads a system status bit to determine the mode of the system.

Note

If **HRESETn** is active for a short enough period of time, the contents of the SDRAM memory might not be corrupted.

Restarting the system when it has locked up using hard reset, possible data loss

The following hard reset procedure can be used to restart the system when it has locked up:

1. Power management unit performs a hard reset, by asserting both **nPOR** and **HRESETn**.

Note

During reset the MPMC does not perform auto-refresh commands and the memory contents of SDRAM might be corrupted.

2. The power management unit exits hard reset, by deasserting **nPOR** and **HRESETn**.
3. On deassertion of **HRESETn** the MPMC can be used.

Note

The MPMC must be initialized before accessing dynamic memory. Specifically the auto-refresh counter must be re-initialized as early as possible to avoid SDRAM data loss. The memory devices do not have to be initialized.

4. The CPU reads a system status bit to determine the mode of the system.

5. The CPU initializes the MPMC.

———— **Note** ————

If **HRESETn** is active for a short enough period of time, the contents of the SDRAM memory might not be corrupted.

—————

Restarting the system when it has locked up using soft reset, retaining SDRAM data

The following soft reset procedure can be used to restart the system when it has locked up:

1. Power management unit forces the MPMC into self-refresh mode.
2. Power management unit performs a soft reset, by asserting **HRESETn**.
3. Power management unit exits soft reset, by deasserting **HRESETn**.
4. On deassertion of **HRESETn** the MPMC can be used.

———— **Note** ————

The MPMC and memory devices do not have to be initialized.

—————

5. The CPU reads a system status bit to determine the mode of the system.
6. Self-refresh mode is exited by one of the following:
 - performing a memory transaction
 - using the self-refresh hardware signals
 - using the software register fields.

7.1.7 EBI connectivity

The memory interface signals can be shared with other peripherals by using an *External Bus Interface* (EBI) block. The EBI block enables a number of peripherals to share the same pads. The EBI contains arbitration logic to decide which peripheral drives the external bus.

For more information on the EBI, see the *ARM PrimeCell External Bus Interface* (PL220) Technical Reference Manual.

Note

The MPMC does not perform any commands or memory transactions unless it has been granted the bus by the EBI.

EBI connectivity is described in the following sections:

- *Pin sharing*
- *EBI signals*
- *EBI connectivity options*
- *Example EBI interface transaction on page 7-17.*

Pin sharing

The EBI enables the address and data signals to be shared. The MPMC control signals are not shared and must be directly connected to the pads.

EBI signals

The MPMC provides the following signals to interface to the EBI:

MPMCEBIREQ

This signal is an output to the EBI and requests the external bus.

MPMCEBIGNT

This signal is an input from the EBI and indicates that the EBI has granted the MPMC the bus.

MPMCEBIBACKOFF

This signal is an input from the EBI and indicates that the MPMC must complete its current transaction and no longer request the bus, so that another peripheral can be granted the bus.

EBI connectivity options

There are two options available for EBI connectivity:

EBI required

If the use of the EBI is required, the MPMC EBI interface signals, **MPMCEBIREQ**, **MPMCEBIGNT**, and **MPMCBACKOFF** must be connected to the EBI.

The address signals, **MPMCADDRROUT[27:0]** and data signals, **MPMCDATAOUT[31:0]** and **MPMCDATAIN[31:0]**, must also be connected to the EBI (see Figure 7-7 on page 7-17).

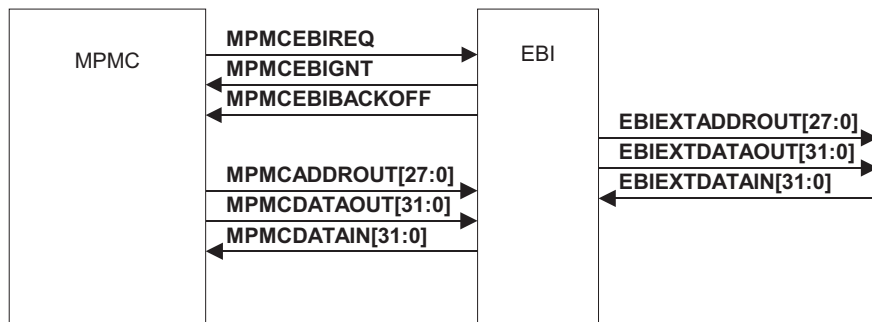


Figure 7-7 EBI connectivity, EBI required

EBI not required

If the EBI is not required, the **MPMCEBIGNT** signal must be tied HIGH, indicating that the bus is always granted. The **MPMCEBIBACKOFF** signal must be tied LOW, indicating that the MPMC must not back off from the bus. The **MPMCEBIREQ** signal can be left unconnected (see Figure 7-8).

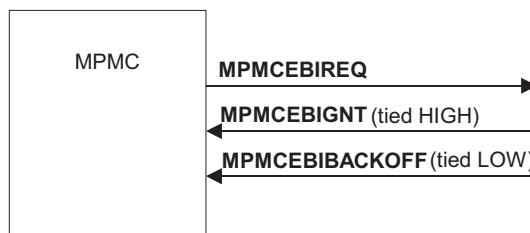


Figure 7-8 EBI connectivity, EBI not required

Example EBI interface transaction

Figure 7-9 on page 7-18 and Table 7-2 on page 7-18 show example EBI signal cycles.

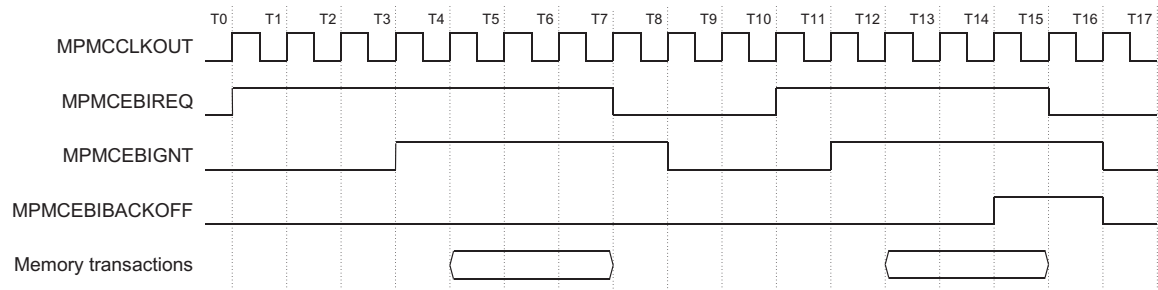


Figure 7-9 EBI signal timing

Table 7-2 EBI signal cycles

Cycle	Description
T0-T1	The MPMC requests the external bus, by asserting the MPMCEBIREQ signal.
T1-T2	MPMC idle waiting to be granted the bus.
T2-T3	MPMC idle waiting to be granted the bus.
T3-T4	The EBI grants the MPMC the external bus.
T4-T5	The MPMC performs transactions.
T5-T6	The MPMC performs transactions.
T6-T7	The MPMC performs transactions.
T7-T8	The MPMC no longer requires the bus, and deasserts the MPMCEBIREQ signal.
T8-T9	The EBI degrades the memory bus by deasserting the MPMCEBIGNT signal.
T9-T10	MPMC idle.
T10-T11	The MPMC requests the external bus by asserting the MPMCEBIREQ signal.
T11-T12	The EBI grants the MPMC the external bus.
T12-T13	The MPMC performs transactions.
T13-T14	The MPMC performs transactions.

Table 7-2 EBI signal cycles

Cycle	Description
T14-T15	The MPMC performs transactions. The EBI wants to re-arbitrate the memory bus to enable another peripheral to perform transactions, and raises the MPMCBACKOFF signal.
T15-T16	The memory deasserts the MPMCCEBIREQ signal as soon as it is able to.
T16-T17	The EBI de-grants the memory bus by deasserting the MPMCCEBIGNT signal.

7.1.8 Endianness

The endianness of the MPMC can be controlled on power-on reset by the **MPMCBIGENDIAN** tie-off. The value of this tie-off must not be changed while the MPMC is in normal operation. The power-on reset value of this signal can be determined by reading the endian mode (N) field of the MPMCCConfig Register. The tie-off value can be overridden by writing to the endian mode (N) field by software. The register then shows the updated value.

Table 7-3 shows the tie-off signal.

Table 7-3 MPMCCConfig Register

Configuration input signal	MPMCCConfig register field	Configuration option
MPMCBIGENDIAN	Endian mode (N)	Set the endian mode: 0 = little endian mode 1 = big endian mode.

———— Note ————

If this tie-off is not required the signal must be tied LOW.

7.1.9 Static memory configuration

The static memory chip selects can be configured on power-on reset by using tie-off signals.

Booting from static memory

The MPMC can be configured to boot from static memory.

Static memory chip select 1 contains additional functionality enabling it to be booted from. This chip select to be configured on power-on reset by setting a number of tie-offs. The value of these tie-offs must not be changed during normal operation.

The tie-offs enable the following to be configured:

- polarity of the chip select
- polarity of the byte lane strobe
- memory width.

The power-on reset value of these signals can be determined by reading the appropriate fields in the MPMCStaticConfig1 Register. The tie-off values can be overridden by writing to the appropriate register field fields. The register then shows the updated value.

Table 7-4 shows the tie-off signals.

Table 7-4 MPMCStaticConfig1 Register

Configuration input signal	MPMCStaticConfig register field	Configuration option
MPMCSTCS1MW[1:0]	Memory width (MW)	Chip select 1 data bus width: 00 = 8-bit 01 = 16-bit 10 = 32-bit 11 = reserved.
MPMCSTCS1PB	Byte lane state (PB)	Chip select 1 byte lane state: 0 = nMPCSBSL SOUT[3:0] signals are HIGH for reads and LOW for writes 1 = nMPMCBSL SOUT[3:0] signals are LOW for reads and LOW for writes.
MPMCSTCS1POL	Chip select polarity (PC)	Chip select 1 chip select polarity: 0 = active LOW chip select 1 = active HIGH chip select.

———— Note —————

If these tie-offs are not required these signals must be tied LOW.

Configuring chip selects 0, 2, and 3 on power-on reset

Chip selects 0, 2, and 3 chip select polarity can be configured by the use of tie-offs to minimize the risk of bus clash. The value of these tie-offs must not be changed while the MPMC is in normal operation. The power-on reset value of these signals can be determined by reading the appropriate chip select polarity (PC) field of the MPMCStaticConfig Register. The tie-off values can be overridden by writing to the chip select polarity (PC) field. The register then shows the updated value.

Table 7-5 shows the tie-off signals.

Table 7-5 MPMCStaticConfig Register

Configuration input signal	MPMCStaticConfig register field	Configuration option
MPMCSTCS0POL	Chip select polarity (PC)	Chip select 0 chip select polarity: 0 = active LOW chip select 1 = active HIGH chip select.
MPMCSTCS2POL	Chip select polarity (PC)	Chip select 2 chip select polarity: 0 = active LOW chip select 1 = active HIGH chip select.
MPMCSTCS3POL	Chip select polarity (PC)	Chip select 3 chip select polarity: 0 = active LOW chip select 1 = active HIGH chip select.

Note

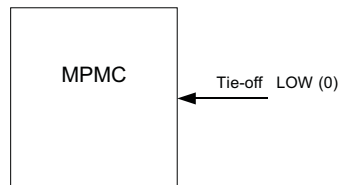
If these tie-offs are not required these signals must be tied LOW.

7.1.10 Methodologies in setting configuration tie-offs

The power-on reset configuration tie-off signals can be set in several ways.

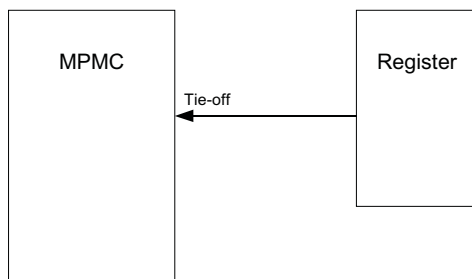
Hard coding tie-off value

If the exact values of the tie-offs are known, or the relevant tie-off is not used, the tie-off can be tied HIGH or LOW. Figure 7-10 on page 7-22 shows a hard coding tie-off.

**Figure 7-10 Hard coding tie-off**

Setting tie-off value by a register

If the tie-off pin value is system-dependent, the tie-off value can be set by a register in another block. Figure 7-11 shows a setting tie-off value by a register.

**Figure 7-11 Setting tie-off value by a register**

Setting tie-off value externally

If the tie-off pin value is system-dependent, the tie-off value can be set externally to the ASIC. However, this does use a pin on the ASIC. Figure 7-12 on page 7-23 shows a setting tie-off value external to the chip.

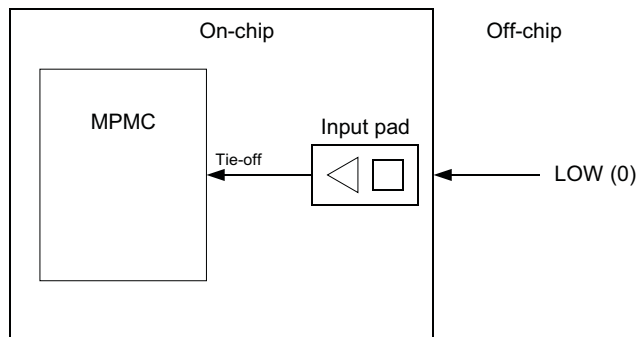


Figure 7-12 Setting tie-off value externally

Setting tie-off value externally using a shared pin

If the tie-off pin value is system-dependent, the tie-off value can be set externally to the ASIC. The ASIC pin can be shared by registering the input value on power-on reset, in for example, the PMU block. Figure 7-12 shows a setting tie-off value external to the chip using a shared pin.

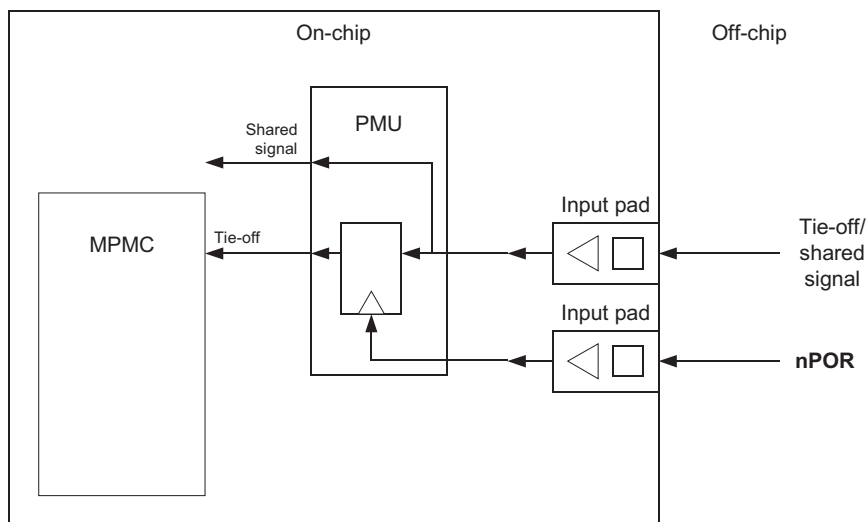


Figure 7-13 Setting tie-off value externally using a shared pin

7.2 Self-refresh entry

The *Power Management Unit* (PMU) self-refresh signals must be connected to the **MPMCSREFREQ** and **MPMCSREFACK** signals. When **MPMCSREFREQ** is asserted, the controller closes any open memory banks, flushes the buffers, and then puts the memory into self-refresh mode. The **MPMCSREFACK** signal indicates to the PMU that the external memories are in self-refresh state. The system must ensure that the memory subsystem is idle before asserting **MPMCSREFREQ**. Deasserting **MPMCSREFREQ** returns the memory to normal operation. See the memory data sheet for refresh requirements.

If the PMU does not provide the required signals to automatically enter self-refresh mode, the **MPMCSREFREQ** signal must be tied LOW. Self-refresh mode can instead be entered manually by programming the self-refresh (SR) bit in the MPMCDynamicControl Register.

This section describes:

- *Power Management Unit self-refresh entry*
- *Manual self-refresh entry* on page 7-26
- *Forcing AHB masters idle* on page 7-26.

———— Note ————

- After self-refresh exit some devices require a number of auto-refresh cycles to take place. If this is the case the auto-refresh cycles must be generated by reprogramming the refresh period and waiting for the required number of refresh commands to occur.
- A PMU is not supplied with the MPMC.

7.2.1 Power Management Unit self-refresh entry

PMU self-refresh entry uses a PMU to handshake with the MPMC to enter self-refresh mode. Figure 7-14 on page 7-25 shows these signals.

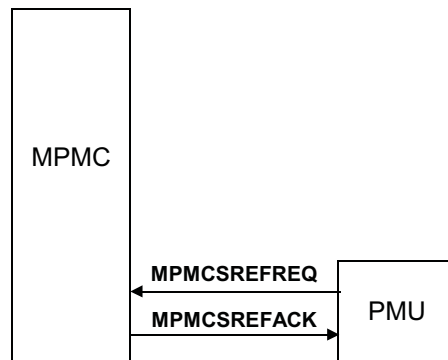


Figure 7-14 PMU self-refresh

When the PMU requires the SDRAM memory to enter self-refresh mode, it must first ensure that there are no masters performing transactions to the MPMC. Various methodologies to perform this are described in *Forcing AHB masters idle* on page 7-26.

When the AHB masters cannot perform transfers to the MPMC, the PMU can then assert the self-refresh request signal, **MPMCSREFREQ**. The MPMC then flushes its buffers and puts the SDRAM memory into self-refresh mode. When the memory is placed into self-refresh mode the MPMC asserts the self-refresh acknowledge signal, **MPMCSREFACK**.

The PMU can then gate **HCLK** as required. See *a in Figure 7-15 on page 7-26.

For exit from self-refresh, the PMU re-enables the clocks, and then brings **MPMCSREFREQ** LOW. The SDRAM controller then exits self-refresh mode. When self-refresh is exited, the MPMC does not permit any access to the SDRAM until the values in the MPMCDynamicctSREX and MPMCDynamicctXSR Registers time out. When the SDRAM controller has exited self-refresh, it lowers the **MPMCSREFACK** signal. The PMU can then enable the AHB masters to access the AHB bus. This can be performed in several ways:

- By performing a soft reset.
- By generating an interrupt.
- By enabling a peripheral transfer to complete. This stalls the ARM processor.
- By lowering the signal to the arbiter so that a dummy AHB master is no longer granted the bus.

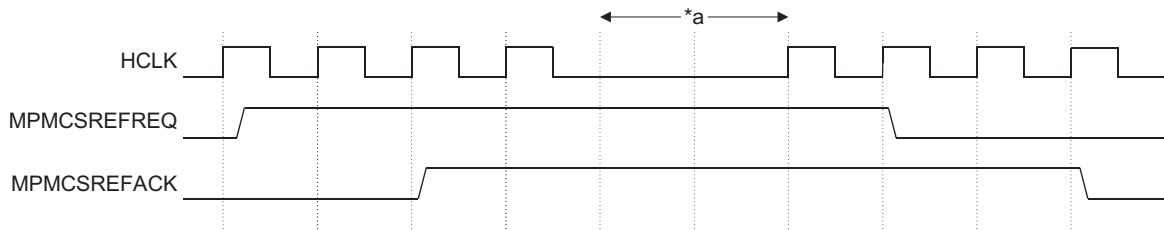


Figure 7-15 PMU self-refresh waveforms

7.2.2 Manual self-refresh entry

Self-refresh can be entered manually in systems where the ARM processor can execute code from internal memory, or from another memory controller.

To enter self-refresh mode the processor must first ensure that there are no masters currently performing transactions to the MPMC. Various methodologies to perform this are described in *Forcing AHB masters idle*.

While the ARM processor is executing code from internal memory or another memory controller it must set the self-refresh request (SR) bit in the MPMCDynamicControl Register. The MPMC then flushes its buffers and puts the memory into self-refresh mode. When the memory is placed into self-refresh mode the MPMC asserts the self-refresh acknowledge (SA) bit in the MPMCStatus Register. The ARM processor must wait until this bit goes active, before continuing the power-down procedure. The PMU gates **HCLK** as necessary.

For exit from self-refresh, the PMU re-enables the clocks and the ARM processor starts executing code. The ARM processor then clears the SR bit in the MPMCDynamicControl Register. The SDRAM controller exits self-refresh mode. This takes about 200 clock cycles. When the SDRAM controller has exited self-refresh it clears the SA bit in the MPMCStatus Register. The ARM processor can then reprogram the AHB masters to enable them to perform transactions to the MPMC.

7.2.3 Forcing AHB masters idle

AHB masters can be forced idle by the following methods:

- The ARM processor can program or interrupt the other masters to stop performing transfers. The ARM processor must then either enter a forever loop, or alternatively read a register in the PMU that waits the transfer until the processor has to re-awake.

- The PMU can assert a signal to the AHB arbiters that puts a dummy AHB master on the bus. For multi-layer AHB systems, a MuxM2S component is required. Masters with AHB-Lite interfaces also require an AHB-Lite to AHB wrapper. See Figure 7-16.

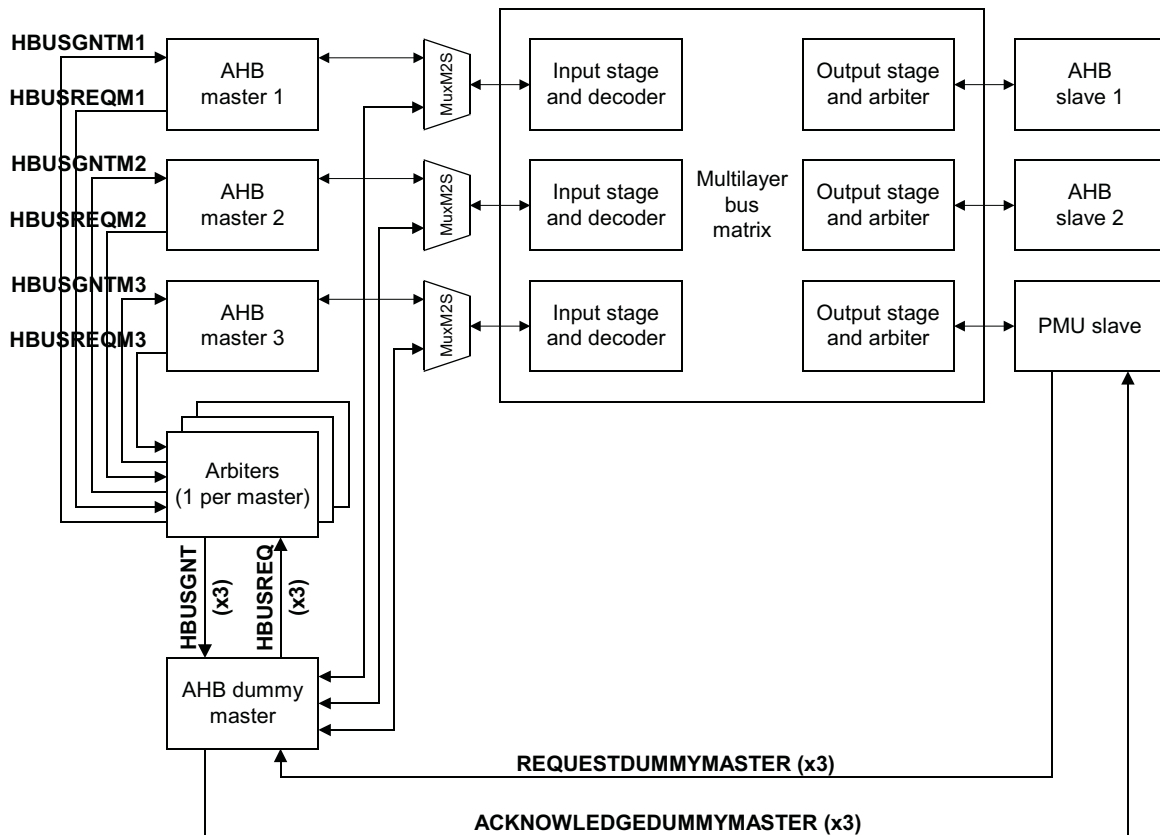


Figure 7-16 Forcing AHB masters idle in a multi-layer system

7.3 Example system

This section describes an example system showing the MPMC integration issues. See Figure 7-17 for a block diagram of the example system.

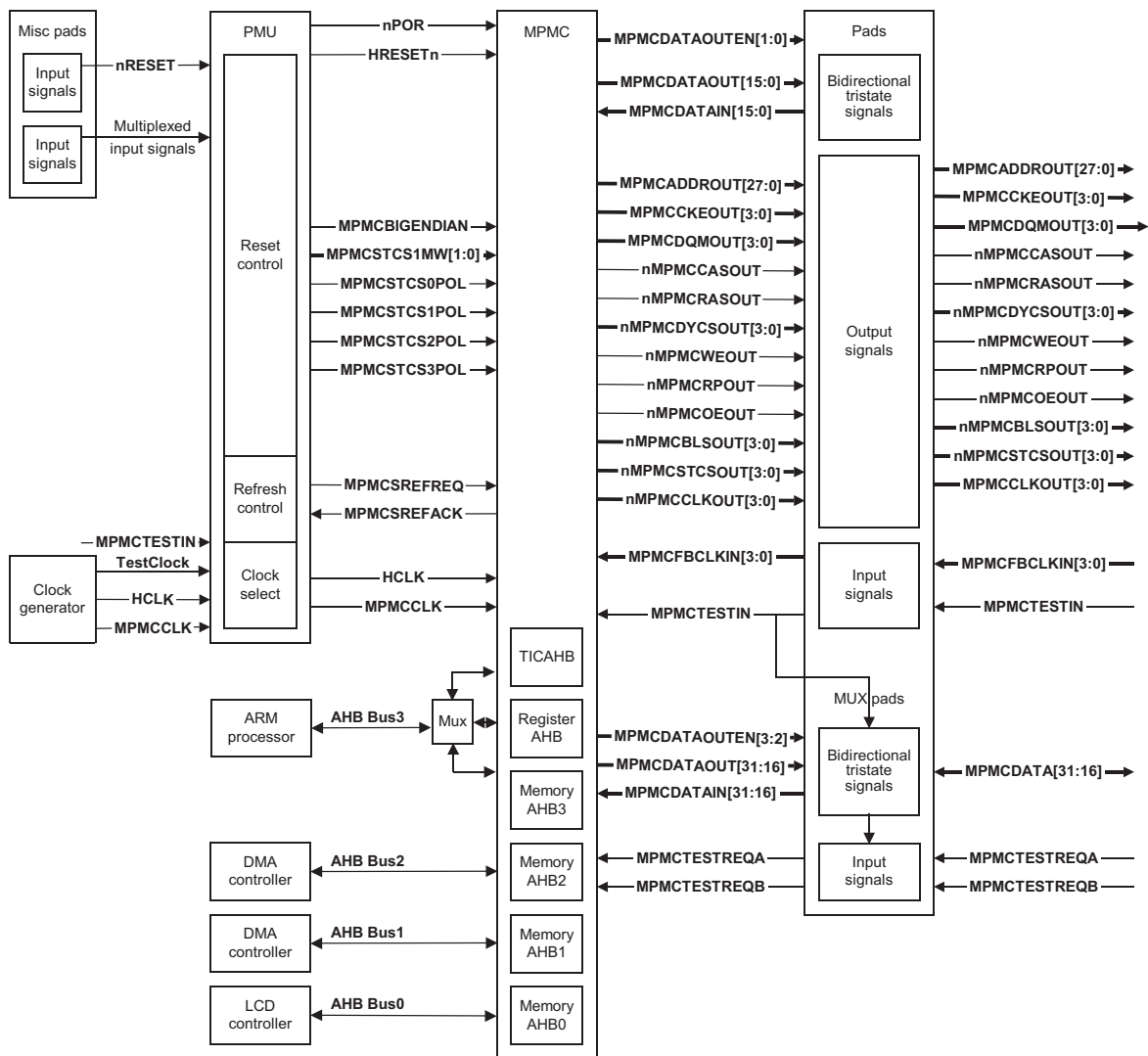


Figure 7-17 Example system

7.3.1 AHB bus 0

The highest priority MPMC AHB memory port, port 0, is connected to high priority, low latency peripheral, in this case an LCD controller.

7.3.2 AHB bus 1

The next highest priority MPMC AHB memory port, port 1, is connected to the next highest priority peripheral, in this case a DMA controller.

7.3.3 AHB bus 2

MPMC AHB memory port 2, is connected to a DMA controller.

7.3.4 AHB bus 3

The AHB memory port (lowest priority), the AHB register port, and the AHB TIC port of the MPMC are all connected to AHB bus 3. The ARM processor is also connected to this bus. The ARM processor is therefore able to program the MPMC and to access external memory.

7.3.5 AHB TIC bus

The MPMC AHB TIC bus is connected to the ARM processor TIC interface.

———— **Note** —————

Some ARM processors use the TIC interface for production test.

7.3.6 Power Management Unit

The major components of the *Power Management Unit* (PMU) are:

- *Reset controller*
- *Refresh controller* on page 7-30
- *Clock select* on page 7-30.

Reset controller

The reset controller generates the power-on reset, **nPOR**, and AMBA reset signal, **HRESETn**.

The **MPMCBIGENDIAN**, **MPMCSTCS1MW[1:0]**, and **MPMCSTCS[3:0]POL** signals are generated by registering the value of a number of inputs to the chip when the power-on reset goes inactive. These input signals are reused during normal operation.

Refresh controller

The refresh controller can place the SDRAM memory devices into self-refresh mode by using the **MPMCSREFREQ** and **MPMCSREFACK** signals.

Clock select

The clock select logic is used to select the various clocks. In TIC test mode, when **MPMCTESTIN** is HIGH, **HCLK** and **MPMCCLK** must operate at no more than 10MHz.

7.3.7 MPMC off-chip signals

The MPMC off-chip signals are:

- *Functional mode signals*
- *Test mode signals.*

Functional mode signals

The system has a 16-bit external data bus, **MPMCDATA[15:0]**. **MPMCDATAOUT[15:0]** and **MPMCDATAIN[15:0]** are connected to bidirectional tristate pads. The direction of these pads is controlled by **MPMCDATAEN[1:0]**.

It is recommended that clock pads are used, if available, for the clock signals **MPMCCLKOUT[3:0]** and **MPMCFBCLKIN[3:0]**.

The output signals, for example **MPMCADDROUT[27:0]**, must be skew balanced with the data bus. This can be performed by selecting appropriate pad cells. One possible way is to use the same bidirectional pads for the output signals as used for the data signals.

The input signals do not have to be skew balanced, so an appropriate input pad must be chosen.

Test mode signals

This example system requires the use of the TIC for ARM production test. The TIC requires a 32-bit data bus, **MPMCDATA[31:0]**, and three control signals, **MPMCTESTREQA**, **MPMCTESTREQB**, and **MPMCTESTACK**. **MPMCTESTACK** is multiplexed with the **nMPMCWEOUT** signal.

MPMCTESTREQA, **MPMCTESTREQB**, and the upper 16 bits of the data bus signal, **MPMCDATA[31:16]**, are required in test mode. To minimize the number of pads used, these signals can be multiplexed with other signals.

Chapter 8

Off-chip Connectivity

This chapter provides examples of how external memory can be connected to the MPMC. It contains the following sections:

- *Memory device selection* on page 8-2
- *Off-chip signals* on page 8-4
- *Clock relationships* on page 8-9
- *Pad interface timing* on page 8-11
- *Pad interface methodology* on page 8-13
- *Clock delay methodology* on page 8-14
- *Command delay methodology* on page 8-19
- *Clock feedback in MPMC designs* on page 8-31
- *Memory clock and feedback clock strategy* on page 8-34.

8.1 Memory device selection

Ensure that all the memory devices connected to the MPMC work together on the same external memory bus.

8.1.1 DC characteristics

All the memory devices connected to the MPMC external memory bus must have the same DC characteristics. See Table 8-1 for a list of the DC characteristics for currently available memory devices.

Table 8-1 Memory device family DC characteristics

Device	3.3V	2.5V	1.8V
Asynchronous static memory	Yes	Yes	Yes
Asynchronous page mode static memory	Yes	Yes	Yes
Synchronous static memory	Yes	Yes	Yes
SDR-SDRAM	Yes	Not currently available	Not currently available
Low-power SDR-SDRAM	Yes	Yes	Yes
Micron SyncFlash	Yes	Not currently available	Not currently available

Note
Synchronous static memory is not supported by the MPMC.

8.1.2 Input and output cell family

Assuming that the DC characteristics of the external memory devices are the same it might be possible to connect LVTTTL and LVCMOS devices together. However, the specific drive characteristics of I/O cells for the ASIC and memory devices must be compared to ensure that they are able to work together. See Table 8-2 for a list of memory devices and the I/O cell family.

Table 8-2 Memory device I/O cell family

Device	LVTTTL pads	LVCMOS pads
Asynchronous static memory	Yes	Yes
Asynchronous page mode static memory	Yes	Yes
Synchronous static memory	Yes	Yes
SDR-SDRAM	Yes	Not currently available
Low-power SDR-SDRAM	Yes	Yes
Micron SyncFlash	Yes	Not currently available

8.2 Off-chip signals

For a 32-bit wide external data bus, 86 pins are required. For a 16-bit external data bus, 70 pins are required.

The **MPMCTESTREQA** and **MPMCTESTREQB** signals are only required in TIC test mode and can therefore be multiplexed with other signals.

The **MPMCRPVHHOUT** signal only has to be bonded out if:

- Micron SyncFlash support is required
- high voltage VHH nRP support is required
- the high voltage generator is not on the chip.

8.2.1 Pin count reduction by reducing data bus width

The data bus width can be reduced from 32 bits to 16 bits. Using a **HCLK** to **MPMCCLK** ratio of 1:1, with a 16-bit bus, impacts performance. Using a **MPMCCLK** that is twice the frequency of **HCLK**, with a 16-bit bus, provides similar performance to a design with a 32-bit data bus.

8.2.2 Pin count reduction by removing functionality

The pin count can be reduced if certain functionality is not required. The pin count reductions that can be achieved by removing functionality are described in:

- *Dynamic and static memory and TIC* on page 8-5
- *Dynamic and static memory* on page 8-5
- *Dynamic memory and TIC* on page 8-5
- *Static memory and TIC* on page 8-6
- *Dynamic memory* on page 8-6
- *Static memory* on page 8-7.

Note

- The signals that are normally multiplexed or not taken off chip are not counted in the following tables. These signals are **MPMCRPVHHOUT**, **MPMCTESTREQA**, and **MPMCTESTREQB**.
 - The output and feedback clocks are not counted on the table because various clocking strategies can be used.
-

Dynamic and static memory and TIC

Table 8-3 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic and static memory and a TIC.

Table 8-3 Pin counts for system with dynamic and static memory and TIC

Data bus	Control	Address	Total
16	26	28	70
32	26	28	86

Dynamic and static memory

Table 8-4 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic and static memory.

Table 8-4 Pin counts for system with dynamic and static memory

Data bus	Control	Address	Total
16	25	28	69
32	25	28	85

The **MPMCTESTIN** signals are not required.

Dynamic memory and TIC

Table 8-5 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic memory and TIC.

Table 8-5 Pin counts for system with dynamic memory and TIC

Data bus	Control	Address	Total
16	16	15	47
32	16	15	63

The **MPMCADDR[27:15]** address signals are not required.

The following static memory signals are not required:

- **nMPMPMCSTCSOUT[3:0]**
- **nMPMCOEOUT**
- **nMPMCBLSOUT[3:0]**

- **nMPMCWEOUT.**

Static memory and TIC

Table 8-6 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing static memory and a TIC.

Table 8-6 Pin counts for system with static memory and TIC

Data bus	Control	Address	Total
16	11	28	55
32	11	28	71

The following dynamic memory signals are not required:

- **nMPMCDYCSOUT[3:0]**
- **MPMCCKEOUT[3:0]**
- **nMPMCRASOUT**
- **nMPMCCASOUT**
- **nMPMCRPOUT**
- **MPMCDQMOUT[3:0].**

Dynamic memory

Table 8-7 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing dynamic memory.

Table 8-7 Pin counts for system with dynamic memory

Data bus	Control	Address	Total
16	15	15	46
32	15	15	62

The **MPMCADDRROUT[27:15]** address signals are not required.

The following static memory signals are not required:

- **nMPMPMCSTCSOUT[3:0]**
- **nMPMCOEOUT**
- **nMPMCBLSOUT[3:0]**
- **nMPMCWEOUT.**

The **MPMCTESTIN** test signal is not required.

Static memory

Table 8-8 shows the pin counts that can be achieved for 32-bit and 16-bit systems containing static memory.

Table 8-8 Pin counts for system with static memory

Data bus	Control	Address	Total
16	13	28	57
32	13	28	73

The following dynamic memory signals are not required:

- **nMPMCDYCSOUT[3:0]**
- **MPMCCKEOUT[3:0]**
- **nMPMCRASOUT**
- **nMPMCCASOUT**
- **nMPMCRPOUT.**

The **MPMCTESTIN** test signals are not required.

8.2.3 Address pin reduction

Depending on the size of the memory parts, not all the address pins have to be bonded out.

8.2.4 Chip select pin reduction

Depending on the number of memory chip selects required, not all of the chip selects have to be bonded out.

8.2.5 Device support

If the Micron SyncFlash devices do not have to be supported, the **nMPMCRPOUT** pin is not required.

8.2.6 Multiplexing static and dynamic memory pins

SDRAM memories use a multiplexed address. These devices use up to 15 address pins.

Static memories do not perform address multiplexing and so require more address pins.

For systems where the SDRAM memory chip select data bus is wider than the static memory chip select data bus, the upper static memory address bits can be multiplexed with the upper bits of the SDRAM data bus to reduce the pin count.

For example, in a system where the static memory devices use a 16-bit data bus and 24 address pins, and the SDRAM memory devices use a 32-bit data bus and 15 address pins, the static memory controller uses more address pins than the dynamic memory controller, and the dynamic memory controller uses more data pins than the static memory controller. Because only one transaction can be active at a time, it is possible to multiplex the top eight bits of the SDRAM data pins with the top eight bits of the static memory address pins. This can be performed by ORing the relevant **MPMCADDRROUT** and **MPMCDATAOUT** signals externally.

———— **Note** —————

If the above optimization is to be performed, appropriate testing is required, ensuring that the pads are driven in the correct direction for static and dynamic memory accesses.

8.2.7 Multiplexing MPMC pins

Total pin count can be reduced by multiplexing the many controller signals with another device. This is normally performed using an EBI. For more information on the EBI, see *EBI connectivity* on page 7-15.

———— **Note** —————

Multiplexing MPMC pins has a detrimental effect on pad interface performance. This is because both on and off-chip signal loading is increased. For many systems this tradeoff is acceptable.

8.3 Clock relationships

HCLK and **MPMCCLK** can have a 1:1 or a 1:2 ratio. All the clocks must have a 50:50 duty cycle for maximum performance.

The positive edge of **HCLK** and **MPMCCLK** must be aligned.

The **MPMCCLKDELAY** signal is used in the command delayed pad interface methodology. **MPMCCLKDELAY** can be tied LOW if not required. See *Pad interface methodology* on page 8-13 for more information on **MPMCCLKDELAY** timing.

Figure 8-1 shows the 1:1 **HCLK** and **MPMCCLK** clock relationship. Figure 8-2 shows the 1:2 **HCLK** and **MPMCCLK** clock relationship.

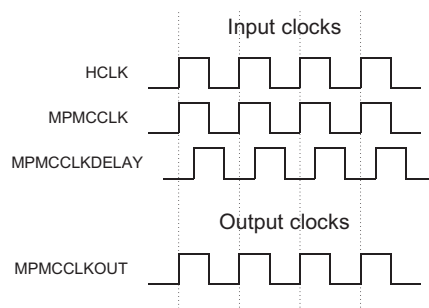


Figure 8-1 1:1 HCLK and MPMCCLK clock relationship

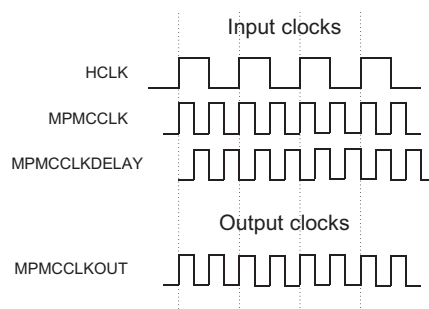


Figure 8-2 1:2 HCLK and MPMCCLK clock relationship

8.3.1 Clock ratios

Table 8-9 shows the clock frequencies required by the MPMC to enable the various clock ratios.

Table 8-9 Clock frequencies

Clock ratio HCLK:MPMCCLK	Required clock frequencies
1:1 (MPMCCLK = HCLK frequency)	HCLK
1:2 (MPMCCLK = 2 x HCLK frequency)	HCLK, HCLK x 2

MPMCCLK must have an equally spaced duty cycle.

1:1 Clock ratio

HCLK must be connected to the MPMC HCLK input, and the MPMCCLK input.

1:2 Clock ratio

HCLK must be connected to the MPMC HCLK input. A clock twice the frequency of HCLK must be connected to the MPMC MPMCCLK input.

————— Note —————

HCLK and MPMCCLK must be synchronous, and the positive edges of the clocks must be aligned.

8.4 Pad interface timing

The on-chip path (the path from the MPMC to the pads), and the off-chip path (the path from the chips I/O pins to the memory devices) are critical for high-speed operation.

8.4.1 On-chip timing path

To enable high-speed operation, the skew between the MPMC pad interface output signals must be minimized.

- This is achieved by the MPMC:
 - providing four output clocks, **MPMCCLKOUT[3:0]**, to reduce clock signal loading
 - registering all the output signals before they are sent off chip to minimize skew between the signals.
- Ensure that:
 - the MPMC is situated as close to the pads as possible to minimize signal delay and skew
 - appropriate strength pads are used for the input signals.

The skew between the MPMC pad interface input signals must be minimized.

- This is achieved by the MPMC:
 - using a feedback clock that is ideally feedback from the memory device clock pin
 - providing four feedback clocks, **MPMCFBCLKIN[3:0]**, to better track the delays of the return data path
 - registering the input SDR-SDRAM read data, using the **MPMCFBCLKIN** signals.
- Ensure that:
 - the MPMC is situated as close to the pads as possible to minimize signal delay and skew
 - appropriate strength pads are used for the input signals.

8.4.2 Off-chip timing path

The skew between the various signals during PCB routing must be minimized. The off-chip path from the ASIC pads to the dynamic memory devices must be as lightly loaded as possible to enable high-speed operation. Therefore, the number of memory devices must be minimized for high-speed operation.

For systems that require a large number of memory devices, additional buffering of the I/O signals might be required. For example, the signals to the static memory devices can be buffered. This adds additional delay when accessing static memories, but enables the dynamic devices to run at a higher speed.

8.5 Pad interface methodology

The SDRAM clock must not be coincident with the SDRAM command signals, address, or write data, otherwise the SDRAM setup and hold requirements are violated. To meet these requirements the MPMC supports two pad interface methodologies:

- *Clock delay methodology* on page 8-14
- *Command delay methodology* on page 8-19.

8.6 Clock delay methodology

The clock methodology is described in the following sections:

- *Clock delay*
- *Clock delay advantages* on page 8-17
- *Clock delay disadvantages* on page 8-18
- *Data capture requirements* on page 8-18
- *Generation of the delayed clock* on page 8-18.

8.6.1 Clock delay

The clock delay methodology requires the output clock to be delayed externally to the MPMC. The output clock must be delayed by more than the SDRAM setup time. However, the output clock must not be delayed too much because the SDRAM hold requirements must also be met.

This methodology is most suited to systems where the SDRAM clock frequency is not high, and where the worst case read data delay is less than a cycle.

This clock delay methodology can be selected by programming the SDR-SDRAM read data strategy (SRD) field of the MPMCDynamicReadConfig Register.

Figure 8-3 on page 8-15 shows the clock delayed pad interface read timing.

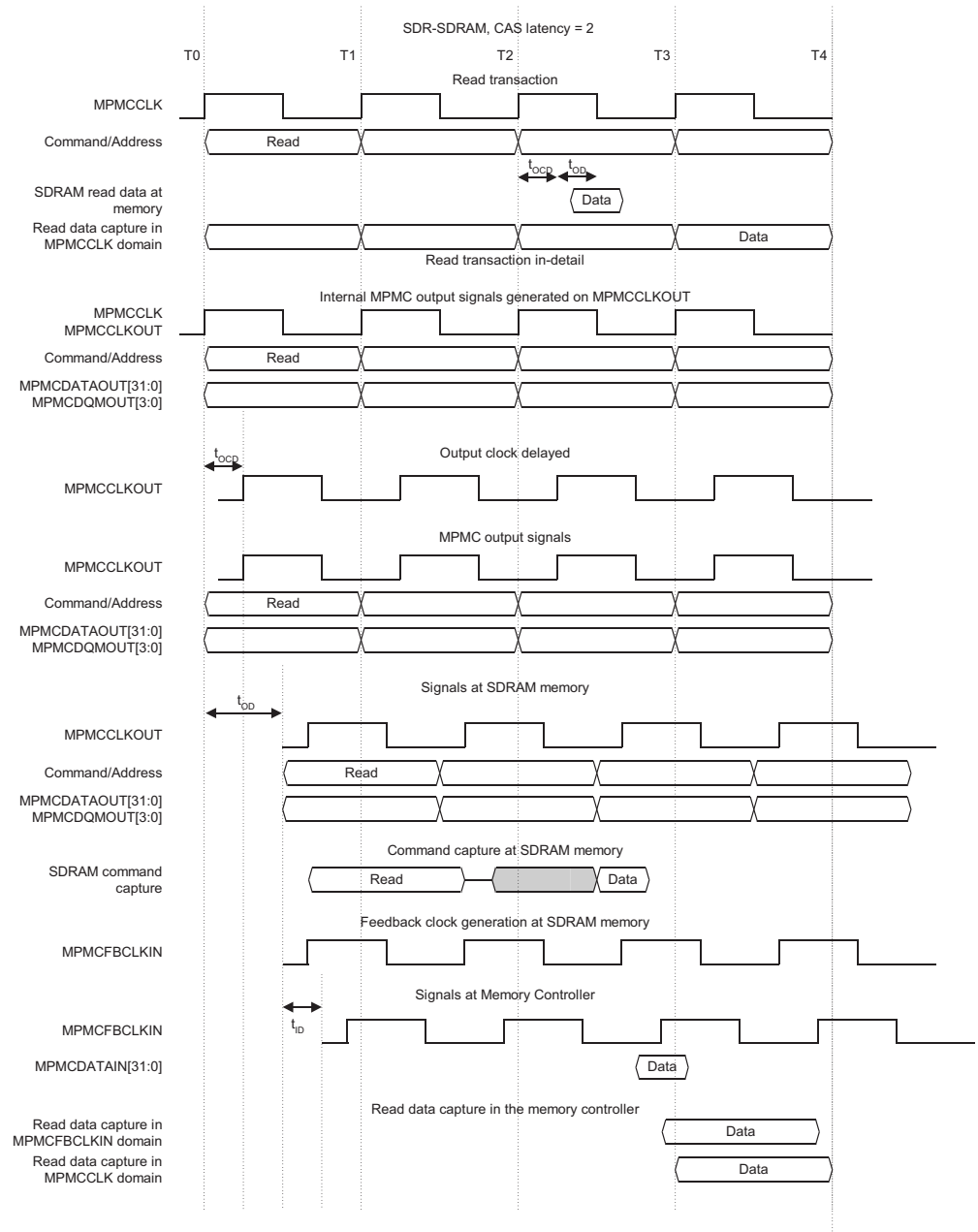


Figure 8-3 Clock delayed pad interface read timing

Figure 8-3 on page 8-15 is described in the following sections:

- *Read transaction*
- *Internal MPMC output signals generated on MPMCCLKOUT*
- *Output clock delayed*
- *MPMC output signals*
- *Signals at SDRAM memory* on page 8-17
- *Command capture at SDRAM memory* on page 8-17
- *Feedback clock generation at SDRAM memory* on page 8-17
- *Signals at MPMC* on page 8-17
- *Read data capture in the MPMC* on page 8-17.

Read transaction

The SDRAM command, address, and clock are posted to the SDRAM during cycle T0-T1.

The SDRAM memory provides the read data (at the memory) during cycle T2-T3.

The read data is captured in the MPMC, in the **MPMCCLK** domain at T3.

Internal MPMC output signals generated on MPMCCLKOUT

Internally the logic of the MPMC is clocked by **MPMCCLK**. This clock also generates the external clock **MPMCCLKOUT**.

To meet SDRAM memory setup and hold requirements the SDRAM clock and command transitions must not be coincident.

Output clock delayed

In the clock delayed pad interface methodology, the output clock **MPMCCLKOUT** must be delayed externally to the MPMC. This is to ensure that the commands meet the SDRAM setup and hold requirements.

Generally SDR-SDRAM memory devices require about 1.5ns of setup and hold time.

In Figure 8-3 on page 8-15 this *Output Clock Delay* (OCD) is indicated as t_{OCD} .

MPMC output signals

Figure 8-3 on page 8-15 shows the resulting signals provided to the pads before they go off chip. The command and address are not delayed.

The output clock is delayed by t_{OCD} . These signals now meet the SDRAM memory requirements.

Signals at SDRAM memory

The output signals (clock, address, and commands) are subjected to a delay t_{OD} (Output Delay) going to the SDRAM memory. t_{OD} is composed of the pad delay and the PCB delay.

Command capture at SDRAM memory

The SDRAM read command is captured on the positive edge of **MPMCCLKOUT** at the SDRAM memory.

For CAS 2 latency SDR-SDRAM reads, the read data is valid on the second positive edge of the memory clock after the read command is captured. SDR-SDRAM provides valid read data a few nanoseconds before and after the SDRAM clock edge to make capturing the read data easier.

Feedback clock generation at SDRAM memory

The feedback clock is ideally generated from an output clock with a similar PCB and pad delay as the SDRAM clock. This is so that the positive edge of the feedback clock is aligned in the middle of the time when the read data is valid.

Signals at MPMC

The input signals (read data and feedback clock) are subjected to a delay t_{ID} (Input Delay) from the SDRAM to the MPMC. t_{ID} is composed of a PCB delay and a pad delay.

Read data capture in the MPMC

The read data is captured using the feedback clock **MPMCFBCLKIN**. This ensures that the read data is captured cleanly. The read data is then captured in the **MPMCCLK** clock domain.

8.6.2 Clock delay advantages

The clock delay methodology provides data a cycle earlier for a read access than the command delay methodology. This improves the read burst access latency by a cycle compared to the command delay methodology.

8.6.3 Clock delay disadvantages

The main disadvantage with the clock delay methodology is that the read capture timing is more tightly constrained.

The read delay time must be less than a clock period.

Another disadvantage is that the changing, dynamic, read skew can be affected by the jitter added by the delayed clock.

8.6.4 Data capture requirements

Both the worst case and best case dynamic skew and read data delay must meet the following requirements, otherwise the MPMC cannot capture the read data correctly.

Maximum read data delay

The maximum read data delay is:

Maximum read data delay = **HCLK** clock period - $t_{\text{setupFBCLK}}$ - $t_{\text{setupHCLK}}$

Alternatively this can be expressed as:

$(t_{\text{OCD}} + t_{\text{OD}} + t_{\text{ID}} + t_{\text{setupFBCLK}} + t_{\text{setupHCLK}}) < \text{HCLK clock period}$

Where t_{OCD} is the output clock delay, t_{OD} is the output delay, and t_{ID} is the input delay.

Minimum read data delay

The minimum read data delay is t_{holdHCLK}

Maximum dynamic skew

The maximum dynamic skew permitted in this pad methodology is:

Maximum dynamic skew = maximum read delay - minimum read data delay

Maximum dynamic skew = **HCLK** clock period - $t_{\text{setupFBCLK}}$ - $t_{\text{setupHCLK}}$ - t_{holdHCLK}

8.6.5 Generation of the delayed clock

To meet both the setup and hold requirements for the SDRAM, a 180 degree phase shifted output clock can be used. A simple way to generate this is to invert the output clock externally to the MPMC. This adds an additional half a clock skew to the return read data. This methodology means there is no additional clock dynamic skew.

8.7 Command delay methodology

The command delay methodology is described in the following sections:

- *Command delay plus zero*
- *Command delay advantages* on page 8-22
- *Command delay disadvantages* on page 8-23
- *Generation of the delayed clock* on page 8-23
- *Command delay plus zero SDRAM timing diagrams* on page 8-23
- *Command delay plus one* on page 8-25
- *Command delay plus one SDRAM timing diagrams* on page 8-27
- *Command delay plus two* on page 8-28
- *Command delay plus two SDRAM timing diagrams* on page 8-30.

8.7.1 Command delay plus zero

The command delay methodology requires the output signals (the address, control signals and write data) to be delayed compared to the output clock **MPMCCLK**. You can delay the output signals using the **MPMCCLKDELAY** clock signal. The output signals must be delayed by more than the SDRAM hold time. The output signals must not be delayed too much because the SDRAM setup requirements must also be met.

This methodology is most suited to systems where the SDRAM clock frequency is high. Select the command delay methodology by programming the SDR-SDRAM read data strategy (SRD) field of the MPMCDynamicReadConfig Register. Figure 8-4 on page 8-20 shows the command delayed pad interface read timing.

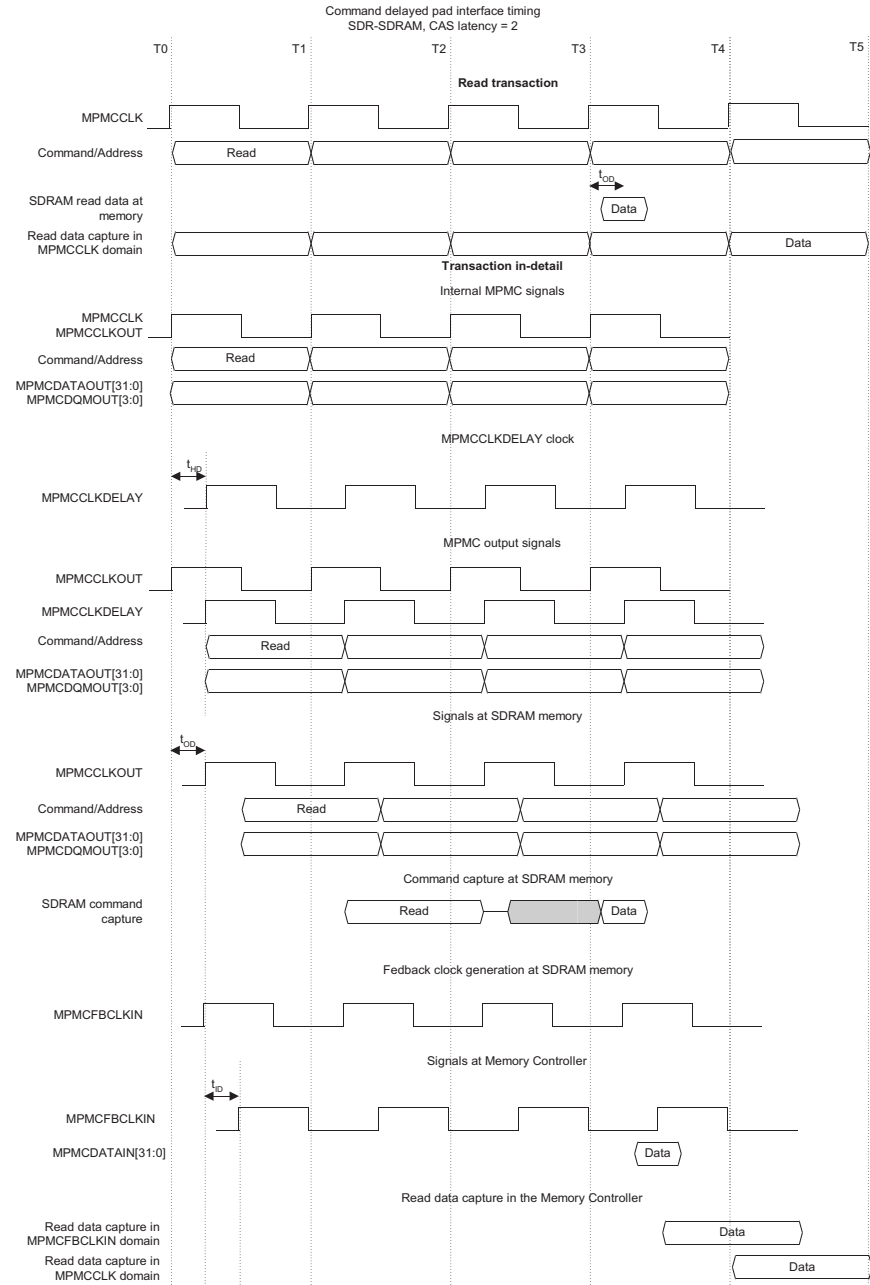


Figure 8-4 Command delayed pad interface timing

Figure 8-4 on page 8-20 is described in the following sections:

- *Read transaction*
- *Internal MPMC output signals generated on MPMCCLKOUT*
- *MPMCCLKDELAY clock*
- *MPMC output signals*
- *Signals at SDRAM memory* on page 8-22
- *Command capture at SDRAM memory* on page 8-22
- *Feedback clock generation at SDRAM memory* on page 8-22
- *Signals at MPMC* on page 8-22
- *Read data capture in the MPMC* on page 8-22.

Read transaction

The SDRAM command, address, and clock are posted to the SDRAM at time T0-T1.

The SDRAM memory provides the read data (at the memory) during cycle T3-T4.

The read data is captured in the MPMC, in the **MPMCCLK** domain at T4.

Internal MPMC output signals generated on MPMCCLKOUT

Internally the MPMC logic is clocked by **MPMCCLK**. This clock also generates the external clock **MPMCCLKOUT**.

To meet SDRAM memory setup and hold requirements, the SDRAM clock and command transitions must not be coincident.

MPMCCLKDELAY clock

In the command delayed pad interface methodology, the delayed clock **MPMCCLKDELAY** is used to clock the output address, commands, and write data. This ensures that the commands meet the SDRAM setup and hold requirements.

Generally SDR-SDRAM memory devices require approximately 1.5ns of setup and hold time. This delay is indicated as t_{HD} (**MPMCCLK** Delay).

MPMC output signals

Figure 8-4 on page 8-20 shows the resulting signals provided to the pads before they go off chip. The command, address, and write data are delayed by t_{HD} , and the output clock is not delayed. These signals now meet the SDRAM memory requirements.

Signals at SDRAM memory

The output signals (clock, address, and commands) are subjected to a delay t_{OD} (Output Delay) going to the SDRAM memory. t_{OD} is composed of the pad delay and the PCB delay.

Command capture at SDRAM memory

The SDRAM read command is captured on the positive edge of **MPMCCLKOUT** at the SDRAM memory.

For CAS 2 latency SDR-SDRAM reads, the read data is valid on the second positive edge of the memory clock after the read command has been captured. SDR-SDRAM provides valid read data a few nanoseconds before and after the SDRAM clock edge to make capturing the read data easier.

Feedback clock generation at SDRAM memory

The feedback clock is ideally generated from an output clock with a similar PCB and pad delay as the SDRAM clock, so that the feedback clocks positive edge is aligned in the middle of the time when the read data is valid.

Signals at MPMC

The input signals (read data and feedback clock) are subjected to a delay t_{ID} (Input Delay) from the SDRAM to the MPMC. t_{ID} is composed of a PCB delay and a pad delay.

Read data capture in the MPMC

The read data is captured using the feedback clock **MPMCFBCLKIN**. This ensures that the read data is captured cleanly. The read data is then captured in the **MPMCCLK** clock domain.

8.7.2 Command delay advantages

The main advantage of the command delay methodology is that the read data can be more easily captured than for the clock delay methodology.

For command delay plus zero mode the read data delay must be less than one **MPMCCLK** cycle. For command delay plus one mode the read data delay must be less than two **MPMCCLK** cycles. For command delay plus two mode the read data delay must be less than three **MPMCCLK** cycles.

The worst case maximum dynamic read skew is half a **MPMCCLK** period.

8.7.3 Command delay disadvantages

The command delay methodology plus 0 provides the read data a cycle later than the clock delay methodology, increasing the read burst access latency by a clock cycle.

The command delay methodology plus 1 provides the read data two cycles later than the clock delay methodology, increasing the read burst access latency by two clock cycles.

The command delay methodology plus 2 provides the read data three cycles later than the clock delay methodology, increasing the read burst access latency by three clock cycles.

8.7.4 Generation of the delayed clock

To meet both the setup and hold requirements for the SDRAM, a 180 degree phase shifted **MPMCCLKDELAY** can be used. A simple way to generate this is to invert the output clock externally to the MPMC.

8.7.5 Command delay plus zero SDRAM timing diagrams

This section describes the command delay plus zero SDR-SDRAM timing diagrams and data capture requirements.

Figure 8-5 on page 8-24 shows the command delay plus zero methodology.

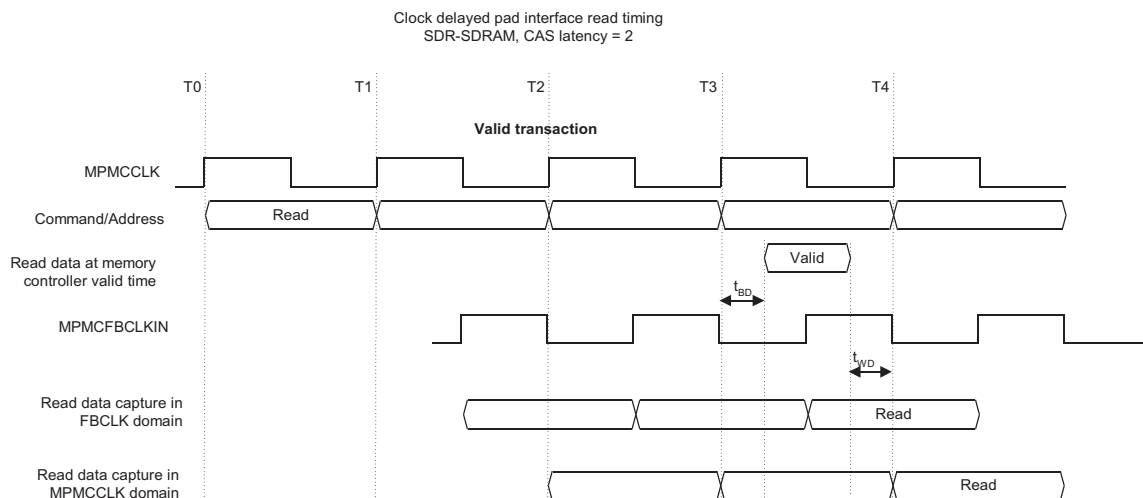


Figure 8-5 Command delay plus zero SDR-SDRAM

Data capture requirements, data captured on positive edge of HCLK

Both the worst case and best case dynamic skew, and read data delay must meet the following requirements, otherwise the MPMC cannot capture the read data correctly.

Maximum read data delay

The maximum read data delay = **HCLK** clock period - $t_{\text{setupFBCLK}}$ - $t_{\text{setupHCLK}}$

Alternatively this can be expressed as:

$$(t_{OD} + t_{ID} + t_{\text{setupFBCLK}} + t_{\text{setupHCLK}}) < \text{HCLK clock period}$$

Where t_{OD} is the output delay and t_{ID} is the input delay.

Minimum read data delay

The minimum read data delay is t_{holdHCLK}

Maximum dynamic skew

The minimum read delay and maximum read delay must not overlap the positive edge of **MPMCCLK**. Therefore the maximum worst case dynamic skew is half a clock cycle.

8.7.6 Command delay plus one

This scheme is similar to the command delay plus zero pad interface methodology except that the data is captured on the next **MPMCCLK** clock cycle. This provides an additional cycle for the read data to return from the SDRAM memory.

Figure 8-6 on page 8-26 shows a read transaction in detail.

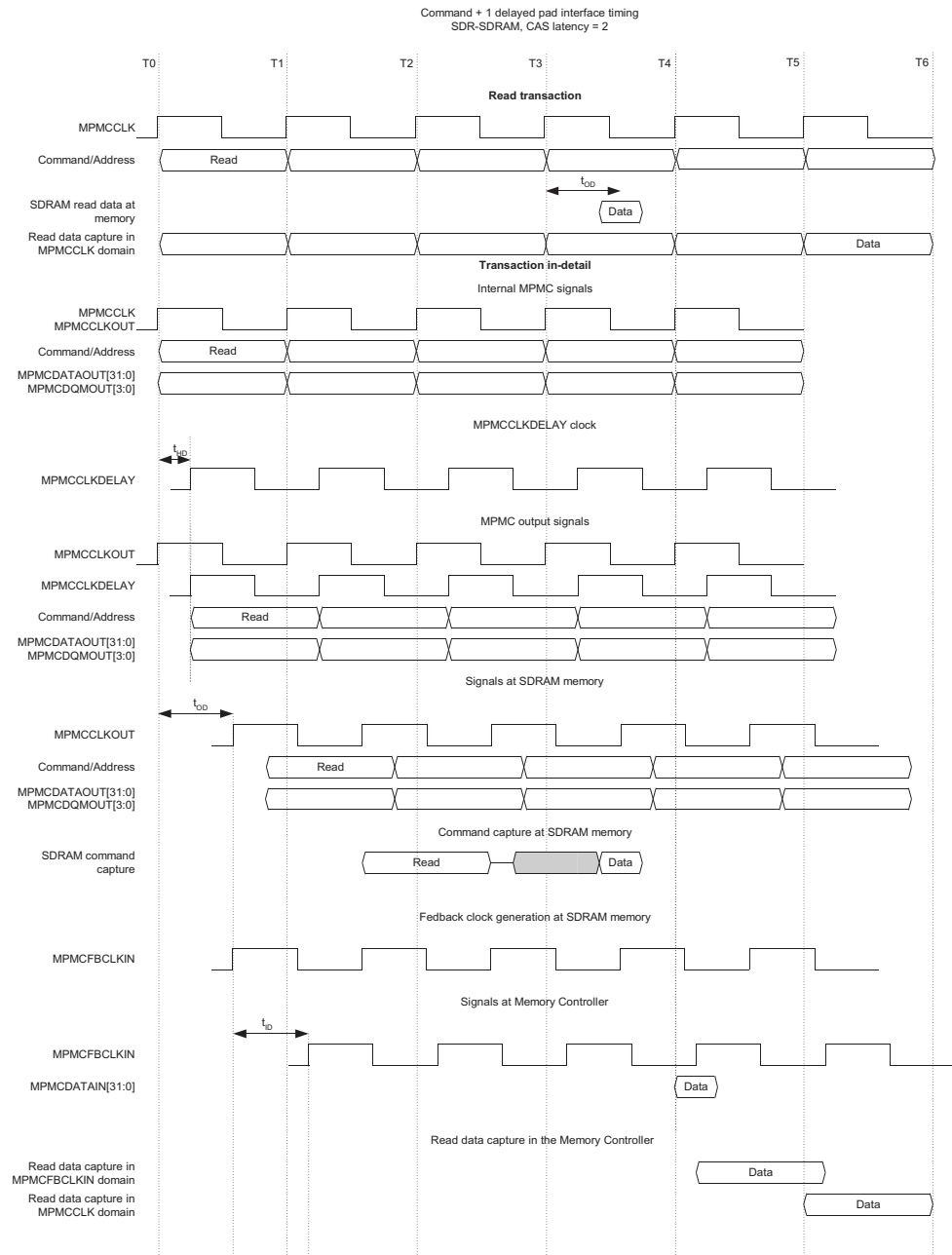


Figure 8-6 Command delayed plus one pad interface timing

8.7.7 Command delay plus one SDRAM timing diagrams

This section describes the command delay plus one SDR-SDRAM timing diagrams and data capture requirements.

Data capture requirements

Figure 8-7 shows the command delay plus one methodology.

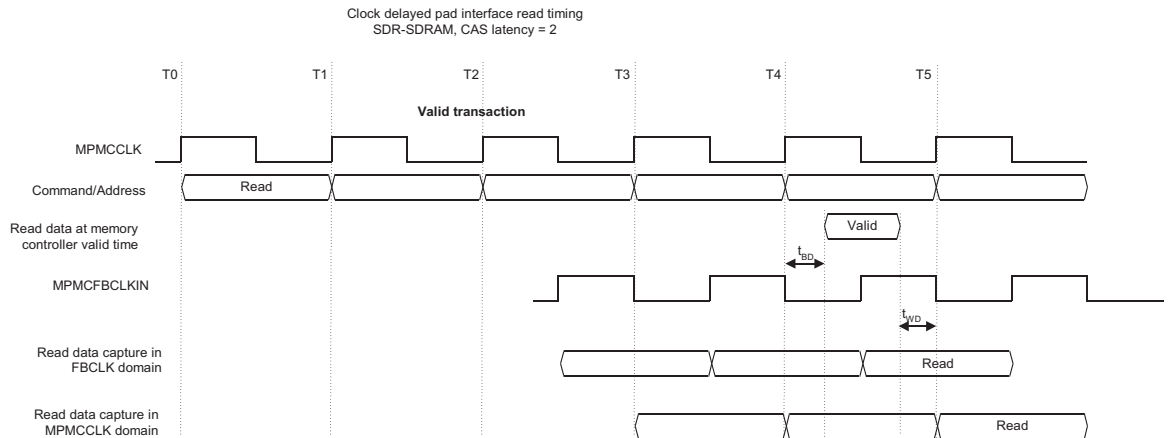


Figure 8-7 Command delay plus one SDR-SDRAM

Both the worst case and best case dynamic skew and read data delay must meet the following requirements, otherwise the MPMC cannot capture the read data correctly.

Maximum read data delay

The maximum read data delay is:

$$(\text{two MPMCCLK clock period}) - t_{\text{setupFBCLK}} - t_{\text{setupMPMCCLK}}$$

Minimum read data delay

The minimum read data delay is:

$$(\text{MPMCCLK clock period}) - t_{\text{holdMPMCCLK}}$$

Maximum dynamic skew

The minimum read delay and maximum read delay must not overlap the positive edge of **MPMCCLK**. Therefore the maximum worst case dynamic skew is half a clock cycle.

8.7.8 Command delay plus two

This scheme is similar to the command delay plus zero pad interface methodology except that the data is captured two **MPMCCLK** clock cycles later than the command plus zero scheme. This provides two additional cycles for the read data to return from the SDRAM memory. Figure 8-8 on page 8-29 shows a read transaction in detail.

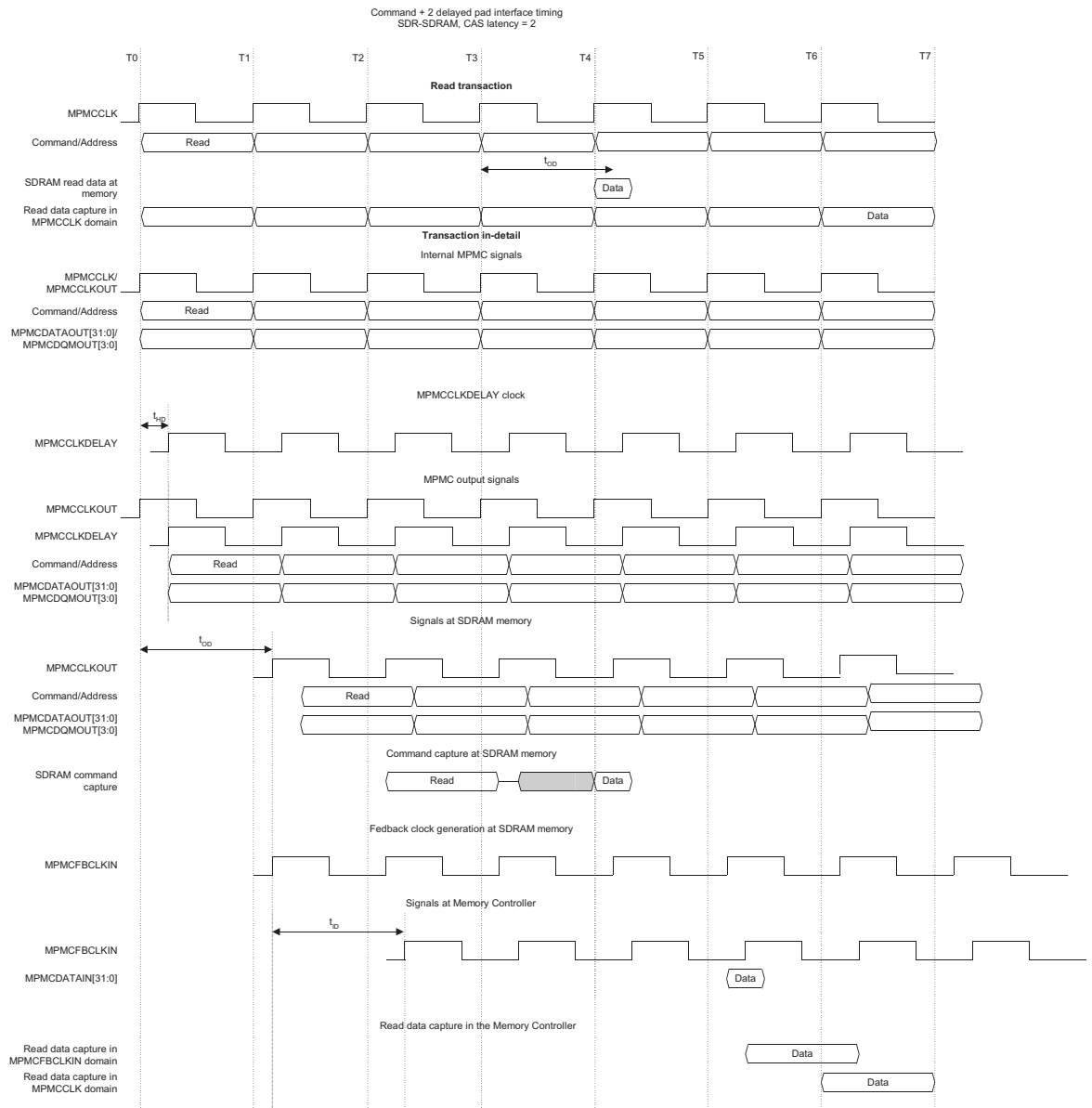


Figure 8-8 Command delayed plus two pad interface timing

8.7.9 Command delay plus two SDRAM timing diagrams

This section describes the command delay plus two SDR-SDRAM timing diagrams and data capture requirements.

Data capture requirements

Figure 8-9 shows the command delay plus two methodology.

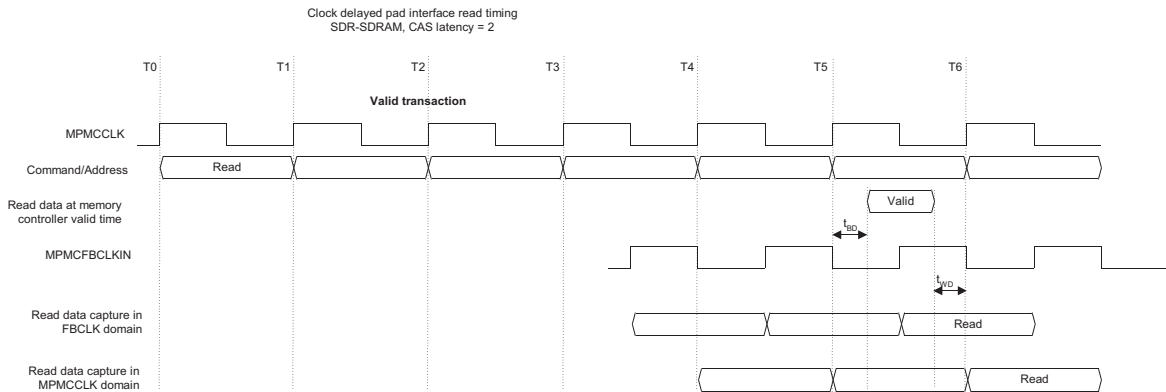


Figure 8-9 Command delay plus two SDR-SDRAM

Both the worst case and best case dynamic skew and read data delay must meet the following requirements, otherwise the MPMC cannot capture the read data correctly.

Maximum read data delay

The maximum read data delay is:

(two and a half **MPMCCLK** clock period) - $t_{\text{setupFBCLK}}$ - $t_{\text{setupMPMCCLK}}$

Minimum read data delay

The minimum read data delay is:

(one and a half **MPMCCLK** clock period) + $t_{\text{holdMPMCCLK}}$

Maximum dynamic skew

The minimum read delay and maximum read delay must not overlap the negative edge of **MPMCCLK**. Therefore the maximum worst case dynamic skew is half a clock cycle.

8.8 Clock feedback in MPMC designs

There are three methods of clock feedback generation:

- *Generating feedback clock off-chip*
- *Generating feedback clock from bidirectional pad*
- *Generating feedback clock on-chip on page 8-32.*

8.8.1 Generating feedback clock off-chip

In this methodology the feedback clock is generated off-chip from the output clock. It is recommended that the feedback clock is generated near to the memory device as possible, and follows the data bus back on-chip.

This methodology takes into account the output pad loading, output pad delay, PCB trace, and input pad delay.

This methodology is the best way to generate the feedback clock, however this methodology requires additional input pins.

Figure 8-10 shows how the output clock is fed back to the MPMC.

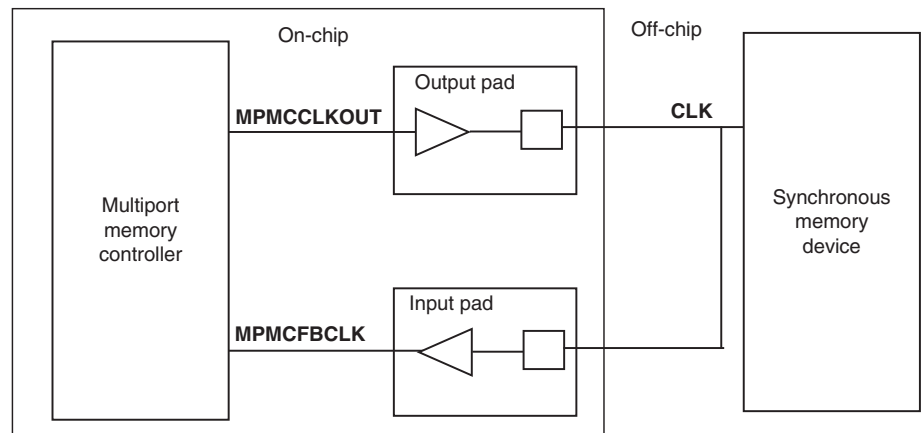


Figure 8-10 Generating feedback clock off-chip

8.8.2 Generating feedback clock from bidirectional pad

In this methodology, the feedback clock is generated on-chip, using the input of the tristate pad that outputs the memory clock.

This methodology takes into account the output pad loading and output pad delay. It is normally the best way to generate the feedback clock without generating the feedback clock externally off-chip.

Figure 8-11 shows how the output clock is fed back from a bidirectional pad.

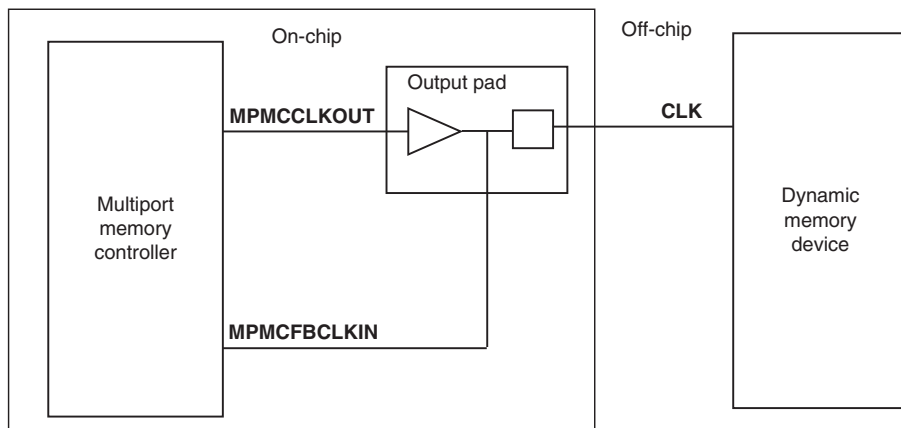


Figure 8-11 Generating feedback clock from bidirectional pad

8.8.3 Generating feedback clock on-chip

In this methodology, the feedback clock is generated on-chip by, for example, inverting the output clock.

This methodology does not take into account the loading of the output pad. It is the least accurate way to generate the feedback clock.

Figure 8-12 on page 8-33 shows how the output clock is inverted to generate the feedback clock.

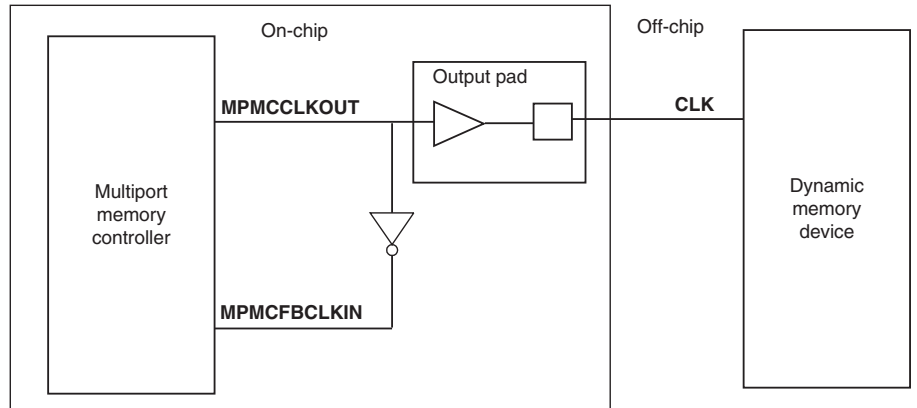


Figure 8-12 Generating feedback clock on-chip

8.9 Memory clock and feedback clock strategy

The MPMC has four output clocks and four feedback clocks:

- The output clocks are used to clock the external dynamic memory devices.
- The feedback clocks register the read data from the memory devices.

Depending on the system architecture and system performance, the output clocks and feedback clocks can be used in a number of different ways.

8.9.1 Output clocks

Providing four output clocks, **MPMCCLKOUT[3:0]**, enables the load to the clock signals to be reduced by, for example, connecting each output clock to a different memory device.

For systems where performance is less of an issue, or where there are a small number of dynamic memory devices, fewer output clocks can be bonded out.

8.9.2 Feedback clocks for SDRAM

There are four feedback clocks. Each feedback clock is associated with eight bits of the data bus:

- **MPMCFBCLKIN[0]** registers bits MPMCDATAIN[7:0]
- **MPMCFBCLKIN[1]** registers bits MPMCDATAIN[15:8]
- **MPMCFBCLKIN[2]** registers bits MPMCDATAIN[23:16]
- **MPMCFBCLKIN[3]** registers bits MPMCDATAIN[31:24].

It is recommended that the feedback clock signals are generated off-chip from an output clock near the furthest memory device from the ASIC. The feedback clock signals must then follow the same path as the read data associated with them so that they are delayed by the same amount. This ensures that the read data is registered correctly.

The ideal situation is where:

- the feedback clock follows a similar path on the PCB to the associated data signals
- the loading on the data bus and the feedback clock is similar.

For high-performance systems the feedback clocks are generated as described above, from an output clock near the furthest memory device from the ASIC. The feedback clock must then be routed with the data it is associated with on the PCB to match the path read data. Higher performance systems can use four separate feedback clock signals so that each byte of the data bus is matched individually.

Medium performance systems can use fewer feedback clocks. These feedback clocks can be connected to the **MPMCFBCLKIN[3:0]** signals on chip as necessary.

For systems where performance is less important, or where there are a small number of dynamic memory devices, the feedback clock can be generated on chip by delaying the output clock.

8.9.3 Signal termination

The off-chip signals must be terminated appropriately.

Series termination is used in lower performance systems. The signal can normally only be tapped at the end of the wire and cannot be connected to form a loop.

Parallel termination is used in higher performance systems. The signal can be tapped at various points and can be connected to form a loop (for example **MPMCCLKOUT** to **MPMCFBCLKIN**).

Termination is beyond the scope of this document. However, care must be taken at high clock speeds to terminate the signals appropriately. In addition, signal drive capabilities, voltage swings, and circuit board signal characteristics must all be taken into account.

8.9.4 High performance systems

This subsection describes the following devices:

- *x8 SDRAM memory devices*
- *x16 SDRAM memory devices* on page 8-38
- *x32 SDRAM memory devices* on page 8-39
- *Mixed width SDRAM memory devices* on page 8-40
- *SRAM and SDRAM memory devices* on page 8-41.

x8 SDRAM memory devices

Figure 8-13 on page 8-37 shows an example of a way of connecting of x8 SDRAM devices. Chip select 4 has four x8 SDRAM devices providing a 32-bit data bus, and chip select 5 has two x8 SDRAM devices providing a 16-bit data bus.

MPMCCLKOUT[0] clocks the memory devices providing the lower eight bits of data. **MPMCFBCLKIN[0]** is generated from **MPMCCLKOUT[0]**. The memory devices that provide the next eight bits of data are clocked by **MPMCCLKOUT[1]**. The read data is registered by **MPMCFBCLKIN[1]**. The same scheme is used for the other bytes of data.

This methodology reduces the load on the output clocks and enables the read data to be registered accurately.

———— **Note** ————

Figure 8-13 on page 8-37 shows parallel termination on the feedback clocks.

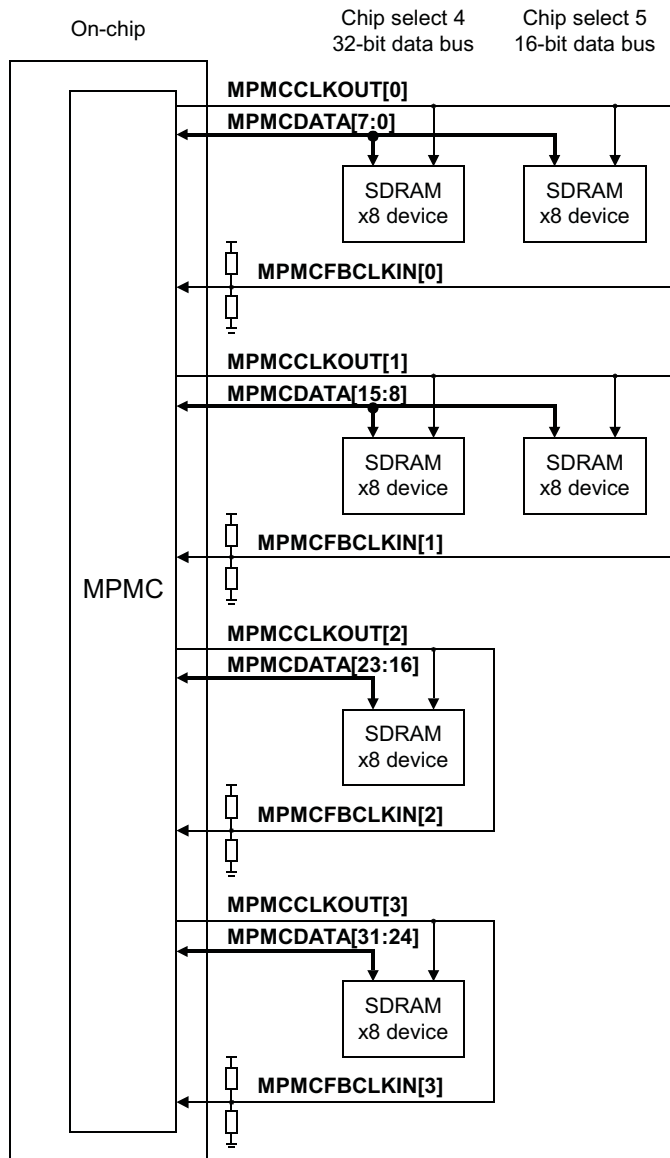


Figure 8-13 x8 SDRAM device

x16 SDRAM memory devices

Figure 8-14 shows an example of a way of connecting x16 SDRAM devices. Chip select 4 has two x16 SDRAM devices providing a 32-bit data bus, and chip select 5 has one x16 SDRAM device providing a 16-bit data bus:

- **MPMCCLKOUT[0]** clocks the memory devices providing the lower 16 bits of data. **MPMCFBCLKIN[0]** and **MPMCFBCLKIN[1]** are generated from **MPMCCLKOUT[1]**.
- **MPMCCLKOUT[2]** clocks the memory devices providing the upper 16 bits of data. **MPMCFBCLKIN[2]** and **MPMCFBCLKIN[3]** are generated from **MPMCCLKOUT[3]**.

————— **Note** —————

MPMCFBCLKIN[3:0] can be connected together on-chip as required.

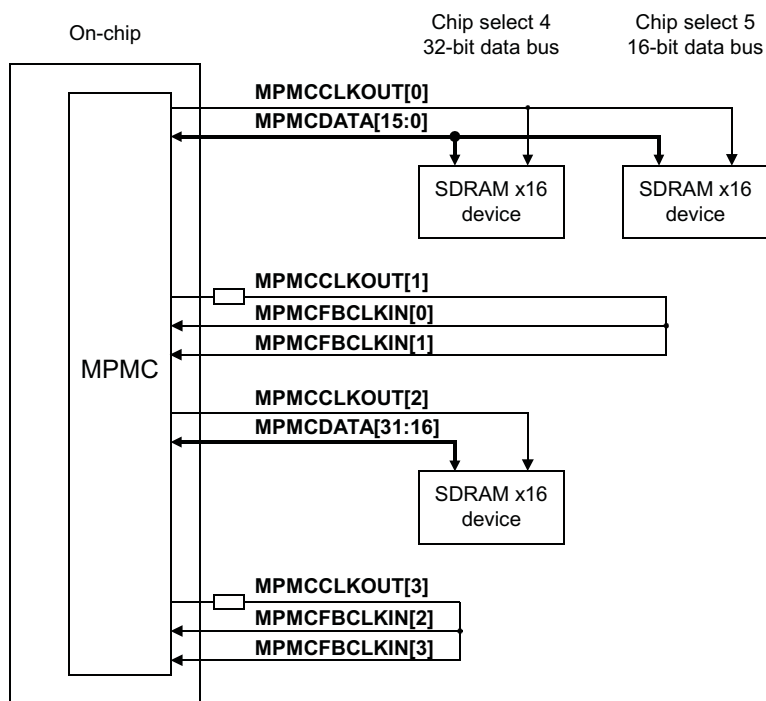


Figure 8-14 x16 SDRAM connection

Figure 8-14 shows series termination on the output clocks used to generate the feedback clocks.

x32 SDRAM memory devices

Figure 8-15 shows an example of a way of connecting x32 SDRAM devices. Chip select 4 and chip select 5 each have one x32 SDRAM device providing a 32-bit data bus.

MPMCCLKOUT[0] clocks the two memory devices. Two feedback clocks are generated from **MPMCCLKOUT[1]**. These feedback clocks generate **MPMCFBCLKIN[3:0]** internally.

Note

MPMCFBCLKIN[3:0] can be connected together on-chip as required.

Figure 8-15 shows series termination on the output clocks used to generate the feedback clocks.

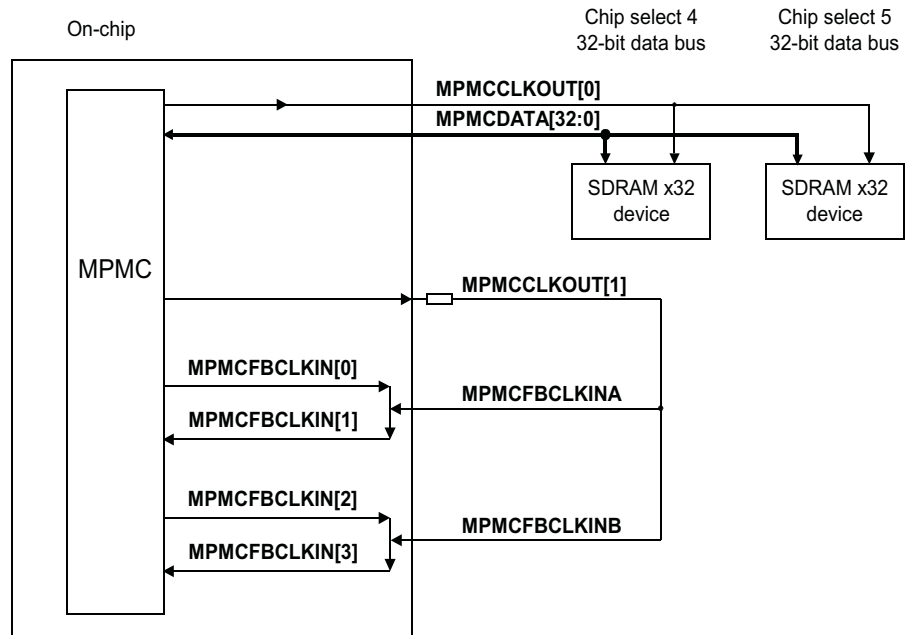


Figure 8-15 x32 SDRAM interconnection

Mixed width SDRAM memory devices

Figure 8-16 shows an example of a way of connecting mixed width memory devices. Chip select 4 has two x8 SDRAM devices providing a 16-bit data bus, and chip select 5 has one x16 SDRAM device also providing a 16-bit data bus:

- **MPMCCLKOUT[0]** clocks the lower 8-bit memory part of chip select 4
- **MPMCCLKOUT[1]** clocks the memory device of chip select 5
- **MPMCCLKOUT[2]** clocks the high order memory part of chip select 4
- **MPMCFBCLKIN[0]** and **MPMCFBCLKIN[1]** are generated from **MPMCCLKOUT[3]**.

This spreads the load on the output clocks and ensures that the feedback clock delay matches the read data delay.

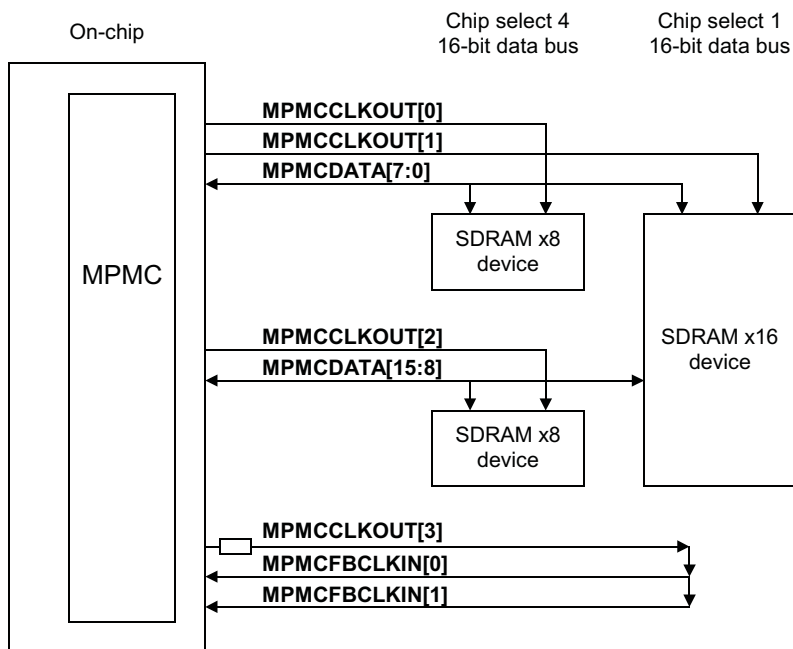


Figure 8-16 Mixed width SDRAM memory devices

Figure 8-16 shows series termination on the output clocks used to generate the feedback clocks.

SRAM and SDRAM memory devices

Figure 8-17 on page 8-42 shows an example of a way of connecting SDRAM and SRAM memory devices. Chip select 4 has four x8 SDRAM devices providing a 32-bit data bus, and chip select 1 has four x8 SRAM devices providing a 32-bit data bus.

MPMCCLKOUT[0] is used to clock the memory devices providing the lower eight bits of data. **MPMCFBCLKIN[0]** is generated from **MPMCCLKOUT[0]**. The memory devices that provide the next eight bits of data are clocked by **MPMCCLKOUT[1]**. The read data is registered by **MPMCFBCLKIN[1]**. The same scheme is used for the other bytes of data.

This methodology reduces the load on the output clocks and enables the read data to be accurately registered by the feedback clocks.

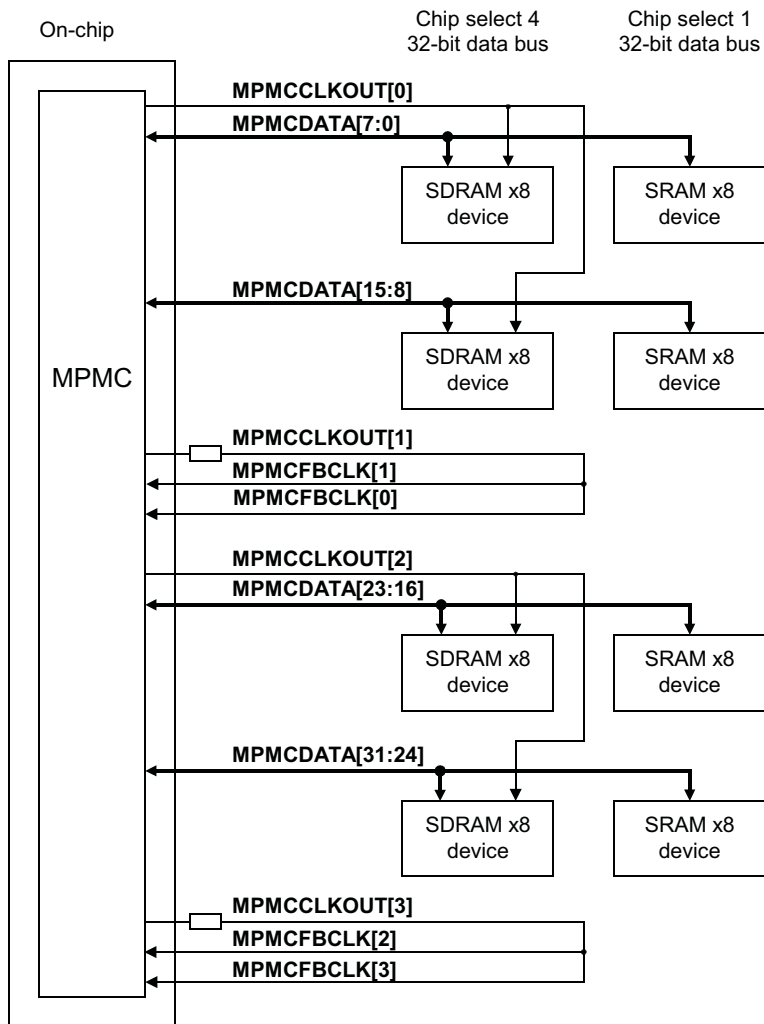


Figure 8-17 SRAM and SDRAM memory devices

Figure 8-17 shows series termination on the output clocks used to generate the feedback clocks.

Medium performance systems

For medium-performance systems, or for systems where the number of pads available is limited, a single output clock and feedback clock can be used.

MPMCCLKOUT clocks all the memory devices. The single feedback clock **MPMCFBCLKIN** generates the four **MPMCFBCLKIN[3:0]** signals on chip.

Figure 8-18 shows parallel termination on the feedback clocks.

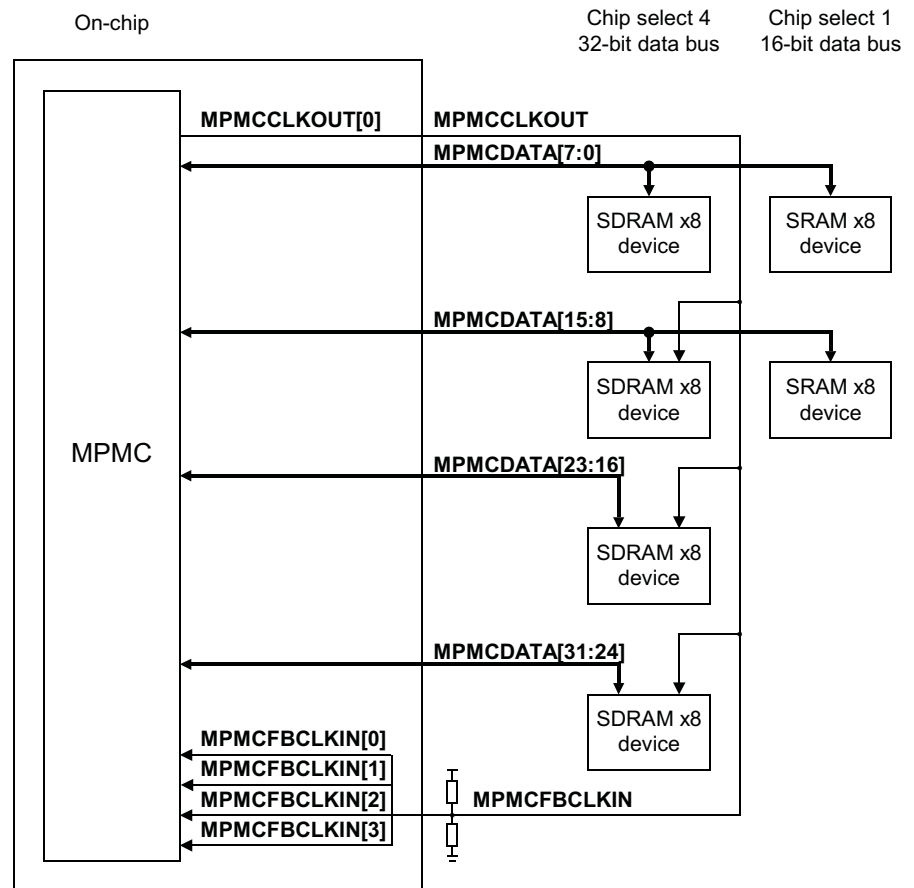


Figure 8-18 Medium performance systems

Lower performance systems

For lower-performance systems, or for systems where the number of pads available is limited, a single output clock can be used. The feedback clock can be generated on-chip by delaying the output clock.

The feedback clock delay can be generated by a number of delay elements, or by using a more complex temperature and voltage compensated delay block, for example a *Delay Locked Loop (DLL)*.

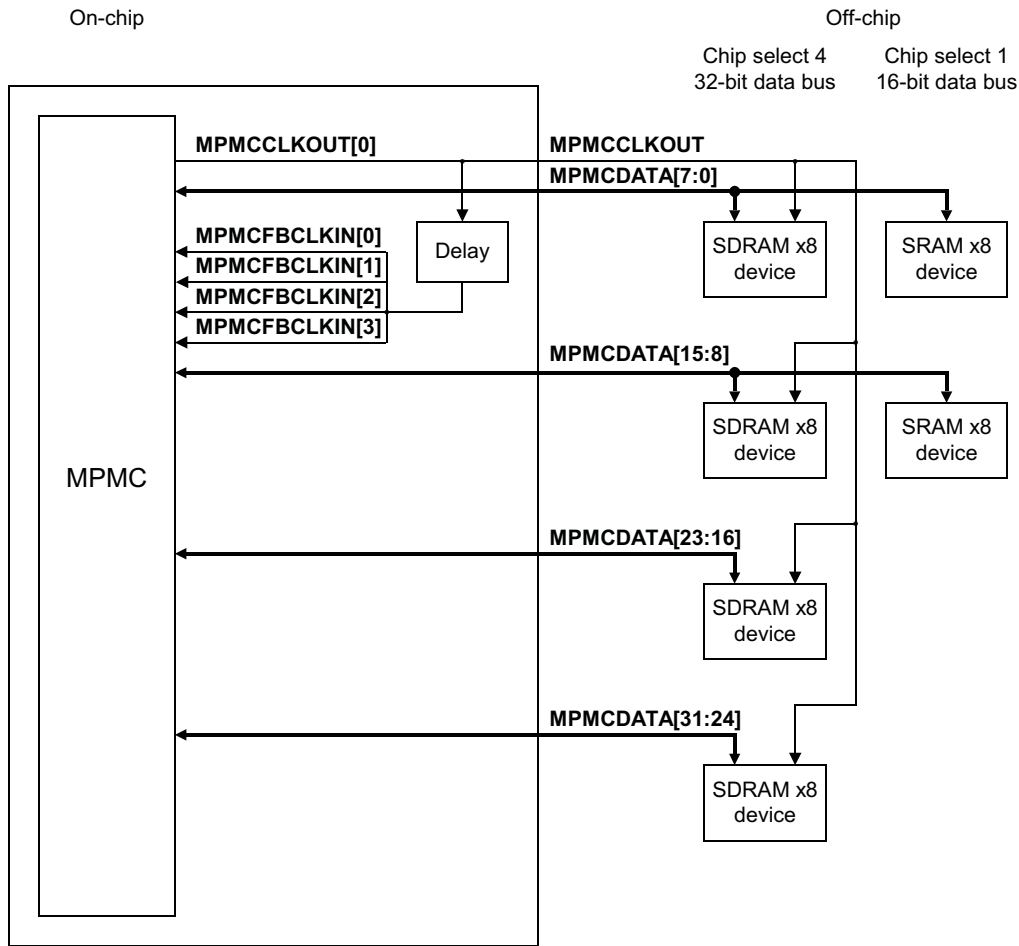


Figure 8-19 Lower performance systems

Chapter 9

Power Strategy

This chapter describes the power strategy aspects relating to the MPMC, and also describes ways to reduce power consumption. It contains the following section:

- *ASIC silicon process and cell library* on page 9-2
- *ASIC design decisions* on page 9-3
- *Memory device selection* on page 9-6
- *MPMC* on page 9-9.

9.1 ASIC silicon process and cell library

The silicon process and cell library used by the ASIC has a direct impact on the maximum frequency of the design and the amount of power consumed by the ASIC.

Power is consumed both when there is no activity in the design (static power consumption), and while the design is active (dynamic power consumption).

———— **Note** ————

Generally a process optimized for low-power consumption does not operate as fast as non-power optimized processes.

—————

9.1.1 Static power consumption (static leakage current)

The static power consumption is because of leakage current in the process. Static power consumption is proportional to:

- the size of the design
- the silicon process.

9.1.2 Dynamic power consumption (gate transitioning)

Dynamic power consumption is because of gate transitioning. Dynamic power consumption is proportional to:

- the voltage of the design
- the frequency of operation
- the size of the design
- the silicon process.

9.2 ASIC design decisions

This section describes ways to reduce the power consumption of the ASIC design. It contains the following subsections:

- *Clock gating*
- *Power management controller* on page 9-4
- *Improving system efficiency* on page 9-4.

9.2.1 Clock gating

Clock gating can be performed at a number of levels:

High-level clock gating

High-level clock gating involves gating the clock before it reaches the block. This type of clock gating is normally performed in the PMU or in the clock generator

High-level clock gating provides the coarsest form of clock control. When the clock is gated the block is disabled, and there is no power consumed in the clock tree or the blocks to which the clock is connected.

Module-level (architectural) clock gating

Architectural clock gating involves gating the clock automatically to modules in a block when the clock is not required. This type of clock gating is performed internally to a block.

Architectural clock gating provides a more fine-grained form of clock control. When the clock is gated only the specific unused module is disabled in the block.

Low-level clock gating

Low-level clock gating involves grouping a number of d-type flip-flops together and only enabling a clock to them when required. This type of clock gating is normally performed using EDA tools, for example Synopsys Power Compiler.

Low-level clock gating provides the most fine-grained form of clock control. The clock enabling and disabling is performed automatically. When the clock is gated the d-types in the group are disabled and the rest of the design functions as normal.

Note

Many systems use all three approaches to reduce power consumption.

9.2.2 Power management controller

Power management controllers normally contain the following functionality:

- Disable and enable clocks in the design when necessary.
- To provide lower frequency clocks when the system is not required to operate at the maximum frequency.
- To put devices in low-power modes when they are not required, for example by placing SDRAM into self-refresh mode.

Providing a large number of clock outputs from the PMU enables finer grained control over the blocks that are enabled or disabled, at the expense of increasing design complexity of the PMU.

9.2.3 Improving system efficiency

System designs that perform tasks more efficiently, generally have lower power consumption.

Designs that make use of memory more efficiently (by reducing the average memory latency) complete a given task in fewer clock cycles and reduce power consumption. This can be performed in a number of ways:

- by increasing processor cache size
- the addition of on chip memory
- by using blocks that access the external memory more efficiently.

———— **Note** ————

Memory transactions on-chip are much more power efficient than off-chip transactions.

Cache and cache size

Using a cached processor, or increasing the cache size reduces the number of external memory transactions and reduces the average transfer latency. This means a given task is completed in fewer clock cycles, and the power consumption is reduced. In addition, memory transactions to on-chip memory consume less power than off-chip memory transactions.

However, having a large cache increases the caches static power consumption, and increases the amount of power consumed for each cache transaction because of additional cache loading. Normally this increase in power consumption is offset by the power saved elsewhere in the system.

Additional cache levels

Additional cache levels can reduce the number of external memory transactions and reduce the average transfer latency for both processors and peripherals (assuming the cache can be shared). This means a given task is completed in fewer clock cycles, and the power consumption is reduced. In addition, memory transactions to on-chip memory consume less power than off-chip memory transactions.

However, having additional levels of cache increases the static power consumption of the ASIC. Dynamic power consumption is not as high as having a single large level 1 cache, because the cache loading is smaller for the majority of the transactions. Normally the increase in power consumption is offset by the power saved elsewhere in the system.

On-chip memory

On-chip memory reduces the number external memory transactions and reduces the average transfer latency for both processors and peripherals (assuming the cache can be shared). This means a given task is completed in fewer clock cycles, and the power consumption is reduced. In addition, memory transactions to on-chip memory consume less power than off-chip memory transactions.

However, on-chip memory increases the static power consumption of the ASIC. Normally the increase in power consumption is offset by the power saved elsewhere in the system.

Efficient transaction patterns

Blocks that perform efficient external memory access patterns (transactions that take into account the external memories characteristics) have lower average transfer latency, complete the task in fewer clock cycles, and reduce power consumption.

9.3 Memory device selection

The number and type of memory devices in a system impacts power consumption.

9.3.1 Number of memory devices

Having a large number of external memory devices connected to the MPMC increases the external load that the pads require to drive. Therefore higher drive strength pads are required and this increases power consumption.

In addition, a system with a very large external load might mean that the memory interface is unable to be clocked at the frequency required.

9.3.2 Memory device DC characteristics

Memory devices have specific DC (voltage) requirements. These requirements must be similar between all the specified memory devices otherwise they cannot operate on the same memory interface.

———— **Note** ————

Lower voltage memory devices generally consume less power, and are slower than higher voltage parts.

SDRAM memory device characteristics

SDRAM memory has a number of features to reduce power consumption:

Clock enable (CKE)

The SDRAM clock can be gated internally to the memory device by enabling or disabling the SDRAM **CKE** signal. This reduces power consumption of the SDRAM memory when the device is idle.

The MPMC supports dynamic enabling and disabling of CKE, by programming the MPMCDynamicControl Register, dynamic memory clock enable (CE) field.

Self-refresh mode

If the SDRAM memory does not require to be accessed, but the contents of the memory must be retained, the memory can be placed in self-refresh mode. In this mode the SDRAM memory performs the required refresh sequences automatically. The ASIC can then be put in a low-power mode, and the clocks can be disabled.

The MPMC supports the self-refresh mode by the use of the self-refresh request (SR) field in the MPMCDynamicControl Register, the self-refresh acknowledge (SA) field in the MPMCStatus Register, or by hardware by using the **MPMCSREFREQ** and **MPMCSREFACK** signals.

SDRAM clock gating

The MPMC supports another low-power mode where the SDRAM memory clock is gated in the MPMC. This provides additional power reduction over using the CKE methodology, because the heavily loaded external SDRAM clock signal is disabled before it goes off chip. This means that the output pad and PCB are not required to toggle unnecessarily.

The MPMC supports SDRAM output clock gating by using the memory clock static enable (MSE), memory clock dynamic enable (MDE), and memory clock self-refresh enable (MRE) fields of the MPMCDynamicControl Register.

Note

Programming both the dynamic enabling and disabling of CKE and the external clock signal reduces power consumption during normal operation to a minimum.

JEDEC low-power SDRAM

JEDEC low-power SDRAM has several additional power reduction features compared to standard SDRAM:

- One of the main ways that low-power SDRAM memory parts reduce power consumption over standard SDRAM parts is that they operate at a lower voltage and therefore consume less power than standard SDRAM memories.
- Low-power SDRAM parts are designed to work in systems where power consumption is an important consideration, and where the clock frequency of the design might not be particularly high. These devices therefore generally permit lower CAS latency values, for example CAS latency 1, compared to standard SDRAM.
- Self-refresh mode power consumption is also reduced for low-power SDRAM devices. The devices enable the ambient temperature to be programmed into the device. The frequency of the self-refresh commands generated internally by the SDRAM are then scheduled accordingly. To use this functionality the ASIC must

be able to measure the temperature. This temperature value must then be programmed into the SDRAM memory device (using the MPMC) before self-fresh mode is entered.

- Low-power SDRAM provides a facility for selecting the memory banks to refresh in self-refresh mode. The memory banks that are not refreshed lose their data. When using this feature it is beneficial to use the bank, row, column (BRC) address-mapping scheme in the MPMC, so that a linear area of memory is not mapped over multiple SDRAM memory banks.
- Finally low-power SDRAM memories also support deep sleep mode. In this mode the SDRAM memory is powered down, and the content of the memory is lost.

SDR-SDRAM memory power analysis

For information on the power consumption of a particular low-power SDR-SDRAM memory device, see the respective data sheet.

Low power SDR-SDRAM memory power analysis

For information on the power consumption on a particular low-power SDR-SDRAM memory device, see the respective data sheet.

9.4 MPMC

The MPMC is designed to reduce external memory consumption by enabling the memory clocks to be gated and supporting the low power features of the memory devices.

Power consumption can be reduced in the MPMC by disabling the clocks to the MPMC when they are not required. This clock gating must be performed cleanly. Choose appropriate pads to drive the external signals. Using pads with larger drive strength than required wastes power unnecessarily. However, using pads with too small a drive strength might mean that the system might not be able to be clocked at the required frequency.

9.4.1 MPMC clock gating

Clocks that are not required by a specific configuration can be tied off, or gated:

Note

Clock gating must be performed with care and fully verified. If clock gating is performed incorrectly the system might not function.

HCLK

This signal clocks the AMBA interface signals, the static memory controller, and the TIC. This is required for SDRAM auto-refresh commands.

If the memory does not have to be accessed, and the SDRAM memory does not have to be auto-refreshed, for example when the memory device is in self-refresh mode, the clock can be gated.

MPMCCLK

This signal clocks the dynamic memory controller logic. This is required for SDRAM auto-refresh commands.

If the memory does not have to be accessed, and the SDRAM memory does not have to be refreshed, the clock can be gated.

MPMCCLKDELAY

This signal is required if the command delayed pad interface mode is used in the design. If the clock-delayed mode is not used, or if the memory does not have to be accessed, this clock can be disabled or tied off.

9.4.2 Memory clock gating

The SDRAM clock signal **MPMCCLKOUT** can be disabled automatically when the SDRAM memory is not in use, or by software by programming the MPMCDynamicControl Register appropriately.

9.4.3 Memory clock enable

The SDRAM clock enable signal, **MPMCCKEOUT**, enables or disables the SDRAM memory internal clock gating.

This signal can be disabled automatically when the SDRAM memory is not in use, or by software by programming the MPMCDynamicControl Register appropriately.

Chapter 10

Pad Interface Timing

This chapter describes the signals that interface with the MPMC block. It contains the following sections:

- *Overview* on page 10-2
- *Signal delay* on page 10-3
- *Method to reduce delay* on page 10-4
- *Methods to reduce skew* on page 10-6
- *Methods to minimize the effects of delay and skew* on page 10-7
- *Example SDRAM memory timing diagram* on page 10-8
- *SDRAM memory timing paths* on page 10-9.

10.1 Overview

The performance of the MPMC is normally limited by the speed that data, address, and control information can be sent to, and data can be read from, the SDRAM memory devices.

There are two factors that limit pad interface performance:

- signal delay to and from the SDRAM memory devices
- skew between the signals.

Note

Process shrinks do not generally improve the performance of the pads. Therefore a memory controller whose performance is limited by the pad interface does not generally run any faster when a smaller process geometry is used.

10.2 Signal delay

The pads add a large delay and skew to the signals going both off and on chip. This delay is, in part, dependent on the load of the PCB interconnect and the devices to be driven.

The PCB interconnect also adds a delay and skew on the signals. Figure 10-1 shows an on-chip to off-chip delay for a single signal.

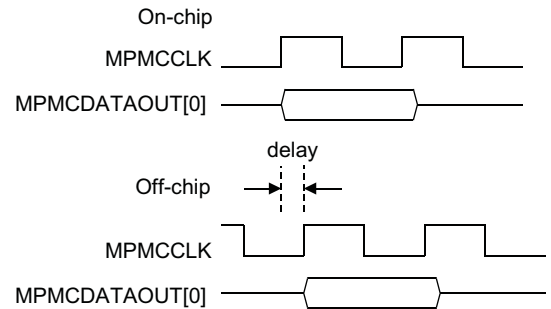


Figure 10-1 On-chip to off-chip delay for a single signal

10.3 Method to reduce delay

Signal delay can be reduced by using appropriately sized pads, and minimizing the load on the pads, for example by reducing the number of memory devices in the system.

If more than one output clock, **MPMCCLKOUT**, is used, the load on the clock can be spread.

10.3.1 Signal skew

Signal skew can be split into static, dynamic, and environmental effects.

10.3.2 Static effects

The types of static effects are:

Routing mismatch

The paths of the different signals might be different.

Different loading

The signals might be loaded differently.

Source timing different

The signals might be generated at different times.

10.3.3 Dynamic effects

The types of dynamic effects are:

- Data pattern. The data pattern might affect the time when the signal becomes valid. For example, a signal changing from 0 to 1 might take a different time from a signal changing from 1 to 0.
- *Simultaneously Switching Outputs (SSO)*. If a large number of outputs change at the same time, it might take more time to reach the valid value than when a small number of signals change.

Figure 10-2 on page 10-5 shows on-chip to off-chip delay with skew for a single bit of the data bus **MPMCDATAOUT[0]**. Registering this signal on the rising edge of the clock, in this case, captures the data correctly.

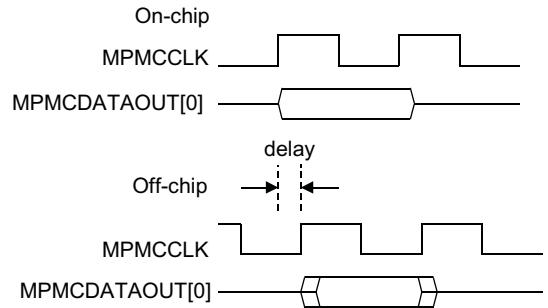


Figure 10-2 On-chip to off-chip delay with skew for a single signal

Figure 10-3 shows on-chip to off-chip delay with skew for multiple signals, **MPMCDATAOUT[31:0]**. The time when all the signals on the data bus are valid at the same time is smaller. In this case, registering the group of signals on the rising edge of the clock does not capture the data correctly.

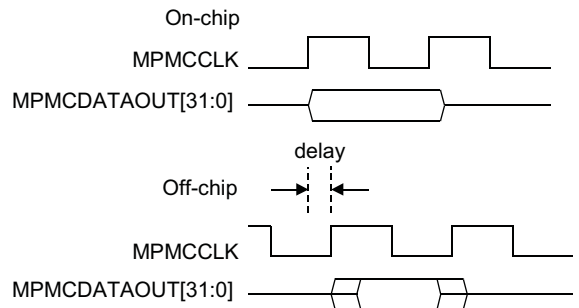


Figure 10-3 On-chip to off-chip delay with skew for multiple signals

10.3.4 Environmental effects

The above sources of skew are affected by both temperature and voltage. This adds additional skew, because ideally the design works at a high voltage and low temperature, and low voltage and high temperature.

10.4 Methods to reduce skew

Signal skew can be reduced by using appropriate pads. In some systems this might, for example, be performed by using the same tristate bidirectional pads for the data, address, and control signals.

The PCB routing for each of the signals, and the loading of the signals must ideally be similar.

10.5 Methods to minimize the effects of delay and skew

The effects of delay and skew can be minimized on the read data path by using feedback clocks to register the data in the center of the data valid window. There are four feedback clocks, **MPMCFBCLKIN[3:0]**. Each feedback clock is associated with eight bits of the data bus:

- **MPMCFBCLKIN[0]** is associated with **MPMCDATA[7:0]**
- **MPMCFBCLKIN[1]** is associated with **MPMCDATA[15:8]**
- **MPMCFBCLKIN[2]** is associated with **MPMCDATA[23:16]**
- **MPMCFBCLKIN[3]** is associated with **MPMCDATA[32:24]**.

Ideally the feedback clocks are generated off chip from the output clock. They must then be routed with their associated data to the ASIC. Therefore, the feedback clock is delayed by a similar amount as the read data.

10.6 Example SDRAM memory timing diagram

Figure 10-4 shows an SDRAM timing diagram for the MT48LC4M16A2 Micron device, speed grade -7E, CAS=2. The control, address, and write (output) data all have the same setup and hold timing values. The read data has different timing parameters to the other signals. The read data is valid 2.1ns before the positive edge of the clock, and 2.7ns after the clock edge. Therefore a feedback clock generated from the clock at the memory device is aligned with the center of the read data valid window.

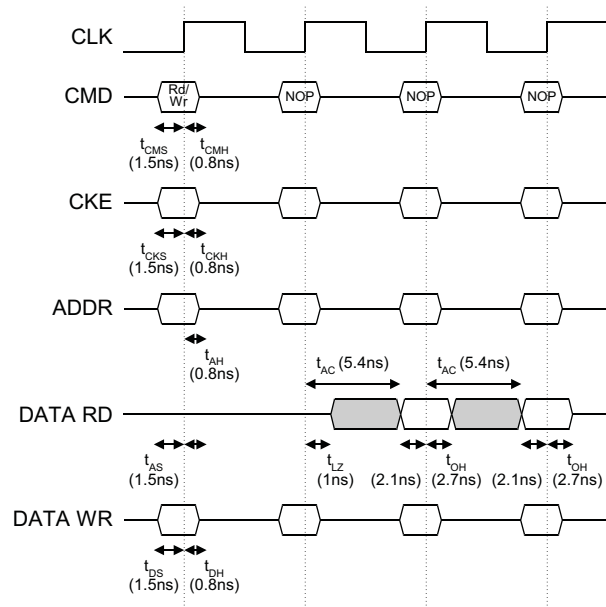


Figure 10-4 SDRAM timing diagram

10.7 SDRAM memory timing paths

There are two main timing paths:

- MPMC to the SDRAM memory (control, address, and write data transfers)
- SDRAM memory to the MPMC (read data transfers).

10.7.1 MPMC to SDRAM memory

The command, address, and write data are generated by an on-chip clock. These signals are sent off chip to the SDRAM memory. The signals arrive at the SDRAM memory delayed by t_{PID} (output pad and interconnect delay), see Figure 10-5.

Note

Figure 10-5 assumes there is no skew. To ensure that the command, address, and write data are captured correctly, either they or the clock must be delayed to ensure that the clock edge occurs in the capture window.

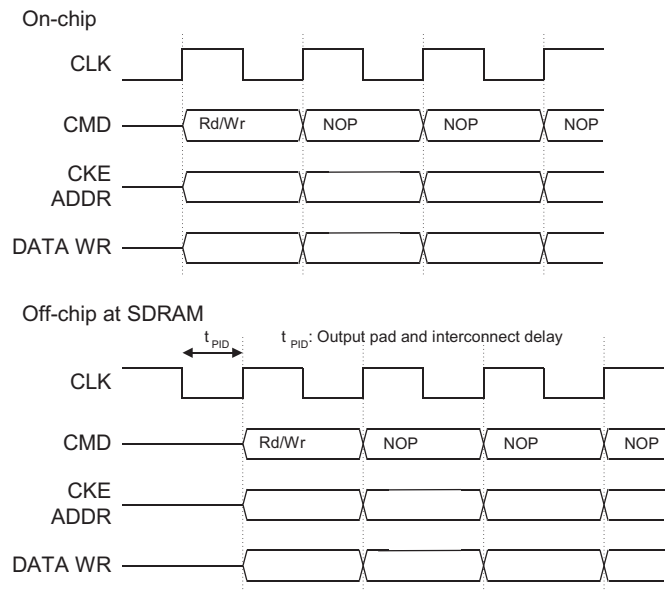


Figure 10-5 MPMC to SDRAM memory timing path

10.7.2 SDRAM memory to MPMC

The feedback clock is generated from the output clock. Figure 10-6 shows how the feedback clock and read data are sent on chip to the MPMC, delayed by t_{IPD} (interconnect and input pad delay). The on-chip feedback clock edge occurs in the center of the read data capture window because it is delayed by the same amount as the read data.

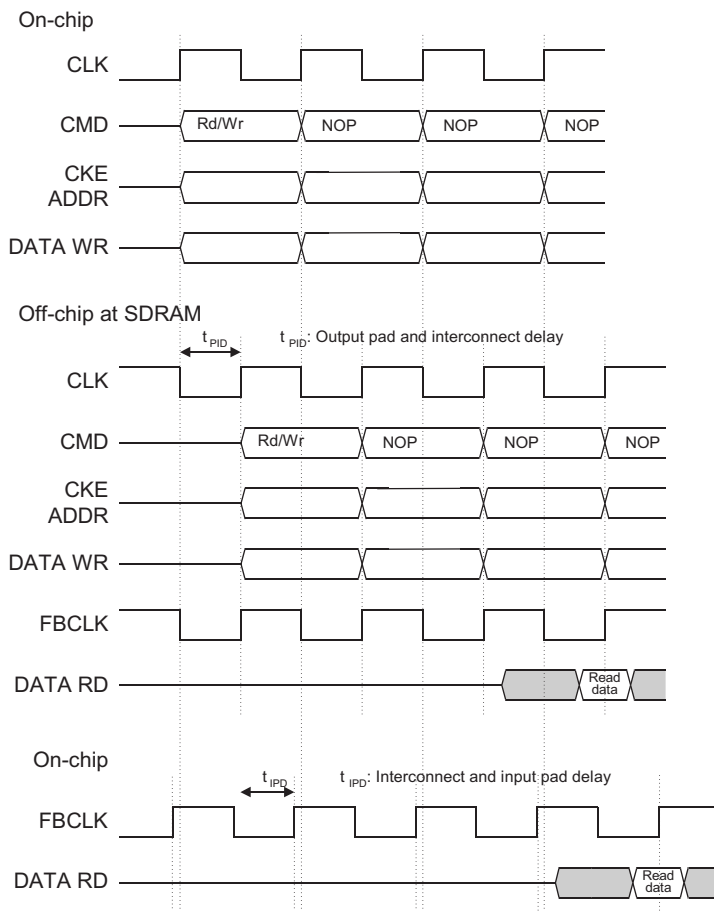


Figure 10-6 SDRAM memory to MPMC timing path

Chapter 11

Troubleshooting

This chapter describes how to troubleshoot the MPMC block. It contains the following sections:

- *Software* on page 11-2
- *System* on page 11-6.

11.1 Software

This section covers typical software problems. See Table 11-1 for a list of problems and their suggested remedies.

Table 11-1 Software troubleshooting

Problem	Suggested remedy
Dynamic memory does not work. However, static memory operates correctly.	Ensure that the SDRAM memory is initialized correctly. The appropriate address mapping, burst size, and CAS latency must be programmed. ^a
	Ensure that the buffers are enabled during normal operation (MPMCDynamicConfig, B = 1).
	If the memory is being clocked at twice the frequency of HCLK , ensure that HCLK and MPMCCLK are synchronous, and that their positive clock edges are aligned.
Mode programming and initialization issues.	The auto-refresh value programmed in the MPMCDyRef Register must be greater than the precharge period and the time required for applying the auto-refresh requests to all the dynamic memory chip selects: Auto-refresh Value > (MpmcDytRPQ + 1) + (MpmcDytRFCQ + 1) + 4) where MpmcDytRPQ is the precharge period, MpmcDytRFCQ is the auto-refresh latency, and the four extra clock cycles are required to apply the auto-refresh cycles. A value programmed equal to or less than this always keeps the refresh state machine running and the MPMC ends up giving only refresh cycles. The SDRAM memory initialization can take two possible variations: • NOP, PALL, MODE, NORMAL (normal SDRAM sequence) • MODE, NORMAL (Micron SyncFlash sequence). In both cases, there must be at least one idle cycle between the first and second steps. For normal SDRAMs, a 200µS delay must be maintained between the first and second steps. Therefore, the one-clock idle is automatically satisfied. Nothing specific is mentioned in the SyncFlash initialization sequence. This restriction comes because of the necessity of the mode state machine to be able to restart the clock and CKE, after exit from the deep-sleep state.^b
The MPMC does not seem to be programmed correctly.	If a cached processor is being used to initialize the MPMC, ensure that the MPMC register address space is marked as noncacheable and nonbufferable.
Reads to flash memory are fine, however flash memory writes do not function correctly.	Ensure that the flash memory space is marked as noncacheable and nonbufferable when writing to or controlling the flash memory.
	Ensure that the flash chip selects buffers are disabled when writing to or programming the flash memory.

Table 11-1 Software troubleshooting (continued)

Problem	Suggested remedy
How do I initialize the SDRAM memory?	Initialization sequence: 1. Program the MPMCDyConfig Register for disabling the buffers. 2. Keep the CE and CS bit in MpmcDyControl HIGH. 3. Perform the mode initialization. During the mode-programming process the MpmcDyControl Register must be written to repeatedly (for altering the SdramInit field). Ensure that the CE and CS bits (residing in the same register) are always kept at 11 value. 4. After mode programming is finished, reprogram to enable or disable the buffers as required. 5. Program the CS and CE bits to the required configuration, when the mode programming is done. If the CE and CS bits are not kept HIGH, the refresh state machine switches off the clock or clock-enable during the mode programming sequence. Mode-register programming is done only at the very beginning (after reset). Multiple mode programming is required only if the controller is programmed for deep-sleep mode entry. If there is no deep-sleep entry, no multiple-mode programming is allowed, and is not necessary. After the controller exits deep-sleep mode, all the chip-selects must be mode-programmed (in case a few chips are deep-sleep SDRAMs and the rest are normal SDRAMs).
The MPMC is in self-refresh mode on power-on reset.	The MPMC sets the self-refresh request and self-refresh acknowledge software bits HIGH on power-on reset. This enables SDRAM memory that has been placed into self-refresh mode to be held if the ASIC is powered down. The self-refresh request bit must be programmed LOW to enable the SDRAM memory to be accessed. This takes the SDRAM CKE signal HIGH, bringing the SDRAM memory out of self-refresh. The MPMC then ensures that the SDRAM self-refresh exit time is adhered to. If the SDRAM memory was not previously placed into self-refresh mode, this sequence has no effect on the memory.
Transactions to a peripheral connected to a static memory chip select do not function correctly.	Ensure that the peripheral memory space is marked as noncacheable and nonbufferable. Ensure that the MPMC buffers are disabled for the relevant chip select.
The MPMC initialization code that has been developed does not work.	Examine the initialization sequences provided in this document. Examine the device driver code provided in the delivery.

Table 11-1 Software troubleshooting (continued)

Problem	Suggested remedy
Static memory page mode accesses do not function correctly. However, normal static memory accesses function correctly.	Ensure that the address output from the MPMC, MPMCADDRROUT, is connected to the appropriate address bits on the static memory. The MPMC supports two modes of operation: • If the MPMCREL1CONFIG input signal is tied HIGH, the connection of the MPMC address depends on the width of the static memory chip select. For 32-bit static memory chip selects MPMCADDRROUT[2] must be connected to the least significant address of the static memory. For 16-bit static memory chip selects MPMCADDRROUT[1] must be connected to the least significant address of the static memory. For 8-bit static memory chip selects MPMCADDRROUT[0] must be connected to the least significant address of the static memory. • If the MPMCREL1CONFIG input signal is tied LOW, the least significant address bit of the MPMC output address must be connected to the least significant address bit of the static memory device. The MPMC address output signal must be connected unscrambled to the memory devices address bus.
Accessing a byte wide static memory chip select functions correctly. However, accessing 16-bit or 32-bit chip selects do not function correctly.	Ensure that the address output signal, MPMCADDRROUT is connected to the correct address bits on the static memory. The MPMC supports two modes of operation: • If the MPMCREL1CONFIG input signal is tied HIGH, the connection of the MPMC address depends on the width of the static memory chip select. For 32-bit static memory chip selects MPMCADDRROUT[2] must be connected to the least significant address of the static memory. For 16-bit static memory chip selects MPMCADDRROUT[1] must be connected to the least significant address of the static memory. For 8-bit static memory chip selects MPMCADDRROUT[0] must be connected to the least significant address of the static memory. • If the MPMCREL1CONFIG input signal is tied LOW, the least significant address bit of the MPMC output address must be connected to the least significant address bit of the static memory device.
The SDRAM memory does not seem to be initialized correctly.	Ensure that the SDRAM memory space is marked as noncacheable and nonbufferable when programming the SDRAM memory. Ensure the buffers for the relevant chip select are disabled when programming the SDRAM memory.
Static memory extended wait transfer issues.	The static memory transfer time programmed in the MPMCStExdWt Register must be small enough to enable the dynamic memory to be refreshed at regular intervals. Using extremely long transfer times might mean that SDRAM devices are not refreshed correctly.

Table 11-1 Software troubleshooting (continued)

Problem	Suggested remedy
The SDRAM read data returns earlier or later than expected.	Ensure that the MPMC and SDRAM memory have been programmed with the same CAS latency. Ensure that the clocking scheme chosen (see the MPMCDynamicReadConfig Register) is the same as the scheme chosen in the testbench/system.
The bank-row-column or row-bank-column SDRAM addressing scheme works, however the other addressing scheme does not function correctly.	Ensure that the correct address mapping scheme is used when programming SDRAM memory.
The MPMC seems to be performing additional accesses to SDRAM memory or the SDRAM memory contents appear to be incorrect.	Ensure that the MPMC has been initialized correctly. SDRAM memory devices connected to a chip select with a 32-bit wide data bus must be programmed to perform a burst of four. SDRAM memory devices connected to a chip select with a 16-bit wide data bus must be programmed to perform a burst of eight. Ensure the value used to program the SDRAM memory is correct. The value used to program the SDRAM memory depends on the address mapping scheme chosen.
Static memory byte lane state issues.	For most static memory chip selects constructed from 8-bit or non byte-partitioned memory devices, the byte lane state (PB) bit must be cleared to 0 in the respective MPMCStConfig Register. For most static memory chip selects constructed from 16 or 32-bit memory devices, the byte lane select (PB) bit must be set to 1 in the respective MPMCStConfig Register.
When can the MPMC be enabled and disabled?	The MPMCEnable bit of the MPMCControl Register can be made LOW only when: - the busy bit of MPMCStatus Register is LOW - the buffer status(S) bit of MPMCStatus Register is LOW - the memory control and configuration registers (both dynamic and static) can only be initialized during system initialization or when the MPMCEnable bit is disabled.
a. During initialization the buffers (MPMCDynamicConfig, B = 0) must be disabled, and CKE (MPMCDynamicControl, CE = 1) must be HIGH. b. The normal sequence for SDRAMs can also be applied to SyncFlash memory parts (especially in the situation where the chip selects are populated with both SyncFlash and SDRAM devices). However, if only SyncFlash devices are used in a system, and SDRAM is not used, the SyncFlash-sequence can be used.	

11.2 System

This section covers typical system problems. See Table 11-2 for a list of problems and their suggested remedies.

Table 11-2 System troubleshooting

Problem	Suggested remedy
When can the self-refresh signal go LOW?	The MPMCSREFREQ pin, or the self-refresh request (SR) field in the MPMCDynamicControl Register must not go LOW until the MPMCSREFACK pin and the self-refresh acknowledge (SA) bit of the MPMCStatus Register are asserted by the MPMC.
What is the SynchFlash sequence?	During SynchFlash initialization, the RP line is handled entirely by software. The nMPMCRPOUT and MPMCRPVHHOUT outputs are directly controlled by the respective bits in the MPMCDyControl Register. The flash-command-set operations (as against the normal array read mode, similar to SDRAMs) must be performed in the unbuffered mode.
What is the low-power deep-sleep sequence?	Before setting the deep-sleep bit in the MPMC, ensure the dynamic memory controller is idle, reflected by the busy-bit in the MPMCStatus Register. The low-power, deep-sleep exit is not automatic. That is, the low-power, deep-sleep bit has to be explicitly reset by software and only then can normal accesses be started.
How must HSELMPMCxG and HSELMPMCxS[7:0] function?	HSELMPMCxG must go active whenever there is an access to the MPMC. At the same time the relevant bit of HSELMPMCxS[7:0] must go active. HSELMPMCxS[7:0] indicates the memory chip select that the transfer is for.
What is the power-on reset memory map?	On power-on reset, memory chip select 1 is mirrored onto memory chip select 0 and chip select 4. Therefore, any transactions to memory chip select 0 or chip select 4 (or chip select 1) access memory chip select 1. The boot code must be located in the memory device connected to chip select 1. Clearing the address mirror bit (M) in the MPMCControl Register disables address mirroring. Memory chip select 0, chip select 4, and memory chip select 1 can then be accessed as normal.
How must the self-refresh signals be connected?	The MPMCSREFREQ pin must be connected to the <i>Power Management Unit</i> (PMU) in the system. The PMU must assert the MPMCSREFREQ signal to enter the self-refresh mode. When asserted, MPMCSREFREQ pin must not go LOW until the MPMCSREFACK pin is asserted by the MPMC. If the PMU does not support this mechanism, the self-refresh mode can be entered manually by software, by setting the MPMCSREFREQ bit of the MPMCDyControl Register and polling the MPMCSREFACK bit of the MPMCStatus Register.

Table 11-2 System troubleshooting (continued)

Problem	Suggested remedy
How must the static memory write enable signals be connected?	For static memory chip selects constructed from 8-bit or non byte-partitioned memory devices, the nMPMCBLSOUT[3:0] signals must be connected to write enable (nWE) inputs of each 8-bit memory. The nMPMCWEOUT signal from the MPMC must not be used. For static memory chip selects constructed from 16-bit or 32-bit memory devices, the nMPMCBLSOUT[3:0] signals must be connected to the byte select inputs of each memory device. The nMPMCWEOUT signal from the MPMC must be connected to the write enable inputs of each memory device.
What are the requirements to enter test mode?	The TIC must be the highest priority master on the respective AHB bus. When the MPMC has entered test mode, it is assumed that there are no normal mode accesses. The MPMC enters test mode only if the sequence of events indicated below is followed. The following sequence of events describes how to use the TIC of the MPMC to apply test patterns: 1. Apply reset (HRESETn) asynchronously. 2. Assert the MPMCTESTIN test input to enter test mode. 3. When the reset is removed (synchronously), the MPMC enters TIC test mode. 4. The TIC block asserts its HBUSREQTIC signal to the arbiter, requesting access to the AHB bus. Because the TIC is the highest priority, HGRANTTIC is asserted and the granted TIC takes ownership of the AHB bus. The MPMCTESTIN signal is usually fed to the clock generation logic so that HCLK can be switched to test speed on entry into test mode.
What are the power-on reset values of input pins?	On power-on reset, the static memory chip select polarity pins, MPMCSTCS[0-3]POL must indicate the polarity of the respective chip selects (0 for active LOW and 1 for active HIGH). These values are used until the MPMCSConfig0-3 Registers are written into. When the configuration register of corresponding chip select is programmed, the register value of the chip select polarity bit in the register determines the chip select polarity. On power-on reset, the static memory chip select 1 data bus width pin MPMCSTCS1MW[1:0] must indicate the width of the chip select (00 for 8-bit, 01 for 16-bit, and 10 for 32-bit data bus width). This value is used until the MPMCSConfig1 Register is written into. When the register is programmed, the memory width bits of the register determine the memory width. On power-on reset, the endian mode of the MPMC must be driven on the MPMCBIGENDIAN pin. This value is used until the MPMCCConfig Register is written into. When this register is programmed, the register value of the Endian mode bit determines the endian mode. On power-on reset, the address mirror bit in the MPMCControl Register is asserted.

Table 11-2 System troubleshooting (continued)

Problem	Suggested remedy
How must the static memory address signals be connected?	The static memory address signals can be right-aligned or non right-aligned. Setting the MPMCREL1CONFIG signal HIGH on power-on reset means that the static memory address is not right aligned: • For a 32-bit data bus width static memory chip select, the output address bits MPMCADDRROUT[1:0] are not used. In other words MPMCADDRROUT[n+2:2] must be connected to memory device address A[n:0]. • For a 16-bit data bus width static memory chip select, the output address bit MPMCADDRROUT[0] is not used. In other words MPMCADDRROUT[n+1:1] must be connected to memory device address A[n:0]. • For an 8-bit data bus width static memory chip select, all the low order output address bits are used. In other words MPMCADDRROUT[n:0] must be connected to memory device address A[n:0] Setting the MPMCREL1CONFIG signal LOW on power-on reset means that the static memory address is right aligned. This can reduce pin count compared to not aligning the address: For an 8, 16, or 32-bit data bus width static memory chip select, all the low order output address bits are used. In other words MPMCADDRROUT[n:0] must be connected to memory device address A[n:0].

Index

The items in this index are listed in alphabetical order, with symbols and numerics appearing at the end. The references given are to page numbers.

A

- Access sequencing and memory width
 - dynamic memory 4-3
 - static memory 3-6
- Access to memory banks
 - 16 or 32-bit devices 3-40
 - 8-bit devices 3-40
- Address mapping 4-75
- Address mappings
 - 16-bit data bus BRC DDR-SDRAM 4-78
 - 16-bit data bus BRC SDR-SDRAM 4-78
 - 16-bit data bus RBC DDR-SDRAM 4-78
 - 16-bit data bus RBC SDR-SDRAM 4-78
 - 32-bit data bus BRC SDR-SDRAM 4-77
 - 32-bit data bus RBC SDR-SDRAM 4-76
- AHB memory interface 7-4

- AHB register interface 7-2
- Asynchronous page mode read ROM 3-18

B

- Bus turnaround 3-30
- Byte lane control 3-39
 - address not right justified 3-53
 - address right justified 3-47

C

- Clock feedback
 - bidirectional pad 8-32
 - off-chip 8-31
 - on-chip 8-32
- Connecting
 - mixed width memory devices 8-40
 - SDRAM and SRAM memory devices 8-41

- Connecting (continued)
 - x16 SDRAM devices 8-38
 - x32 SDRAM devices 8-39
 - x8 SDRAM devices 8-35

- Conventions
 - numerical xix
 - signal naming xviii
 - timing diagram xviii
 - typographical xvii

- Customization
 - dynamic memory controller 2-24
 - memory data bus width 2-22
 - memory device support 2-27
 - number of AHB ports 2-21
 - static memory controller 2-23
 - TIC 2-23
 - TIC and dynamic memory controller 2-26
 - TIC and static memory controller 2-25

D

- Data bus steering
 - address not right justified 3-53
 - address right justified 3-47
- Dynamic memory controller 4-1
 - address mapping 4-75
 - command descriptions 4-80

E

- Elimination of floating bytes 3-38
- Entry into TIC test mode 6-3
- Environment variables
 - AMBA 2-30
 - DIRS_VERILOG 2-30
 - FILES_VERILOG 2-30
 - GLOBAL 2-28
 - HDL_NETL 2-29
 - HDL_SOURCE 2-29
 - LIB_SYNOP 2-30
 - LIB_VHDL 2-29
 - PERIPH 2-28
 - SIMULATOR 2-28
 - TEST_ENV 2-29
 - TEST_METH 2-28
- Extended wait transfers 3-4

F

- Flash memory 3-30
- Four fed-back clocks 8-34

G

- Generic SDRAM initialization 4-81

H

- High performance systems 8-35

I

- Initialization
 - generic SDRAM 4-81
 - low-power SDRAM 4-86
 - SDRAM 4-83
 - static memory 3-6
 - SyncFlash 4-90
- Integration 2-2
 - AMBA bus connection 2-4
 - clocks 2-7
 - endianness 2-9
 - non-AMBA intra-chip connection 2-4
 - power management 2-10
 - primary input/output connections 2-6
 - resets 2-9
 - scan signals 2-9
 - supported design flow 2-2
 - synchronization 2-9
- Integration test
 - code 2-16
 - strategy 2-16

L

- Low-power SDRAM initialization 4-86

M

- Memory connection diagram 3-40
- Memory mapped peripherals 3-5
- Micron MT28F4M16S2 SyncFlash
 - initialization example 4-90
- Micron MT48LC4M16A2 SDRAM
 - initialization example 4-83
- Micron SyncFlash commands 4-93
- MPMC timing 8-11

N

- Numerical conventions xix

O

- Off-chip signals 8-4
- Off-chip timing 8-11
- On-chip signals 7-2
- On-chip timing 8-11
- Output clocks 8-34
- Output enable programmable delay 3-7

P

- Pad interface timing 10-1
- Pin count reduction 8-4
- Power management connectivity 7-24
- Product revision status xvi
- Production test 2-18

R

- Reducing data bus width 8-4
- Removing functionality 8-4
- Revision
 - status xvi

S

- SDRAM initialization 4-83
- Signal naming conventions xviii
- Static memory
 - device selection 3-2
 - read control 3-7
 - write control 3-22
- Static memory controller 3-1
- Static memory initialization 3-6
- SyncFlash
 - commands 4-93
 - initialization 4-90
- Synthesis
 - constraints 2-11
 - scripts 2-13
- System interconnection diagram 7-3

T

- Termination of off-chip signals 8-35
- Test Interface Controller (TIC) 6-2
- TIC test connectivity 7-4
- TIC test mode 6-3
- Timing
 - MPMC 8-11
 - off-chip 8-11
 - on-chip 8-11
- Timing diagram conventions xviii
- Troubleshooting
 - software 11-2
 - system 11-6
- Typographical conventions xvii

V

- Verification
 - functional 2-19
 - timing 2-19

W

- Wait state generation 3-6
- Write enable programmable delay 3-22
- Write-protection
 - dynamic memory 4-2
 - static memory 3-3

